Dart 3 · Flutter 3.x · October 2026 edition

The Complete Flutter Book

From your first widget to engine internals, architecture, testing, performance, security and releases, with interview questions and answers in every chapter.

41 chapters · 7 parts · 350+ interview questions with answers · 110 real errors with fixes

Contents

PART I — FOUNDATIONS

1. What Is Flutter? (The Big Picture)
2. Setup, Tooling, Project Structure & the Flutter CLI
3. Dart Language, Part 1: The Basics
4. Dart Language, Part 2: OOP, Generics & Modern Dart 3 Features
5. Asynchronous Dart: Event Loop, Futures, Streams, Isolates & Zones

PART II — BUILDING UI

6. Widgets Deep Dive: Stateless, Stateful, Lifecycle, BuildContext, Keys & the Three Trees
7. Layout: Constraints, Flex, Stack, Responsive Design & Overflow Errors
8. The Widget Catalog: Text, Images, Buttons, Inputs, Forms, Scaffold, Dialogs & More
9. Scrolling, Lists, Grids & Slivers
10. Theming, Material 3, Design Systems, Assets, Fonts, Localization & Accessibility
11. Navigation & Routing: Navigator 1.0, Navigator 2.0, go_router, Deep Links
12. Animations: Implicit, Explicit, Hero, Physics, Staggered, Lottie/Rive
13. Gestures, Focus & Keyboard, CustomPainter and Custom RenderObjects

PART III — STATE MANAGEMENT

14. State Management, Part 1: Concepts, setState, InheritedWidget, ValueNotifier, ChangeNotifier & Provider
15. State Management, Part 2: Riverpod, Bloc/Cubit, GetX, MobX, Redux, Signals

PART IV — DATA & PLATFORM

16. Networking: HTTP, Dio, JSON Serialization, REST, GraphQL, WebSockets, Error Handling & Offline
17. Local Storage: Preferences, Secure Storage, Files, SQLite (sqflite/Drift), NoSQL & Caching
18. Firebase, Backend Services & Common Integrations (Auth, Push, Analytics, Crashlytics, Payments, Maps, Media, Permissions, Background Work)
19. Talking to Native: Platform Channels, Pigeon, FFI, Plugins, Platform Views, Add-to-App & Web Interop

PART V — EXPERT ENGINEERING

20. Flutter Internals: Bindings, the Frame Pipeline, Build/Layout/Paint/Composite, Layers, Impeller, the Dart VM & Hot Reload
21. App Architecture for Large Projects: MVVM, Clean Architecture, Repositories, DI, Modularization & Design Patterns
22. Testing: Unit, Widget, Golden, Integration, Mocking, Coverage & TDD
23. Performance Optimization & DevTools: Jank, Rebuilds, Memory Leaks, Startup Time & App Size
24. Debugging Mastery & the Flutter Error Encyclopedia
25. Mobile App Security for Flutter (OWASP MASVS in Practice)
26. Flavors, Signing, Building, Store Release, CI/CD & Code Push

27. Flutter Web, Desktop, Embedded, TV, Wearables, Games & AI Features
28. Working on a Big Flutter Project in a Company: Standards, Code Review, Git, Upgrades, Collaboration & Playbooks

PART VI — INTERVIEW PREPARATION

29. Interview Question Bank, Part 1: Freshers & Junior (0–2 years)
30. Interview Question Bank, Part 2: Intermediate (2–5 years)
31. Interview Question Bank, Part 3: Senior, Lead & Architect (5+ years) + Mobile System Design
32. Coding Rounds: Dart Problems, Output Puzzles, Machine-Coding Apps & Behavioral Questions
33. Cheat Sheets, Deprecated-API Migration Table, Glossary & Learning Roadmap

PART VII — DEEP DIVES WITH FULL CODE

34. Camera, Image Picking, Video & Audio (Full Code)
35. Maps & Location: Google Maps, OpenStreetMap, Geolocation, Geocoding, Geofencing & Background Tracking
36. In-App Purchases, Subscriptions & Payment Gateways (Full Code)
37. State Restoration, Custom Slivers & Advanced Widgets (Tables, Overlays, Search, Stepper, Autocomplete)
38. Code Generation Deep Dive (build_runner, build.yaml, writing a generator) & Publishing Packages to pub.dev
39. Release Automation: fastlane (Android & iOS), Codemagic, GitHub Actions for iOS, and Patrol E2E Tests
40. Games with Flame, Advanced FFI (structs, callbacks, ffigen) & Custom Stream Transformers
41. Interview Question Bank, Part 4: 120 Extra Questions (All Levels)

PART I

Foundations

Chapter 1 — What Is Flutter? (The Big Picture)

Goal of this chapter: By the end you will be able to explain what Flutter is, how it is different from other ways of building apps, how it draws pixels on the screen, and why companies choose it — the way you would explain it to a child *and* the way you would explain it to a senior interviewer.

1.1 The "5-year-old" explanation

Imagine you want to draw the same picture on **six different pieces of paper**: one is an iPhone, one is an Android phone, one is a web browser, one is a Windows PC, one is a Mac, and one is a Linux computer.

- **Old way:** You hire six different artists. Each artist speaks a different language (Swift, Kotlin, JavaScript, C#...). Each artist draws the picture a little bit differently. When you want to change the colour of the sun, you must tell all six artists. Expensive and slow.
- **Flutter way:** You hire **one artist** (you!) who speaks **one language (Dart)**. You give this artist a **magic paintbrush (the Flutter engine)** that works on every piece of paper. The artist paints *every single pixel* themselves, so the picture looks *exactly* the same everywhere.

That magic paintbrush is the key idea: **Flutter does not ask Android or iOS to draw buttons for it. Flutter draws its own buttons**, pixel by pixel, on a blank canvas the operating system gives it.

1.2 The grown-up definition

Flutter is an open-source **UI toolkit / SDK** created by **Google** for building **natively compiled**, multi-platform applications from a **single codebase**. It targets:

Platform	Status
Android	Stable
iOS	Stable
Web	Stable (since Flutter 2.0, March 2021)
Windows	Stable (since Flutter 2.10, Feb 2022)
macOS	Stable (since Flutter 3.0, May 2022)
Linux	Stable (since Flutter 3.0, May 2022)
Embedded (cars, kiosks, TVs)	Through custom embedders (e.g., Toyota, LG webOS, Sony)

Key facts interviewers like:

- **Language:** Dart (also created by Google).

- **First announced:** 2015 as "Sky" (Dart Developer Summit). **Flutter 1.0** released **4 December 2018.**
- **Flutter 2.0** (March 2021): web stable + sound null safety.
- **Flutter 3.0** (May 2022): all 6 platforms stable, Material 3 begins.
- **Flutter 3.10** (May 2023): Dart 3 (100% sound null safety, records, patterns, sealed classes), Impeller default on iOS.
- **Flutter 3.16** (Nov 2023): Material 3 becomes the default (`useMaterial3: true`).
- **Flutter 3.22** (May 2024): WebAssembly (Wasm) web compilation stable.
- **Flutter 3.27** (Dec 2024): Impeller default on Android API 29+ (Android 10+).
- **Flutter 3.29** (Feb 2025): Skia opt-out removed on iOS (Impeller only on iOS).
- **Flutter 3.38 / Dart 3.10** (Nov 2025): Dart *dot shorthands*, widget previews in IDEs, web hot reload by default.
- **Flutter 3.44** (May 2026): Swift Package Manager default for iOS/macOS, freezing of `material.dart` / `cupertino.dart` in the framework ahead of decoupling.
- **Flutter 3.47 / Dart 3.13** (Aug 2026): standalone `material_ui` and `cupertino_ui` packages on pub.dev (1.0); Dart **primary constructors** stable.

⚠️ Release history moves fast. Always check your actual version with `flutter --version`. This book focuses on concepts that stay true across versions, and flags things that changed recently so you are never surprised in an interview or in a codebase that uses an older SDK.

1.3 Why does Flutter exist? (The problem it solves)

Before Flutter, there were three main ways to build a mobile app:

1.3.1 Native development

- Android with **Kotlin/Java**, iOS with **Swift/Objective-C**.
- ✅ Best access to platform features, best performance.
- ❌ Two teams, two codebases, features drift apart, double cost.

1.3.2 Web-view / hybrid apps (Cordova, Ionic, Capacitor)

- Your app is a website inside a native "browser box" (WebView).
- ✅ Web developers can build apps.
- ❌ Often feels slower, less "native", limited by the WebView.

1.3.3 JavaScript-bridge frameworks (React Native — classic architecture, Xamarin.Forms)

- You write JavaScript; it tells the platform to create **real native widgets** (UIButton, android.widget.Button).
- ✅ Native look.

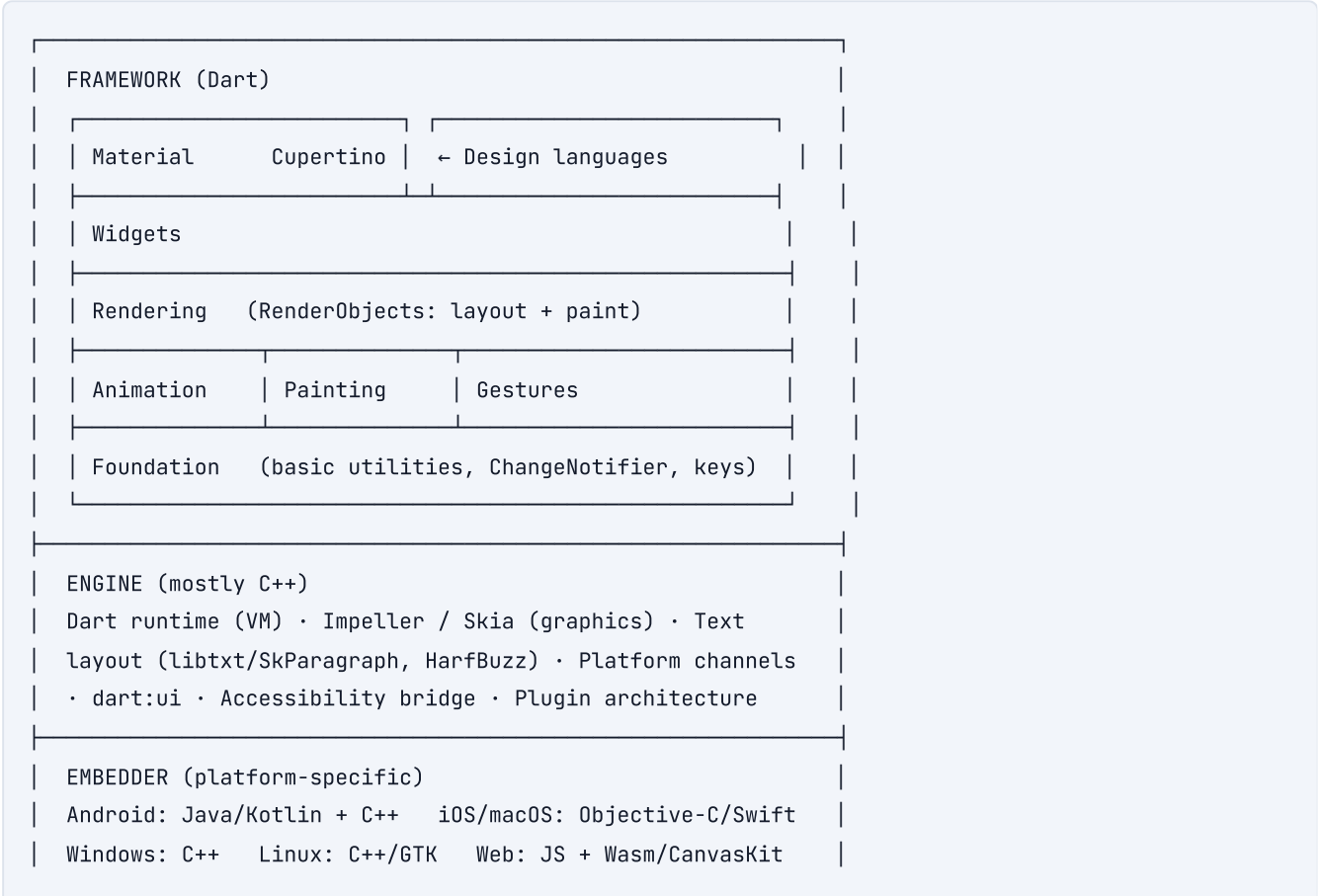
- ❌ Historically, a "bridge" serialised messages between JS and native — a bottleneck. (React Native's *New Architecture* with JSI/Fabric reduces this, but the model is still "control native widgets".)
- ❌ Each platform's widgets behave differently → subtle cross-platform bugs.

1.3.4 Flutter's approach

- You write **Dart**. It is compiled **ahead-of-time (AOT)** to **native ARM / x64 machine code** for release builds.
- Flutter **does not use platform widgets**. It ships its own rendering engine and **paints every pixel itself** onto a surface (a "canvas") provided by the OS.
- ✅ Pixel-perfect consistency across platforms.
- ✅ No JS bridge for UI work — the UI code runs directly as machine code.
- ✅ Hot reload: change code and see it in under a second, state preserved.
- ❌ Larger minimum app size (engine is bundled; a "hello world" release APK is a few MB).
- ❌ Must re-implement platform look-and-feel (Material, Cupertino) — and keep up when Apple/Google change their design.
- ❌ Platform features still need plugins (which use **platform channels** or **FFI** to talk to native code).

1.4 Flutter's architecture — the layered cake 🎂

Think of Flutter as a **three-layer cake**. You mostly eat the top layer, but the other layers hold it up.



```
|  Provides: rendering surface, input events, threads,         |
|  event loop, app lifecycle, plugin registration              |
└──────────────────────────────────────────────────────────────┘
```

1.4.1 Embedder (the plate under the cake)

The **embedder** is the native app that "hosts" Flutter. On Android it is an `Activity` (`FlutterActivity`) containing a `FlutterView`; on iOS it is a `FlutterViewController`. The embedder:

- Creates the drawing **surface** (an OpenGL/Vulkan/Metal surface).
- Sets up **threads** (task runners).
- Forwards **touch, keyboard, mouse** events into Flutter.
- Forwards **lifecycle** events (app paused/resumed).
- Hosts **plugins** (native code).

1.4.2 Engine (the sponge)

Written mostly in **C++**. It contains:

- **Dart runtime** — runs your Dart code (JIT in debug, AOT in release).
- **Graphics renderer — Impeller** (the modern default on iOS and Android API 29+) or **Skia** (the older renderer still used in some configurations such as older Android devices, and historically on desktop/web via CanvasKit).
- **Text layout** — shaping, line breaking.
- `dart:ui` — the lowest-level Dart library, exposing the engine to the framework: `Canvas`, `Paint`, `Path`, `Picture`, `Scene`, `PlatformDispatcher`, etc.

1.4.3 Framework (the icing — where you work)

Written in **Dart**. This is the `package:flutter/...` you import. Layers, bottom to top:

1. **Foundation** — `ChangeNotifier`, `Key`, `debugPrint`, `kReleaseMode`, `compute`.
2. **Animation, Painting, Gestures** — `AnimationController`, `Tween`, `TextPainter`, `GestureDetector` internals.
3. **Rendering** — the `RenderObject` tree that does layout & painting.
4. **Widgets** — the declarative layer: `StatelessWidget`, `StatefulWidget`, `Row`, `Text`.
5. **Material & Cupertino** — ready-made design systems. (Since Flutter 3.47 also published as standalone `material_ui` and `cupertino_ui` packages — see Chapter 10.)

> 🎯 **Interview soundbite:** "Flutter is layered: the embedder hosts it on each platform, the C++ engine runs Dart and renders using Impeller or Skia, and the Dart framework gives us widgets which build an element tree, which manages a render-object tree that performs layout and paint. Because Flutter paints its own pixels, it gets consistent UI across platforms."

1.5 How does Flutter get pixels on the screen? (Simple version)

1. **You** write widgets: "I want a blue box with the text Hello in the middle."
2. Flutter **builds** a tree of widgets → creates **elements** → creates **render objects**.
3. Render objects **lay out** (decide sizes and positions) and **paint** (record drawing commands).
4. The painted commands are put into **layers** and sent as a **scene** to the engine.
5. The engine's **raster thread** uses **Impeller/Skia** to turn the commands into GPU instructions.
6. The GPU draws the pixels. The OS shows them on the screen.

This happens up to **60 times per second** on a 60 Hz screen (16.67 ms per frame), or **120 times per second** on 120 Hz screens (8.33 ms per frame). If a frame takes longer, the user sees a stutter called **jank**. Chapter 20 goes deep into this pipeline; Chapter 23 shows how to fix jank.

1.6 Dart: why did Flutter pick it?

Interviewers *love* this question.

1. **JIT + AOT compilation.**
   - **JIT (Just-In-Time)** in debug mode → enables **hot reload** (inject changed code into a running VM).
   - **AOT (Ahead-Of-Time)** in release mode → fast startup, predictable performance, native machine code. Few languages do *both* well.
2. **Fast allocation & garbage collection.** Flutter creates and throws away thousands of tiny widget objects every frame. Dart's **generational GC** has a very fast "young generation" (scavenger) that is optimised exactly for short-lived objects.
3. **Single-threaded event loop + isolates.** No shared-memory threads means **no locks needed** in UI code; widgets can be rebuilt without race conditions. Heavy work goes to **isolates** (separate memory heaps communicating via messages).
4. **Sound null safety & strong typing** → fewer runtime crashes.
5. **Familiar syntax** for Java / C# / JavaScript / Kotlin / Swift developers.
6. **Google controls both** → the Dart team can add features Flutter needs (e.g., hot reload support, records, patterns, dot shorthands for enums like `.center`).
7. **Tree shaking** — unused code is removed from release builds.

1.7 Build modes (debug, profile, release)

Mode	Command	Compilation	Use for	Features
Debug	`flutter run`	**JIT**	**Development**	**Hot reload, assertions on, debug banner, service extensions, slow**
Profile	`flutter run --profile`	**AOT**	**Performance measurement**	**Close to release speed, DevTools performance tools work, no hot reload, real device only**
Release	`flutter run --release` / `flutter build ...`	**AOT**	**Shipping to users**	**Fastest, smallest, assertions removed, no debugging**

Useful constants from `package:flutter/foundation.dart`:

```dart
import 'package:flutter/foundation.dart';

if (kDebugMode) print('only in debug');
if (kProfileMode) { /* ... */ }
if (kReleaseMode) { /* ... */ }
if (kIsWeb) { /* running on web */ }
```

⚠️ **Common mistake:** Measuring performance in debug mode. Debug mode is *much* slower (JIT, assertions, extra checks). Always measure jank in **profile mode on a real device**.

🔎 Note on the web: web has no "JIT/AOT to machine code" in the same sense. Debug web builds use the `dartdevc` (DDC) compiler to JavaScript modules; release builds use `dart2js` (optimised JavaScript) or `dart2wasm` (WebAssembly) when you build with `--wasm`.

1.8 Hot reload vs hot restart vs full restart

This is asked in almost **every** Flutter interview.

	Hot Reload (`r`)	Hot Restart (`R`)	Full restart (stop & run)
Speed	~sub-second	a few seconds	slow (recompile native)
Keeps app state?	✅ Yes	❌ No (state reset)	❌ No
How it works	Injects updated source into running Dart VM, then rebuilds widget tree	Recompiles, restarts the Dart isolate, re-runs `main()`	Rebuilds native app
Re-runs `main()` / `initState()`?	❌ No	✅ Yes	✅ Yes

When hot reload does NOT reflect your change (memorise this list):

1. Changes to `main()` or to code only run once at startup.
2. Changes to `initState()` of an already-created widget (it won't run again).
3. Changes to **global variables and static fields initialisers** (they're already initialised).
4. Changing a `StatelessWidget` into a `StatefulWidget` (or vice versa) — may produce errors; use hot restart.
5. Changing **enum** types to classes, or **generic type** declarations — may require hot restart.
6. **Native code** changes (Kotlin, Swift, `AndroidManifest.xml`, `Info.plist`, Gradle, Podfile) — need a full rebuild.
7. Adding **new assets** to `pubspec.yaml` or new **packages** — usually needs restart/rebuild.
8. Changes in code that a **compile error** blocked.

1.9 Everything is a widget

In Flutter, almost everything you see — and many things you don't — are **widgets**:

- Visible things: `Text`, `Image`, `Icon`, `ElevatedButton`.
- Layout: `Row`, `Column`, `Padding`, `Center`, `Stack`.
- Behaviour: `GestureDetector`, `Dismissible`, `Draggable`.
- Styling: `Theme`, `DefaultTextStyle`.
- Even the whole app: `MaterialApp`.

A **widget** is just an **immutable description** of part of the UI — like a **blueprint** or a **recipe card**. It is *not* the actual thing on screen. When anything changes, Flutter builds new, cheap blueprints and compares them with the old ones to decide what to actually change. Chapter 6 explains this deeply.

Your first Flutter app, explained line by line

```dart
import 'package:flutter/material.dart'; // 1. Bring in Material widgets.

void main() {                // 2. Every Dart program starts at main().
  runApp(const MyApp());     // 3. runApp() attaches the root widget to the screen.
}

class MyApp extends StatelessWidget {   // 4. A widget with no changing state.
  const MyApp({super.key});              // 5. const constructor → can be cached.

  @override
  Widget build(BuildContext context) {   // 6. Describes the UI.
    return MaterialApp(                  // 7. Sets up theme, navigation, etc.
      title: 'Hello App',
      theme: ThemeData(colorSchemeSeed: Colors.blue),
      home: const HomePage(),
    );
  }
}

class HomePage extends StatefulWidget {  // 8. A widget whose state can change.
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState(); // 9. Creates its State object.
}

class _HomePageState extends State<HomePage> {
  int _count = 0;                        // 10. The changing data (state).

  void _increment() {
    setState(() {                        // 11. "Hey Flutter, my data changed —
      _count++;                          //      please call build() again."
    });
```

```
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(                    // 12. Basic page layout (app bar, body, FAB).
      appBar: AppBar(title: const Text('Counter')),
      body: Center(
        child: Text(
          'You pressed $_count times',
          style: Theme.of(context).textTheme.headlineMedium,
        ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _increment,
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

What happens when you tap the button?

1. `_increment` runs → `setState` marks this State's element as **dirty**.
2. On the next frame, Flutter calls `build()` again for this widget.
3. A new `Text` widget with the new number is created.
4. Flutter compares it with the old one: same type, same position → **reuse** the existing element & render object, just **update** the text.
5. Only that text is re-laid-out and repainted.

1.10 Declarative vs imperative UI

Imperative (old Android/iOS style): *"Find the label. Change its text to 5. Make the button red."* You tell the UI **how** to change, step by step.

```
// Imperative (Android Views)
TextView label = findViewById(R.id.label);
label.setText("5");
```

Declarative (Flutter, SwiftUI, Jetpack Compose, React): *"When count is 5, the UI looks like this."* You describe **what** the UI should look like for a given state; the framework figures out the changes.

```
// Declarative (Flutter)
Text('$count')
```

Formula Flutter docs use: **UI = f(state)** — the UI is a function of the state. `build()` is that function.

Benefits: fewer bugs from forgetting to update a view, easier reasoning, easy testing.

1.11 Flutter vs alternatives (interview comparison table)

Aspect	Flutter	React Native	Native (Kotlin/Swift)	Kotlin Multiplatform (KMP) / Compose Multiplatform
Language	Dart	JavaScript/TypeScript	Kotlin / Swift	Kotlin
UI rendering	Own engine draws every pixel	Native platform widgets	Native widgets	KMP: share logic, native UI; Compose MP: own rendering via Skia (Skiko)
Code sharing	UI + logic	UI + logic	none	logic (+ UI with Compose MP)
Performance	Near-native, AOT	Good; JS engine (Hermes)	Best	Native
Hot reload	Yes, stateful	Fast Refresh	Limited (Compose previews / Live Edit, SwiftUI previews)	Compose hot reload (newer)
Look & feel	Identical everywhere unless adapted	Native per platform	Native	Native / custom
App size	Larger baseline	Medium	Smallest	Small/medium
Web	Yes (canvas-based; not SEO-friendly by default)	React Native Web	No	Wasm (Compose)

When NOT to choose Flutter (senior candidates should know):

- App is mostly **content/SEO websites** → use a web framework.
- App relies heavily on **cutting-edge platform-specific UI** (e.g., new iOS widgets/Live Activities, deep OS integrations) — Flutter can still do it but you write native code anyway.
- Extremely **size-constrained** apps (instant apps, App Clips have size limits — Flutter is possible but tight).
- Team already has strong native teams and features differ completely per platform.

When Flutter shines: startups/MVPs, branded custom UI, apps needing identical UX across iOS/Android, internal enterprise apps, apps that also want desktop/web from the same code, design-heavy apps, fintech/e-commerce dashboards. Known users: Google Pay, Google Ads, Google Classroom (parts), BMW (My BMW app), Alibaba (Xianyu), ByteDance products, Nubank, Toyota (infotainment), eBay Motors, Philips Hue, Reflectly, Credit Agricole, LG webOS, etc.

1.12 Threads in Flutter (simple version)

Flutter's engine uses several **task runners** (threads):

Runner	Job
Platform thread (main thread)	OS events, plugin calls (platform channel handlers on native side run here by default)
UI thread	Runs your Dart code (root isolate): build, layout, paint (recording), animations, event handling
Raster thread (formerly "GPU thread")	Takes the layer tree and rasterises it with Impeller/Skia, talks to the GPU
IO thread	Expensive I/O like decoding images, uploading textures

📌 Recent change: Starting around Flutter 3.29 on iOS and Android, the engine **merged the platform and UI threads** by default (Dart now runs on the platform's main thread). This enables synchronous calls to native code (useful for FFI and future direct native interop). Conceptually, you still think of "the UI thread running Dart" — but don't be surprised if an interviewer mentions this.

Key consequence: **all your Dart code runs on one thread** (per isolate). If you do heavy work (parse a 20 MB JSON, image processing) on it, frames are missed → **jank**. Solution: `Isolate.run()` / `compute()` (Chapter 5 & 23).

1.13 Summary checklist ✅

- ☐ Flutter = Google's UI toolkit, one Dart codebase, 6+ platforms.
- ☐ Draws its own pixels using Impeller/Skia (not platform widgets).
- ☐ Three layers: Embedder → Engine (C++) → Framework (Dart).
- ☐ Debug = JIT + hot reload; Release = AOT machine code.
- ☐ Hot reload keeps state; hot restart resets state; native changes need full rebuild.
- ☐ Declarative: UI = f(state); widgets are immutable blueprints.
- ☐ Single UI thread for Dart → use isolates for heavy work.

1.14 Interview Questions for this chapter

Q1. What is Flutter? How is it different from other cross-platform frameworks? A: Flutter is Google's open-source UI SDK for building natively compiled apps for mobile, web, desktop and embedded from one Dart codebase. Its key difference is that it doesn't wrap native UI components — it ships its own rendering engine (Impeller/Skia) and draws every pixel, giving consistent UI and high performance. Release builds are compiled AOT to native machine code, so there is no JavaScript bridge for UI.

Q2. Why does Flutter use Dart? A: Dart supports both JIT (for hot reload in development) and AOT (fast, predictable native code in production); has a generational GC optimised for many short-lived objects (widgets); a single-threaded event loop with isolates so UI code needs no locks; sound null safety; tree shaking; and Google owns both so the language evolves with Flutter's needs.

Q3. Explain Flutter's architecture. A: Embedder (platform-specific host: surface, threads, input, lifecycle, plugins) → Engine (C++: Dart runtime, Impeller/Skia, text layout, dart:ui, platform channels) → Framework (Dart: foundation, animation/painting/gestures, rendering, widgets, Material/Cupertino).

Q4. Difference between hot reload and hot restart? A: Hot reload injects changed code into the running Dart VM and rebuilds the widget tree, preserving state; it doesn't rerun `main()` or `initState()`. Hot restart recompiles and restarts the app's Dart state from `main()`, losing state. Native changes need a full rebuild.

Q5. What are Flutter build modes? A: Debug (JIT, hot reload, assertions, slow), Profile (AOT, profiling enabled, real device only), Release (AOT, optimised, minimal). Measure performance only in profile mode.

Q6. Is Flutter "native"? A: The compiled code is native machine code (AOT ARM/x64) and it uses native rendering APIs (Metal/Vulkan/OpenGL), but the UI widgets are not platform-native widgets; they are Flutter re-implementations. Native platform features are accessed via plugins using platform channels or FFI.

Q7. What is Impeller and why was it created? A: Impeller is Flutter's newer rendering engine replacing Skia on mobile. Skia compiled shaders at runtime the first time an effect was used, causing "shader compilation jank" on first animations. Impeller precompiles a smaller, fixed set of shaders at engine build time, so there is no runtime shader compilation, giving predictable frame times. It targets Metal on iOS and Vulkan (with OpenGL ES fallback) on Android.

Q8. What is the difference between declarative and imperative UI? A: Imperative: you mutate UI objects step by step. Declarative: you describe the UI for the current state (UI = f(state)) and the framework diffs and updates. Flutter is declarative.

Q9. What threads does Flutter use? A: Platform, UI (Dart), Raster, and IO task runners. Dart code runs on one thread per isolate; heavy computations should move to another isolate.

Q10. What are the disadvantages of Flutter? A: Larger app size, non-native widgets (must mimic platform design), web output is canvas-based (SEO and text selection need care), needing native code for some platform features, plugin quality varies, and Dart talent pool smaller than JS.

Q11. What is `runApp()`? A: It inflates the given widget and attaches it to the screen, making it the root of the widget tree. It ensures the `WidgetsFlutterBinding` is initialized and schedules the first frame. Calling it again replaces the root.

Q12. What is `WidgetsFlutterBinding.ensureInitialized()` and when do you need it? A: The binding is the glue between the framework and the engine. `runApp` initialises it automatically, but if you need to use platform channels/plugins *before* `runApp` (e.g., `await Firebase.initializeApp()`, `SharedPreferences.getInstance()`, setting orientations), you must call `WidgetsFlutterBinding.ensureInitialized()` first, otherwise you get "Binding has not yet been initialized".

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp();
  runApp(const MyApp());
}
```

Chapter 2 — Setup, Tooling, Project Structure & the Flutter CLI

Goal: Install Flutter correctly, understand every file in a new project, master the CLI, pub (package manager), versioning, and IDE tools. Most "it doesn't build!" problems in companies come from this chapter's topics.

2.1 Installing Flutter (all platforms)

2.1.1 What you need

You want to build for	You need
Android	Android Studio (or command-line tools), Android SDK, a JDK (Android Studio bundles one), an emulator or device
iOS / macOS	A Mac with Xcode, plus CocoaPods (still used by many plugins) — Swift Package Manager is increasingly the default
Web	Chrome (for debugging)
Windows	Visual Studio (not VS Code) with "Desktop development with C++" workload
Linux	clang, cmake, ninja-build, pkg-config, libgtk-3-dev

You **cannot** build an iOS app on Windows/Linux locally. Companies use Macs or cloud CI Macs (Codemagic, GitHub Actions macOS runners, Bitrise).

2.1.2 Steps (conceptually the same on every OS)

1. Download the Flutter SDK (zip from docs.flutter.dev, or `git clone https://github.com/flutter/flutter.git -b stable`). The VS Code Flutter extension can also download the SDK for you.
2. Add `flutter/bin` to your **PATH**.
3. Run `flutter doctor` — it checks everything and tells you what is missing.
4. Accept Android licenses: `flutter doctor --android-licenses`.
5. Install IDE plugins: **Flutter** and **Dart** plugins for VS Code or Android Studio / IntelliJ.

2.1.3 `flutter doctor` explained like you're 5

`flutter doctor` is a **doctor that checks your computer's health for Flutter**. Each line is a check:

```
[✓] Flutter (Channel stable, 3.x.x, ...)
[✓] Android toolchain - develop for Android devices
[✗] Xcode - develop for iOS and macOS        ← missing!
[✓] Chrome - develop for the web
```

```
[!] Android Studio (version ...)             ← warning
[✓] Connected device (2 available)
[✓] Network resources
```

- `✓` healthy · `!` warning (may work) · `✗` broken. Use `flutter doctor -v` for details.

2.2 Channels & versions

Flutter has **release channels**:

Channel	What	Who should use
stable	**Tested releases, roughly quarterly + hotfixes**	**Production apps — ALWAYS**
beta	**Monthly-ish preview of next stable**	**Trying new features**
main	**Latest commits, may break**	**Contributors**

(The old `dev` channel was retired in 2022.)

```
flutter channel            # show channels
flutter channel stable     # switch
flutter upgrade            # upgrade current channel
flutter downgrade          # go back to previous version on that channel
flutter --version          # show Flutter, engine, Dart versions
```

2.2.1 Version pinning in teams — FVM

In a company, **everyone must use the same Flutter version**; otherwise "works on my machine" bugs appear (different lint rules, generated code, Gradle versions).

FVM (Flutter Version Management) solves this:

```
dart pub global activate fvm
fvm install 3.x.x
fvm use 3.x.x          # writes .fvmrc in the project
fvm flutter run        # use the pinned version
```

Commit `.fvmrc` to git; add `.fvm/` (the symlink folder) to `.gitignore`. CI uses the same version (e.g., `subosito/flutter-action` in GitHub Actions reading the version from the file).

Alternatives: asdf/mise version managers, or specifying `flutter:` constraint in `pubspec.yaml` `environment:` (which only *checks* — it does not install).

2.3 Creating a project

```
flutter create my_app                                # default counter app
flutter create --org com.mycompany my_app            # sets bundle id / package name prefix
flutter create --platforms=android,ios my_app        # only some platforms
flutter create -t package my_package                 # pure Dart/Flutter package
flutter create -t plugin --platforms=android,ios my_plugin   # plugin with native code
flutter create -t plugin_ffi my_ffi_plugin           # FFI plugin
flutter create -e my_app                             # "empty" minimal app (no counter comments)
flutter create -t module my_module                   # add-to-app module
```

Tip: Always set `--org` up front. Changing the Android `applicationId` and iOS bundle identifier later is tedious (but possible — see Chapter 26).

2.4 Project structure — every file explained

```
my_app/
├── android/                 ← Native Android project (Gradle)
│   ├── app/
│   │   ├── build.gradle(.kts)  ← app module config: applicationId, minSdk, signing, flavors
│   │   └── src/main/
│   │       ├── AndroidManifest.xml  ← permissions, activities, deep links, app name
│   │       ├── kotlin/.../MainActivity.kt  ← extends FlutterActivity
│   │       └── res/          ← icons (mipmap), launch background, styles
│   ├── build.gradle(.kts)    ← project-level Gradle
│   ├── settings.gradle(.kts) ← plugin management, Flutter Gradle plugin, AGP & Kotlin versions
│   └── gradle/wrapper/gradle-wrapper.properties ← Gradle version
├── ios/                     ← Native iOS project (Xcode)
│   ├── Runner/
│   │   ├── AppDelegate.swift ← FlutterAppDelegate
│   │   ├── Info.plist        ← permissions descriptions, URL schemes, display name
│   │   └── Assets.xcassets   ← app icons, launch images
│   ├── Runner.xcworkspace    ← OPEN THIS in Xcode (not .xcodeproj) when CocoaPods is used
│   └── Podfile               ← CocoaPods deps (platform :ios, 'xx.0')
├── lib/                     ← YOUR DART CODE lives here
│   └── main.dart             ← entry point
├── test/                    ← unit & widget tests (files must end with _test.dart)
├── integration_test/        ← integration tests (you create it)
├── web/                     ← index.html, manifest.json, icons for web
├── windows/ macos/ linux/    ← desktop runners
├── assets/                  ← (you create) images, fonts, json
├── pubspec.yaml             ← project manifest: name, version, deps, assets, fonts
├── pubspec.lock             ← exact resolved versions (COMMIT for apps)
├── analysis_options.yaml    ← lint rules
├── .metadata                ← used by flutter tool for migrations; commit it
```

```
├── .dart_tool/              ← generated tool cache (DON'T commit)
├── build/                   ← build outputs (DON'T commit)
└── README.md
```

2.4.1 `pubspec.yaml` deeply explained

```yaml
name: my_app                        # package name: lowercase_with_underscores
description: "A new Flutter project."
publish_to: 'none'                  # prevents accidental `pub publish` of an app

version: 1.2.3+45                   # versionName 1.2.3 + build number 45
                                    # Android: versionName=1.2.3, versionCode=45
                                    # iOS: CFBundleShortVersionString=1.2.3, CFBundleVersion=45

environment:
  sdk: ^3.5.0                       # Dart SDK constraint (also enables language features by
version!)
  flutter: ">=3.24.0"               # optional minimum Flutter

dependencies:                       # needed at runtime
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  http: ^1.2.0                      # caret syntax: >=1.2.0 <2.0.0
  my_local_pkg:
    path: ../packages/my_local_pkg # local path dependency
  some_fork:
    git:
      url: https://github.com/me/some_fork.git
      ref: v1.0.1                   # branch, tag or commit — pin it!
      path: packages/some_fork      # subdirectory, optional

dev_dependencies:                   # only for development: tests, code generation, lints
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0
  build_runner: ^2.4.0

dependency_overrides:               # FORCE a version (use sparingly, document why!)
  intl: 0.19.0

flutter:
  uses-material-design: true        # includes Material Icons font
  generate: true                    # needed for gen-l10n (localization) in many setups
  assets:
    - assets/images/                # folder (NOT recursive into subfolders!)
    - assets/config/app.json        # single file
  fonts:
```

```
    - family: Inter
      fonts:
        - asset: assets/fonts/Inter-Regular.ttf
        - asset: assets/fonts/Inter-Bold.ttf
          weight: 700
```

Semantic versioning (SemVer) & caret syntax

`MAJOR.MINOR.PATCH` → `2.4.1`

- **MAJOR** — breaking changes.
- **MINOR** — new features, backwards compatible.
- **PATCH** — bug fixes.

`^1.2.3` means `>=1.2.3 <2.0.0`. **Special case for 0.x:** `^0.2.3` means `>=0.2.3 <0.3.0` (because in 0.x, minor bumps may break).

Other constraints: `any` (avoid!), `">=1.0.0 <3.0.0"`, `1.2.3` (exact).

`environment: sdk:` lower bound is important

The **lower bound** of your Dart SDK constraint decides the **language version** your package uses. Example: features like records/patterns need `sdk: ^3.0.0`; dot shorthands need `^3.10.0`; primary constructors need `^3.13.0`. If your lower bound is too low, the analyzer says *"This requires the 'xyz' language feature to be enabled."*

2.4.2 `pubspec.lock` — commit or not?

- **Apps: COMMIT IT.** Everyone and CI gets the *exact same* versions → reproducible builds.
- **Packages (libraries): historically not committed**, because consumers resolve their own versions. (Modern guidance: committing it in packages is OK for your own CI reproducibility; it is ignored by consumers anyway.)

2.4.3 Assets gotcha

```
assets:
  - assets/images/
```

This includes files **directly** inside `assets/images/` but **not** `assets/images/icons/`. You must list subfolders separately. This is a top-3 beginner bug: *"Unable to load asset"*.

Also: resolution-aware images:

```
assets/images/logo.png        ← 1.0x
assets/images/2.0x/logo.png   ← used on 2x screens
assets/images/3.0x/logo.png   ← used on 3x screens
```

You still reference `assets/images/logo.png`; Flutter picks the best variant.

2.5 Pub — the package manager

```
flutter pub get                    # download deps per pubspec (creates/updates lock)
flutter pub add dio                # add latest compatible dep
flutter pub add --dev mocktail     # add dev dependency
flutter pub add 'dio:^5.4.0'       # with constraint
flutter pub remove dio
flutter pub upgrade                # upgrade within constraints (updates lock)
flutter pub upgrade --major-versions  # bump constraints to latest majors (review!)
flutter pub outdated               # show what's outdated (Current/Upgradable/Resolvable/Latest)
flutter pub deps                   # dependency tree
flutter pub cache repair           # fix corrupted cache
flutter pub cache clean            # wipe global cache
dart pub global activate <pkg>     # install CLI tools
```

2.5.1 Choosing a package on pub.dev (organisational checklist)

Before adding any dependency to a company app, check:

1. **Publisher** — verified publisher (e.g., `dart.dev`, `flutter.dev`, `firebase.google.com`, well-known companies).
2. **Likes / Pub points / Downloads (popularity)**.
3. **Last published date** — abandoned packages are a risk.
4. **Platform support** badges (Android, iOS, Web…).
5. **Null safety / Dart 3 compatibility** (all current packages are null-safe; Dart 3 requires null safety).
6. **Open issues & how maintainers respond**.
7. **License** (MIT/BSD/Apache OK for most companies; GPL may be a problem — ask legal).
8. **Transitive dependencies** — does it pull in 30 others?
9. **Native code quality** for plugins: permissions it adds to your manifest, minimum SDK, size impact.
10. Could you write it yourself in 50 lines? Then maybe don't add a dependency.

2.5.2 Version conflict ("version solving failed")

```
Because app depends on a ^2.0.0 which depends on intl ^0.18.0,
and app depends on b ^3.0.0 which depends on intl ^0.19.0, version solving failed.
```

Fixes, in order of preference:

1. `flutter pub upgrade` / bump `a` or `b` to versions compatible with each other.
2. Check package changelogs for a version supporting the newer dependency.
3. Temporarily `dependency_overrides:` with a **comment** explaining why and a ticket to remove it.
4. Fork & fix, depend via `git:` with a pinned `ref`.

2.6 Essential Flutter CLI commands (cheat sheet)

```
flutter devices                    # list connected devices/emulators
flutter emulators                  # list emulators
flutter emulators --launch <id>
flutter run                        # run on device (debug)
flutter run -d chrome              # web
flutter run -d <deviceId>
flutter run --release / --profile
flutter run --flavor dev -t lib/main_dev.dart        # flavors (Chapter 26)
flutter run --dart-define=API_URL=https://dev.api    # compile-time env vars
flutter run --dart-define-from-file=config/dev.json  # many vars from a JSON file

flutter build apk --release                  # Android APK (universal)
flutter build apk --split-per-abi            # smaller per-ABI APKs
flutter build appbundle                      # AAB for Play Store
flutter build ipa                            # iOS archive for App Store
flutter build web --release                  # web (add --wasm for WebAssembly)
flutter build windows | macos | linux

flutter test                                  # run tests
flutter test --coverage                       # generate coverage/lcov.info
flutter test test/foo_test.dart --name "adds"
flutter analyze                               # static analysis (lints)
dart format .                                 # format code
dart fix --dry-run / dart fix --apply         # automated migrations & lint fixes
flutter clean                                 # delete build/ and .dart_tool/ (fixes many weird
issues)
flutter gen-l10n                              # generate localization classes
flutter pub run build_runner build --delete-conflicting-outputs   # older form
dart run build_runner build --delete-conflicting-outputs           # modern form
dart run build_runner watch                                        # continuous codegen
flutter logs                                  # device logs
flutter attach                                # attach to running app (add-to-app)
flutter screenshot
flutter config --enable-web / --no-analytics  # global config
flutter precache                              # download engine artifacts
flutter symbolize -i <stacktrace> -d <symbols-dir>  # de-obfuscate stack traces
```

2.6.1 Keys while `flutter run` is running

Key	Action
`r`	**Hot reload**
`R`	**Hot restart**
`p`	**Toggle debug paint (layout boundaries)**
`P`	**Performance overlay**
`o`	**Toggle platform (Android ↔ iOS look)**

Key	Action
`w`	**Dump widget tree**
`t`	**Dump render tree**
`L`	**Dump layer tree**
`S`	**Dump semantics tree**
`i`	**Toggle widget inspector**
`v`	**Open DevTools**
`M`	**Write SkSL shaders (legacy Skia only)**
`q`	**Quit**
`h`	**Help**

2.7 `--dart-define` and environment configuration

```dart
// Reading compile-time constants:
const apiUrl = String.fromEnvironment('API_URL', defaultValue: 'https://prod.api');
const isStaging = bool.fromEnvironment('STAGING');
const retries = int.fromEnvironment('RETRIES', defaultValue: 3);
```

- These are **compile-time constants** — `const` required; changing them needs rebuild (not hot reload).
- ⚠️ They are **embedded in the binary. Never put secrets** (private keys) here — anyone can extract strings from an APK/IPA. API keys meant to be public (e.g., Firebase client keys, Maps keys restricted by app signature) are acceptable.
- `--dart-define-from-file=env/dev.json` (JSON or `.env` file) is the clean approach for many variables.

Alternative: `flutter_dotenv` (reads a bundled asset `.env` at runtime — same security caveat: it's in the bundle), `envied` (code generation with optional obfuscation — obfuscation is *not* real security).

2.8 analysis_options.yaml & lints

```yaml
include: package:flutter_lints/flutter.yaml   # or
package:very_good_analysis/analysis_options.yaml

analyzer:
  exclude:
    - "**/*.g.dart"
    - "**/*.freezed.dart"
  language:
    strict-casts: true          # no implicit dynamic casts
    strict-inference: true
    strict-raw-types: true
```

```
  errors:
    missing_required_param: error
    missing_return: error
    todo: ignore

linter:
  rules:
    - prefer_const_constructors
    - prefer_final_locals
    - avoid_print
    - always_use_package_imports    # or prefer_relative_imports — pick ONE in your team
    - require_trailing_commas
    - unawaited_futures
    - cancel_subscriptions
    - close_sinks
    - use_build_context_synchronously
```

Lint packages:

- `lints` — Dart core/recommended sets.
- `flutter_lints` — Flutter's recommended set (default in new projects).
- `very_good_analysis` — strict set used by Very Good Ventures, popular in companies.
- **DCM (Dart Code Metrics)** — commercial tool with metrics (cyclomatic complexity, etc.) and extra rules.
- Custom lint rules: `custom_lint` package, or the newer official **analyzer plugin** system (Dart 3.10+).

Suppress for one line: `// ignore: avoid_print`; for a file: `// ignore_for_file: avoid_print`.

2.9 IDEs & productivity

VS Code

- Extensions: **Flutter**, **Dart**. Optional: Error Lens, Pubspec Assist, Awesome Flutter Snippets, bloc / riverpod snippets.
- Shortcuts: `Ctrl/Cmd + .` → **Quick actions**: *Wrap with widget, Wrap with Padding/Center/Column, Remove widget, Extract widget, Extract method, Convert to StatefulWidget*.
- `stless` / `stful` snippets create widgets.
- Run & Debug panel, breakpoints, `launch.json` for flavors and dart-defines:

```
{
  "configurations": [
    {
      "name": "dev",
      "request": "launch",
      "type": "dart",
```

```
      "program": "lib/main_dev.dart",
      "args": ["--flavor", "dev", "--dart-define-from-file=env/dev.json"]
    }
  ]
}
```

Android Studio / IntelliJ

- Flutter + Dart plugins. Flutter Inspector, Flutter Outline, Performance tabs.
- `Alt+Enter` (Win/Linux) / `Option+Enter` (Mac) → quick actions (wrap, extract).
- Good for native Android debugging (Logcat, Gradle).

Xcode

Needed for iOS signing, capabilities (Push, Sign in with Apple, App Groups), archive, and debugging native crashes.

Widget Previews

Newer Flutter versions (3.35 experimental, IDE integration in 3.38+) support **widget previews** via a `@Preview` annotation so you can see widgets in the IDE without running the app (similar to Compose/SwiftUI previews). Check your version's docs as the API is evolving.

DevTools

Browser-based suite: Inspector, Performance, CPU Profiler, Memory, Network, Logging, App Size, Deep Links validator, Provider/Riverpod/Bloc extensions. Opened with `v` in `flutter run`, from IDE, or `dart devtools`. Chapter 23 covers it deeply.

2.10 Android-specific setup knowledge (comes up in every company!)

2.10.1 Gradle files (modern Kotlin DSL — default for new projects since Flutter 3.29)

`android/settings.gradle.kts` (simplified):

```
pluginManagement {
    val flutterSdkPath = run { /* reads local.properties flutter.sdk */ }
    includeBuild("$flutterSdkPath/packages/flutter_tools/gradle")
    repositories { google(); mavenCentral(); gradlePluginPortal() }
}
plugins {
    id("dev.flutter.flutter-plugin-loader") version "1.0.0"
    id("com.android.application") version "8.x.x" apply false   // AGP version
    id("org.jetbrains.kotlin.android") version "2.x.x" apply false
```

```
}
include(":app")
```

`android/app/build.gradle.kts`:

```
plugins {
    id("com.android.application")
    id("kotlin-android")
    id("dev.flutter.flutter-gradle-plugin") // must come after Android & Kotlin plugins
}
android {
    namespace = "com.mycompany.myapp"
    compileSdk = flutter.compileSdkVersion
    defaultConfig {
        applicationId = "com.mycompany.myapp"
        minSdk = 23                    // or flutter.minSdkVersion
        targetSdk = flutter.targetSdkVersion
        versionCode = flutter.versionCode
        versionName = flutter.versionName
    }
    buildTypes {
        release {
            signingConfig = signingConfigs.getByName("release")
            isMinifyEnabled = true      // R8 shrinking of Java/Kotlin code
            isShrinkResources = true
        }
    }
}
```

Older projects use Groovy `build.gradle` with `apply plugin:` "imperative apply" — Flutter 3.16+ migrated to the **declarative `plugins {}` block**; old style now shows deprecation warnings / errors. Migration guide: "Deprecated imperative apply of Flutter's Gradle plugins".

2.10.2 Version compatibility triangle

Gradle ↔ Android Gradle Plugin (AGP) ↔ Java (JDK) ↔ Kotlin must be compatible.

- AGP 8.x needs Java 17.
- Each AGP version needs a minimum Gradle version.
- Flutter tool warns if versions are too old. Typical error: *"Unsupported class file major version 65"* → your JDK is too new for your Gradle (65 = Java 21). Fix: upgrade Gradle wrapper (`gradle-wrapper.properties`) or point Flutter to a different JDK: `flutter config --jdk-dir=/path/to/jdk17`.

2.10.3 minSdk / targetSdk / compileSdk

- **minSdk** — oldest Android version supported. Many plugins require ≥ 21 or higher (Firebase etc.). Flutter itself has raised its minimum over time (currently API 21+, with recent releases moving the default higher — check your version).
- **targetSdk** — the Android behaviour version you've tested against; Google Play requires a recent targetSdk (typically within one year of the latest Android release).
- **compileSdk** — SDK used to compile; must be ≥ what plugins need.

2.10.4 MultiDex

Only needed for minSdk < 21. Flutter tool can enable it automatically (`flutter run --multidex` historically). With minSdk 21+ it's native.

2.11 iOS-specific setup knowledge

- Open `ios/Runner.xcworkspace` in Xcode.
- **Signing & Capabilities** tab: Team, Bundle Identifier, capabilities (Push Notifications, Background Modes, Associated Domains for universal links).
- `Podfile`: `platform :ios, '13.0'` — many plugins require a minimum iOS version; mismatches cause `pod install` errors: *"The plugin X requires a higher minimum iOS deployment version"*.
- Common fixes:

```
cd ios
pod deintegrate         # if very broken
rm -rf Pods Podfile.lock
pod repo update         # update specs
pod install
cd ..
flutter clean && flutter pub get
```

- Apple Silicon (M1+) historical issue: `ffi` gem / running `arch -x86_64 pod install` — no longer needed on modern CocoaPods.
- **Swift Package Manager (SwiftPM)**: Flutter added support (opt-in 3.24, increasingly default; default for iOS/macOS in Flutter 3.44). Plugins that ship `Package.swift` integrate via SPM; CocoaPods is still used as a fallback for plugins that don't support SPM yet. Enable/disable with `flutter config --enable-swift-package-manager` / `--no-enable-swift-package-manager`, or per project in `pubspec.yaml` under `flutter: config:` (check your version's docs).
- `Info.plist` must contain **usage descriptions** for every permission (`NSCameraUsageDescription`, `NSLocationWhenInUseUsageDescription`, `NSPhotoLibraryUsageDescription`, ...) — missing ones cause **instant crash** when requesting the permission and **App Store rejection**.

2.12 Common setup problems & fixes (company survival kit)

Symptom	Cause	Fix
`flutter: command not found`	PATH not set	Add `<flutter>/bin` to PATH, restart terminal
`Android license status unknown`	licenses not accepted	`flutter doctor --android-licenses`
`cmdline-tools component is missing`	SDK Manager missing tools	Android Studio → SDK Manager → SDK Tools → Android SDK Command-line Tools
`Gradle task assembleDebug failed with exit code 1`	Generic; read the error above it	Run `cd android && ./gradlew assembleDebug --stacktrace` for details
`Unsupported class file major version XX`	JDK vs Gradle mismatch	Upgrade Gradle wrapper or `flutter config --jdk-dir`
`Execution failed for task ':app:checkDebugAarMetadata'`	Plugin needs higher compileSdk	Raise `compileSdk`
`Manifest merger failed: uses-sdk:minSdkVersion X cannot be smaller than version Y declared in library`	Plugin needs higher minSdk	Raise `minSdk`
`CocoaPods not installed` / `pod: command not found`		`sudo gem install cocoapods` or `brew install cocoapods`
`CocoaPods could not find compatible versions for pod "Firebase/..."`	Specs repo outdated / version conflict	`pod repo update`, delete Podfile.lock, `pod install`
`Module 'xyz' not found` (Xcode)	Opened .xcodeproj or pods not installed	Open .xcworkspace, `pod install`
`Unable to load asset`	Asset not declared or wrong path / subfolder	Check pubspec indentation & paths, full restart
`Waiting for another flutter command to release the startup lock`	Stale lock	Kill flutter processes; delete `<flutter>/bin/cache/lockfile`
Strange errors after upgrading	stale build caches	`flutter clean && flutter pub get` (+ `pod install`)
`version solving failed`	dep conflicts	see §2.5.2
`The current Dart SDK version is X. Because app requires SDK version >=Y`	Your Flutter is older than a package needs	Upgrade Flutter or downgrade package
Emulator very slow	No hardware acceleration	Enable HAXM/Hypervisor/KVM, use x86_64/arm64 system image matching host
`No connected devices`	USB debugging off / drivers	Enable Developer options → USB debugging; `adb devices`; accept RSA prompt

2.13 Interview Questions

Q1. What is `pubspec.yaml`? The project manifest: name, version, SDK constraints, dependencies, dev_dependencies, dependency_overrides, assets, fonts and Flutter-specific settings.

Q2. Difference between `dependencies` and `dev_dependencies`? Dependencies are needed by the app at runtime and are compiled in; dev_dependencies are tools used during development (tests, code generators, lints) and are not included in the app or passed on to consumers of a package.

Q3. What is `pubspec.lock` and should it be committed? It records the exact resolved versions of all (transitive) dependencies. Commit it for applications for reproducible builds.

Q4. What does `^1.2.3` mean? `>=1.2.3 <2.0.0`. For `^0.2.3` it's `>=0.2.3 <0.3.0`.

Q5. What is `dependency_overrides`? Forces a package version regardless of other constraints — useful for temporary conflicts or testing a local fork, but risky; document and remove ASAP.

Q6. `flutter pub get` vs `flutter pub upgrade`? `get` resolves deps respecting the existing lock file (only changing what's needed); `upgrade` tries to move every dependency to the latest version allowed by constraints and rewrites the lock file.

Q7. What does `flutter clean` do? Deletes the `build/` folder and `.dart_tool/` (and some ephemeral files), forcing a full rebuild — fixes stale artifact issues.

Q8. How do you manage different environments (dev/staging/prod)? Flavors (Android productFlavors, iOS schemes/configurations) + separate entrypoints (`main_dev.dart`) and/or `--dart-define(-from-file)`; never put secrets in the binary.

Q9. What is FVM and why use it? Flutter Version Management pins a Flutter SDK version per project so the entire team and CI use the same version.

Q10. What is the version format in pubspec and how does it map to stores? `x.y.z+n` → Android versionName/versionCode, iOS CFBundleShortVersionString/CFBundleVersion. Build number must increase on each store upload.

Q11. What are Flutter channels? stable, beta, main; use stable in production.

Q12. How do you add assets/fonts? Declare under `flutter: assets:` / `fonts:` in pubspec with correct indentation; subdirectories must be listed separately; use `Image.asset`, `rootBundle.loadString`, `fontFamily`.

Q13. What is `analysis_options.yaml`? Configures the Dart analyzer: included lint sets, enabled/disabled rules, strict modes, excluded files, error severities.

Q14. Package vs plugin? A package is pure Dart (possibly Flutter-dependent) code; a plugin is a package that includes platform-specific native code (Kotlin/Swift/C++/JS) accessed via platform channels or FFI.

Chapter 3 — Dart Language, Part 1: The Basics

Goal: Learn Dart from zero: variables, types, null safety, operators, control flow, functions and collections. Every Flutter interview has Dart questions — `final` vs `const`, null safety, `late`, `dynamic` vs `var` vs `Object` — so read carefully.

You can try every example at **dartpad.dev** without installing anything.

3.1 Hello Dart

```dart
void main() {
  print('Hello, Dart!');
}
```

- `main()` is the entry point.
- `void` means it returns nothing.
- Statements end with `;`.
- `print()` writes to the console. (In Flutter apps prefer `debugPrint()` or a logger — `print` is flagged by the `avoid_print` lint.)

Comments:

```dart
// single line
/* multi
   line */
/// Documentation comment — shows up in IDE hover & dart doc.
/// Use [ClassName] to link.
```

3.2 Variables — boxes with labels 📦

A **variable** is a labelled box that holds a value.

```dart
var name = 'Asha';        // type inferred as String
String city = 'Pune';     // explicit type
int age = 25;
double height = 5.6;
bool isStudent = true;
```

3.2.1 `var`, `final`, `const`, `late`, `dynamic`, `Object`

Keyword	Can reassign?	When value is known	Example
`var`	✅ yes	runtime	`var x = 1; x = 2;`
`final`	❌ no (set once)	runtime	`final now = DateTime.now();`
`const`	❌ no	compile time	`const pi = 3.14159;`
`late`	depends (`late final` = once)	initialised later, lazily	`late final String token;`
`dynamic`	✅ and type can change; no static checks	runtime	`dynamic d = 1; d = 'hi'; d.foo(); // compiles, crashes at runtime`
`Object` / `Object?`	✅ yes, any type, but type-checked	runtime	`Object o = 1; o.foo(); // compile error`

`final` vs `const` — the most asked Dart question

5-year-old version:

- `const` is a toy that is **glued shut at the factory**. Before the shop even opens (compile time), everyone knows exactly what is inside, and **every identical const toy is literally the same toy** (shared in memory).
- `final` is a box you can **fill once when the shop is open** (runtime), then tape shut forever.

```
final time = DateTime.now();   // ✅ known at runtime
// const time = DateTime.now(); // ❌ error: not a compile-time constant

const list = [1, 2, 3];        // the list itself is deeply immutable
// list.add(4);                // ❌ runtime error: Unsupported operation: Cannot add to an
unmodifiable list

final list2 = [1, 2, 3];       // the VARIABLE can't be reassigned...
list2.add(4);                  // ✅ ...but the list contents CAN change!
// list2 = [];                 // ❌ compile error
```

Key differences:

1. **Time:** `const` = compile-time constant; `final` = assigned once at runtime.
2. **Depth:** `const` makes the object **deeply/transitively immutable**; `final` only freezes the **reference**.
3. **Canonicalisation:** identical `const` expressions share **one instance** in memory:

   ```
   print(identical(const [1, 2], const [1, 2])); // true
   print(identical([1, 2], [1, 2]));             // false
   ```

4. **Instance fields:** instance fields can be `final`, not `const` (only `static const` fields allowed in classes).

5. **Flutter impact:** `const` widgets are created once and **reused**; when Flutter sees the *same instance* during rebuild it can skip rebuilding that subtree. That's why the linter suggests `prefer_const_constructors`.

`late`

Use `late` when:

1. A non-nullable variable will be initialised **after** declaration but **before** use (e.g., in `initState`).
2. You want **lazy initialisation** — the initialiser runs only when first read.

```
class _MyState extends State<MyWidget> {
  late final AnimationController controller; // set in initState

  @override
  void initState() {
    super.initState();
    controller = AnimationController(vsync: this, duration: const Duration(seconds: 1));
  }
}

late final config = loadHeavyConfig(); // runs only when first accessed
```

⚠️ If you read a `late` variable before it's set: `LateInitializationError: Field 'controller' has not been initialized.` If you assign a `late final` twice: `LateInitializationError: Field 'x' has already been initialized.`

`dynamic` vs `Object?` vs `var`

- `var` is **not a type**; it just means "infer the type". `var x = 1;` → `x` is `int` forever. `var x;` (no initializer) → `dynamic`.
- `dynamic` **turns off** the type checker. Any method call compiles; errors happen at runtime (`NoSuchMethodError`).
- `Object?` accepts any value but you must **check/cast** before calling type-specific methods. Safer.

Rule: avoid `dynamic` except when dealing with JSON (`Map<String, dynamic>`) or interop.

3.3 Built-in types

3.3.1 Numbers

```
int a = 7;            // 64-bit integer on native; on web it's a JS number (53-bit precise)
double b = 3.5;       // 64-bit floating point (IEEE 754)
num c = 5;            // supertype of int and double
c = 5.5;              // allowed
```

```dart
print(7 / 2);   // 3.5   (// always returns double)
print(7 ~/ 2);  // 3     (integer division)
print(7 % 2);   // 1     (remainder)
print(-7 % 2);  // 1     (Dart % is always non-negative for positive divisor)
print((-7).remainder(2)); // -1 (sign follows the dividend)

print(int.parse('42'));          // 42
print(int.tryParse('abc'));      // null (safe)
print(double.parse('3.14'));
print(3.14159.toStringAsFixed(2)); // "3.14"
print(10.clamp(0, 5));           // 5
print(0.1 + 0.2);                // 0.30000000000000004 (floating point!)
```

💰 **Money tip:** never use `double` for money in real apps → use integer cents (`int`) or a decimal package (`decimal`).

3.3.2 Strings

```dart
var s1 = 'single';
var s2 = "double";
var multi = '''
Line 1
Line 2''';
var raw = r'C:\path\no\escape';     // raw string
var name = 'Ravi';
print('Hi $name, length ${name.length}'); // interpolation
print('a' + 'b');                // concatenation (prefer interpolation)
print('abc' * 2);                // abcabc

// Useful methods
'Hello'.toUpperCase();  'Hello'.toLowerCase();
'  hi  '.trim();        'a,b,c'.split(',');      // [a, b, c]
'hello'.contains('ell'); 'hello'.startsWith('he');
'hello'.replaceAll('l', 'L');  'hello'.substring(1, 3); // "el"
'hello'.indexOf('l');   // 2
'5'.padLeft(3, '0');    // "005"
```

Strings are **immutable** and **UTF-16** encoded. For user-perceived characters (emoji, accents) use the `characters` package (`'👨‍👩‍👧'.characters.length == 1`).

`StringBuffer` for building big strings efficiently:

```dart
final sb = StringBuffer();
for (var i = 0; i < 3; i++) { sb.write(i); }
print(sb.toString()); // 012
```

3.3.3 Booleans

Only `true` and `false`. Dart does **not** have "truthy" values: `if (1)` or `if (list)` is a **compile error**. Use `if (list.isNotEmpty)`.

3.3.4 Other core types

- `List`, `Set`, `Map` (section 3.8), `Iterable`.
- `Runes` (Unicode code points), `Symbol` (`#foo`), `Null`, `Never` (a function that never returns: always throws/loops), `void`, `Record` (Chapter 4), `Function`, `Future` / `Stream` (Chapter 5), `DateTime`, `Duration`, `Uri`, `RegExp`, `BigInt`.

```
final now = DateTime.now();
final tomorrow = now.add(const Duration(days: 1));
print(tomorrow.difference(now).inHours); // 24
final utc = DateTime.utc(2025, 1, 31);
final parsed = DateTime.parse('2025-01-31T10:00:00Z');
print(parsed.isUtc);              // true
print(parsed.toLocal());
```

3.4 Null safety — the seat belt 🚗

Since Dart 2.12 (and **mandatory** since Dart 3.0), Dart has **sound null safety**.

5-year-old version: Before null safety, any box could secretly be **empty**, and when you reached in to grab the toy — 💥 crash ("null pointer exception"). Now each box has a label: **"always has a toy"** (`String`) or **"might be empty"** (`String?`). The compiler won't let you grab from a "might be empty" box without checking first.

```
String name = 'Ana';      // can NEVER be null
String? nickname;         // may be null (defaults to null)

// print(nickname.length);    // ❌ compile error
print(nickname?.length);      // ✅ null-aware access → prints null
print(nickname ?? 'none');    // ✅ if-null operator → 'none'
nickname ??= 'Annie';         // assign only if null
print(nickname!.length);      // ✅ "bang" — I PROMISE it's not null (throws if it is!)
```

3.4.1 Null-aware operators (memorise)

Operator	Name	Meaning
`?` in type	nullable type	`int?` may be null
`?.`	null-aware access	call only if not null, else null
`??`	if-null	left if not null, else right
`??=`	null-aware assignment	assign if currently null

Operator	Name	Meaning
`!`	null assertion ("bang")	cast away nullability; throws `TypeError` if null
`?..`	null-aware cascade	`obj?..a()..b()`
`...?`	null-aware spread	`[...?maybeList]`
`?[`	null-aware index	`list?[0]`
`?` element (Dart 3.8+)	null-aware collection element	`[?maybeItem]` adds only if non-null; `{?key: value}`

```
String? maybe = getValue();
final items = [1, 2, ?maybeInt];           // Dart 3.8+: adds maybeInt only if non-null
final map = {'a': 1, if (maybe != null) 'b': maybe}; // older equivalent using collection-if
```

3.4.2 Type promotion (flow analysis)

Dart is smart: after a null check, the variable is **promoted** to non-null.

```
void greet(String? name) {
  if (name == null) return;
  print(name.length);   // ✅ name is promoted to String
}
```

⚠️ **Promotion does NOT work on (non-private or non-final) fields** because another piece of code (or a getter override) could change them between the check and the use:

```
class User {
  String? email;
  void send() {
    if (email != null) {
      // print(email.length); // ❌ error: 'email' is a public field, can't promote
      final e = email;        // ✅ copy to local variable
      if (e != null) print(e.length);
    }
  }
}
```

Since Dart 3.2, **private final fields** *can* be promoted (`final String? _email;`).

3.4.3 Why avoid `!`

`!` is a promise to the compiler. Break the promise → runtime crash (`Null check operator used on a null value` — one of the most common Flutter crash messages in Crashlytics!). Prefer:

- `if (x != null)` / local variable promotion
- `??` defaults
- pattern matching: `if (x case final value?) { ... }`

3.4.4 `required` named parameters

```
void createUser({required String name, int? age, String country = 'IN'}) {}
```

3.5 Operators

```
// Arithmetic
+  -  *  /  ~/  %  -expr   ++  --
// Equality & relational
==  !=  >  <  >=  <=
// Type test
is  is!  as
// Logical
&&  ||  !
// Bitwise
&  |  ^  ~  <<  >>  >>>
// Assignment
=  +=  -=  *=  /=  ~/=  %=  ??=  &=  |=  ^=  <<=  >>=
// Conditional
cond ? a : b      a ?? b
// Cascade
..   ?..
// Spread
...  ...?
```

Cascade `..` — "and also do this to the same object"

```
final paint = Paint()
  ..color = Colors.red
  ..strokeWidth = 4
  ..style = PaintingStyle.stroke;
// Equivalent to creating paint, then setting 3 properties, then returning paint.

final list = <int>[]..add(1)..add(2); // [1, 2]
```

`is`, `as`

```
Object x = 'hello';
if (x is String) print(x.toUpperCase()); // promoted
final s = x as String;  // cast; throws TypeError if wrong
```

`==` vs `identical()`

- `==` → **equality**, can be overridden (value equality).
- `identical(a, b)` → **same object in memory** (reference equality).

3.6 Control flow

```
// if / else
if (score >= 90) {
  grade = 'A';
} else if (score >= 75) {
  grade = 'B';
} else {
  grade = 'C';
}

// for
for (var i = 0; i < 5; i++) { print(i); }
// for-in
for (final fruit in ['apple', 'mango']) { print(fruit); }
// forEach (don't use with async callbacks!)
[1, 2, 3].forEach(print);
// while / do-while
while (n > 0) { n--; }
do { n++; } while (n < 3);
// break / continue / labels
outer:
for (var i = 0; i < 3; i++) {
  for (var j = 0; j < 3; j++) {
    if (j == 1) continue outer;
    if (i == 2) break outer;
  }
}
```

Switch (Dart 3 — powerful!)

Old style statement (no fall-through needed for non-empty cases since Dart 3 — `break` is optional):

```
switch (command) {
  case 'open':
    open();
  case 'close' || 'exit':   // logical-or pattern
    close();
  default:
    unknown();
}
```

Switch expression (returns a value):

```
final label = switch (statusCode) {
  200 => 'OK',
```

```
  404 => 'Not found',
  >= 500 && < 600 => 'Server error',   // relational patterns
  _ => 'Other',                          // wildcard = default
};
```

Patterns are covered fully in Chapter 4.

Collection if / for (super useful in Flutter widget lists!)

```
final isAdmin = true;
final menu = [
  'Home',
  'Profile',
  if (isAdmin) 'Admin Panel',
  for (final n in [1, 2]) 'Item $n',
];
// ['Home', 'Profile', 'Admin Panel', 'Item 1', 'Item 2']

Column(children: [
  const Text('Title'),
  if (loading) const CircularProgressIndicator() else const Text('Done'),
  ...items.map((e) => Text(e)),
]);
```

Assert

```
assert(age >= 0, 'Age must be non-negative');
```

Runs **only in debug mode** (stripped from release). Flutter framework uses many asserts to give helpful debug errors.

3.7 Functions

3.7.1 Basics

```
int add(int a, int b) {
  return a + b;
}
int multiply(int a, int b) => a * b;   // arrow syntax for a single expression
```

Every function returns something; if no return statement, it returns `null` (with return type `void` / `dynamic`).

3.7.2 Parameters

```
// Positional required
void a(int x, int y) {}

// Optional positional — in []
void b(int x, [int y = 0, int? z]) {}
b(1); b(1, 2); b(1, 2, 3);

// Named — in {} ; optional by default; use `required` or defaults
void c({required String name, int age = 0, String? email}) {}
c(name: 'A', email: 'x@y.com');

// Mixed: positional first, then named
void d(int id, {bool verbose = false}) {}
```

You can't mix `[]` and `{}` in the same function.

Since Dart 2.17, **named arguments can appear anywhere** in the call:

```
d(verbose: true, 42);  // allowed
```

3.7.3 Functions are first-class objects

```
int square(int x) => x * x;
final f = square;              // store in a variable
print([1, 2, 3].map(square));  // pass as argument

void runTwice(void Function() action) { action(); action(); }
runTwice(() => print('hi'));   // anonymous function (lambda/closure)

typedef Compare = int Function(int a, int b);   // type alias
Compare desc = (a, b) => b - a;
```

3.7.4 Closures — functions that remember 🧠

A closure "captures" variables from the scope where it was created:

```
Function makeCounter() {
  var count = 0;
  return () => ++count;   // remembers `count`
}
final counter = makeCounter();
print(counter()); // 1
print(counter()); // 2
```

Flutter callbacks like `onPressed: () => setState(() => _count++)` are closures capturing `this` and fields.

⚠️ Closure capture in loops:

```
final callbacks = <void Function()>[];
for (var i = 0; i < 3; i++) {
  callbacks.add(() => print(i));
}
for (final c in callbacks) c();  // prints 0, 1, 2 (Dart creates a fresh `i` per iteration — unlike old JS `var`)
```

3.7.5 Lexical scope

Variables are visible inside the `{}` block where they are declared and nested blocks.

3.7.6 Getters & setters, tear-offs

```
class Rect {
  double width, height;
  Rect(this.width, this.height);
  double get area => width * height;     // getter
  set side(double v) { width = v; height = v; } // setter
}
// Tear-off: referencing a method/constructor without calling it
final parse = int.parse;               // method tear-off
final makeList = List<int>.empty;      // constructor tear-off (Dart 2.15+)
['1','2'].map(int.parse).toList();
```

3.7.7 Wildcard variables (Dart 3.7+)

`_` is now a non-binding wildcard; multiple `_` parameters are allowed:

```
list.forEach((_) => count++);
builder: (_, _, child) => child!,   // multiple wildcards allowed in Dart 3.7+
```

3.8 Collections — List, Set, Map, Iterable

3.8.1 List (ordered, allows duplicates, indexed)

```
final nums = <int>[3, 1, 2];
nums.add(4);                 // [3,1,2,4]
nums.addAll([5, 6]);
nums.insert(0, 0);
nums.remove(1);              // removes first occurrence of value 1
nums.removeAt(0);
```

```dart
nums.removeWhere((n) => n.isEven);
nums.sort();                 // in place
nums.sort((a, b) => b.compareTo(a)); // descending
print(nums.first); print(nums.last); print(nums.length);
print(nums.isEmpty); print(nums.isNotEmpty);
print(nums.contains(3)); print(nums.indexOf(3));
print(nums.reversed.toList());
print(nums.sublist(0, 2));

// Fixed-length & filled
final zeros = List.filled(3, 0);          // [0,0,0] — fixed length!
final gen = List.generate(5, (i) => i * i); // [0,1,4,9,16]
final copy = List.of(nums);               // shallow copy
final unmod = List.unmodifiable(nums);
```

⚠️ `List.filled` with objects shares the same instance:

```dart
final grid = List.filled(3, <int>[]);   // all 3 rows are THE SAME list!
grid[0].add(1);
print(grid); // [[1], [1], [1]]  😱
final ok = List.generate(3, (_) => <int>[]); // ✅ separate lists
```

3.8.2 Set (unique items, unordered conceptually — default LinkedHashSet keeps insertion order)

```dart
final ids = <int>{1, 2, 3};
ids.add(2);                 // no duplicate
print(ids.contains(2));     // fast O(1)
final a = {1, 2, 3}, b = {2, 3, 4};
print(a.union(b));          // {1,2,3,4}
print(a.intersection(b));   // {2,3}
print(a.difference(b));     // {1}
final unique = [1, 1, 2].toSet().toList(); // dedupe
```

⚠️ `{}` alone is a **Map**, not a Set. Empty set: `<int>{}`.

3.8.3 Map (key → value)

```dart
final ages = <String, int>{'ann': 30, 'bob': 25};
ages['cat'] = 22;
print(ages['ann']);              // 30
print(ages['zzz']);              // null (lookup returns V?)
ages.putIfAbsent('dan', () => 40);
ages.update('ann', (v) => v + 1, ifAbsent: () => 0);
ages.remove('bob');
print(ages.containsKey('ann'));
print(ages.keys); print(ages.values);
```

```dart
for (final MapEntry(:key, :value) in ages.entries) {  // pattern destructuring (Dart 3)
  print('$key = $value');
}
ages.forEach((k, v) => print('$k:$v'));
final upper = ages.map((k, v) => MapEntry(k.toUpperCase(), v));
```

Variants: `HashMap` (no order), `LinkedHashMap` (insertion order — default `{}`), `SplayTreeMap` (sorted by key).

3.8.4 Iterable & functional methods

`List` and `Set` are `Iterable`s. Most functional methods return **lazy** Iterables (computed only when iterated!).

```dart
final nums = [1, 2, 3, 4, 5];
nums.map((n) => n * 2);               // (2,4,6,8,10) — lazy Iterable!
nums.where((n) => n.isOdd);           // filter
nums.firstWhere((n) => n > 3);        // 4 (throws StateError if none)
nums.firstWhere((n) => n > 9, orElse: () => -1);
nums.any((n) => n > 4);               // true
nums.every((n) => n > 0);             // true
nums.reduce((a, b) => a + b);         // 15 (throws on empty list)
nums.fold<int>(0, (sum, n) => sum + n); // 15 (safe for empty)
nums.take(2); nums.skip(2);
nums.takeWhile((n) => n < 3);
nums.expand((n) => [n, n]);           // flatMap
nums.join(', ');                      // "1, 2, 3, 4, 5"
nums.whereType<int>();
nums.indexed;                          // Dart 3: (0,1),(1,2)... records
```

From `package:collection` (official, very common):

```dart
import 'package:collection/collection.dart';
nums.firstWhereOrNull((n) => n > 9);   // null instead of throwing
nums.groupListsBy((n) => n.isEven);    // {false: [1,3,5], true: [2,4]}
nums.sum;  nums.average;  nums.max;
nums.sortedBy((n) => -n);
const ListEquality().equals([1,2], [1,2]); // true (deep equality)
```

Since Dart 3.0 the core libraries also provide `nums.firstOrNull`, `lastOrNull`, `singleOrNull`, `elementAtOrNull`, `indexed` and `nonNulls` as extension members on `Iterable` (no import needed).

⚠️ Lazy iterable bug

```dart
var counter = 0;
final doubled = [1, 2, 3].map((n) { counter++; return n * 2; });
print(counter);         // 0 — nothing ran yet!
doubled.toList();
```

```
doubled.toList();
print(counter);         // 6 — ran twice!
```

Fix: call `.toList()` once and store it if you'll iterate multiple times. In Flutter, `children: items.map(...).toList()` (or spread `...items.map(...)`) is required because `children` expects a `List<Widget>`.

3.8.5 Spread operator

```
final a = [1, 2];
final b = [0, ...a, 3];        // [0,1,2,3]
List<int>? maybe;
final c = [...?maybe];         // [] if null
final merged = {...mapA, ...mapB}; // later keys win
```

3.8.6 Immutability helpers

```
final frozen = List.unmodifiable([1, 2]);
final view = UnmodifiableListView(myList); // dart:collection — read-only VIEW (changes in myList show through)
```

3.9 Exceptions & error handling

```
try {
  final n = int.parse('abc');
} on FormatException catch (e) {
  print('Bad number: ${e.message}');
} on Exception catch (e, stackTrace) {
  print('Some exception: $e\n$stackTrace');
} catch (e) {
  print('Anything else (including Errors): $e');
  rethrow;                     // re-throw preserving stack trace
} finally {
  print('always runs');
}

throw FormatException('Invalid');
throw ArgumentError.value(x, 'x', 'must be positive');
throw StateError('Not initialised');
throw UnimplementedError();
```

`Exception` vs `Error` (interview favourite)

- `Exception` — expected, recoverable problems the program should handle (network down → `SocketException`, bad input → `FormatException`, `TimeoutException`).

- `Error` — **programmer bugs** that should not be caught normally; fix the code instead (`TypeError`, `RangeError`, `NoSuchMethodError`, `StateError`, `AssertionError`, `LateInitializationError`, `StackOverflowError`, `OutOfMemoryError`).

Custom exception:

```
class ApiException implements Exception {
  final int statusCode;
  final String message;
  const ApiException(this.statusCode, this.message);
  @override
  String toString() => 'ApiException($statusCode): $message';
}
```

You *can* throw any object (`throw 'oops';`) but **don't** — throw `Exception`/`Error` subtypes (lint: `only_throw_errors`).

3.10 Libraries, imports & visibility

```
import 'dart:math';                          // core library
import 'package:http/http.dart' as http;     // prefix to avoid name clashes
import 'package:my_app/utils.dart' show formatDate;   // only this name
import 'package:my_app/legacy.dart' hide OldThing;    // all except
import 'big_feature.dart' deferred as big;   // lazy load (web: code splitting)
await big.loadLibrary(); big.run();

export 'src/widgets.dart';                   // re-export (barrel files)
part 'user.g.dart';                          // code-gen part file
part of 'user.dart';                         // in the generated file
```

Privacy: Dart has **no `public/private/protected` keywords.** An identifier starting with `_` is **private to its library (file)** — not to its class! Two classes in the same file can see each other's `_fields`.

Common libraries: `dart:core` (auto-imported), `dart:async`, `dart:math`, `dart:convert` (JSON, UTF-8, base64), `dart:io` (files, sockets, HTTP — **not on web**), `dart:isolate`, `dart:typed_data`, `dart:collection`, `dart:ui` (Flutter engine), `dart:ffi`, `dart:js_interop` (web).

Conditional imports (to support web and mobile with different implementations):

```
import 'storage_stub.dart'
  if (dart.library.io) 'storage_io.dart'
  if (dart.library.js_interop) 'storage_web.dart';
```

3.11 Dart coding conventions (Effective Dart)

- `UpperCamelCase` for classes, enums, typedefs, extensions, type parameters.
- `lowerCamelCase` for variables, functions, parameters, named constants (`const defaultTimeout`, NOT `DEFAULT_TIMEOUT`).
- `lowercase_with_underscores` for file names, packages, directories, import prefixes.
- Prefer `final` for locals that don't change.
- Use `///` doc comments for public APIs.
- Format with `dart format` (since Dart 3.7 the formatter uses a new "tall" style and decides trailing commas itself, depending on your language version).
- Avoid `print`; prefer `debugPrint`/`log` from `dart:developer`/`package:logging`.

3.12 Common Dart bugs & fixes

Bug	Cause	Fix
`Null check operator used on a null value`	`!` on null	Avoid `!`; check null; trace where value should be set
`type 'Null' is not a subtype of type 'String'`	JSON field missing/null cast to non-null	Use `String?` + defaults; validate JSON
`type 'List<dynamic>' is not a subtype of type 'List<String>'`	JSON decode gives `List<dynamic>`	`(json['tags'] as List).cast<String>()` or `List<String>.from(...)` or `.map((e) => e as String).toList()`
`_Map<String, dynamic>` not subtype of `Map<String, String>`	same	`Map<String, String>.from(json)`
`LateInitializationError`	read before set	initialise in `initState`, or make nullable
`RangeError (index)`	index out of bounds	check `length`, use `elementAtOrNull`
`Bad state: No element`	`first`/`firstWhere`/`single`/`reduce` on empty	use `firstOrNull`, `firstWhereOrNull`, `fold`
`Concurrent modification during iteration`	modifying list while looping	iterate a copy (`List.of(list)`) or use `removeWhere`
`Unsupported operation: Cannot add to an unmodifiable list`	mutating const / unmodifiable list	copy: `[...list, item]` or `List.of(list)`
`0.1 + 0.2 != 0.3`	floating point	compare with tolerance, use ints for money
`List.filled` shared objects	same instance	`List.generate`

3.13 Interview Questions — Dart basics

Q1. Difference between `final` and `const`? `final`: single assignment, value determined at runtime, only reference is immutable. `const`: compile-time constant, deeply immutable, canonicalised (identical consts share one instance). Instance fields can be `final` but not `const`. Const widgets help Flutter skip rebuilds.

Q2. `var` vs `dynamic` vs `Object?` `var` infers a static type from the initializer (fixed afterwards). `dynamic` disables static type checking — any member access compiles and may fail at runtime. `Object` is the root type of non-nullable values; members other than Object's must be accessed after type checks/casts.

Q3. What is null safety? What is "sound" null safety? Types are non-nullable by default; nullable types are marked `?`. "Sound" means the compiler *guarantees* a non-nullable variable can never be null at runtime (when all code is null-safe), allowing the compiler to optimise away null checks. Dart 3 requires 100% sound null safety.

Q4. What does the `late` keyword do? Declares a non-nullable variable initialised later (runtime check on access) and/or makes the initialiser lazy. Throws `LateInitializationError` on read-before-write or double-write for `late final`.

Q5. What is the `!` operator and its risk? Null assertion: casts `T?` to `T`, throwing at runtime if null. Overuse causes "Null check operator used on a null value" crashes.

Q6. What is the difference between `??` and `?.`? `a ?? b` returns `a` if non-null else `b`. `a?.b` accesses `b` only if `a` is non-null, otherwise the whole expression is null.

Q7. What are named vs positional parameters? Positional parameters are passed by position (`[]` makes them optional); named parameters `{}` are passed by name and are optional unless `required`. Named improve readability and are standard in Flutter widget constructors.

Q8. What is a closure? A function object that captures variables from its enclosing lexical scope and can use them later even after the scope has ended.

Q9. What's the difference between `Exception` and `Error`? Exceptions represent anticipated runtime failures meant to be caught; Errors represent programming bugs that generally shouldn't be caught.

Q10. Difference between `==` and `identical`? `==` is overridable value equality; `identical` checks reference identity.

Q11. What is the cascade operator? `..` performs a sequence of operations on the same object and returns the object, e.g., building `Paint()..color=..` .

Q12. What is the spread operator? `...` inserts all elements of a collection into another collection literal; `...?` handles null.

Q13. What are collection if/for? Conditionally or iteratively include elements inside list/set/map literals — used heavily to build widget children declaratively.

Q14. Are `map`/`where` eager or lazy? Lazy; they return Iterables that compute on iteration. Iterating multiple times recomputes. Call `toList()` to materialise.

Q15. How is privacy achieved in Dart? With a leading underscore — private to the library (file), not the class. `part` files share the same library privacy.

Q16. What's the difference between `List.filled` and `List.generate`? `filled` puts the *same* value in every slot (dangerous for mutable objects) and is fixed-length by default; `generate` calls a function per index.

Q17. What is `typedef`? A type alias, e.g., `typedef JsonMap = Map<String, dynamic>;` or function types `typedef Callback = void Function(int);`.

Q18. What does `int ~/ int` do? Integer (truncating) division.

Q19. What happens if you `print` in release mode? It still executes (prints to device log) — that's why `debugPrint`/loggers and `avoid_print` are preferred; `debugPrint` also throttles output to avoid Android log dropping. To fully remove logs in release, guard with `kDebugMode` or use a logger with levels.

Q20. What is type promotion and why doesn't it work on public fields? Flow analysis narrows a variable's type after checks (`is`, `!= null`). Public or non-final fields could be overridden by a getter or modified between check and use, so they cannot be soundly promoted; copy to a local first (private final fields are promotable since Dart 3.2).

Chapter 4 — Dart Language, Part 2: OOP, Generics & Modern Dart 3 Features

Goal: Master classes, constructors (all kinds!), inheritance, abstract classes, interfaces, mixins, extensions, enums, generics, class modifiers, sealed classes, records, patterns, extension types, dot shorthands and primary constructors. These power every Flutter codebase and are standard interview material.

4.1 Classes and objects — cookie cutters 🍪

A **class** is a **cookie cutter**; an **object (instance)** is a **cookie** made with it. Each cookie can have different sprinkles (field values), but the same shape (structure & behaviour).

```dart
class Dog {
  // Fields (instance variables)
  final String name;
  int age;

  // Constructor
  Dog(this.name, this.age);

  // Method
  void bark() => print('$name says Woof!');

  // Getter
  bool get isPuppy => age < 1;

  @override
  String toString() => 'Dog($name, $age)';
}

void main() {
  final rex = Dog('Rex', 3);   // `new` keyword is optional (and discouraged)
  rex.bark();
  rex.age++;
  print(rex);                  // Dog(Rex, 4)
}
```

4.2 Constructors — every kind (interview gold)

4.2.1 Generative constructor with initializing formals

```
class Point {
  final double x, y;
  Point(this.x, this.y);           // "this.x" = initializing formal
}
```

4.2.2 Named constructors

Dart has **no constructor overloading**; use named constructors:

```
class Point {
  final double x, y;
  Point(this.x, this.y);
  Point.origin() : x = 0, y = 0;
  Point.fromJson(Map<String, dynamic> json)
      : x = (json['x'] as num).toDouble(),
        y = (json['y'] as num).toDouble();
}
final p = Point.fromJson({'x': 1, 'y': 2});
```

4.2.3 Initializer list (runs before the body; can set final fields; can assert)

```
class Temperature {
  final double celsius;
  final double fahrenheit;
  Temperature(this.celsius)
      : assert(celsius >= -273.15),
        fahrenheit = celsius * 9 / 5 + 32;
}
```

4.2.4 Redirecting constructor

```
class Point {
  final double x, y;
  Point(this.x, this.y);
  Point.onXAxis(double x) : this(x, 0);
}
```

4.2.5 Const constructor

All fields must be `final`, no body:

```dart
class Color {
  final int r, g, b;
  const Color(this.r, this.g, this.b);
}
const red = Color(255, 0, 0);
print(identical(const Color(255, 0, 0), red)); // true — canonicalised
```

Every Flutter widget you write should have a `const` constructor when possible.

4.2.6 Factory constructor

A `factory` constructor **doesn't have to create a new instance**. It can return a cached instance, a subtype, or do logic before creating:

```dart
class Logger {
  static final Map<String, Logger> _cache = {};
  final String name;
  Logger._internal(this.name);                // private named constructor

  factory Logger(String name) =>
      _cache.putIfAbsent(name, () => Logger._internal(name));
}

abstract class Shape {
  factory Shape(String type) => switch (type) {
        'circle' => Circle(),
        'square' => Square(),
        _ => throw ArgumentError(type),
      };
  double area();
}
```

Factory vs generative:

	Generative	Factory
Must create new instance	✅	❌ **(can return existing/subtype)**
Has access to `this`	✅ **(in body)**	❌
Initializer list	✅	❌
Can be `const`	✅	✅ **only as redirecting factory**
Common use	**normal**	**singletons, caching, `fromJson`, returning subclasses**

4.2.7 Singleton pattern in Dart

```dart
class ApiClient {
  ApiClient._();                              // private constructor
  static final ApiClient instance = ApiClient._();
```

```
  // or: factory ApiClient() => instance;
}
```

(In big apps prefer dependency injection over global singletons — easier to test. See Chapter 21.)

4.2.8 Super parameters (Dart 2.17+)

```
class Animal {
  final String name;
  Animal(this.name);
}
class Cat extends Animal {
  final bool indoor;
  Cat(super.name, {this.indoor = true});   // instead of Cat(String name) : super(name)
}
// In Flutter: const MyWidget({super.key});
```

4.2.9 Primary constructors (Dart 3.13+)

Dart 3.13 (released August 2026) added **primary constructors**: declare the constructor and its fields directly in the class header. They add no new semantics — just less boilerplate. Requires `environment: sdk: ^3.13.0`.

```
// Before
class Point {
  final int x;
  final int y;
  const Point(this.x, this.y);
}

// With a primary constructor: `var`/`final` parameters become fields
class Point(var int x, var int y);
```

Dart 3.13 also allows `new` in place of the class name to declare a (non-factory) constructor in the body, and `factory` alone for a factory constructor. A class with a primary constructor cannot have other *non-redirecting generative* constructors.

> Because this is very new, most existing codebases (and interview code) still use classic constructors. Check the official Dart docs for the exact syntax variants (named/optional parameters, `const`, initializer bodies) before adopting team-wide.

4.3 Inheritance — `extends`

```
class Animal {
  void speak() => print('...');
}
class Dog extends Animal {
  @override
  void speak() {
    super.speak();          // call parent
    print('Woof');
  }
}
```

- Dart has **single inheritance** (one superclass).
- Every class implicitly extends `Object`.
- `@override` annotation documents and lets the analyzer check that you're overriding.
- Constructors are **not** inherited.

Polymorphism

```
final List<Animal> zoo = [Dog(), Cat()];
for (final a in zoo) a.speak();   // each calls its own version
```

4.4 Abstract classes & interfaces

Abstract class

Can't be instantiated; may contain abstract methods (no body) and concrete methods.

```
abstract class Repository {
  Future<List<String>> fetchAll();        // abstract
  void log(String m) => print('[repo] $m'); // concrete
}
```

Implicit interfaces — every class is an interface!

In Dart there is (traditionally) **no `interface` keyword**. Every class defines an interface. With `implements`, you must implement **all** members (no code inherited):

```
class FakeRepository implements Repository {
  @override
  Future<List<String>> fetchAll() async => ['a', 'b'];
  @override
  void log(String m) {}     // must implement this too
}
```

Since Dart 3, you can write `interface class` / `abstract interface class` to control this explicitly (see 4.9).

`extends` vs `implements` vs `with` (very common question)

	`extends`	`implements`	`with`
How many	**one**	**many**	**many**
Inherits code	✅	❌ **(must implement everything)**	✅ **(mixin code copied in)**
Calls `super`	✅	❌	✅ **(to the class before it in chain)**
Relationship	**"is-a"**	**"can act like / contract"**	**"has capability"**

4.5 Mixins — sticker packs 🏷️

5-year-old version: A mixin is a **sticker pack of abilities.** You can stick "CanSwim" and "CanFly" on a Duck. The Duck gets those abilities without being *born from* a Swimmer.

```
mixin CanSwim {
  void swim() => print('Swimming');
}
mixin CanFly {
  int altitude = 0;           // mixins can have state
  void fly() => print('Flying at $altitude');
}
class Duck extends Animal with CanSwim, CanFly {}
Duck()..swim()..fly();
```

Restricting a mixin with `on`

```
mixin Logging on State {          // only classes that extend State can use it
  void log(String msg) => debugPrint('${widget.runtimeType}: $msg');
}
```

Flutter's famous mixins

- `SingleTickerProviderStateMixin` / `TickerProviderStateMixin` — provide `vsync` for `AnimationController`.
- `AutomaticKeepAliveClientMixin` — keep list item state alive when scrolled away.
- `WidgetsBindingObserver` — app lifecycle (it's an abstract mixin class).
- `RestorationMixin` — state restoration.
- `ChangeNotifier` can be used as a mixin (`with ChangeNotifier`).

Linearization (order matters!)

`class C extends A with M1, M2` builds a chain: `A → A+M1 → A+M1+M2 → C`. If M1 and M2 both define `hello()`, **the last one (M2) wins**, and `super.hello()` in M2 calls M1's.

```dart
mixin A { String hi() => 'A'; }
mixin B { String hi() => 'B'; }
class X with A, B {}
print(X().hi()); // B
```

Mixin vs abstract class

- Mixins can't have generative constructors.
- Since Dart 3, a class can't be used as a mixin unless declared `mixin` or `mixin class`.

4.6 Extension methods — adding new tricks to old toys 🧸

Add methods to existing types (even ones you don't own) without subclassing:

```dart
extension StringX on String {
  String get capitalized =>
      isEmpty ? this : '${this[0].toUpperCase()}${substring(1)}';
  bool get isValidEmail => RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$').hasMatch(this);
}

extension ContextX on BuildContext {
  ThemeData get theme => Theme.of(this);
  Size get screenSize => MediaQuery.sizeOf(this);
  void showSnack(String msg) =>
      ScaffoldMessenger.of(this).showSnackBar(SnackBar(content: Text(msg)));
}

print('hello'.capitalized);   // Hello
context.showSnack('Saved!');
```

⚠️ Extensions are resolved **statically** (by the declared type) — they don't work on `dynamic` and can't be overridden polymorphically.

4.7 Enums

Simple enum

```dart
enum Status { idle, loading, success, error }
final s = Status.loading;
print(s.name);         // "loading"
print(s.index);        // 1
print(Status.values);  // [Status.idle, ...]
Status.values.byName('error'); // Status.error (throws if not found)
```

Enhanced enums (Dart 2.17+) — fields, constructors, methods, interfaces

```
enum Planet implements Comparable<Planet> {
  mercury(mass: 3.303e+23, radius: 2.4397e6),
  earth(mass: 5.976e+24, radius: 6.37814e6);

  const Planet({required this.mass, required this.radius});
  final double mass;
  final double radius;

  double get surfaceGravity => 6.67300E-11 * mass / (radius * radius);

  @override
  int compareTo(Planet other) => mass.compareTo(other.mass);
}

enum HttpStatus {
  ok(200), notFound(404), serverError(500);
  const HttpStatus(this.code);
  final int code;
  static HttpStatus fromCode(int c) => values.firstWhere((e) => e.code == c);
}
```

Enums work great with `switch` — the compiler checks **exhaustiveness** (forget a case → error).

Dot shorthands (Dart 3.10+)

When the type is known from context, you can omit it:

```
Status s = .loading;              // instead of Status.loading
Column(
  mainAxisAlignment: .center,     // instead of MainAxisAlignment.center
  crossAxisAlignment: .start,
  children: [...],
);
switch (s) {
  case .idle: ...
  case .loading: ...
  // ...
}
EdgeInsets padding = .all(16);    // static members & constructors too
```

Requires `sdk: ^3.10.0`. Older code uses the full names — both are valid.

4.8 Generics — boxes that say what's inside 🎁

5-year-old version: A generic is a box with a **label slot**: `Box<Apple>` only holds apples, `Box<Toy>` only toys. Same box design, safe contents.

```
class Box<T> {
  final T value;
  Box(this.value);
  T open() => value;
}
final b = Box<int>(5);
final c = Box('hi');           // T inferred as String

T firstOrDefault<T>(List<T> items, T fallback) =>
    items.isEmpty ? fallback : items.first;

// Bounded generics
class NumberBox<T extends num> {
  final T value;
  NumberBox(this.value);
  T doubleIt() => (value * 2) as T;
}

// Generic result type used in real apps:
sealed class Result<T> {
  const Result();
}
final class Ok<T> extends Result<T> {
  final T data;
  const Ok(this.data);
}
final class Err<T> extends Result<T> {
  final Object error;
  const Err(this.error);
}
```

Dart generics are **reified** — type info is kept at runtime (`list is List<int>` works), unlike Java's erasure.

Covariance gotcha: Dart's generic classes are covariant: a `List<int>` *is a* `List<Object>`. This can cause runtime errors:

```
List<Object> objs = <int>[1, 2];
objs.add('x');  // compiles, but throws TypeError at runtime!
```

4.9 Class modifiers (Dart 3) — `abstract`, `base`, `interface`, `final`, `sealed`, `mixin`

Dart 3 added modifiers controlling how a class can be used **outside its library**:

Declaration	Construct?	Extend outside lib?	Implement outside lib?	Mix in?	Exhaustive switch?
`class`	✅	✅	✅	❌	❌
`base class`	✅	✅ (subclasses must be base/final/sealed)	❌	❌	❌
`interface class`	✅	❌	✅	❌	❌
`final class`	✅	❌	❌	❌	❌
`sealed class`	❌ (implicitly abstract)	❌	❌	❌	✅
`abstract class`	❌	✅	✅	❌	❌
`abstract interface class`	❌	❌	✅	❌	❌
`mixin`	❌	❌	✅	✅	❌
`mixin class`	✅	✅	✅	✅	❌

Inside the **same library** (file), these restrictions don't apply.

Sealed classes — the "closed family" 👨‍👩‍👧

A `sealed` class's direct subtypes must all be in the **same library**. So the compiler **knows all possible subtypes** and can check that a `switch` handles every one:

```
sealed class AuthState {}
class AuthInitial extends AuthState {}
class AuthLoading extends AuthState {}
class AuthSuccess extends AuthState { final User user; AuthSuccess(this.user); }
class AuthFailure extends AuthState { final String message; AuthFailure(this.message); }

Widget build(BuildContext context) {
  final state = ...;
  return switch (state) {
    AuthInitial() => const LoginForm(),
    AuthLoading() => const CircularProgressIndicator(),
    AuthSuccess(:final user) => HomePage(user: user),
    AuthFailure(:final message) => ErrorView(message),
    // No default needed; add a new subclass → compiler errors here until handled ✅
  };
}
```

This is the modern replacement for many `freezed` union types and is widely used for Bloc states, API results, and navigation events.

4.10 Records (Dart 3) — anonymous mini-objects

A **record** is a lightweight, immutable, **anonymous** bundle of values — like a tuple.

```dart
(String, int) getUser() => ('Asha', 30);          // positional
({String name, int age}) getUser2() => (name: 'Asha', age: 30); // named

final user = getUser();
print(user.$1);  // Asha  ($1, $2 for positional fields)
print(user.$2);  // 30
final u2 = getUser2();
print(u2.name);

// Destructuring with patterns
final (name, age) = getUser();
final (:name, :age) = getUser2();   // shorthand for (name: name, age: age)

// Records have structural value equality
print((1, 'a') == (1, 'a')); // true

// Swap
var a = 1, b = 2;
(a, b) = (b, a);
```

Use records for **returning multiple values** from functions; use classes when the data has behaviour or is passed widely.

4.11 Patterns (Dart 3) — shape-matching 🔍

Patterns let you **check the shape of a value and pull out pieces** at the same time.

Pattern kinds

Pattern	Example
Constant	`case 404:`
Variable / wildcard	`case var x:` / `_`
Relational	`case >= 200 && < 300:`
Logical or / and	`case 'y' || 'yes':` / `case > 0 && < 10:`
Null-check	`case final v?:` **(match if non-null, bind)**
Null-assert	`case final v!:` **(throw if null)**
Cast	`case final n as int:`
List	`case [first, ...rest]:` / `case [_, _]:`
Map	`case {'type': 'user', 'id': int id}:`
Record	`case (int x, int y):`

Pattern	Example
Object	`case Point(x: 0, :final y):`
Parenthesized	`case (1 || 2) && != 3:`

Where patterns appear

```
// 1. switch statements & expressions
String describe(Object o) => switch (o) {
      int n when n < 0 => 'negative int',     // `when` guard clause
      int() => 'int',
      String s when s.isEmpty => 'empty string',
      String() => 'string',
      [] => 'empty list',
      [final first, ...] => 'list starting with $first',
      {'name': String name} => 'map with name $name',
      (int a, int b) => 'pair $a,$b',
      _ => 'something else',
    };

// 2. if-case
final json = {'user': {'name': 'Ravi', 'age': 30}};
if (json case {'user': {'name': String name, 'age': int age}}) {
  print('$name is $age');      // safe JSON validation + extraction in one go!
}

// 3. variable declarations / destructuring
final [a, b, c] = [1, 2, 3];
final Point(:x, :y) = point;

// 4. for-in loops
for (final (index, item) in items.indexed) { print('$index: $item'); }
```

Exhaustiveness

`switch` **expressions** must be exhaustive. With `bool`, enums, sealed types and nullable types, the compiler can verify completeness. Statement switches over these types are also checked.

4.12 Extension types (Dart 3.3+) — zero-cost wrappers

An **extension type** wraps an existing type with a **new static type** and a different API, with **no runtime cost** (it's erased at runtime).

```
extension type UserId(String value) {
  bool get isValid => value.startsWith('usr_');
}
extension type Meters(double value) {
```

```
  Meters operator +(Meters other) => Meters(value + other.value);
}

void loadUser(UserId id) {}
loadUser(UserId('usr_1'));   // ✅
// loadUser('usr_1');        // ❌ compile error — prevents mixing up IDs
```

Uses: type-safe IDs, units, and **JS interop** (`dart:js_interop` uses extension types heavily for web). Caveat: since it's erased at runtime, `is UserId` checks the underlying type — it's a static-only safety net.

4.13 Equality & hashCode — value objects

By default, `==` compares **identity**. For data classes compare **values**:

```
class User {
  final String id;
  final String name;
  const User(this.id, this.name);

  @override
  bool operator ==(Object other) =>
      identical(this, other) ||
      other is User && other.id == id && other.name == name;

  @override
  int get hashCode => Object.hash(id, name);   // Object.hashAll(list) for lists
}
```

Rule: if you override `==`, you **must** override `hashCode` consistently (equal objects ⇒ equal hash codes) — otherwise `Set` / `Map` break.

Writing this by hand is error-prone. Teams use:

- `equatable` — extend `Equatable`, list `props`.
- `freezed` — code generation of `==`, `hashCode`, `copyWith`, `toString`, JSON, unions.
- **records** — have built-in structural equality.
- Data class macros were planned but **the Dart team cancelled macros in January 2025**; code generation (`build_runner`) remains the standard. (Interviewers sometimes ask — know that it was cancelled.)

`copyWith` pattern (immutable updates)

```
class Todo {
  final String title;
  final bool done;
```

```
  const Todo({required this.title, this.done = false});

  Todo copyWith({String? title, bool? done}) =>
      Todo(title: title ?? this.title, done: done ?? this.done);
}
final t2 = t1.copyWith(done: true);
```

⚠️ Gotcha: with `??` you can't set a nullable field *back to null*. Freezed handles that; manually you can use a sentinel or `ValueGetter<T?>? field` parameter.

4.14 Static members, operators, callable classes, `noSuchMethod`

```
class MathUtils {
  static const double pi = 3.14159;
  static int square(int x) => x * x;
}
MathUtils.square(3);

class Vector {
  final double x, y;
  const Vector(this.x, this.y);
  Vector operator +(Vector o) => Vector(x + o.x, y + o.y);
  Vector operator -() => Vector(-x, -y);
  double operator [](int i) => i == 0 ? x : y;
}

class Multiplier {
  final int factor;
  Multiplier(this.factor);
  int call(int x) => x * factor;   // callable object
}
final triple = Multiplier(3);
print(triple(5)); // 15
```

4.15 Composition over inheritance (Flutter's philosophy)

Flutter widgets are built by **composing** small widgets rather than subclassing big ones. You don't extend `ElevatedButton` to make a "PrimaryButton" — you create a new `StatelessWidget` that *returns* a configured `ElevatedButton`. Interviewers ask "why does Flutter prefer composition?" → flexibility, smaller units, easier reuse, avoids fragile base class problems, deep hierarchies.

4.16 Interview Questions — Dart OOP & modern features

Q1. Does Dart support multiple inheritance? No — single inheritance via `extends`; but a class can `implement` multiple interfaces and use multiple mixins (`with`).

Q2. What is a mixin? How is it different from inheritance and interfaces? A mixin is a reusable chunk of class members that can be "mixed into" multiple class hierarchies with `with`. Unlike `extends` you can apply many; unlike `implements` you get the implementation. Restrict with `on`. Order matters (last wins).

Q3. What is a factory constructor? Give use cases. A constructor that may return an existing instance or a subtype instead of always creating a new one: singletons, caches, `fromJson`, choosing implementation subclasses.

Q4. What's the difference between `extends` and `implements`? `extends` inherits implementation (single); `implements` only adopts the interface and requires implementing all members (multiple allowed).

Q5. What are extension methods? Statically-resolved methods added to existing types without modifying/subclassing them.

Q6. What are sealed classes and why are they useful? Classes whose direct subtypes are all declared in the same library, allowing the compiler to check exhaustive switches over them; great for modelling states, events, results.

Q7. What are records? Immutable, anonymous, fixed-size aggregates with positional and/or named fields and structural equality; useful for multiple return values.

Q8. What are patterns? Syntax for matching values against shapes (constants, types, lists, maps, records, objects, relational/logical operators) and destructuring them, usable in switch, if-case, declarations and loops.

Q9. What is an extension type? A compile-time-only wrapper giving a new static type and interface to an existing representation type with zero runtime overhead.

Q10. Difference between `abstract class` and `interface class`? Abstract can't be instantiated but can be extended and implemented. `interface class` can be instantiated and implemented but not extended outside its library. `abstract interface class` = pure interface.

Q11. What does `base` / `final` class modifier do? `base`: can be extended but not implemented outside the library (guarantees inherited implementation). `final`: neither extended nor implemented outside the library.

Q12. Why must `hashCode` be overridden with `==`? Hash-based collections use hashCode to find buckets; equal objects with different hash codes break Set/Map lookups.

Q13. Why use `const` constructors? Compile-time canonical instances, less memory/GC, and Flutter can skip rebuilding identical const widget subtrees.

Q14. What are named constructors and why does Dart need them? Dart lacks overloading; named constructors (`Point.origin()`) provide multiple ways to construct with clear names.

Q15. What are super parameters? `super.x` in constructor parameters forwards to the superclass constructor, e.g., `const MyWidget({super.key})`.

Q16. What's an initializer list? Code after `:` in a constructor that runs before the body, used for initializing final fields, asserts and calling super/redirecting.

Q17. What are generics, and are Dart generics reified? Parameterised types for type-safe reuse; yes, Dart keeps type arguments at runtime.

Q18. What happened to Dart macros? They were an experimental metaprogramming feature (e.g., `@JsonCodable`); the Dart team stopped work on them in early 2025, focusing instead on improving code generation (build_runner speed) and smaller language features like augmentations-related improvements, dot shorthands and primary constructors.

Q19. What are dot shorthands? Dart 3.10 syntax allowing `.value` instead of `Type.value` where the type is inferable from context — e.g., `mainAxisAlignment: .center`.

Q20. Explain `mixin class`. A declaration usable both as a regular class (instantiate/extend) and as a mixin; required in Dart 3 if you want both behaviours.

Chapter 5 — Asynchronous Dart: Event Loop, Futures, Streams, Isolates & Zones

Goal: Truly understand how Dart does many things "at once" with one thread. This is one of the most heavily tested interview areas (event loop, microtasks, Future vs Stream, isolates, `compute`) and the source of many production bugs.

5.1 The 5-year-old story: one chef, many orders 👨‍🍳

Dart (per isolate) has **one chef** (one thread). The chef can only do one thing at a time. But when the chef puts a cake in the oven (a slow network request), they **don't stand and stare at the oven** — they write a sticky note "when oven beeps, take cake out" and go do other orders. When the oven beeps, the note goes into a **queue**, and the chef handles it when free.

- The sticky-note system = **event loop**.
- "When oven beeps, do X" = a **Future** with a callback / `await`.
- A conveyor belt delivering many cakes over time = a **Stream**.
- Hiring a **second chef in a separate kitchen** (no shared bowls, only passing notes through a window) = an **Isolate**.

Important: if the chef decides to **knead dough for 10 minutes without stopping** (heavy synchronous computation), *nobody* else is served — in Flutter that means **frozen UI / jank**. `async` does NOT make heavy CPU work run in the background! Only isolates do.

5.2 The event loop in detail

Each isolate has:

1. **Microtask queue** — high priority, small internal tasks (`scheduleMicrotask`, `Future.microtask`, completion callbacks of already-completed futures, `then` callbacks).
2. **Event queue** — I/O events, timers, gestures, frame callbacks, messages from other isolates, `Future(...)`, `Future.delayed`.

Algorithm:

```
run main() synchronously to completion
loop forever:
    while microtask queue not empty:
        run next microtask
    if event queue not empty:
        run next event (then go back and drain microtasks again)
```

Microtasks always run before the next event. Too many microtasks → starve events (UI freezes).

The classic interview puzzle

```
import 'dart:async';

void main() {
  print('1 - sync start');

  Future(() => print('2 - event queue (Future)'));
  Future.delayed(Duration.zero, () => print('3 - event queue (delayed 0)'));
  scheduleMicrotask(() => print('4 - microtask'));
  Future.microtask(() => print('5 - microtask (Future.microtask)'));
  Future.value(1).then((_) => print('6 - then on completed future (microtask)'));
  Future.sync(() => print('7 - Future.sync runs immediately'));

  print('8 - sync end');
}
```

Output:

```
1 - sync start
7 - Future.sync runs immediately
8 - sync end
4 - microtask
5 - microtask (Future.microtask)
6 - then on completed future (microtask)
2 - event queue (Future)
3 - event queue (delayed 0)
```

Explanation:

- Synchronous code first: 1, 7 (`Future.sync` runs its callback synchronously), 8.
- Then **all microtasks** in FIFO order: 4, 5, 6.
- Then **events** in order: 2, 3.

Another puzzle with async/await

```
Future<void> task() async {
  print('A');
  await null;          // suspends; rest continues as a microtask
  print('B');
}
void main() {
  print('1');
  task();
  print('2');
```

```
}
// 1, A, 2, B
```

Key rule: an `async` function runs **synchronously until the first `await`**. Then it returns a `Future` to the caller, and the rest of the function continues later.

5.3 Future — a promise of one value later 🎟️

A `Future<T>` is like a **restaurant buzzer**: you get it now, and it will buzz later with either your food (**value**) or bad news (**error**).

States: **uncompleted → completed with value** OR **completed with error**.

5.3.1 Creating futures

```
Future<int> a = Future.value(42);
Future<int> b = Future.error(Exception('fail'));
Future<int> c = Future.delayed(const Duration(seconds: 2), () => 42);
Future<int> d = Future(() => compute42());   // schedule on event queue

// Completer — manually complete (useful for wrapping callback APIs)
Future<String> wrapCallbackApi() {
  final completer = Completer<String>();
  legacyApi(onSuccess: completer.complete, onError: completer.completeError);
  return completer.future;
}
```

5.3.2 Consuming futures: `then` vs `async/await`

```
// then-style
fetchUser().then((user) => print(user.name))
           .catchError((e) => print('Error $e'))
           .whenComplete(() => print('done'));

// async/await style (preferred: reads like sync code)
Future<void> load() async {
  try {
    final user = await fetchUser();
    print(user.name);
  } catch (e, st) {
    print('Error $e $st');
  } finally {
    print('done');
  }
}
```

5.3.3 Running futures in parallel vs sequentially

```
// ❌ Sequential — total ≈ 3s
final a = await fetchA(); // 1s
final b = await fetchB(); // 1s
final c = await fetchC(); // 1s

// ✅ Parallel — total ≈ 1s
final results = await Future.wait([fetchA(), fetchB(), fetchC()]);

// ✅ Dart 3 records: typed parallel waits
final (user, posts) = await (fetchUser(), fetchPosts()).wait;
```

By default `Future.wait` waits for **all** futures to finish and then completes with the **first error** that occurred; pass `eagerError: true` to complete with an error as soon as any future fails (the others keep running regardless — futures can't be cancelled). The records `.wait` extension throws a `ParallelWaitError` that contains both the successful values and the errors.

Other helpers:

```
await Future.any([f1, f2]);                    // first to complete
await fetch().timeout(const Duration(seconds: 10)); // throws TimeoutException
await Future.forEach(items, (item) async => await upload(item)); // sequential async loop
await Future.doWhile(() async { ... return keepGoing; });
```

5.3.4 ⚠️ `forEach` with async — classic bug

```
// ❌ Does NOT wait! forEach ignores returned futures.
items.forEach((item) async {
  await upload(item);
});
print('All uploaded?'); // prints immediately, uploads still running

// ✅ Sequential
for (final item in items) {
  await upload(item);
}
// ✅ Parallel
await Future.wait(items.map(upload));
```

5.3.5 Unawaited futures & errors

If a Future errors and nobody listens, it becomes an **uncaught async error** (reported to the zone / `PlatformDispatcher.instance.onError`). Lint `unawaited_futures` catches missing awaits. When you intentionally don't await, use `unawaited(future)` from `dart:async` to document it.

5.3.6 `FutureOr<T>`

A type meaning "either `T` or `Future<T>`", used in APIs that accept sync or async callbacks.

5.4 Streams — a conveyor belt of values 🛤️

A `Stream<T>` delivers **many values over time** (plus possibly errors) and then optionally a **done** event. Examples: user typing, location updates, WebSocket messages, Firestore snapshots, Bloc states.

	Future	Stream
Number of values	**one**	**zero or many**
Analogy	**buzzer**	**conveyor belt / YouTube live**
Consume	`await`, `then`	`await for`, `listen`
Flutter widget	`FutureBuilder`	`StreamBuilder`

5.4.1 Single-subscription vs broadcast

- **Single-subscription** (default): **only one listener**, ever. Data is buffered until listened. Example: reading a file, HTTP response body. Listening twice → `Bad state: Stream has already been listened to.`
- **Broadcast: many listeners**; events not buffered — late listeners miss earlier events. Example: button clicks, `StreamController.broadcast()`.

```
final broadcast = myStream.asBroadcastStream();
```

5.4.2 Creating streams

```
// 1. async* generator
Stream<int> countDown(int from) async* {
  for (var i = from; i >= 0; i--) {
    await Future.delayed(const Duration(seconds: 1));
    yield i;                  // emit a value
  }
}
// yield* forwards all values from another stream:
Stream<int> twice() async* { yield* countDown(3); yield* countDown(3); }

// 2. StreamController
final controller = StreamController<String>();      // or .broadcast()
controller.sink.add('hello');      // or controller.add
controller.addError(Exception('x'));
await controller.close();          // ALWAYS close to avoid leaks
controller.stream.listen(print);

// 3. Built-ins
Stream.periodic(const Duration(seconds: 1), (i) => i);
```

```
Stream.fromIterable([1, 2, 3]);
Stream.fromFuture(fetch());
Stream.value(1); Stream.error(e); Stream.empty();
```

Synchronous generator (`sync*`) produces an `Iterable` lazily:

```
Iterable<int> naturals(int n) sync* { for (var i = 0; i < n; i++) yield i; }
```

5.4.3 Listening

```
final StreamSubscription<int> sub = countDown(5).listen(
  (value) => print(value),
  onError: (e) => print('error $e'),
  onDone: () => print('done'),
  cancelOnError: false,
);
sub.pause(); sub.resume();
await sub.cancel();   // ALWAYS cancel in dispose()!

// await for (inside async function)
await for (final v in countDown(3)) {
  print(v);
}
```

5.4.4 Transforming streams

```
stream.map((x) => x * 2)
      .where((x) => x > 2)
      .distinct()                 // skip consecutive duplicates
      .take(10).skip(1)
      .asyncMap((x) => fetchDetails(x))  // async map, sequential
      .handleError((e) => log(e))
      .timeout(const Duration(seconds: 5));
final first = await stream.first;
final list = await stream.toList();
```

Debounce/throttle (for search boxes) are not in core — use `package:rxdart` (`debounceTime`, `switchMap`, `BehaviorSubject`, `combineLatest`) or `stream_transform` (`debounce`, `throttle`, `switchMap`), or implement with a `Timer`.

5.4.5 RxDart subjects (common in older/enterprise code)

- `PublishSubject` — broadcast, no replay.
- `BehaviorSubject` — remembers the **latest** value; new listeners get it immediately (like a "current state").
- `ReplaySubject` — replays all (or N) previous values.

5.4.6 Stream memory leak bug

```
class _ChatState extends State<Chat> {
  StreamSubscription? _sub;

  @override
  void initState() {
    super.initState();
    _sub = chatService.messages.listen((m) => setState(() => _messages.add(m)));
  }

  @override
  void dispose() {
    _sub?.cancel();   // ❗ forget this → leak + "setState() called after dispose()"
    super.dispose();
  }
}
```

5.5 Isolates — a second kitchen 🏠🏠

5.5.1 What is an isolate?

An **isolate** is Dart's unit of concurrency: its **own memory heap**, its **own event loop**, and its **own thread** (when running). Isolates **do not share mutable memory → no data races, no locks**. They communicate by **passing messages** through **ports** (`SendPort` / `ReceivePort`).

The Flutter app's Dart code runs in the **root isolate** (main isolate).

5.5.2 When to use one

Any CPU-heavy work that would take more than a few milliseconds (remember: 16 ms per frame at 60 fps):

- Parsing/encoding big JSON (> ~100 KB… measure!)
- Image processing, compression, encryption, hashing.
- Sorting/filtering huge lists, search indexing.
- Heavy database transformations, PDF generation.

Not needed for: network requests or file I/O themselves — those are already asynchronous (the waiting happens outside Dart).

5.5.3 `Isolate.run` (Dart 2.19+) — the easiest way

```
import 'dart:convert';
import 'dart:isolate';

Future<List<Photo>> parsePhotos(String body) {
```

```
  return Isolate.run(() {
    final list = jsonDecode(body) as List<dynamic>;
    return list.map((e) => Photo.fromJson(e as Map<String, dynamic>)).toList();
  });
}
```

5.5.4 Flutter's `compute()`

```
import 'package:flutter/foundation.dart';

List<Photo> _parse(String body) { /* ... */ }   // top-level or static function
final photos = await compute(_parse, responseBody);
```

`compute` takes a callback and one message argument; on native it uses `Isolate.run` internally; **on the web it runs on the same thread** (web has no Dart isolates — use Web Workers for true parallelism).

5.5.5 Long-lived isolates with ports

```
Future<void> startWorker() async {
  final receivePort = ReceivePort();
  await Isolate.spawn(_worker, receivePort.sendPort);

  late SendPort workerPort;
  receivePort.listen((message) {
    if (message is SendPort) {
      workerPort = message;            // handshake
      workerPort.send('job 1');
    } else {
      print('Result: $message');
    }
  });
}

void _worker(SendPort mainPort) {
  final port = ReceivePort();
  mainPort.send(port.sendPort);        // send our port back
  port.listen((job) {
    mainPort.send('done: $job');
  });
}
```

5.5.6 Rules and gotchas

- Messages are **copied** (deep copy) between isolates — big messages cost time. `Isolate.exit()` (used by `Isolate.run`) can transfer the result **without copying**. `TransferableTypedData` transfers byte buffers efficiently.

- Since Dart 2.15, isolates spawned with `Isolate.spawn` share an **isolate group** (same code), so spawning is fast and closures can be sent (with restrictions — no sockets, ReceivePorts, etc.).
- **Platform channels / plugins in background isolates:** historically not possible; since Flutter 3.7 you can use `BackgroundIsolateBinaryMessenger.ensureInitialized(rootIsolateToken)` to call plugins from background isolates:

```
final token = RootIsolateToken.instance!;
Isolate.run(() async {
  BackgroundIsolateBinaryMessenger.ensureInitialized(token);
  final prefs = await SharedPreferences.getInstance(); // works now
});
```

- You can't touch the widget tree / `BuildContext` / UI from another isolate.
- Each isolate has its own copy of globals/singletons! A singleton initialised in main isolate is *not* initialised in a spawned one.
- Isolates have memory overhead; for many small tasks, use a pool (packages like `isolate_manager`, `squadron`, `worker_manager`).

5.5.7 Concurrency vs parallelism

- **Concurrency** — dealing with many things at once (interleaving) → event loop + async.
- **Parallelism** — doing many things *simultaneously* on multiple cores → isolates.

5.6 Zones — invisible bubbles around your code 🫧

A **Zone** is an execution context that persists across async callbacks. It can:

- Catch **uncaught async errors** in that zone.
- Store zone-local values (like thread-locals).
- Intercept `print`, timers, microtasks (used by test frameworks / `fake_async`).

```
runZonedGuarded(() {
  runApp(const MyApp());
}, (error, stack) {
  FirebaseCrashlytics.instance.recordError(error, stack, fatal: true);
});
```

⚠️ **Zone mismatch warning:** if you call `WidgetsFlutterBinding.ensureInitialized()` in one zone and `runApp` in another, Flutter prints "Zone mismatch". Keep them in the same zone.

Modern recommended global error handling (Flutter 3.3+):

```
void main() {
  WidgetsFlutterBinding.ensureInitialized();

  // 1. Errors thrown by the Flutter framework (build/layout/paint callbacks)
```

```
  FlutterError.onError = (details) {
    FlutterError.presentError(details);  // keeps console output in debug
    crashReporter.recordFlutterError(details);
  };

  // 2. Uncaught async errors and errors outside Flutter callbacks
  PlatformDispatcher.instance.onError = (error, stack) {
    crashReporter.recordError(error, stack, fatal: true);
    return true; // handled
  };

  // 3. Errors in background isolates (optional)
  Isolate.current.addErrorListener(RawReceivePort((pair) async {
    final List<dynamic> errorAndStack = pair;
    crashReporter.recordError(errorAndStack.first,
StackTrace.fromString(errorAndStack.last.toString()));
  }).sendPort);

  // 4. What to show when a widget's build throws (instead of red screen in release)
  ErrorWidget.builder = (details) => const Center(child: Text('Something went wrong'));

  runApp(const MyApp());
}
```

5.7 Timers

```
final t = Timer(const Duration(seconds: 2), () => print('once'));
final p = Timer.periodic(const Duration(seconds: 1), (timer) {
  if (timer.tick == 5) timer.cancel();
});
t.cancel(); // ALWAYS cancel timers in dispose()
```

Debounce with a timer (search box):

```
Timer? _debounce;
void onSearchChanged(String q) {
  _debounce?.cancel();
  _debounce = Timer(const Duration(milliseconds: 400), () => search(q));
}
```

5.8 Async in Flutter widgets — the dangerous parts

5.8.1 `setState() called after dispose()`

```
Future<void> _load() async {
  final data = await api.fetch();   // user navigated away during this await!
  if (!mounted) return;             // ✅ guard
  setState(() => _data = data);
}
```

5.8.2 Using `BuildContext` across async gaps

```
onPressed: () async {
  await save();
  if (!context.mounted) return;     // ✅ Flutter 3.7+: BuildContext.mounted
  Navigator.of(context).pop();
  ScaffoldMessenger.of(context).showSnackBar(...);
}
```

Lint: `use_build_context_synchronously`. Alternative: capture `final navigator = Navigator.of(context);` **before** the await.

5.8.3 `FutureBuilder` re-firing the future on every rebuild

```
// ❌ BAD: a new future is created on every build → infinite loading / repeated API calls
FutureBuilder(future: api.fetchUser(), builder: ...);

// ✅ GOOD: create once (initState / state management) and reuse
late final Future<User> _userFuture;
@override
void initState() {
  super.initState();
  _userFuture = api.fetchUser();
}
FutureBuilder<User>(
  future: _userFuture,
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const CircularProgressIndicator();
    }
    if (snapshot.hasError) return Text('Error: ${snapshot.error}');
    return Text(snapshot.data!.name);
  },
);
```

`ConnectionState` values: `none`, `waiting`, `active` (streams emitting), `done`.

`StreamBuilder` similarly — don't create the stream inside `build`.

5.9 Error handling in async code — full reference

```dart
// Errors thrown inside async functions become errors of the returned Future.
Future<void> f() async => throw Exception('x');

// Catch with try/await:
try { await f(); } catch (e) { ... }

// ❌ Not caught! The future isn't awaited inside the try.
try { f(); } catch (e) { /* never runs */ }

// Streams: errors go to onError / handleError; await-for rethrows.
```

5.10 Interview Questions — Async

Q1. Explain the Dart event loop. Each isolate runs main synchronously, then loops: drains the microtask queue entirely, then processes one event from the event queue, repeat. Futures/timers/I-O schedule events; `scheduleMicrotask` and future completions schedule microtasks.

Q2. Future vs Stream? Future = single async result (value or error). Stream = sequence of async events (zero or more values/errors, then done).

Q3. Is Dart single-threaded? How does Flutter avoid freezing? Each isolate is single-threaded with an event loop. I/O is non-blocking so `await` doesn't block; heavy CPU work must be moved to another isolate (`Isolate.run`, `compute`) to avoid janking the UI.

Q4. Does `async` make code run in a background thread? No. It only allows suspension at `await` points; the code still runs on the same isolate's thread.

Q5. What is the difference between microtask and event queues? Microtasks have higher priority and are all processed before the next event; used for short internal follow-ups. Events include I/O, timers, user input, frames.

Q6. What's a Completer? An object to create and manually complete a Future; used to adapt callback APIs to futures.

Q7. Single-subscription vs broadcast streams? Single-subscription allows one listener and buffers events until listened; broadcast allows many listeners and doesn't buffer.

Q8. How do you cancel a stream subscription and why? Store the `StreamSubscription` and call `cancel()` (e.g., in `dispose`) to free resources and avoid callbacks on disposed widgets.

Q9. `async*` vs `sync*`? `async*` returns a Stream and can `await`; `sync*` returns a lazily evaluated Iterable. Both use `yield` / `yield*`.

Q10. What is an isolate? How do isolates communicate? Independent worker with own memory and event loop; communicate by message passing through SendPort/ReceivePort; messages are copied (or transferred on exit).

Q11. `compute` vs `Isolate.run`? `compute` is Flutter's wrapper (one argument, web fallback runs on main thread); `Isolate.run` is Dart's API for running a closure in a new isolate and returning its result.

Q12. How do you run futures in parallel? `Future.wait([...])` or record `.wait` — start them before awaiting.

Q13. What is a Zone? An execution context propagated across async callbacks, used for error handling (`runZonedGuarded`), zone-local values, and intercepting async ops in tests.

Q14. How do you handle uncaught errors globally in Flutter? `FlutterError.onError` for framework errors, `PlatformDispatcher.instance.onError` for other uncaught errors, optionally `runZonedGuarded`, and `ErrorWidget.builder` for build failures.

Q15. Why shouldn't you create a future inside `build` for FutureBuilder? `build` runs often; each rebuild starts a new future, re-triggering the request and resetting the builder to waiting.

Q16. What does `await` do under the hood? It registers continuation callbacks on the future; the function is suspended and returns control to the event loop; when the future completes, the continuation is scheduled (as a microtask) to resume the function.

Q17. How to implement debounce for search? Cancel/restart a `Timer` on each keystroke, or use rxdart's `debounceTime` on a stream of queries, combined with `switchMap` to discard stale responses.

Q18. Can you use plugins inside isolates? Since Flutter 3.7, yes, via `BackgroundIsolateBinaryMessenger.ensureInitialized(RootIsolateToken)` (with plugin support caveats).

Q19. Can isolates share memory? Not mutable Dart objects; they share code within an isolate group and can share immutable data efficiently, and can transfer buffers. Shared-memory is possible only via FFI/native memory.

Q20. What is `FutureOr`? Union type `T | Future<T>`, allowing APIs to accept sync or async values.

PART II

Building UI

Chapter 6 — Widgets Deep Dive: Stateless, Stateful, Lifecycle, BuildContext, Keys & the Three Trees

Goal: Understand *exactly* how widgets work. This chapter is the heart of Flutter and of Flutter interviews: "What is BuildContext?", "Explain the StatefulWidget lifecycle", "What are keys and when do you need them?", "Widget vs Element vs RenderObject".

6.1 What is a widget really?

5-year-old: A widget is a **LEGO instruction card** 🖨️. The card says "put a red 2×4 brick here". The card is not the brick. Cards are cheap — you can print new ones all day. The actual bricks (what you see) are built by Flutter based on the cards.

Grown-up: A `Widget` is an **immutable configuration object** describing part of the UI. Its fields are `final`. When something changes, Flutter **creates new widget objects** (cheap) and compares them with the old ones to decide what work to do on the long-lived objects underneath (Elements and RenderObjects).

```
@immutable
abstract class Widget extends DiagnosticableTree {
  const Widget({this.key});
  final Key? key;
  @protected
  Element createElement();
  static bool canUpdate(Widget oldWidget, Widget newWidget) {
    return oldWidget.runtimeType == newWidget.runtimeType
        && oldWidget.key == newWidget.key;
  }
}
```

That `canUpdate` method is **the most important rule in Flutter** — memorise it: *an element can be updated with a new widget if the runtime type and key are the same.*

6.2 The three trees 🌳🌳🌳

Tree	Object	Lifetime	Job	Analogy
Widget tree	`Widget`	**Very short (rebuilt often)**	**Configuration / blueprint**	**Instruction cards**

Tree	Object	Lifetime	Job	Analogy
Element tree	`Element`	**Long-lived**	**Manages lifecycle, holds position in tree, connects widget ↔ render object, holds `State`**	**The site manager who reads the cards and keeps track of what's built where**
Render tree	`RenderObject`	**Long-lived**	**Layout, painting, hit testing**	**The actual bricks**

```
Widget tree             Element tree                 RenderObject tree
Container ──────►  StatelessElement
 └ Padding ──────►  SingleChildRenderObjectElement ──► RenderPadding
    └ Text ──────►  StatelessElement
       └ RichText ► MultiChildRenderObjectElement ──► RenderParagraph
```

Not every widget has a RenderObject. **StatelessWidget/StatefulWidget** only *compose* other widgets (they create `ComponentElement`s). **RenderObjectWidgets** (`Padding`, `Opacity`, `RichText`, `Row`, `SizedBox`...) create RenderObjects.

Why three trees? (interview question)

- Widgets are **cheap and immutable** → easy declarative programming; rebuild freely.
- Elements & RenderObjects are **expensive and mutable** → kept and **updated in place** when possible.
- Diffing new widgets against existing elements lets Flutter **reuse** expensive objects and do the minimum layout/paint work.

Reconciliation (how Flutter "diffs")

When a parent rebuilds and returns a new child widget, the element calls `updateChild(oldChildElement, newWidget)`:

1. `newWidget == null` → remove (unmount) old child element.
2. Old element is null → **inflate** new widget (create new element + maybe render object).
3. `Widget.canUpdate(old.widget, newWidget)` is true → **update** existing element with new widget (`element.update(newWidget)`), keeping State and RenderObject.
4. Otherwise → **deactivate old element**, inflate a **new** one (State is lost!).

Also, if the new widget is `identical` to the old one (same instance — e.g., a `const` widget), Flutter **skips rebuilding** that subtree completely. That's why `const` matters.

For children **lists** (`Row`, `Column`, `ListView`...), `updateChildren` matches old and new children from the top and bottom by type+key, and uses **keys** to match moved children in the middle. Without keys, matching is **positional**.

6.3 StatelessWidget

Use when the widget depends only on its **constructor arguments** and **inherited data** (Theme, MediaQuery...), and has no internal mutable state.

```
class Greeting extends StatelessWidget {
  const Greeting({super.key, required this.name});
  final String name;

  @override
  Widget build(BuildContext context) {
    return Text('Hello, $name!', style: Theme.of(context).textTheme.titleLarge);
  }
}
```

When is a StatelessWidget's `build` called?

1. First time it's inserted in the tree.
2. When its parent rebuilds and provides a new widget instance (not identical).
3. When an `InheritedWidget` it depends on changes (e.g., `Theme.of(context)`, `MediaQuery.of(context)`).

"Stateless" doesn't mean it never rebuilds — it means *it has no State object of its own*.

6.4 StatefulWidget

A StatefulWidget has two classes: the **widget** (immutable config) and the **State** (mutable, long-lived, attached to the element).

```
class Counter extends StatefulWidget {
  const Counter({super.key, this.initial = 0});
  final int initial;

  @override
  State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  late int _count;

  @override
  void initState() {
    super.initState();
    _count = widget.initial;   // access the widget's config via `widget`
  }

  @override
```

```
  Widget build(BuildContext context) {
    return TextButton(
      onPressed: () => setState(() => _count++),
      child: Text('$_count'),
    );
  }
}
```

Why split into two classes? Because widgets are recreated on every parent rebuild, but the State must **survive** those rebuilds. The State lives with the Element, and gets a *new* `widget` reference each time the parent rebuilds.

6.5 The StatefulWidget lifecycle — complete & precise ♻️

```
createState()
   |
   ▼
[mounted = true]
initState()                      ← once
   |
   ▼
didChangeDependencies()          ← right after initState, and whenever an InheritedWidget it
depends on changes
   |
   ▼
build()  ◄──────────────────────────┐
   |                                |
   ├── setState() ──────────────────┤ (marks dirty → build on next frame)
   ├── didUpdateWidget(oldWidget) ──┤ (parent rebuilt with new config of same type & key;
build always follows)
   ├── didChangeDependencies() ─────┤
   |                                |
   ▼
deactivate()      ← removed from tree (may be re-inserted in same frame via GlobalKey)
   |
   ├── activate()  ← re-inserted (GlobalKey reparenting) → build()
   ▼
dispose()         ← permanently removed; release resources
[mounted = false]
```

Method	When	What to do	What NOT to do
`createState()`	**Framework creates element**	**Return new State**	**Logic**

Method	When	What to do	What NOT to do
`initState()`	Once, after State inserted	Init controllers, subscriptions, start one-time loads; call `super.initState()` first	`Theme.of(context)` / `MediaQuery.of(context)` / `dependOnInheritedWidgetOfExactType` (use didChangeDependencies); calling async and expecting result synchronously
`didChangeDependencies()`	After initState and when inherited deps change	Work depending on InheritedWidgets (e.g., read locale, start something based on `Provider.of`)	Heavy work without checking whether it's needed (it can be called many times)
`build()`	Often	Return widgets; be pure & fast	Side effects, network calls, `setState`, creating controllers
`didUpdateWidget(old)`	Parent provided new widget config	Compare `old.x` vs `widget.x`; update controllers/subscriptions	Forgetting to call `super.didUpdateWidget(old)`
`setState(fn)`	You call it	Synchronously mutate state inside fn, then marks dirty	Calling during build, after dispose, or with async fn
`deactivate()`	Removed from tree	Rarely used	—
`activate()`	Reinserted after deactivate	Rarely used	—
`dispose()`	Permanently removed	Dispose controllers, cancel subscriptions/timers, remove listeners; call `super.dispose()` last	`setState`, using context to look up ancestors
`reassemble()`	On hot reload (debug only)	Re-init things during development	Production logic

`mounted`

`mounted` is `true` between `initState` and `dispose`. Check it after `await`s before calling `setState`.

What exactly does `setState` do?

```
@protected
void setState(VoidCallback fn) {
  // (asserts in debug: not disposed, fn not async, etc.)
  final Object? result = fn() as dynamic;
  assert(result is! Future, 'setState() callback argument returned a Future.');
  _element!.markNeedsBuild();
}
```

1. Runs your callback **synchronously**.
2. Calls `markNeedsBuild()` on the element → added to the **dirty elements list** and a frame is scheduled.
3. On the next frame, `BuildOwner.buildScope` rebuilds dirty elements (in depth order) → `build()` is called.

So `setState` does **not** rebuild immediately and multiple `setState` calls in the same frame are **batched into one build.**

The callback is only for clarity — technically you can mutate state before calling `setState(() {})` and it works, but convention is to mutate inside the callback.

`setState` rebuilds the whole `build` method of that State (and child widgets whose instances changed). Keep stateful widgets **small and low in the tree** to minimise rebuild cost.

6.6 BuildContext — "where am I in the tree?" 📍

5-year-old: BuildContext is your **home address** in the big family tree. With your address you can ask "who is my grandparent that has the Theme?" or "where is the nearest Navigator?"

Grown-up: `BuildContext` is an **interface implemented by `Element`**. Each widget's `build` receives *its own element* as context. It gives:

- Lookup of ancestors: `Theme.of(context)`, `Navigator.of(context)`, `context.findAncestorStateOfType<T>()`, `context.findAncestorWidgetOfExactType<T>()`, `context.dependOnInheritedWidgetOfExactType<T>()`, `context.getInheritedWidgetOfExactType<T>()` (no dependency).
- Size of render object after layout: `context.size` (only after layout, careful).
- `context.mounted` (Flutter 3.7+).
- `findRenderObject()`.

The #1 BuildContext bug: using the wrong context

```
class MyPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: ElevatedButton(
        onPressed: () {
          // This `context` belongs to MyPage — ABOVE the Scaffold.
          Scaffold.of(context).openDrawer();  // ❌ "Scaffold.of() called with a context that does not contain a Scaffold."
        },
        child: const Text('Open'),
      ),
    );
  }
}
```

Fixes:

```
// 1. Builder gives a context BELOW the Scaffold
body: Builder(
  builder: (innerContext) => ElevatedButton(
    onPressed: () => Scaffold.of(innerContext).openDrawer(),
    child: const Text('Open'),
  ),
),
// 2. Extract the button into its own widget (its context is below Scaffold).
// 3. Use a GlobalKey<ScaffoldState>.
```

Same bug class: `Provider.of<X>(context)` where the provider is created in the same build method; `Navigator.of(context)` above `MaterialApp`; `MediaQuery.of(context)` above `MaterialApp` ("No MediaQuery widget ancestor found").

`of` vs `maybeOf`

`Theme.of(context)` always returns something (falls back to default). `Navigator.of` throws if none; `Navigator.maybeOf` returns null. Many classes provide both.

6.7 InheritedWidget — sharing data down the tree efficiently

5-year-old: It's like a **radio tower** 📻 at the top of the tree. Any widget below can tune in (`dependOn...`). When the tower broadcasts a new message, **only the widgets that tuned in** rebuild.

```
class UserScope extends InheritedWidget {
  const UserScope({super.key, required this.user, required super.child});
  final User user;

  static User of(BuildContext context) {
    final scope = context.dependOnInheritedWidgetOfExactType<UserScope>();
    assert(scope != null, 'No UserScope found in context');
    return scope!.user;
  }

  @override
  bool updateShouldNotify(UserScope oldWidget) => user != oldWidget.user;
}
```

- `dependOnInheritedWidgetOfExactType` — O(1) lookup (elements keep a map of inherited widgets) **and registers a dependency**: when `updateShouldNotify` returns true, dependents get `didChangeDependencies` + rebuild.
- `getInheritedWidgetOfExactType` — O(1) lookup **without** dependency (no rebuilds).
- `Theme`, `MediaQuery`, `Directionality`, `DefaultTextStyle`, `Localizations`, `Navigator`-ish scopes, `Provider` — all built on InheritedWidget (or InheritedNotifier/InheritedModel).

InheritedModel — rebuild only for the aspect you care about

`MediaQuery` became an `InheritedModel` (Flutter 3.10). `MediaQuery.of(context)` rebuilds you on **any** change (keyboard opening changes viewInsets → rebuild!). Use aspect methods:

```
final size = MediaQuery.sizeOf(context);          // rebuild only when size changes
final padding = MediaQuery.paddingOf(context);
final insets = MediaQuery.viewInsetsOf(context);  // keyboard
final textScaler = MediaQuery.textScalerOf(context);
final brightness = MediaQuery.platformBrightnessOf(context);
```

👉 Performance tip asked in senior interviews: prefer `MediaQuery.sizeOf(context)` over `MediaQuery.of(context).size`.

InheritedNotifier

Combines InheritedWidget with a `Listenable` (e.g., `ChangeNotifier`, `ValueNotifier`): dependents rebuild when the notifier notifies. Basis of simple DI/state patterns (Chapter 14).

6.8 Keys — name tags for widgets 🏷️

5-year-old: Imagine kids sitting in chairs. If no one has a **name tag**, and the first kid leaves, the teacher just looks at chair positions: "chair 1 is still here, chair 2 is still here..." and gets confused about who's who. With **name tags**, the teacher knows exactly which kid moved where.

When do you NEED keys?

When you have **a collection of stateful widgets of the same type** that can be **reordered, inserted, or removed**, and you want each one's state to follow it.

The classic demo

```
class ColorTile extends StatefulWidget {
  const ColorTile({super.key});
  @override
  State<ColorTile> createState() => _ColorTileState();
}
class _ColorTileState extends State<ColorTile> {
  final color = Color(Random().nextInt(0xFFFFFFFF));   // state lives HERE
  @override
  Widget build(BuildContext context) => Container(width: 100, height: 100, color: color);
}

// Parent swaps tiles:
tiles = [tiles[1], tiles[0]];
```

Without keys: `Row` compares by position → same type at index 0 → `canUpdate` true → **element (and its State with the color) stays at index 0** → colours **don't swap!** 😱 With `ColorTile(key: UniqueKey())` or `ValueKey(id)`: elements are matched by key → states move → colours swap ✅.

(If the colour were passed in from the parent as a constructor parameter — i.e., a StatelessWidget — swapping works without keys because the *widget* carries the data.)

Types of keys

Key	Equality based on	Use case
`ValueKey<T>(value)`	`value ==`	**Items with unique id:** `ValueKey(todo.id)`
`ObjectKey(object)`	`identical(object)`	**When the object identity is unique but no simple id**
`UniqueKey()`	**only equal to itself**	**Force a brand-new element every build (⚠️ creating UniqueKey in build = state reset each rebuild!)**
`PageStorageKey`	**value**	**Preserve scroll position in PageStorage across tabs/routes**
`GlobalKey<T>`	**unique across the entire app**	**Access State/context from outside, move widget between parents keeping state, forms**
`GlobalObjectKey`	**object identity, global**	**rare**

Local keys (ValueKey, ObjectKey, UniqueKey) only need to be unique **among siblings**.

GlobalKey

```
final _formKey = GlobalKey<FormState>();
Form(key: _formKey, child: ...);
if (_formKey.currentState!.validate()) { _formKey.currentState!.save(); }

final scaffoldKey = GlobalKey<ScaffoldState>();
scaffoldKey.currentState?.openDrawer();
```

GlobalKey gives: `currentState`, `currentContext`, `currentWidget`. Moving a widget with a GlobalKey to another parent **in the same frame** reparents the element and **keeps its State** (deactivate → activate).

⚠️ GlobalKey downsides: expensive-ish (global registry), must be unique (error: *"Multiple widgets used the same GlobalKey"* — usually from creating a key in `build()` of a widget used twice, or two routes showing the same widget), encourage imperative coupling. Use sparingly.

Where to put the key

Put the key **on the outermost widget of the list item**, i.e., the direct child of the list:

```
ListView(children: [
  for (final todo in todos)
    TodoTile(key: ValueKey(todo.id), todo: todo),   // ✅ top-level of item
```

```
]);
// ❌ Wrong: Padding(child: TodoTile(key: ValueKey(todo.id)))  — the Padding is the sibling being matched!
```

Keys and `ListView.builder`

Lazy lists reuse elements by index. If items can be removed/reordered and are stateful (e.g., checkboxes with local state, text fields, animations), give them `ValueKey(id)`. Use `findChildIndexCallback` in `ListView.builder` / `SliverChildBuilderDelegate` to help Flutter find moved keyed items efficiently.

Using keys to *reset* state on purpose

```
// When userId changes, give the form a new key → fresh state
ProfileForm(key: ValueKey(userId), userId: userId);
```

6.9 Widget categories you must know

1. **ComponentWidgets** (no render object): `StatelessWidget`, `StatefulWidget`, `ProxyWidget` (`InheritedWidget`, `ParentDataWidget`).
2. **RenderObjectWidgets:**
   - `LeafRenderObjectWidget` (no children): `RawImage`, `ErrorWidget`.
   - `SingleChildRenderObjectWidget`: `Padding`, `Opacity`, `SizedBox`, `Align`, `ClipRRect`, `Transform`.
   - `MultiChildRenderObjectWidget`: `Row`, `Column`, `Stack`, `Flex`, `Wrap`, `Flow`, `RichText`, `CustomMultiChildLayout`.
3. **ParentDataWidget**: configures how a child is laid out by its parent: `Expanded` / `Flexible` (must be direct child of Flex: Row/Column), `Positioned` (must be child of Stack), `TableCell`, `LayoutId`. Misuse error: **"Incorrect use of ParentDataWidget"** — e.g., `Expanded` wrapped in a `Padding` inside a `Row` is fine only if the `Expanded` is the outer one: `Expanded` must be the *direct* child of a `Row` / `Column` / `Flex`, and `Positioned` must be a direct child of a `Stack` (wrap the inner widget instead: `Expanded(child: Padding(...))`).

Container is a convenience widget

`Container` is a StatelessWidget composing `Padding`, `ConstrainedBox`, `DecoratedBox`, `Align`, `Transform`, `ColoredBox`, `ClipPath`, `LimitedBox` depending on which properties you set. Interview Q: *"Container vs SizedBox?"* → `SizedBox` is a lightweight single render object for fixed size/spacing and can be `const`; Container is heavier and can't be const with many params. Use `SizedBox` for gaps and fixed sizes, `ColoredBox` for just colour, `DecoratedBox` for decoration, `Padding` for padding.

⚠️ `Container(color: X, decoration: BoxDecoration(...))` → **assertion error**: "Cannot provide both a color and a decoration". Put color inside the decoration.

6.10 Extract widgets, not helper methods (important best practice)

```
// ❌ Helper method
Widget _buildHeader() => Text('Header $count');

// ✅ Separate widget class
class Header extends StatelessWidget {
  const Header({super.key, required this.count});
  final int count;
  @override
  Widget build(BuildContext context) => Text('Header $count');
}
```

Why classes are better:

1. **Own BuildContext** → correct `of(context)` lookups (fixes the Scaffold.of bug).
2. **Rebuild optimisation**: a `const` widget or unchanged instance can be skipped; helper methods always re-execute with the parent.
3. Separate element → only it rebuilds when its InheritedWidget dependency changes (not the whole parent).
4. DevTools shows it in the widget tree; easier testing; reusable. Helper methods are fine for tiny, non-reused snippets but in interviews say "prefer widget classes".

6.11 Builder widgets & callbacks

- `Builder` — new context.
- `LayoutBuilder` — gives parent **constraints** (Chapter 7).
- `StatefulBuilder` — local setState inside dialogs/bottom sheets:

```
showDialog(context: context, builder: (_) => StatefulBuilder(
  builder: (context, setLocalState) => Checkbox(
    value: checked,
    onChanged: (v) => setLocalState(() => checked = v!),
  ),
));
```

- `ValueListenableBuilder` — rebuild only when a `ValueNotifier` changes.
- `ListenableBuilder` / `AnimatedBuilder` — rebuild when any `Listenable` changes (AnimatedBuilder is the same thing, named for animations).
- `FutureBuilder`, `StreamBuilder` (Chapter 5).
- `OrientationBuilder`, `TweenAnimationBuilder`.

`child` parameter optimisation:

```
AnimatedBuilder(
  animation: controller,
  child: const ExpensiveWidget(),             // built ONCE
  builder: (context, child) => Transform.rotate(
    angle: controller.value * 2 * pi,
    child: child,                             // reused every frame
  ),
);
```

6.12 App lifecycle (whole app, not widget)

```
class _AppState extends State<App> with WidgetsBindingObserver {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
  }
  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    super.dispose();
  }
  @override
  void didChangeAppLifecycleState(AppLifecycleState state) {
    switch (state) {
      case AppLifecycleState.resumed:  // visible & interactive
      case AppLifecycleState.inactive: // visible but not receiving input (iOS app switcher, phone call overlay)
      case AppLifecycleState.hidden:   // (3.13+) not visible; all platforms report it before paused
      case AppLifecycleState.paused:   // in background (mobile)
      case AppLifecycleState.detached: // engine running without a view (being destroyed / not yet attached)
    }
  }
}
```

Flutter 3.13 added `AppLifecycleListener` — easier API with `onResume`, `onPause`, `onHide`, `onShow`, `onInactive`, `onDetach`, `onRestart`, `onExitRequested` (desktop exit handling):

```
late final AppLifecycleListener _listener = AppLifecycleListener(
  onResume: _refresh,
  onPause: _saveDraft,
);
```

```
@override
void dispose() { _listener.dispose(); super.dispose(); }
```

Other `WidgetsBindingObserver` callbacks: `didChangeMetrics` (rotation/resize/keyboard), `didChangePlatformBrightness`, `didChangeLocales`, `didHaveMemoryPressure`, `didChangeTextScaleFactor` / `didChangeAccessibilityFeatures`, `didPopRoute` / `didPushRouteInformation` (back button/deep links at root).

6.13 Common widget-level bugs & fixes

Error message	Cause	Fix
`setState() called after dispose()`	async completes after widget removed / un-cancelled stream/timer	check `mounted`; cancel subscriptions in dispose
`setState() or markNeedsBuild() called during build`	calling setState (or a notifier that triggers rebuild) inside build / initState synchronously that triggers ancestor rebuild	move to event handler; use `WidgetsBinding.instance.addPostFrameCallback((_){...})`
`dependOnInheritedWidgetOfExactType<...>() was called before initState() completed`	`Theme.of(context)` in initState	move to `didChangeDependencies` or `build`
`Looking up a deactivated widget's ancestor is unsafe`	using `context` after widget disposed (e.g., in async callback, in dispose)	capture needed objects earlier (`final nav = Navigator.of(context)`), check `mounted`
`Multiple widgets used the same GlobalKey`	same GlobalKey in two places/routes	create keys once per instance in State, not static/global; ensure single use
`Duplicate keys found`	two siblings with same ValueKey	ensure IDs are unique
`A RenderFlex overflowed by X pixels`	Chapter 7	Chapter 7
`Incorrect use of ParentDataWidget`	Expanded/Flexible/Positioned not a direct child of Flex/Stack	restructure
State not updating when parent passes new value	You copied `widget.value` into State in `initState` only	handle in `didUpdateWidget`, or read `widget.value` directly in build
State "sticks" to wrong list item after delete	no keys on stateful list items	`ValueKey(item.id)`
Text field loses focus/text on rebuild	`TextEditingController` created in `build`, or UniqueKey in build	create controller in initState; avoid UniqueKey in build

Example of the `didUpdateWidget` bug:

```
class _PriceState extends State<Price> {
  late double _displayed;
  @override
  void initState() { super.initState(); _displayed = widget.price; }
```

```
  @override
  void didUpdateWidget(covariant Price oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.price != widget.price) {
      _displayed = widget.price;   // ✅ sync state with new config
    }
  }
  ...
}
```

6.14 Interview Questions — Widgets

Q1. What is a widget? An immutable description of part of the UI; a configuration for an Element.

Q2. StatelessWidget vs StatefulWidget? Stateless has no mutable state of its own and rebuilds only from parent/inherited changes; Stateful has a separate long-lived `State` object that can call `setState` to rebuild.

Q3. Explain the StatefulWidget lifecycle. createState → initState → didChangeDependencies → build → (setState / didUpdateWidget / didChangeDependencies → build)* → deactivate → (activate → build) → dispose. Plus reassemble on hot reload.

Q4. What does setState do internally? Runs the callback synchronously, then marks the element dirty (`markNeedsBuild`), scheduling a frame; dirty elements rebuild in the next frame's build phase. Multiple calls are batched.

Q5. What is BuildContext? A handle to the widget's location in the tree — actually the Element. Used to look up ancestors and inherited widgets.

Q6. Why can't you use `Theme.of(context)` in initState? Inherited dependencies aren't safe to establish there; use `didChangeDependencies` (or build). Reading with `listen: false`-style APIs that don't register dependencies may work, but the rule is to use didChangeDependencies.

Q7. Widget vs Element vs RenderObject? Widget = immutable config; Element = instantiation in the tree, manages lifecycle, holds State, links widget to render object; RenderObject = does layout, painting, hit testing.

Q8. What are keys? When are they needed? Identifiers to preserve/match element state when widgets of the same type move, are added or removed among siblings; needed for stateful items in reorderable/modified lists, to force state reset, to preserve scroll positions, and GlobalKeys for cross-tree access.

Q9. ValueKey vs ObjectKey vs UniqueKey vs GlobalKey? See table §6.8.

Q10. What is InheritedWidget? A widget that efficiently propagates data down the tree; descendants depending on it are rebuilt when `updateShouldNotify` returns true.

Q11. What is `didUpdateWidget` used for? Reacting to new configuration from the parent (e.g., re-subscribe if a stream parameter changed, update animation targets).

Q12. What's the difference between `deactivate` and `dispose`? Deactivate: element removed from tree, possibly re-inserted in the same frame (GlobalKey reparenting). Dispose: permanently removed; release resources.

Q13. Why is `build` called so many times, and is it expensive? Build runs on any state/config/dependency change; it should be pure and cheap — creating widgets is cheap, while elements/render objects are reused. Costly work belongs elsewhere.

Q14. Why prefer widget classes over helper methods? Own context, rebuild isolation, const-ability, better DevTools/testing.

Q15. What does `const` do for widgets? Canonical instance; identical instance on rebuild lets Flutter skip updating that subtree.

Q16. What does `mounted` mean? Whether the State is currently in the tree (between initState and dispose).

Q17. How do you run code after the first frame/build? `WidgetsBinding.instance.addPostFrameCallback((_) { ... })` in initState (e.g., to show a dialog or measure sizes).

Q18. Can a StatelessWidget have mutable fields? It shouldn't — widgets are `@immutable`; the analyzer warns. Mutable state belongs in a State or external state holder.

Q19. What is `Widget.canUpdate`? Returns true if runtimeType and key match — decides whether an element is updated or replaced.

Q20. How does Flutter know which widgets to rebuild? Only elements marked dirty (via setState, inherited dependency change, or listenable builders) and descendants receiving new widget instances get rebuilt; subtrees with identical widget instances are skipped.

Chapter 7 — Layout: Constraints, Flex, Stack, Responsive Design & Overflow Errors

Goal: Never be confused by a layout again. The single rule **"Constraints go down. Sizes go up. Parent sets position."** explains 90% of layout behaviour and 90% of layout bugs (yellow-black overflow stripes, "unbounded height", "RenderBox was not laid out").

7.1 The golden rule 🥇

Constraints go down. Sizes go up. Parent sets position.

5-year-old version: A parent box tells its child: *"You can be **between** this small and this big."* (constraints go down). The child thinks and says: *"OK, I'll be **this** big."* (size goes up). Then the parent decides **where** to put the child (position). The child **never** decides its own position, and can't be bigger than allowed.

Step by step (from the official docs):

1. A widget gets its **constraints** from its parent: min width, max width, min height, max height.
2. It goes through its children, telling each its constraints (can differ per child), and asks each child its size.
3. It positions children (x, y) one by one.
4. It tells its parent its own size (within original constraints).

Consequences (memorise these)

- A widget **can't know its position** on screen during layout; the parent decides.
- A widget **can't have any size it wants**; it must obey constraints. `Container(width: 100)` inside a parent that forces full width will be full width!
- Final size depends on the parent too — you can't define size "in isolation".
- **Alignment** decisions are made by the parent (`Center`, `Align`, `Row`'s alignment).

7.2 BoxConstraints

```
BoxConstraints(minWidth: 0, maxWidth: 300, minHeight: 0, maxHeight: 600)
```

Terminology:

- **Tight** constraints: min == max (exact size forced). e.g., `BoxConstraints.tight(Size(100, 100))`. The **screen** gives the root widget tight constraints = screen size.

- **Loose** constraints: min = 0, max = something. Child can be anywhere from 0 to max. e.g., `Center` loosens constraints for its child.
- **Bounded**: max is finite.
- **Unbounded**: max is `double.infinity`. Given by scrollables in their scroll direction and by `Row` (width) / `Column` (height) to non-flex children.

Experiments (official "Understanding constraints" examples, summarised)

```
// 1. Screen forces tight constraints → Container fills screen even with width 100
Container(width: 100, height: 100, color: Colors.red)   // as root → FULL SCREEN red

// 2. Center loosens constraints → now 100×100
Center(child: Container(width: 100, height: 100, color: Colors.red))

// 3. Container with no child and no size → as big as possible (fills Center's max)
Center(child: Container(color: Colors.red))   // full screen

// 4. Container with child and no size → wraps child size
Center(child: Container(color: Colors.red, child: const Text('Hi')))

// 5. ConstrainedBox can't override tighter parent constraints
Center(
  child: ConstrainedBox(
    constraints: const BoxConstraints(minWidth: 70, minHeight: 70, maxWidth: 150, maxHeight: 150),
    child: Container(color: Colors.red, width: 10, height: 10),   // becomes 70×70
  ),
)

// 6. UnconstrainedBox lets child be any size (may overflow → stripes)
UnconstrainedBox(child: Container(width: 4000, height: 50, color: Colors.red))

// 7. OverflowBox lets child be bigger without warning
// 8. LimitedBox: limits size only when incoming constraints are unbounded
// 9. FittedBox scales child to fit
FittedBox(child: Text('Some very long text that would overflow'))
// 10. FractionallySizedBox: size as fraction of parent
FractionallySizedBox(widthFactor: 0.5, child: ...)
```

Container's sizing algorithm (when there is no `width` / `height`)

1. No child, no constraints, no alignment → tries to be **as big as possible** (unless incoming constraints are unbounded, then as small as possible).
2. Has `alignment` → as big as possible, child aligned inside.
3. Has child, no alignment → **size of child** (plus padding). (This is why "Container behaves strangely" is a common confusion.)

7.3 Row & Column (Flex) — the most used layout widgets

```
Row(
  mainAxisAlignment: MainAxisAlignment.spaceBetween, // along the row (horizontal)
  crossAxisAlignment: CrossAxisAlignment.center,     // perpendicular (vertical)
  mainAxisSize: MainAxisSize.max,                    // take all width (default) or .min
  textDirection: TextDirection.ltr,                  // affects start/end
  verticalDirection: VerticalDirection.down,
  spacing: 8,                                        // (Flutter 3.27+) gap between children
  children: const [Icon(Icons.star), Text('4.5'), Text('(120)')],
)
```

- **Main axis**: Row → horizontal, Column → vertical.
- **Cross axis**: the other one.

`MainAxisAlignment`: `start`, `end`, `center`, `spaceBetween` (no space at edges), `spaceAround` (half space at edges), `spaceEvenly` (equal everywhere). `CrossAxisAlignment`: `start`, `end`, `center`, `stretch` (fill cross axis), `baseline` (needs `textBaseline: TextBaseline.alphabetic`).

How Flex lays out (important for debugging!)

1. Lay out **non-flexible** children first with **unbounded** main-axis constraints (Row gives each child `maxWidth: infinity`!) and cross-axis constraints from parent (loose, or tight if `stretch`).
2. Remaining main-axis space is divided among **flexible** children (`Expanded` / `Flexible`) according to their `flex` factor.
3. Position children per `mainAxisAlignment`.

That's why:

- A long `Text` in a `Row` **overflows** (it was told "you can be infinitely wide"). Fix: wrap with `Expanded` / `Flexible`.
- A `ListView` (horizontal) inside a `Row` crashes: unbounded width inside unbounded width.

Expanded vs Flexible vs Spacer

```
Row(children: [
  Expanded(flex: 2, child: Container(color: Colors.red)),   // takes 2/3 of free space, MUST fill it
  Flexible(flex: 1, child: Container(color: Colors.blue, width: 20)), // up to 1/3, can be smaller (loose)
  const Spacer(),                                           // empty Expanded
])
```

- `Expanded` = `Flexible(fit: FlexFit.tight)` — **must** fill the allotted space.
- `Flexible` (default `fit: FlexFit.loose`) — **can** be smaller than the allotted space.
- `Spacer(flex: 1)` — invisible expanded gap.

Gaps between children

- `SizedBox(width: 8)` / `SizedBox(height: 8)` between children (classic).
- `spacing:` parameter on Row/Column (Flutter 3.27+).
- `Gap(8)` from the `gap` package (works in Row/Column/scrollables).
- `Wrap(spacing:, runSpacing:)`.

7.4 Stack & Positioned — layers on top of each other 🥞

```
Stack(
  alignment: Alignment.center,       // for non-positioned children
  clipBehavior: Clip.none,           // allow children outside bounds to be visible (default
hardEdge)
  fit: StackFit.loose,               // non-positioned children constraints
  children: [
    Image.asset('assets/bg.png'),                          // base layer, sizes the stack
    const Positioned(top: 8, right: 8, child: Icon(Icons.favorite)),
    Positioned.fill(child: Container(color: Colors.black26)), // fills entire stack
    const Align(alignment: Alignment.bottomCenter, child: Text('Caption')),
  ],
)
```

- Stack's size = size of the largest **non-positioned** child (within constraints). If all children are positioned → Stack tries to be as big as possible.
- `Positioned` must be a **direct child** of Stack.
- Hit testing: children outside the Stack bounds (with `Clip.none`) are **visible but not tappable** — a classic bug ("my badge button outside the card doesn't respond"). Fix: make the Stack bigger (padding) so the child lies within bounds.
- `IndexedStack` shows only one child by index but keeps all alive (state preserved) — used for bottom navigation tabs.

7.5 Other layout widgets you must know

Widget	Purpose
`Padding`	**Space inside around child:** `EdgeInsets.all(16)`, `.symmetric(horizontal: 8)`, `.only(left: 4)`, `.fromLTRB`, `EdgeInsetsDirectional.only(start: 8)` **(RTL aware)**
`Center` / `Align`	**Position child;** `Alignment(-1..1, -1..1)`, `Alignment.topRight`, `AlignmentDirectional.centerStart`; `widthFactor` / `heightFactor`
`SizedBox`	**Fixed size / gap;** `SizedBox.expand()`, `SizedBox.shrink()`, `SizedBox.square(dimension:)`
`ConstrainedBox`	**Add min/max constraints**
`UnconstrainedBox`, `OverflowBox`, `LimitedBox`	**Advanced constraint control**

Widget	Purpose
`AspectRatio`	Size child to a ratio (e.g., 16/9 video)
`FittedBox`	Scale/fit child (`BoxFit.contain/cover/fill/scaleDown`)
`FractionallySizedBox`	Size as fraction of available space
`Expanded`, `Flexible`, `Spacer`	Flex children
`Wrap`	Like Row/Column but wraps to next line (chips, tags)
`Flow`	Efficient custom positioning with transforms (advanced)
`Table`	Grid of rows with column widths (`FlexColumnWidth`, `FixedColumnWidth`, `IntrinsicColumnWidth`)
`GridView`	Scrolling grid (Chapter 9)
`IntrinsicHeight` / `IntrinsicWidth`	Size children to their intrinsic size (e.g., make Row children same height for a divider). ⚠️ Expensive (O(N²) potential) — avoid in lists
`Baseline`	Align by text baseline
`CustomSingleChildLayout`, `CustomMultiChildLayout`	Custom layout delegates
`LayoutBuilder`	Build based on parent constraints
`OrientationBuilder`	Portrait/landscape
`SafeArea`	Avoid notches, status bar, home indicator
`Offstage`	Lay out but don't paint (state kept)
`Visibility`	Show/hide with options to maintain state/size/animation
`Opacity`	Transparency (costly when animating — use `FadeTransition` / `AnimatedOpacity`)
`Transform`	Rotate/scale/translate at paint time (doesn't affect layout)
`RotatedBox`	Rotate by quarter turns affecting layout
`ClipRRect`, `ClipOval`, `ClipPath`	Clip children (costly; use `borderRadius` on decoration when possible)
`DecoratedBox` / `BoxDecoration`	Background, border, radius, shadow, gradient, image
`Card`, `ListTile`	Material composites

Visibility vs Offstage vs Opacity vs conditional

Technique	Takes space?	Keeps state?	Paints?	Hit-testable?	Cost
`if (show) Widget()`	No	❌ (removed)	No	No	Cheapest when hidden
`Visibility(visible: false)`	No (default)	❌ by default (`maintainState: true` to keep)	No	No	Cheap
`Offstage(offstage: true)`	No	✅	No	No	Layout still runs
`Opacity(opacity: 0)`	Yes	✅	No (0 is optimised)	Yes (still tappable!)	Opacity between 0–1 costs a layer

Technique	Takes space?	Keeps state?	Paints?	Hit-testable?	Cost
`IgnorePointer` / `AbsorbPointer`	—	—	—	blocks taps	—

7.6 LayoutBuilder & responsive design

LayoutBuilder

```
LayoutBuilder(
  builder: (context, constraints) {
    if (constraints.maxWidth >= 1200) return const DesktopLayout();
    if (constraints.maxWidth >= 600) return const TabletLayout();
    return const MobileLayout();
  },
)
```

LayoutBuilder uses the **parent's constraints** (available space for this widget), not the full screen — great for reusable components.

MediaQuery

```
final size = MediaQuery.sizeOf(context);            // logical pixels of the window
final padding = MediaQuery.paddingOf(context);       // safe area insets (notch)
final viewInsets = MediaQuery.viewInsetsOf(context); // keyboard height in bottom
final dpr = MediaQuery.devicePixelRatioOf(context);
final textScaler = MediaQuery.textScalerOf(context);
final orientation = MediaQuery.orientationOf(context);
```

Logical pixels (dp): Flutter uses logical pixels; ~38 per cm. Physical pixels = logical × devicePixelRatio.

Responsive vs Adaptive

- **Responsive** — the layout adapts to **available space** (one column on phone, two on tablet).
- **Adaptive** — the app adapts to the **platform/input type** (Cupertino vs Material controls, mouse hover, keyboard shortcuts, desktop menus, touch target sizes).

Material 3 window size classes (breakpoints)

Class	Width (dp)	Typical layout
Compact	< 600	Bottom navigation bar, single pane
Medium	600–839	Navigation rail, maybe two panes
Expanded	840–1199	Navigation rail/drawer, two panes
Large	1200–1599	Permanent drawer, list-detail

Class	Width (dp)	Typical layout
Extra-large	≥ 1600	Multiple panes

Responsive best practices:

- Base decisions on **window width** (not device type/orientation) — split-screen and foldables exist.
- Use `LayoutBuilder` for component-level decisions.
- Don't lock orientation unless required (large screens; Android 16+ ignores orientation/resizability restrictions on large screens for apps targeting recent SDKs).
- Constrain content width on big screens: `ConstrainedBox(constraints: BoxConstraints(maxWidth: 600), child: ...)` centered.
- Use `Wrap`, `GridView` with `SliverGridDelegateWithMaxCrossAxisExtent`.
- Packages: `flutter_adaptive_scaffold` (now discontinued/community), `responsive_framework`, `flutter_screenutil` (scales sizes by design dimensions — popular in companies, but beware of scaling text unnaturally and breaking accessibility).

SafeArea

```
Scaffold(body: SafeArea(child: content));  // pads for notch/status bar/home indicator
SafeArea(bottom: false, child: ...);       // selectively
```

On Android 15+, apps targeting SDK 35 are **edge-to-edge by default** — content draws behind system bars, so correct `SafeArea`/padding handling is critical.

7.7 The famous layout errors (and EXACT fixes) 🚨

7.7.1 "A RenderFlex overflowed by 42 pixels on the right."

Yellow/black stripes. A Row/Column's children are bigger than the available space.

```
// ❌
Row(children: [Icon(Icons.person), Text('A very very very long user name that keeps going')]);
// ✅ Fix 1: Expanded/Flexible + overflow handling
Row(children: [
  const Icon(Icons.person),
  Expanded(child: Text(name, overflow: TextOverflow.ellipsis, maxLines: 1)),
]);
// ✅ Fix 2: Wrap instead of Row
// ✅ Fix 3: make it scrollable: SingleChildScrollView(scrollDirection: Axis.horizontal, child: Row(...))
```

Column overflow when keyboard opens (bottom overflowed by N pixels): The Scaffold resizes the body when the keyboard opens (`resizeToAvoidBottomInset: true` default), so a fixed-height Column of fields overflows. Fix: wrap in `SingleChildScrollView` (or use `ListView`). Don't "fix" by setting

`resizeToAvoidBottomInset: false` unless you handle insets yourself — the keyboard would then cover fields.

7.7.2 "Vertical viewport was given unbounded height."

```
// ❌ ListView inside Column — Column gives unbounded height to children; ListView wants to fill infinite height.
Column(children: [Text('Header'), ListView(children: items)]);
// ✅ Fix 1: Expanded
Column(children: [const Text('Header'), Expanded(child: ListView(children: items))]);
// ✅ Fix 2: fixed height
SizedBox(height: 200, child: ListView(...));
// ✅ Fix 3: shrinkWrap (⚠️ lays out ALL children → bad for long lists)
ListView(shrinkWrap: true, physics: const NeverScrollableScrollPhysics(), children: items);
// ✅ Best for complex scrolling pages: CustomScrollView with slivers (Chapter 9)
```

Same error horizontally: "Horizontal viewport was given unbounded width" (horizontal ListView inside Row) → wrap in `Expanded` or give a fixed width; a horizontal ListView inside a Column needs a **fixed height** (`SizedBox(height: 120, child: ListView(scrollDirection: Axis.horizontal, ...))`).

7.7.3 "RenderBox was not laid out" / "BoxConstraints forces an infinite width"

Usually a follow-up error caused by the unbounded constraint problems above. **Scroll to the FIRST error in the console** — later errors are consequences.

```
// ❌ TextField inside Row → "An InputDecorator...cannot have an unbounded width"
Row(children: [TextField()]);
// ✅
Row(children: [Expanded(child: TextField())]);
```

```
// ❌ Expanded inside a SingleChildScrollView > Column → unbounded height can't be divided
SingleChildScrollView(child: Column(children: [Expanded(child: ...)])); // error: "RenderFlex children have non-zero flex but incoming height constraints are unbounded."
```

Fix: remove `Expanded`, or use `CustomScrollView` + `SliverFillRemaining(hasScrollBody: false)`, or `LayoutBuilder` + `ConstrainedBox(minHeight: constraints.maxHeight)` + `IntrinsicHeight` pattern:

```
LayoutBuilder(builder: (context, constraints) => SingleChildScrollView(
  child: ConstrainedBox(
    constraints: BoxConstraints(minHeight: constraints.maxHeight),
    child: IntrinsicHeight(
      child: Column(children: [
        const Text('Top'),
        const Spacer(),            // pushes button to bottom when content is short
        ElevatedButton(onPressed: () {}, child: const Text('Submit')),
      ]),
    ),
```

```
  ),
));
```

7.7.4 "Incorrect use of ParentDataWidget."

`Expanded` not directly under Row/Column/Flex, or `Positioned` not directly under Stack. Fix structure.

7.7.5 Container width ignored

Parent gives tight constraints (e.g., the item of a `ListView` is forced to full width cross-axis; `CrossAxisAlignment.stretch`). Fix: wrap with `Align` / `Center` to loosen constraints.

7.7.6 Text overflow without error but cut off

Use `maxLines`, `overflow: TextOverflow.ellipsis`, `softWrap`, `FittedBox`, or `AutoSizeText` package.

7.7.7 Debugging layout tools

- **Flutter Inspector** → "Layout Explorer" shows flex properties and constraints visually.
- `debugPaintSizeEnabled = true;` (or press `p`) shows boxes, padding, alignment.
- Hover/select a widget in DevTools to see its constraints & size.
- Wrap with `ColoredBox(color: Colors.red.withValues(alpha: 0.3))` temporarily.

7.8 Building a real responsive screen (worked example)

```
class ProductPage extends StatelessWidget {
  const ProductPage({super.key, required this.product});
  final Product product;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text(product.name)),
      body: SafeArea(
        child: LayoutBuilder(
          builder: (context, constraints) {
            final wide = constraints.maxWidth >= 840;
            final image = AspectRatio(
              aspectRatio: 1,
              child: ClipRRect(
                borderRadius: BorderRadius.circular(16),
                child: Image.network(product.imageUrl, fit: BoxFit.cover),
              ),
            );
            final details = _Details(product: product);
```

```
            if (wide) {
              return Center(
                child: ConstrainedBox(
                  constraints: const BoxConstraints(maxWidth: 1200),
                  child: Padding(
                    padding: const EdgeInsets.all(24),
                    child: Row(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: [
                        Expanded(child: image),
                        const SizedBox(width: 32),
                        Expanded(child: SingleChildScrollView(child: details)),
                      ],
                    ),
                  ),
                ),
              );
            }
            return ListView(
              padding: const EdgeInsets.all(16),
              children: [image, const SizedBox(height: 16), details],
            );
          },
        ),
      ),
    );
  }
}

class _Details extends StatelessWidget {
  const _Details({required this.product});
  final Product product;
  @override
  Widget build(BuildContext context) {
    final t = Theme.of(context).textTheme;
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(product.name, style: t.headlineSmall),
        const SizedBox(height: 8),
        Row(children: [
          Text('₹${product.price}', style: t.titleLarge),
          const Spacer(),
          const Icon(Icons.star, size: 18),
          Text(product.rating.toStringAsFixed(1)),
        ]),
        const SizedBox(height: 16),
        Text(product.description),
        const SizedBox(height: 24),
```

```
        SizedBox(
          width: double.infinity,
          child: FilledButton(onPressed: () {}, child: const Text('Add to cart')),
        ),
      ],
    );
  }
}
```

7.9 Interview Questions — Layout

Q1. Explain Flutter's layout algorithm. Constraints go down, sizes go up, parent sets position — single pass (mostly linear) layout where parents pass BoxConstraints to children, children choose sizes within them, and parents position children.

Q2. Tight vs loose constraints? Tight: min == max (exact size). Loose: min 0, child may choose anything up to max.

Q3. Why does my Container ignore its width? Its parent passes tight constraints; Container must obey. Wrap in `Center` / `Align` to loosen.

Q4. Expanded vs Flexible? Expanded forces child to fill allocated flex space (FlexFit.tight); Flexible allows child to be smaller (FlexFit.loose).

Q5. How to fix "Vertical viewport was given unbounded height"? Give the scrollable bounded height — Expanded, SizedBox, or shrinkWrap (costly), or use slivers.

Q6. How do you make a layout responsive? LayoutBuilder/MediaQuery breakpoints (window size classes), flexible widgets (Expanded, Wrap, Flexible, FractionallySizedBox, AspectRatio), constraining max widths, adaptive navigation (bar/rail/drawer), testing on multiple sizes.

Q7. MediaQuery vs LayoutBuilder? MediaQuery provides app window info (size, padding, insets, text scale); LayoutBuilder provides constraints from the parent, better for reusable components.

Q8. Why use `MediaQuery.sizeOf` instead of `MediaQuery.of(context).size`? `sizeOf` subscribes only to size changes (InheritedModel aspects), avoiding rebuilds when unrelated properties (like keyboard insets) change.

Q9. What is IntrinsicHeight and why is it expensive? It sizes its child to the child's intrinsic height by running a speculative layout pass — can be $O(N^2)$ in nested cases; avoid in lists.

Q10. Stack sizing rules? Sized by non-positioned children; positioned children placed relative to Stack edges; overflow clipped by default; out-of-bounds children don't receive hits.

Q11. Difference between `Transform` and `RotatedBox`? Transform applies at paint time (layout unaffected, hit testing transformed); RotatedBox rotates in quarter turns during layout so surrounding layout adapts.

Q12. What is SafeArea? Widget inserting padding to avoid OS intrusions (notches, status bar, home indicator).

Q13. Logical vs physical pixels? Logical = device-independent units; physical = logical × devicePixelRatio.

Q14. How do you push a button to the bottom of a scrollable form when content is short? LayoutBuilder + SingleChildScrollView + ConstrainedBox(minHeight) + IntrinsicHeight + Spacer, or CustomScrollView with SliverFillRemaining(hasScrollBody: false).

Q15. What's the difference between `Opacity(0)`, `Visibility` and `Offstage`? See table §7.5 — Opacity keeps space and hit testing; Visibility removes from layout (optionally maintaining state); Offstage lays out but doesn't paint or hit-test.

Chapter 8 — The Widget Catalog: Text, Images, Buttons, Inputs, Forms, Scaffold, Dialogs & More

Goal: Know the everyday widgets thoroughly — their important properties, gotchas, and the modern (Material 3) replacements for deprecated ones.

8.1 App shells: MaterialApp, CupertinoApp, WidgetsApp

```
MaterialApp(
  title: 'Shop',                          // used by OS task switcher (Android) / web tab
  debugShowCheckedModeBanner: false,      // hide DEBUG ribbon
  theme: lightTheme,
  darkTheme: darkTheme,
  themeMode: ThemeMode.system,            // .light/.dark/.system
  home: const HomePage(),                 // OR routes / onGenerateRoute / router config
  routes: {'/settings': (_) => const SettingsPage()},
  navigatorKey: rootNavigatorKey,         // navigate without context
  scaffoldMessengerKey: messengerKey,     // show snackbars without context
  localizationsDelegates: AppLocalizations.localizationsDelegates,
  supportedLocales: AppLocalizations.supportedLocales,
  locale: const Locale('en'),
  builder: (context, child) => MediaQuery(           // wrap every route
    data: MediaQuery.of(context).copyWith(textScaler: TextScaler.noScaling), // (⚠️ accessibility
— don't disable scaling casually)
    child: child!,
  ),
);
// With go_router or Navigator 2.0:
MaterialApp.router(routerConfig: router);
```

`MaterialApp` provides: `Navigator`, `Theme`, `MediaQuery`, `Localizations`, `Directionality`, `ScaffoldMessenger`, `DefaultTextStyle`, `Overlay`, `Shortcuts`/`Actions`, `HeroController`, etc. Many "No X ancestor found" errors mean you used something **above** MaterialApp.

8.2 Scaffold — the page skeleton 🦴

```
Scaffold(
  appBar: AppBar(
    title: const Text('Inbox'),
    centerTitle: true,
```

```
    leading: const BackButton(),            // usually automatic
    actions: [IconButton(icon: const Icon(Icons.search), onPressed: () {})],
    bottom: const TabBar(tabs: [...]),       // with DefaultTabController
    elevation: 0, scrolledUnderElevation: 3, // M3: tint when content scrolls under
  ),
  body: const InboxList(),
  floatingActionButton: FloatingActionButton.extended(
    onPressed: () {}, icon: const Icon(Icons.edit), label: const Text('Compose'),
  ),
  floatingActionButtonLocation: FloatingActionButtonLocation.endFloat,
  drawer: const AppDrawer(),          // left (start) drawer
  endDrawer: const FilterDrawer(),
  bottomNavigationBar: NavigationBar(  // M3 (BottomNavigationBar is M2 style)
    selectedIndex: index,
    onDestinationSelected: (i) => setState(() => index = i),
    destinations: const [
      NavigationDestination(icon: Icon(Icons.home_outlined), selectedIcon: Icon(Icons.home),
label: 'Home'),
      NavigationDestination(icon: Icon(Icons.person_outline), label: 'Profile'),
    ],
  ),
  bottomSheet: null,
  backgroundColor: null,              // from theme
  resizeToAvoidBottomInset: true,     // shrink body when keyboard shows
  extendBody: false,                  // body behind bottom nav (for translucent bars)
  extendBodyBehindAppBar: false,
);
```

Nested Scaffolds

Each route usually has its own Scaffold. Avoid Scaffold inside Scaffold body unless needed (e.g., tabs with own FAB); inner AppBars then don't handle status bar padding automatically.

8.3 Text & typography

```
Text(
  'Hello world',
  style: Theme.of(context).textTheme.bodyLarge?.copyWith(fontWeight: FontWeight.w600),
  textAlign: TextAlign.center,
  maxLines: 2,
  overflow: TextOverflow.ellipsis,   // .clip/.fade/.ellipsis/.visible
  softWrap: true,
  textScaler: TextScaler.linear(1.0), // usually leave default to respect user setting
  semanticsLabel: 'Greeting',
  selectionColor: null,
)
```

```
Text.rich(TextSpan(
  text: 'By continuing you agree to our ',
  children: [
    TextSpan(
      text: 'Terms',
      style: const TextStyle(decoration: TextDecoration.underline),
      recognizer: _tapRecognizer, // TapGestureRecognizer()..onTap = openTerms — create in
initState & dispose!
    ),
    const TextSpan(text: '.'),
  ],
));

const SelectableText('Copy me');
SelectionArea(child: Column(children: [...]));   // make many Texts selectable (web/desktop)
DefaultTextStyle(style: ..., child: ...);         // default style for subtree
```

- `TextStyle` props: `fontSize`, `fontWeight`, `fontStyle`, `color`, `letterSpacing`, `wordSpacing`, `height` (line height multiplier), `decoration`, `fontFamily`, `fontFeatures` (tabular numbers: `FontFeature.tabularFigures()`), `shadows`, `overflow`, `leadingDistribution`.
- **Text scaling:** `textScaleFactor` is deprecated (Flutter 3.16) in favour of `TextScaler` (supports nonlinear scaling on Android 14+).
- `google_fonts` package downloads fonts at runtime (cache) or bundle them — for production, consider bundling (offline, GDPR/privacy reasons): `GoogleFonts.config.allowRuntimeFetching = false;` with font files in assets.

8.4 Images 🖼️

```
Image.asset('assets/images/logo.png', width: 120, fit: BoxFit.contain);
Image.network(
  url,
  fit: BoxFit.cover,
  loadingBuilder: (context, child, progress) =>
      progress == null ? child : const Center(child: CircularProgressIndicator()),
  errorBuilder: (context, error, stack) => const Icon(Icons.broken_image),
  cacheWidth: 300,                // ⭐ decode at smaller size → saves LOTS of memory
);
Image.file(File(path));
Image.memory(bytes);
const FadeInImage.assetNetwork(placeholder: 'assets/ph.png', image: url);
```

- `BoxFit`: `fill` (stretch), `contain` (fit inside, letterbox), `cover` (fill & crop), `fitWidth`, `fitHeight`, `none`, `scaleDown`.
- **`cached_network_image`** package — disk caching of network images (standard in production apps).

- SVG: `flutter_svg` package (`SvgPicture.asset('assets/icon.svg')`). For performance, `vector_graphics` precompiles SVGs.
- `Image.network` keeps decoded images in `ImageCache` (memory, default 1000 images / 100 MB). Huge images at full resolution cause **OOM crashes** — use `cacheWidth/cacheHeight` or `ResizeImage`.
- `precacheImage(AssetImage('assets/hero.png'), context)` in `didChangeDependencies` to avoid flicker.
- `Ink.image` / `Ink` for images with ink splashes.
- `CircleAvatar(backgroundImage: NetworkImage(url))`.

8.5 Icons

```
const Icon(Icons.favorite, color: Colors.red, size: 24, semanticLabel: 'Favorite');
const Icon(CupertinoIcons.heart);       // import cupertino_icons
Icon(Icons.adaptive.share);             // platform-adaptive icon
```

Custom icon fonts via FlutterIcon.com / `IconData(0xe800, fontFamily: 'MyIcons')`. Material Symbols via `material_symbols_icons` package. Tree shaking of icon fonts happens in release (`--no-tree-shake-icons` disables; required if you construct `IconData` dynamically from non-const values).

8.6 Buttons (Material 3)

Widget	Look	Use
`FilledButton`	solid fill	primary action (M3)
`FilledButton.tonal`	soft fill	secondary emphasis
`ElevatedButton`	raised with shadow/tint	when separation from patterned background needed
`OutlinedButton`	border	medium emphasis
`TextButton`	text only	low emphasis, dialogs
`IconButton` (+ `.filled`, `.filledTonal`, `.outlined`)	icon	toolbar actions
`FloatingActionButton` (`.small`, `.large`, `.extended`)	FAB	main screen action
`SegmentedButton`	segments	choose 1 or many options
`DropdownMenu` / `DropdownButton`	menu	choose one
`MenuAnchor`, `PopupMenuButton`	menus	overflow/context menus
`CupertinoButton`	iOS style	iOS UI

Old `RaisedButton`, `FlatButton`, `OutlineButton` were **removed** (Flutter 2.x deprecated → removed in 3.x). Interviewers may ask: "What replaced RaisedButton?" → `ElevatedButton`.

```dart
FilledButton.icon(
  onPressed: isLoading ? null : submit,     // null onPressed = DISABLED
  icon: const Icon(Icons.send),
  label: const Text('Send'),
  style: FilledButton.styleFrom(
    minimumSize: const Size.fromHeight(48),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
  ),
);

// Per-state styling with WidgetStateProperty (was MaterialStateProperty before Flutter 3.19)
ButtonStyle(
  backgroundColor: WidgetStateProperty.resolveWith((states) {
    if (states.contains(WidgetState.disabled)) return Colors.grey;
    if (states.contains(WidgetState.pressed)) return Colors.blue.shade700;
    return Colors.blue;
  }),
);
```

Disabling a button = pass `onPressed: null`.

Preventing double taps (very common real bug: duplicate orders/payments): disable while loading, or guard with a flag:

```dart
bool _submitting = false;
Future<void> _submit() async {
  if (_submitting) return;
  setState(() => _submitting = true);
  try { await api.placeOrder(); } finally { if (mounted) setState(() => _submitting = false); }
}
```

(And the backend should be idempotent — use idempotency keys.)

InkWell vs GestureDetector

- `InkWell` — Material ripple effect; needs a `Material` ancestor (else ripple is invisible/hidden behind a colored Container — wrap with `Material(color: ...)` or use `Ink` for decoration).
- `GestureDetector` — no visual feedback, many gestures (Chapter 13).

8.7 Text input — TextField & TextFormField ⌨️

```dart
class _LoginState extends State<Login> {
  final _emailCtrl = TextEditingController();
  final _focus = FocusNode();

  @override
```

```
  void dispose() {
    _emailCtrl.dispose();   // ❗ always dispose controllers & focus nodes
    _focus.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return TextField(
      controller: _emailCtrl,
      focusNode: _focus,
      keyboardType: TextInputType.emailAddress,
      textInputAction: TextInputAction.next,       // keyboard action button
      autofillHints: const [AutofillHints.email],   // password managers
      autocorrect: false,
      textCapitalization: TextCapitalization.none,
      obscureText: false,                           // true for passwords
      maxLength: 50,
      maxLines: 1,                                  // null = grows; minLines for multiline
      inputFormatters: [FilteringTextInputFormatter.deny(RegExp(r'\s'))],
      decoration: InputDecoration(
        labelText: 'Email',
        hintText: 'you@example.com',
        prefixIcon: const Icon(Icons.email),
        suffixIcon: IconButton(icon: const Icon(Icons.clear), onPressed: _emailCtrl.clear),
        border: const OutlineInputBorder(),
        errorText: _error,     // shows error state
        helperText: 'We never share it',
        counterText: '',       // hide maxLength counter
      ),
      onChanged: (value) {},
      onSubmitted: (_) => FocusScope.of(context).nextFocus(),
      onTapOutside: (_) => FocusScope.of(context).unfocus(),  // dismiss keyboard on outside tap
    );
  }
}
```

Common input formatters: `FilteringTextInputFormatter.digitsOnly`, `.allow(RegExp(...))`, `LengthLimitingTextInputFormatter(10)`, custom `TextInputFormatter` subclass for masks (phone, card) — or packages like `mask_text_input_formatter`.

Reading/writing controller:

```
_emailCtrl.text;                          // read
_emailCtrl.text = 'a@b.com';              // write (moves cursor to... careful)
_emailCtrl.value = TextEditingValue(text: t, selection: TextSelection.collapsed(offset:
t.length));
_emailCtrl.addListener(() => print(_emailCtrl.text));
```

Hide keyboard: `FocusScope.of(context).unfocus()` or `FocusManager.instance.primaryFocus?.unfocus()`. Programmatic focus: `_focus.requestFocus()`; autofocus: `autofocus: true`.

Common TextField bugs

Bug	Fix
Controller created in `build` → cursor jumps, text resets	Create in `initState` (or as field), dispose in `dispose`
TextField in Row → unbounded width error	Wrap in `Expanded`
Keyboard covers field	`Scaffold(resizeToAvoidBottomInset: true)` + scrollable body; `Scrollable.ensureVisible`; `scrollPadding`
Bottom overflow when keyboard opens	`SingleChildScrollView`
`setState` on every keystroke rebuilding whole form	Keep form in its own widget; use `ValueListenableBuilder` on controller
Losing text when list scrolls (TextField in ListView items)	Keep values in a model/state, not only in item widgets; or keys + AutomaticKeepAlive
Arabic/RTL cursor issues	Use `TextDirection` / `Directionality` & locale correctly

8.8 Forms & validation ✅

```dart
class SignUpForm extends StatefulWidget {
  const SignUpForm({super.key});
  @override
  State<SignUpForm> createState() => _SignUpFormState();
}

class _SignUpFormState extends State<SignUpForm> {
  final _formKey = GlobalKey<FormState>();
  final _password = TextEditingController();
  String? _email;
  bool _agree = false;

  @override
  void dispose() { _password.dispose(); super.dispose(); }

  void _submit() {
    final form = _formKey.currentState!;
    if (!form.validate()) return;   // runs all validators, shows errors
    form.save();                    // calls onSaved for each field
    // call API with _email, _password.text
  }

  @override
  Widget build(BuildContext context) {
```

```
    return Form(
      key: _formKey,
      autovalidateMode: AutovalidateMode.onUserInteraction,  // validate as user types (after
first interaction)
      child: Column(children: [
        TextFormField(
          decoration: const InputDecoration(labelText: 'Email'),
          keyboardType: TextInputType.emailAddress,
          validator: (v) {
            if (v == null || v.trim().isEmpty) return 'Email is required';
            if (!RegExp(r'^[^@\s]+@[^@\s]+\.[^@\s]+$').hasMatch(v)) return 'Enter a valid email';
            return null;   // null = valid
          },
          onSaved: (v) => _email = v!.trim(),
        ),
        TextFormField(
          controller: _password,
          obscureText: true,
          decoration: const InputDecoration(labelText: 'Password'),
          validator: (v) => (v ?? '').length < 8 ? 'At least 8 characters' : null,
        ),
        TextFormField(
          obscureText: true,
          decoration: const InputDecoration(labelText: 'Confirm password'),
          validator: (v) => v != _password.text ? 'Passwords do not match' : null,
        ),
        FormField<bool>(                      // custom form field (checkbox)
          initialValue: _agree,
          validator: (v) => v == true ? null : 'You must accept the terms',
          builder: (state) => Column(children: [
            CheckboxListTile(
              value: state.value,
              onChanged: (v) { state.didChange(v); _agree = v!; },
              title: const Text('I accept the terms'),
            ),
            if (state.hasError) Text(state.errorText!, style: TextStyle(color:
Theme.of(context).colorScheme.error)),
          ]),
        ),
        FilledButton(onPressed: _submit, child: const Text('Sign up')),
      ]),
    );
  }
}
```

- `AutovalidateMode`: `disabled`, `always`, `onUserInteraction`, `onUnfocus` (added in recent Flutter 3.x releases).

- Server-side errors: keep a `String? _serverError` and pass via `InputDecoration(errorText:)`, or use `forceErrorText` on `FormField` / `TextFormField` (available in recent Flutter 3.x releases).
- `form.reset()` resets fields to initial values.
- Packages for complex forms: `flutter_form_builder`, `reactive_forms`, `formz` (validation models, used with Bloc).
- Wrap forms in `AutofillGroup` for password managers; call `TextInput.finishAutofillContext()` after login.

8.9 Selection controls

```
Checkbox(value: checked, onChanged: (v) => setState(() => checked = v!), tristate: false);
CheckboxListTile(value: x, onChanged: ..., title: const Text('Wi-Fi'));
Switch(value: on, onChanged: (v) => setState(() => on = v));
Switch.adaptive(...);                          // Cupertino on iOS
SwitchListTile(...);
Radio<Gender>(value: Gender.male, groupValue: g, onChanged: (v) => setState(() => g = v));
// Note: newer Flutter versions introduce RadioGroup to manage groupValue/onChanged — check your
version.
Slider(value: v, min: 0, max: 100, divisions: 10, label: '${v.round()}', onChanged: ...);
RangeSlider(values: RangeValues(20, 80), onChanged: ...);
ChoiceChip(label: Text('S'), selected: size == 'S', onSelected: ...);
FilterChip(...); InputChip(...); ActionChip(...);
DropdownMenu<String>(
  dropdownMenuEntries: const [DropdownMenuEntry(value: 'in', label: 'India')],
  onSelected: (v) {},
);
DropdownButtonFormField<String>(items: [...], onChanged: ..., validator: ...);
```

Date & time pickers:

```
final date = await showDatePicker(
  context: context,
  initialDate: DateTime.now(),
  firstDate: DateTime(2000),
  lastDate: DateTime(2100),
);
final time = await showTimePicker(context: context, initialTime: TimeOfDay.now());
final range = await showDateRangePicker(context: context, firstDate: ..., lastDate: ...);
// iOS style: CupertinoDatePicker inside showCupertinoModalPopup
```

8.10 Dialogs, bottom sheets, snackbars, overlays 💬

Dialogs

```
final confirmed = await showDialog<bool>(
  context: context,
  barrierDismissible: false,          // tap outside doesn't close
  builder: (dialogContext) => AlertDialog(
    title: const Text('Delete item?'),
    content: const Text('This cannot be undone.'),
    actions: [
      TextButton(onPressed: () => Navigator.pop(dialogContext, false), child: const
Text('Cancel')),
      FilledButton(onPressed: () => Navigator.pop(dialogContext, true), child: const
Text('Delete')),
    ],
  ),
);
if (confirmed == true) { ... }  // could be null if dismissed
```

- Dialogs are **routes** pushed on the Navigator → `Navigator.pop(context, result)` returns a value.
- Use the **dialog's own context** (`dialogContext`) to pop, especially with nested navigators.
- `showAdaptiveDialog` + `AlertDialog.adaptive` for iOS look.
- `SimpleDialog`, `Dialog` (custom content), `Dialog.fullscreen`.
- `showGeneralDialog` for custom transitions.
- Use `useRootNavigator: true` (default for showDialog) to show above nested navigators (bottom nav tabs).

Bottom sheets

```
final choice = await showModalBottomSheet<String>(
  context: context,
  isScrollControlled: true,       // allow > half screen & keyboard handling
  showDragHandle: true,
  useSafeArea: true,
  builder: (ctx) => Padding(
    padding: EdgeInsets.only(bottom: MediaQuery.viewInsetsOf(ctx).bottom), // above keyboard
    child: const ShareOptions(),
  ),
);
// Draggable heights:
DraggableScrollableSheet(initialChildSize: 0.4, minChildSize: 0.2, maxChildSize: 0.9,
  builder: (ctx, scrollController) => ListView(controller: scrollController, children: ...));
// Persistent: Scaffold.of(context).showBottomSheet(...)
```

SnackBars

```
ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: const Text('Item deleted'),
    action: SnackBarAction(label: 'UNDO', onPressed: undo),
    behavior: SnackBarBehavior.floating,
    duration: const Duration(seconds: 4),
  ),
);
ScaffoldMessenger.of(context).hideCurrentSnackBar();
```

`Scaffold.of(context).showSnackBar` was **removed** — use `ScaffoldMessenger` (snackbars survive route changes). For showing snackbars from non-UI code (e.g., a service), use a `GlobalKey<ScaffoldMessengerState>` on MaterialApp.

Also: `MaterialBanner` (`showMaterialBanner`), toasts via packages (`fluttertoast`, `toastification`).

Tooltips, popups & overlays

```
const Tooltip(message: 'Add item', child: Icon(Icons.add));
PopupMenuButton<String>(itemBuilder: (_) => [const PopupMenuItem(value: 'edit', child:
Text('Edit'))], onSelected: ...);
MenuAnchor(menuChildren: [...], builder: (context, controller, child) => IconButton(onPressed:
controller.open, ...));
```

`Overlay` / `OverlayEntry` / **`OverlayPortal`** (Flutter 3.10+, declarative overlay tied to widget tree) for custom popups like autocomplete dropdowns; `CompositedTransformTarget` + `CompositedTransformFollower` to anchor overlays to widgets. `Autocomplete<T>` widget built-in.

8.11 Lists & tiles (intro — scrolling deep dive in Chapter 9)

```
ListTile(
  leading: const CircleAvatar(child: Text('A')),
  title: const Text('Asha'),
  subtitle: const Text('Last seen 2m ago'),
  trailing: const Icon(Icons.chevron_right),
  onTap: () {},
  dense: false,
  contentPadding: const EdgeInsets.symmetric(horizontal: 16),
);
const Divider(height: 1);
Card(child: ListTile(...));
ExpansionTile(title: const Text('FAQ'), children: [const Text('Answer')]);
Dismissible(
  key: ValueKey(item.id),             // REQUIRED unique key
```

```
  direction: DismissDirection.endToStart,
  background: Container(color: Colors.red),
  confirmDismiss: (dir) async => await askConfirm(),
  onDismissed: (_) => setState(() => items.remove(item)), // MUST remove from data or error:
  child: ListTile(title: Text(item.title)),               // "A dismissed Dismissible widget is
still part of the tree."
);
ReorderableListView(onReorder: (oldI, newI) {...}, children: [for (final i in items) ListTile(key:
ValueKey(i.id), ...)]);
```

8.12 Progress, feedback & misc

```
const CircularProgressIndicator();
const LinearProgressIndicator(value: 0.4);     // determinate
CircularProgressIndicator.adaptive();
RefreshIndicator(onRefresh: () async => await reload(), child: ListView(...)); // pull to refresh
(child must be scrollable & AlwaysScrollableScrollPhysics if few items)
const Badge(label: Text('3'), child: Icon(Icons.notifications));
Chip(label: const Text('Flutter'), onDeleted: () {});
Stepper(steps: [...]);
TabBar + TabBarView + DefaultTabController(length: 3, child: ...);
NavigationRail(...); NavigationDrawer(...);
SearchAnchor / SearchBar (M3 search);
Placeholder();   // draws a box with an X — scaffolding
```

`HapticFeedback.lightImpact()`, `SystemSound.play(SystemSoundType.click)`, `Clipboard.setData(ClipboardData(text: 'x'))`, `SystemChrome.setPreferredOrientations([...])`, `SystemChrome.setSystemUIOverlayStyle(...)` / `AnnotatedRegion<SystemUiOverlayStyle>` for status bar icon colours.

8.13 Cupertino (iOS-style) widgets

`CupertinoApp`, `CupertinoPageScaffold`, `CupertinoNavigationBar`, `CupertinoSliverNavigationBar` (large titles), `CupertinoTabScaffold` / `CupertinoTabBar`, `CupertinoButton`, `CupertinoSwitch`, `CupertinoSlider`, `CupertinoTextField`, `CupertinoAlertDialog`, `CupertinoActionSheet`, `CupertinoDatePicker`, `CupertinoPicker`, `CupertinoActivityIndicator`, `CupertinoContextMenu`, `CupertinoListSection` / `CupertinoListTile`, `CupertinoPageRoute` (swipe-back), `CupertinoSegmentedControl` / `CupertinoSlidingSegmentedControl`, `CupertinoSearchTextField`.

Adaptive approach: Material widgets with `.adaptive` constructors (`Switch.adaptive`, `Slider.adaptive`, `CircularProgressIndicator.adaptive`, `AlertDialog.adaptive`, `Checkbox.adaptive`), or branch on `Theme.of(context).platform` / `defaultTargetPlatform`.

8.14 Interview Questions — Common widgets

Q1. What replaced `RaisedButton` / `FlatButton` / `OutlineButton`? `ElevatedButton` / `TextButton` / `OutlinedButton` (and M3 adds `FilledButton`).

Q2. How do you disable a button? Set `onPressed: null`.

Q3. How do you validate a form? Wrap fields in `Form` with a `GlobalKey<FormState>`, give `TextFormField`s `validator`s returning error strings or null, and call `formKey.currentState!.validate()`.

Q4. TextField vs TextFormField? `TextFormField` wraps `TextField` in a `FormField`, integrating with `Form` (validator, onSaved, reset, autovalidate).

Q5. Why must controllers be disposed? They hold listeners/resources (and for TextEditingController, platform text input connections); not disposing leaks memory.

Q6. How do you return data from a dialog? `Navigator.pop(context, value)` and `await showDialog<T>(...)`.

Q7. ScaffoldMessenger vs Scaffold.of for snackbars? ScaffoldMessenger (root-level, persists across routes) replaced Scaffold.showSnackBar.

Q8. InkWell ripple not visible — why? InkWell needs a `Material` ancestor and draws on it; an opaque `Container` color between hides the ink. Use `Material` / `Ink` for colour.

Q9. How do you prevent images from consuming too much memory? `cacheWidth` / `cacheHeight` / `ResizeImage`, appropriately sized server images, caching packages, limit ImageCache size.

Q10. What is `Dismissible` and its common error? Swipe-to-dismiss widget requiring a unique key; after `onDismissed` you must remove the item from the data list, else "A dismissed Dismissible widget is still part of the tree".

Q11. How do you show a bottom sheet that resizes with the keyboard? `showModalBottomSheet(isScrollControlled: true)` + bottom padding = `MediaQuery.viewInsetsOf(context).bottom`.

Q12. What's `WidgetStateProperty`? (Previously `MaterialStateProperty`) a way to resolve style values based on interaction states (pressed, hovered, focused, disabled, selected, error).

Q13. How do you make text selectable? `SelectableText` or wrap in `SelectionArea`.

Q14. How do you dismiss the keyboard? `FocusScope.of(context).unfocus()` / `FocusManager.instance.primaryFocus?.unfocus()`, `onTapOutside`, or `keyboardDismissBehavior: ScrollViewKeyboardDismissBehavior.onDrag` on scroll views.

Chapter 9 — Scrolling, Lists, Grids & Slivers

Goal: Build fast, smooth lists of any size; understand viewports and slivers; implement pagination, pull-to-refresh, sticky headers, collapsing app bars, and fix scrolling bugs.

9.1 The 5-year-old picture: a window over a long paper roll 📜

Imagine a **very long paper roll** with drawings, and a **small window** you look through. You only need to *draw* the parts visible through the window (plus a little extra above and below so scrolling is smooth). That window is the **viewport**; the pieces of paper are **slivers** ("thin slices"); drawing only what's visible is **lazy building**.

9.2 SingleChildScrollView vs ListView

	`SingleChildScrollView`	`ListView` (default ctor)	`ListView.builder` / `.separated`
Builds children	**All at once**	**All at once (children list)**	**Lazily, only visible + cache extent**
Good for	**A form/one page bigger than screen**	**Small fixed lists**	**Long or infinite lists**
Memory	**All children**	**All children**	**Only visible ones**

```
ListView.builder(
  itemCount: items.length,          // null = infinite
  itemExtent: 72,                   // ⭐ fixed height → faster layout & jumping
  // prototypeItem: const ListTile(title: Text('x')), // alternative: measure one prototype
  padding: const EdgeInsets.symmetric(vertical: 8),
  itemBuilder: (context, index) {
    final item = items[index];
    return ListTile(key: ValueKey(item.id), title: Text(item.title));
  },
);

ListView.separated(
  itemCount: items.length,
  separatorBuilder: (_, __) => const Divider(height: 1),
  itemBuilder: (context, i) => ItemTile(item: items[i]),
);
```

Never do `ListView(children: hugeList.map(...).toList())` for hundreds/thousands of items — it builds everything.

Important properties of scroll views

- `scrollDirection: Axis.horizontal`
- `reverse: true` (chat apps — newest at bottom, list starts at bottom)
- `controller: ScrollController()`
- `physics`: `BouncingScrollPhysics` (iOS), `ClampingScrollPhysics` (Android), `NeverScrollableScrollPhysics` (disable), `AlwaysScrollableScrollPhysics` (allow even if content short — needed for RefreshIndicator with few items), `PageScrollPhysics`. Default depends on platform (`ScrollConfiguration`).
- `shrinkWrap: true` — size to content (⚠️ builds all items, avoid for long lists).
- `primary` — use PrimaryScrollController (iOS tap-status-bar-to-top).
- `cacheExtent` — pixels pre-built beyond viewport (default 250).
- `keyboardDismissBehavior: ScrollViewKeyboardDismissBehavior.onDrag`.
- `addAutomaticKeepAlives`, `addRepaintBoundaries` (default true), `addSemanticIndexes`.
- `restorationId` for state restoration.

9.3 ScrollController

```
class _FeedState extends State<Feed> {
  final _controller = ScrollController();

  @override
  void initState() {
    super.initState();
    _controller.addListener(_onScroll);
  }

  void _onScroll() {
    final pos = _controller.position;
    if (pos.pixels >= pos.maxScrollExtent - 300) {   // near bottom
      context.read<FeedCubit>().loadMore();
    }
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  void scrollToTop() => _controller.animateTo(0,
      duration: const Duration(milliseconds: 400), curve: Curves.easeOut);
  ...
}
```

- `jumpTo(offset)`, `animateTo(offset, ...)`, `offset`, `position.maxScrollExtent`, `position.userScrollDirection`, `hasClients` (check before using!).
- Error: *"ScrollController attached to multiple scroll views"* — same controller given to two scrollables. *"ScrollController not attached to any scroll views"* — used before build or after dispose (`if (_controller.hasClients)`).
- `NotificationListener<ScrollNotification>` alternative to controllers: listen to `ScrollStartNotification`, `ScrollUpdateNotification`, `ScrollEndNotification`, `OverscrollNotification`, `UserScrollNotification`; return `true` to stop bubbling.
- Scroll to a specific widget: `Scrollable.ensureVisible(key.currentContext!)`; for index-based scrolling in long lazy lists use `scrollable_positioned_list` / `super_sliver_list` packages (ListView can't scroll to an index of unbuilt variable-height items).

9.4 Infinite scrolling / pagination (production pattern)

```
class PagedList extends StatefulWidget { ... }

class _PagedListState extends State<PagedList> {
  final _items = <Item>[];
  int _page = 1;
  bool _loading = false;
  bool _hasMore = true;
  Object? _error;

  @override
  void initState() { super.initState(); _loadMore(); }

  Future<void> _loadMore() async {
    if (_loading || !_hasMore) return;            // ❗ guard against duplicate calls
    setState(() { _loading = true; _error = null; });
    try {
      final page = await api.fetchItems(page: _page, limit: 20);
      if (!mounted) return;
      setState(() {
        _items.addAll(page);
        _page++;
        _hasMore = page.length == 20;
      });
    } catch (e) {
      if (mounted) setState(() => _error = e);
    } finally {
      if (mounted) setState(() => _loading = false);
    }
  }

  Future<void> _refresh() async {
    setState(() { _items.clear(); _page = 1; _hasMore = true; });
```

```
    await _loadMore();
  }

  @override
  Widget build(BuildContext context) {
    return RefreshIndicator(
      onRefresh: _refresh,
      child: ListView.builder(
        physics: const AlwaysScrollableScrollPhysics(),
        itemCount: _items.length + 1,   // +1 for footer
        itemBuilder: (context, i) {
          if (i < _items.length) {
            if (i == _items.length - 5) _loadMore();  // prefetch near end (side effect in build:
acceptable here because guarded, but many teams prefer a scroll listener)
            return ItemTile(key: ValueKey(_items[i].id), item: _items[i]);
          }
          if (_error != null) return TextButton(onPressed: _loadMore, child: const Text('Retry'));
          if (_hasMore) return const Padding(padding: EdgeInsets.all(16), child: Center(child:
CircularProgressIndicator()));
          return const SizedBox.shrink();
        },
      ),
    );
  }
}
```

⚠️ Calling `_loadMore()` (which calls `setState`) from `itemBuilder` *during build* can trigger "setState() called during build". Safer: schedule it: `WidgetsBinding.instance.addPostFrameCallback((_) => _loadMore());` or use a scroll listener. Packages: **`infinite_scroll_pagination`** (popular, handles all states).

Pagination styles: **offset/limit (page numbers)** (simple, but duplicates/skips when data changes) vs **cursor-based** (`nextCursor` from server; stable for feeds). Senior interview: prefer cursor-based for live feeds.

9.5 GridView

```
GridView.builder(
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2, mainAxisSpacing: 8, crossAxisSpacing: 8, childAspectRatio: 3 / 4,
  ),
  itemCount: products.length,
  itemBuilder: (_, i) => ProductCard(product: products[i]),
);

// Responsive: as many columns as fit with max tile width 200
GridView.builder(
```

```
  gridDelegate: const SliverGridDelegateWithMaxCrossAxisExtent(maxCrossAxisExtent: 200,
childAspectRatio: 1),
  ...
);
GridView.count(crossAxisCount: 3, children: [...]);
GridView.extent(maxCrossAxisExtent: 150, children: [...]);
```

Gotcha: grid tiles have **fixed aspect ratio**; content taller than the tile → overflow. Use `childAspectRatio` or `mainAxisExtent` (fixed height) on the delegate. Staggered/masonry grids: `flutter_staggered_grid_view` package.

9.6 PageView, TabBarView, carousels

```
final _pageController = PageController(viewportFraction: 0.9);  // peek next page
PageView.builder(
  controller: _pageController,
  itemCount: banners.length,
  onPageChanged: (i) => setState(() => _current = i),
  itemBuilder: (_, i) => BannerCard(banners[i]),
);
_pageController.animateToPage(2, duration: const Duration(milliseconds: 300), curve: Curves.ease);
```

- Material 3 `CarouselView` (Flutter 3.24+) for carousels.
- Tabs: `DefaultTabController(length: 3, child: Scaffold(appBar: AppBar(bottom: TabBar(tabs: ...)), body: TabBarView(children: ...)))`. Or a manual `TabController` with `TickerProviderStateMixin`.
- Tabs/pages lose state when scrolled away → keep alive (9.8).

9.7 Slivers & CustomScrollView 🥓

A **sliver** is a piece of a scrollable area that knows how to lay itself out lazily based on the visible portion (`SliverConstraints` in, `SliverGeometry` out — a different protocol from boxes!). `ListView` itself is just a `CustomScrollView` with one `SliverList` (plus padding).

Use **CustomScrollView** when you want **multiple different scrolling things to scroll together as one** (header + grid + list + footer), collapsing app bars, sticky headers — **instead of nesting ListViews with shrinkWrap**.

```
CustomScrollView(
  slivers: [
    SliverAppBar(
      pinned: true,          // stays visible (collapsed) at top
      floating: false,       // reappears immediately on scroll up if true
      snap: false,           // with floating: snaps fully open/closed
```

```
      expandedHeight: 220,
      stretch: true,
      flexibleSpace: FlexibleSpaceBar(
        title: const Text('Goa Trip'),
        background: Image.network(coverUrl, fit: BoxFit.cover),
      ),
    ),
    const SliverToBoxAdapter(child: Padding(padding: EdgeInsets.all(16), child:
Text('Highlights'))),
    SliverPadding(
      padding: const EdgeInsets.symmetric(horizontal: 16),
      sliver: SliverGrid.builder(
        gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(crossAxisCount: 3,
mainAxisSpacing: 8, crossAxisSpacing: 8),
        itemCount: photos.length,
        itemBuilder: (_, i) => PhotoTile(photos[i]),
      ),
    ),
    SliverList.separated(
      itemCount: reviews.length,
      separatorBuilder: (_, __) => const Divider(),
      itemBuilder: (_, i) => ReviewTile(reviews[i]),
    ),
    const SliverFillRemaining(hasScrollBody: false, child: Center(child: Text('The end'))),
  ],
);
```

Sliver catalogue

Sliver	Purpose
`SliverList` / `SliverList.builder` / `.separated` / `.list`	Lazy list
`SliverFixedExtentList`, `SliverPrototypeExtentList`, `SliverVariedExtentList`	Faster lists with known extents
`SliverGrid`	Lazy grid
`SliverAppBar`, `SliverAppBar.medium`/`.large` (M3)	Collapsing app bar
`SliverToBoxAdapter`	Put one normal (box) widget inside
`SliverPadding`	Padding around a sliver
`SliverFillRemaining`	Fill leftover viewport space
`SliverFillViewport`	Children fill viewport (pages)
`SliverPersistentHeader`	Custom sticky/collapsing headers (with delegate)
`SliverOpacity`, `SliverIgnorePointer`, `SliverVisibility`, `SliverOffstage`	Sliver versions of effects
`SliverAnimatedList`	Animated insert/remove
`SliverLayoutBuilder`	Build based on SliverConstraints

Sliver	Purpose
`SliverMainAxisGroup`, `SliverCrossAxisGroup` **(3.13+)**	**Group slivers; sticky header per section with `pinned` SliverPersistentHeader**
`SliverResizingHeader`, `PinnedHeaderSliver`, `SliverFloatingHeader` **(3.24+)**	**Easier headers without delegates**
`DecoratedSliver` **(3.16+)**	**Background decoration for slivers**
`SliverSafeArea`	**Safe area for slivers**

Error: putting a box widget directly in `slivers:` → *"A RenderViewport expected a child of type RenderSliver but received a child of type RenderPadding"* (or similar). Wrap box widgets in `SliverToBoxAdapter`; use `SliverPadding` not `Padding`.

SliverPersistentHeader (sticky header)

```
class _HeaderDelegate extends SliverPersistentHeaderDelegate {
  _HeaderDelegate(this.title);
  final String title;
  @override double get minExtent => 48;
  @override double get maxExtent => 48;
  @override
  Widget build(BuildContext context, double shrinkOffset, bool overlapsContent) =>
      Container(color: Theme.of(context).colorScheme.surface, alignment: Alignment.centerLeft,
          padding: const EdgeInsets.symmetric(horizontal: 16), child: Text(title));
  @override
  bool shouldRebuild(_HeaderDelegate old) => old.title != title;
}
SliverPersistentHeader(pinned: true, delegate: _HeaderDelegate('Section A'));
```

For grouped sticky headers ("contacts A, B, C"), use `SliverMainAxisGroup` with a pinned header per group, or the `sliver_tools` / `flutter_sticky_header` packages.

NestedScrollView

Outer header (e.g., collapsing app bar + TabBar) + inner scrollable per tab, coordinated:

```
NestedScrollView(
  headerSliverBuilder: (context, innerBoxIsScrolled) => [
    SliverOverlapAbsorber(
      handle: NestedScrollView.sliverOverlapAbsorberHandleFor(context),
      sliver: SliverAppBar(pinned: true, expandedHeight: 200, forceElevated: innerBoxIsScrolled,
        bottom: const TabBar(tabs: [Tab(text: 'Posts'), Tab(text: 'Media')])),
    ),
  ],
  body: TabBarView(children: [PostsTab(), MediaTab()]), // each uses SliverOverlapInjector at top
);
```

NestedScrollView has subtle bugs (overlap, scroll position sharing); test carefully.

9.8 Keeping state alive in lists, tabs and pages

Lazy lists **dispose** off-screen items (state lost: text typed, video position, checkbox local state). Options:

1. **Lift state up** into a model / state management (best practice).
2. `AutomaticKeepAliveClientMixin`:

```
class _TabState extends State<MyTab> with AutomaticKeepAliveClientMixin {
  @override
  bool get wantKeepAlive => true;
  @override
  Widget build(BuildContext context) {
    super.build(context);  // ❗ must call
    return ListView(...);
  }
}
```

3. `PageStorageKey` to remember **scroll position** across tab switches: `ListView(key: const PageStorageKey('feed-list'), ...)`.
4. `IndexedStack` for bottom nav tabs (keeps all alive — memory trade-off), or go_router's `StatefulShellRoute` (Chapter 11).

9.9 Animated lists & reordering

```
final _listKey = GlobalKey<AnimatedListState>();
AnimatedList(
  key: _listKey,
  initialItemCount: items.length,
  itemBuilder: (context, i, animation) => SizeTransition(sizeFactor: animation, child:
ItemTile(items[i])),
);
void insert(Item item) { items.insert(0, item); _listKey.currentState!.insertItem(0); }
void remove(int i) {
  final removed = items.removeAt(i);
  _listKey.currentState!.removeItem(i, (ctx, anim) => SizeTransition(sizeFactor: anim, child:
ItemTile(removed)));
}
```

`ReorderableListView.builder` (needs keys), `SliverReorderableList`, `ReorderableDragStartListener` for custom handles. `implicitly_animated_reorderable_list` / `animated_list_plus` packages diff lists automatically.

9.10 Scroll performance checklist ⚡

1. Use `.builder` constructors for long lists.
2. Provide `itemExtent` / `prototypeItem` when heights are fixed.
3. Keep `itemBuilder` cheap: no heavy computation, no synchronous JSON parsing, no creating controllers.
4. Use `const` widgets inside items.
5. Avoid `shrinkWrap: true` + nested ListViews; use slivers.
6. Images: proper sizes (`cacheWidth`), `cached_network_image`, placeholders with fixed sizes (avoid layout jumps).
7. Avoid `Opacity`, `ClipPath`, `BackdropFilter`, `saveLayer`-heavy effects per item; avoid `IntrinsicHeight` in items.
8. Avoid rebuilding the whole list when one item changes: per-item state (e.g., `select` in Provider/Riverpod, `BlocSelector`), keys.
9. Profile in **profile mode** with DevTools performance view (Chapter 23).
10. For **very** large or complex lists: `super_sliver_list` (better extent estimation, jump to index).

9.11 Common scrolling bugs

Bug	Fix
Unbounded height (ListView in Column)	`Expanded` **/ slivers**
Nested scroll views fighting (inner ListView scrolls instead of outer)	**Use single CustomScrollView; or inner** `physics: NeverScrollableScrollPhysics()` **+** `shrinkWrap` **for SHORT lists**
RefreshIndicator doesn't trigger with few items	`physics: AlwaysScrollableScrollPhysics()`
Scroll position resets after setState	**Controller recreated in build, or key changes; keep controller in State; use PageStorageKey**
`ScrollController not attached`	**Check** `hasClients`**; call after first frame**
Horizontal list inside vertical list has no height	**Wrap in** `SizedBox(height: ...)`
Jank when scrolling images	**Resize/cache images,** `RepaintBoundary` **(default), avoid expensive effects**
Lost text in TextFields inside list	**Lift state; keys; keep alive**
Load-more fires many times	**Guard with** `_loading` **flag; debounce**
Overscroll glow/stretch unwanted	`ScrollConfiguration(behavior: ScrollConfiguration.of(context).copyWith(overscroll: false), ...)`
Can't drag-scroll with mouse on web/desktop	**Since Flutter 2.5, mouse drag scrolling is disabled by default; add** `PointerDeviceKind.mouse` **to** `ScrollBehavior.dragDevices` **via a custom** `MaterialScrollBehavior`

9.12 Interview Questions — Scrolling

Q1. ListView vs ListView.builder? Default ListView builds all children upfront; builder lazily builds visible items — use for long/infinite lists.

Q2. What is a sliver? A portion of a scrollable area using the sliver layout protocol (SliverConstraints → SliverGeometry) enabling lazy and custom scroll effects; composed in CustomScrollView.

Q3. When use CustomScrollView? When combining multiple scrollable sections (lists, grids, headers) into one scroll, or for effects like collapsing/sticky headers.

Q4. Why avoid shrinkWrap? It forces the scrollable to lay out all children to compute its size, losing laziness — expensive for long lists.

Q5. How do you implement infinite scroll? ScrollController listener or NotificationListener detecting near-end, guarded loading flag, page/cursor tracking, footer loader & error/retry, refresh resetting state.

Q6. How do you preserve state of tabs/list items? AutomaticKeepAliveClientMixin, PageStorageKey, IndexedStack, StatefulShellRoute, or lifting state out of widgets.

Q7. What does `itemExtent` do? Gives fixed main-axis extent per item allowing O(1) layout calculations and efficient jumps/scrollbar.

Q8. NestedScrollView use case? Coordinating an outer collapsing header with inner scroll views (e.g., profile header + tabs with lists).

Q9. How do you make a sticky header? `SliverAppBar(pinned: true)`, `SliverPersistentHeader(pinned: true)`, `PinnedHeaderSliver`, or grouping with `SliverMainAxisGroup`.

Q10. What is cacheExtent? Area before/after the visible viewport where items are pre-built for smooth scrolling.

Chapter 10 — Theming, Material 3, Design Systems, Assets, Fonts, Localization & Accessibility

Goal: Build apps that look consistent (design systems), support dark mode, multiple languages and RTL, and are accessible to everyone. These topics separate hobby apps from enterprise apps.

10.1 Theme basics — the app's outfit 👗

5-year-old: Instead of telling every button "be blue" one by one, you tell the whole app at the top: "**our team colour is blue.**" Everybody below automatically wears blue. That's a **Theme.**

```
MaterialApp(
  theme: ThemeData(
    colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF0061A4)),
    useMaterial3: true,                // default true since Flutter 3.16
    textTheme: GoogleFonts.interTextTheme(),
    visualDensity: VisualDensity.adaptivePlatformDensity,
  ),
  darkTheme: ThemeData(
    colorScheme: ColorScheme.fromSeed(
      seedColor: const Color(0xFF0061A4),
      brightness: Brightness.dark,
    ),
  ),
  themeMode: ThemeMode.system,
);
```

Reading theme values:

```
final theme = Theme.of(context);
final cs = theme.colorScheme;
Container(color: cs.primaryContainer, child: Text('Hi', style:
theme.textTheme.titleMedium?.copyWith(color: cs.onPrimaryContainer)));
```

Rule: never hard-code colours/text styles in widgets in a company app; always use theme tokens. This makes dark mode, rebranding, and white-labelling trivial.

10.2 Material 3 (Material You)

- M3 became **default in Flutter 3.16** (`useMaterial3: true`). Set `false` to keep M2 temporarily (M2 is being phased out).
- **ColorScheme roles** (memorise the pattern): `primary` / `onPrimary` / `primaryContainer` / `onPrimaryContainer`; same for `secondary`, `tertiary`, `error`; `surface` / `onSurface` / `onSurfaceVariant`; `surfaceContainerLowest` … `surfaceContainerHighest` (tonal surfaces, replaced deprecated `background` / `surfaceVariant` in Flutter 3.22); `outline`, `outlineVariant`, `inverseSurface`, `shadow`, `scrim`.
  - "on" colours are for content (text/icons) placed **on** the matching colour — guarantees contrast.
- `ColorScheme.fromSeed` generates a harmonious tonal palette from one seed colour (dynamic color algorithm). `dynamic_color` package uses Android 12+ wallpaper colours.
- **Elevation** in M3 is shown by **surface tint** colour more than shadows; `scrolledUnderElevation` on AppBar.
- New components: `FilledButton`, `NavigationBar`, `NavigationRail`, `NavigationDrawer`, `SegmentedButton`, `SearchBar` / `SearchAnchor`, `Badge`, `MenuAnchor`, `DropdownMenu`, `CarouselView`, M3 `SliverAppBar.medium/.large`.
- Typography: `displayLarge/Medium/Small`, `headlineLarge/…`, `titleLarge/…`, `bodyLarge/…`, `labelLarge/…` (old names `headline1`, `bodyText1` etc. were removed).

Material & Cupertino moving to packages (Flutter 3.44–3.47+)

The Flutter team **froze** `material.dart` / `cupertino.dart` in the framework in 3.44 and published them as standalone packages `material_ui` and `cupertino_ui` (1.0 published starting with Flutter 3.47). Reason: Material/Cupertino updates can ship on their own schedule instead of waiting for quarterly SDK releases. Migration is designed to be drop-in (mostly import changes, with a `dart fix` migration available). The in-framework libraries are planned to be deprecated and later removed — follow the official breaking-change guide ("Migrate to standalone material_ui and cupertino_ui packages") when your team upgrades. All widget APIs in this book are the same whichever import you use.

10.3 Component themes — design system in code

```
ThemeData buildTheme(Brightness b) {
  final cs = ColorScheme.fromSeed(seedColor: AppColors.brand, brightness: b);
  return ThemeData(
    colorScheme: cs,
    scaffoldBackgroundColor: cs.surface,
    appBarTheme: AppBarTheme(centerTitle: true, backgroundColor: cs.surface, elevation: 0),
    filledButtonTheme: FilledButtonThemeData(
      style: FilledButton.styleFrom(
        minimumSize: const Size(64, 48),
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
      ),
```

```
    ),
    inputDecorationTheme: InputDecorationTheme(
      filled: true,
      border: OutlineInputBorder(borderRadius: BorderRadius.circular(12)),
    ),
    cardTheme: const CardThemeData(margin: EdgeInsets.zero, elevation: 0), // CardThemeData in
recent versions
    snackBarTheme: const SnackBarThemeData(behavior: SnackBarBehavior.floating),
    pageTransitionsTheme: const PageTransitionsTheme(builders: {
      TargetPlatform.android: PredictiveBackPageTransitionsBuilder(), // Android predictive back
      TargetPlatform.iOS: CupertinoPageTransitionsBuilder(),
    }),
    extensions: [AppSpacing.regular, AppBrandColors.light],
  );
}
```

Note: in Flutter 3.27+, `ThemeData.cardTheme`, `dialogTheme`, `tabBarTheme` etc. are being normalized to `*ThemeData` classes (`CardThemeData`, `DialogThemeData`, `TabBarThemeData`). Analyzer/`dart fix` will guide you.

ThemeExtension — your own custom tokens

```
@immutable
class AppBrandColors extends ThemeExtension<AppBrandColors> {
  const AppBrandColors({required this.success, required this.warning});
  final Color success;
  final Color warning;

  static const light = AppBrandColors(success: Color(0xFF2E7D32), warning: Color(0xFFF9A825));
  static const dark  = AppBrandColors(success: Color(0xFF81C784), warning: Color(0xFFFFD54F));

  @override
  AppBrandColors copyWith({Color? success, Color? warning}) =>
      AppBrandColors(success: success ?? this.success, warning: warning ?? this.warning);

  @override
  AppBrandColors lerp(ThemeExtension<AppBrandColors>? other, double t) {
    if (other is! AppBrandColors) return this;
    return AppBrandColors(
      success: Color.lerp(success, other.success, t)!,
      warning: Color.lerp(warning, other.warning, t)!,
    );
  }
}
// Use:
final brand = Theme.of(context).extension<AppBrandColors>()!;
Icon(Icons.check, color: brand.success);
```

`lerp` enables smooth animated theme transitions (MaterialApp animates between themes using `AnimatedTheme`).

Local theme overrides

```
Theme(
  data: Theme.of(context).copyWith(colorScheme: Theme.of(context).colorScheme.copyWith(primary:
Colors.red)),
  child: const DangerZone(),
);
```

Color API changes to know (Flutter 3.27+)

- `Color.withOpacity()` is **deprecated** → use `color.withValues(alpha: 0.5)` (wide-gamut colour support; components are now doubles `r`, `g`, `b`, `a` in 0–1).
- `Color.value`, `.red`, `.green`, `.blue`, `.opacity` deprecated → `toARGB32()`, `.r`, `.g`, `.b`, `.a`.

Dark mode toggle (persisted)

Store `ThemeMode` in your state management + SharedPreferences; MaterialApp listens and rebuilds:

```
ValueListenableBuilder<ThemeMode>(
  valueListenable: themeController,   // ValueNotifier<ThemeMode>
  builder: (_, mode, __) => MaterialApp(themeMode: mode, theme: light, darkTheme: dark, home:
...),
);
```

Design system organisation (enterprise)

```
lib/core/design_system/
├── tokens/ (colors.dart, typography.dart, spacing.dart, radii.dart, durations.dart)
├── theme/ (app_theme.dart, extensions.dart)
└── components/ (app_button.dart, app_text_field.dart, app_card.dart, ...)
```

Many companies put this in a separate **package** in a monorepo (`packages/design_system`) with its own **widgetbook** (Storybook-like catalog: `widgetbook` package) and **golden tests** (Chapter 22). Figma tokens can be exported and generated into Dart.

10.4 Assets, fonts, icons, splash

Loading assets

```
Image.asset('assets/images/logo.png');
final jsonStr = await rootBundle.loadString('assets/config/app.json');
```

```
final bytes = await rootBundle.load('assets/models/model.tflite');
DefaultAssetBundle.of(context).loadString(...);   // testable alternative
```

Type-safe asset references: `flutter_gen` generates `Assets.images.logo.image()` — avoids typos ("Unable to load asset").

Asset **transformers** (Flutter 3.22+): transform assets at build time (e.g., `vector_graphics_compiler` for SVGs) via `transformers:` in pubspec.

Fonts

Declare in pubspec (Chapter 2), then `fontFamily: 'Inter'` or in `ThemeData(fontFamily: 'Inter')`. Variable fonts supported with `FontVariation`. Packaged fonts: `fontFamily: 'Roboto', package: 'my_design_system'`.

App icons & splash screens

- `flutter_launcher_icons` — generates Android adaptive icons and iOS icon sets.
- `flutter_native_splash` — generates native splash (Android 12+ uses the `SplashScreen` API: icon on background colour only; iOS LaunchScreen storyboard). The native splash shows while the engine loads; then your first Flutter frame. Keep it until init finishes: `FlutterNativeSplash.preserve(widgetsBinding: binding)` → `FlutterNativeSplash.remove()`.
- Don't build a fake "splash page" with a timer just to show a logo — it delays users. Do real initialization instead.

10.5 Internationalization (i18n) & Localization (l10n) 🌍

i18n = making the app *able* to support many languages/regions. **l10n** = actually translating & formatting for a specific locale.

Official setup with `gen-l10n`

1. `pubspec.yaml`:

```
dependencies:
  flutter_localizations:
    sdk: flutter
  intl: any          # version pinned by flutter_localizations
flutter:
  generate: true
```

2. `l10n.yaml` at project root:

```
arb-dir: lib/l10n
template-arb-file: app_en.arb
output-localization-file: app_localizations.dart
```

```
# Recent Flutter versions generate into the source tree (no synthetic package):
# output-dir: lib/l10n/gen
nullable-getter: false
```

3. `lib/l10n/app_en.arb`:

```
{
  "@@locale": "en",
  "helloUser": "Hello {name}!",
  "@helloUser": { "placeholders": { "name": { "type": "String" } } },
  "itemsInCart": "{count, plural, =0{Your cart is empty} =1{1 item in cart} other{{count} items in cart}}",
  "@itemsInCart": { "placeholders": { "count": { "type": "int" } } },
  "genderGreeting": "{gender, select, male{Welcome sir} female{Welcome madam} other{Welcome}}",
  "price": "Price: {amount}",
  "@price": { "placeholders": { "amount": { "type": "double", "format": "currency", "optionalParameters": { "symbol": "₹", "decimalDigits": 2 } } } },
  "lastSeen": "Last seen {date}",
  "@lastSeen": { "placeholders": { "date": { "type": "DateTime", "format": "yMMMd" } } }
}
```

`app_hi.arb`, `app_ar.arb` etc. with the same keys. 4. Wire up:

```
MaterialApp(
  localizationsDelegates: AppLocalizations.localizationsDelegates, // includes GlobalMaterialLocalizations etc.
  supportedLocales: AppLocalizations.supportedLocales,
  locale: userSelectedLocale,   // null = follow device
);
// Use:
Text(AppLocalizations.of(context).helloUser('Asha'));
// handy extension: extension L10nX on BuildContext { AppLocalizations get l10n => AppLocalizations.of(this); }
```

Run `flutter gen-l10n` (or it runs automatically on `flutter run`/`build` when `generate: true`).

> The old "synthetic package" approach (`import 'package:flutter_gen/gen_l10n/app_localizations.dart'`) was deprecated in Flutter 3.29-ish; newer projects generate into the source tree (`output-dir`), imported like normal source.

iOS extra step

Add supported languages to `ios/Runner/Info.plist` `CFBundleLocalizations` array, or iOS may not switch.

Formatting with `intl`

```
NumberFormat.currency(locale: 'en_IN', symbol: '₹').format(123456.5); // ₹1,23,456.50
NumberFormat.compact().format(1200000);   // 1.2M
DateFormat.yMMMMd('fr').format(DateTime.now());  // 15 octobre 2026
DateFormat('dd/MM/yyyy HH:mm').format(dt);
```

Call `initializeDateFormatting()` if using DateFormat for non-default locales outside the Flutter localizations delegates.

RTL (Arabic, Hebrew, Urdu)

- Flutter flips layouts automatically based on `Directionality` (set by locale).
- Use `EdgeInsetsDirectional`, `AlignmentDirectional`, `PositionedDirectional`, `start/end` instead of `left/right`.
- Icons like arrows: use `Icons.arrow_back` (auto-mirrored in RTL for certain icons) or `Directionality.of(context)` checks.
- Test with `locale: Locale('ar')`.

Alternatives

`easy_localization` (JSON/CSV/YAML, runtime switching, popular), `slang` (type-safe, codegen), translation management tools (Lokalise, Crowdin, Phrase) that sync ARB files.

Org best practices

- Never concatenate translated strings (`'Hello ' + name`) — word order differs per language; use placeholders.
- Use ICU plurals/selects; plural rules differ (Arabic has 6 forms!).
- Don't put text in images.
- Allow text expansion (German ~30% longer) — avoid fixed-width text containers.
- Pseudo-localization testing.
- Keys named by meaning (`checkoutPayButton`), with descriptions for translators (`"description"` in `@key`).

10.6 Accessibility (a11y) ♿

5-year-old: Some people can't see the screen well, some can't use their hands easily, some need bigger letters. Accessibility means **everyone** can use your app — like ramps next to stairs.

Flutter builds a **Semantics tree** parallel to the render tree, which it sends to the platform accessibility services (TalkBack on Android, VoiceOver on iOS, screen readers on web/desktop).

Key techniques

```
// 1. Labels for icons/images
IconButton(icon: const Icon(Icons.delete), tooltip: 'Delete', onPressed: ...); // tooltip provides semantics label
Image.asset('assets/chart.png', semanticLabel: 'Sales up 20% this month');
const Image(image: AssetImage('decor.png'), excludeFromSemantics: true);   // decorative

// 2. Custom semantics
Semantics(
  label: 'Rating: 4 out of 5 stars',
  button: false,
  child: Row(children: stars),
);
MergeSemantics(child: Row(children: [Checkbox(...), const Text('Remember me')])); // read as one
ExcludeSemantics(child: decorativeWidget);
Semantics(liveRegion: true, child: Text(statusMessage));  // announce updates
SemanticsService.announce('Item deleted', TextDirection.ltr);  // ad-hoc announcement

// 3. Large text: respect user text scaling; test at 200%. Avoid fixed heights for text containers.
// 4. Contrast: WCAG AA ≥ 4.5:1 for normal text, 3:1 for large text — use colorScheme "on" colours.
// 5. Tap targets ≥ 48×48 dp (Android) / 44×44 pt (iOS). Material widgets handle this via MaterialTapTargetSize.padded.
// 6. Focus & keyboard navigation (desktop/web/TV): FocusTraversalGroup, Shortcuts, Actions, visible focus indicators.
// 7. Don't convey info by colour alone (add icons/text).
// 8. Reduce motion: MediaQuery.disableAnimationsOf(context) → shorten/skip animations.
// 9. Ordering: OrdinalSortKey / FocusTraversalOrder to fix reading order.
```

Testing accessibility

- Turn on TalkBack/VoiceOver and navigate your app.
- `showSemanticsDebugger: true` in MaterialApp.
- Widget tests with guidelines:

```
testWidgets('meets a11y guidelines', (tester) async {
  final handle = tester.ensureSemantics();
  await tester.pumpWidget(const MyApp());
  await expectLater(tester, meetsGuideline(androidTapTargetGuideline));
  await expectLater(tester, meetsGuideline(iOSTapTargetGuideline));
  await expectLater(tester, meetsGuideline(labeledTapTargetGuideline));
  await expectLater(tester, meetsGuideline(textContrastGuideline));
  handle.dispose();
});
```

- Accessibility Scanner (Android), Accessibility Inspector (Xcode).

- Web: Flutter web builds the semantics tree lazily; users enable it via a hidden "Enable accessibility" button, or you can call `SemanticsBinding.instance.ensureSemantics()` to force it on (helps automation/SEO tooling too).

Legal context: many countries require accessible apps (ADA in US, European Accessibility Act enforced from June 2025, etc.) — enterprise clients ask for WCAG 2.1/2.2 AA compliance.

10.7 Interview Questions — Theming, i18n, a11y

Q1. How do you implement dark mode? Provide `theme` and `darkTheme` (e.g., `ColorScheme.fromSeed` with brightness) and `themeMode` (system/light/dark), read colours from `Theme.of(context).colorScheme`, persist the user choice.

Q2. What is ColorScheme.fromSeed? Generates a full M3 tonal colour scheme from a single seed colour.

Q3. How do you add custom theme properties? `ThemeExtension<T>` with `copyWith` and `lerp`, registered in `ThemeData.extensions`, accessed with `Theme.of(context).extension<T>()`.

Q4. How do you localize a Flutter app? flutter_localizations + intl, ARB files, gen-l10n generating `AppLocalizations`, delegates and supported locales on MaterialApp, `AppLocalizations.of(context)`; ICU plurals/selects; Info.plist locales for iOS.

Q5. How does Flutter support RTL? Directionality from locale; directional widgets (`EdgeInsetsDirectional`, `AlignmentDirectional`), Row/ListView automatically mirror.

Q6. How do you make a Flutter app accessible? Semantic labels, tooltips, MergeSemantics/ExcludeSemantics, sufficient contrast, 48dp tap targets, support text scaling, focus traversal & keyboard, reduce motion, test with screen readers and `meetsGuideline`.

Q7. What is the Semantics tree? A tree of semantic nodes generated from widgets that describes meaning (labels, actions, states) for accessibility services.

Q8. What changed with Material 3 in Flutter? M3 default from 3.16; new colour roles & tonal surfaces; new components (FilledButton, NavigationBar, etc.); typography names; surface tint elevation.

Q9. `withOpacity` vs `withValues`? `withOpacity` is deprecated (3.27+) in favour of `withValues(alpha:)` due to wide-gamut colour support and float components.

Q10. What are `material_ui` and `cupertino_ui`? Standalone pub.dev packages containing Flutter's Material and Cupertino libraries, decoupled from the SDK (published from Flutter 3.47) so they can release independently.

Chapter 11 — Navigation & Routing: Navigator 1.0, Navigator 2.0, go_router, Deep Links

Goal: Move between screens correctly, pass and return data, handle back buttons (Android predictive back!), nested navigation with bottom tabs, authentication redirects, deep links and web URLs.

11.1 5-year-old picture: a stack of plates 🍽️

The **Navigator** keeps screens like a **stack of plates**. Opening a new screen = **put a plate on top** (`push`). Going back = **take the top plate off** (`pop`). You only see the top plate.

```
push(Details)        pop()
┌─────────┐
| Details | ← top
├─────────┤          ┌─────────┐
|  Home   |          |  Home   |
└─────────┘          └─────────┘
```

11.2 Navigator 1.0 (imperative)

Push and pop

```
// Push a new route
Navigator.of(context).push(
  MaterialPageRoute(builder: (context) => DetailsPage(id: item.id)),
);
// Shorthand
Navigator.push(context, MaterialPageRoute(builder: (_) => const SettingsPage()));

// Pop (go back)
Navigator.of(context).pop();
Navigator.pop(context);
Navigator.maybePop(context);          // pops only if possible
Navigator.canPop(context);
```

`MaterialPageRoute` gives platform-appropriate transitions; `CupertinoPageRoute` gives iOS swipe-back transition; `PageRouteBuilder` for custom transitions; `fullscreenDialog: true` for modal pages (slide-up, close "X").

Passing data forward

Via constructor (type-safe — best):

```
Navigator.push(context, MaterialPageRoute(builder: (_) => DetailsPage(product: product)));
```

Returning data back ↩️

```
// Screen A
final selected = await Navigator.push<Color>(
  context,
  MaterialPageRoute(builder: (_) => const ColorPickerPage()),
);
if (!context.mounted) return;
if (selected != null) setState(() => _color = selected);

// Screen B (ColorPickerPage)
onTap: () => Navigator.pop(context, Colors.red),
```

`selected` is `null` if the user pressed system back.

Replacing & clearing the stack

```
Navigator.pushReplacement(context, MaterialPageRoute(builder: (_) => const HomePage())); //
replace current (e.g., after splash)
Navigator.pushAndRemoveUntil(                                                    // e.g.,
after login/logout
  context,
  MaterialPageRoute(builder: (_) => const LoginPage()),
  (route) => false,          // remove ALL previous routes
);
Navigator.popUntil(context, (route) => route.isFirst);         // back to first
Navigator.popUntil(context, ModalRoute.withName('/home'));
```

Named routes (legacy-ish)

```
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (_) => const HomePage(),
    '/details': (_) => const DetailsPage(),
  },
  onGenerateRoute: (settings) {       // for routes needing arguments/parsing
    if (settings.name == '/product') {
      final id = settings.arguments as String;
      return MaterialPageRoute(builder: (_) => ProductPage(id: id), settings: settings);
    }
```

```
    return null;
  },
  onUnknownRoute: (_) => MaterialPageRoute(builder: (_) => const NotFoundPage()),
);
Navigator.pushNamed(context, '/product', arguments: 'p42');
final args = ModalRoute.of(context)!.settings.arguments;
```

The Flutter team **no longer recommends named routes** for most apps: arguments aren't type-safe, deep link behaviour can't be customised, and they don't support the browser forward button well. Use a router package (go_router) instead.

Navigating without BuildContext

```
final navigatorKey = GlobalKey<NavigatorState>();
MaterialApp(navigatorKey: navigatorKey, ...);
navigatorKey.currentState?.pushNamed('/login');   // e.g., from interceptor on 401
```

Use carefully — prefer routing state (go_router redirect) for auth flows.

Route lifecycle observing

```
class LoggingObserver extends NavigatorObserver {
  @override
  void didPush(Route route, Route? previousRoute) => analytics.logScreen(route.settings.name);
  @override
  void didPop(Route route, Route? previousRoute) {}
}
MaterialApp(navigatorObservers: [LoggingObserver(), routeObserver]);
```

`RouteAware` + `RouteObserver<ModalRoute>` lets a screen know when it becomes visible again (`didPopNext`) — e.g., refresh data when returning.

11.3 Back button handling: `PopScope` (WillPopScope is deprecated)

Android 14+ introduced **predictive back** (preview of the previous screen while swiping). The system must know **in advance** whether back is allowed, so the old `WillPopScope` (async decision) was **deprecated in Flutter 3.12+** and replaced with `PopScope`:

```
PopScope<Object?>(
  canPop: !_hasUnsavedChanges,                        // decided in advance
  onPopInvokedWithResult: (didPop, result) async {    // Flutter 3.24+ (replaces onPopInvoked)
    if (didPop) return;                               // already popped
    final leave = await showDialog<bool>(
      context: context,
      builder: (c) => AlertDialog(
        title: const Text('Discard changes?'),
```

```
        actions: [
          TextButton(onPressed: () => Navigator.pop(c, false), child: const Text('Stay')),
          TextButton(onPressed: () => Navigator.pop(c, true), child: const Text('Discard')),
        ],
      ),
    );
    if (leave == true && context.mounted) Navigator.pop(context);
  },
  child: const EditForm(),
);
```

- `canPop: false` blocks the back gesture/button and calls `onPopInvokedWithResult(didPop: false)`.
- For predictive back animations on Android, `PredictiveBackPageTransitionsBuilder` in the theme and `android:enableOnBackInvokedCallback="true"` in the manifest (the default behaviour changes with targetSdk 35/36 — predictive back is enabled by default for apps targeting Android 15+/16).
- "Double back to exit" pattern: `canPop: false` + track last back press time + `SystemNavigator.pop()`.

11.4 Navigator 2.0 (declarative Router API)

Navigator 2.0 (introduced in Flutter 1.22) lets you set the **whole page stack as a function of app state**, and syncs with the platform (browser URL, deep links).

Pieces:

- **`Page`** — immutable description of a route (`MaterialPage`, `CupertinoPage`, custom `Page`).
- **`Navigator(pages: [...], onDidRemovePage: ...)`** — declarative stack. (`onPopPage` was deprecated in 3.24 in favour of `onDidRemovePage`.)
- **`Router`** — coordinates:
  - **`RouteInformationProvider`** — gets route info (URL) from OS/browser.
  - **`RouteInformationParser`** — parses URL ↔ your app's route configuration (typed).
  - **`RouterDelegate`** — builds the `Navigator` with pages from app state; notifies on change; handles `setNewRoutePath` (deep link) and `currentConfiguration` (URL update).
  - **`BackButtonDispatcher`** — Android back button.

Minimal declarative example:

```
class _AppState extends State<App> {
  Book? _selected;
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Navigator(
        pages: [
```

```
          MaterialPage(key: const ValueKey('list'), child: BookList(onTap: (b) => setState(() =>
_selected = b))),
          if (_selected != null) MaterialPage(key: ValueKey(_selected), child: BookDetails(book:
_selected!)),
        ],
        onDidRemovePage: (page) {
          if (page.key == ValueKey(_selected)) setState(() => _selected = null);
        },
      ),
    );
  }
}
```

Implementing RouterDelegate/Parser by hand is verbose and error-prone → teams use **go_router** (official, maintained by the Flutter team), or **auto_route** (codegen), or **beamer**.

Interview: *"Navigator 1.0 vs 2.0?"* → 1.0 is imperative push/pop; 2.0 is declarative (pages from state), supports deep linking/web URLs/browser history, and complex nested navigation, but is complex; router packages build on 2.0. They can coexist (go_router uses Navigator 2.0 and still allows `Navigator.push` for dialogs).

11.5 go_router — the standard choice 🧭

```
dependencies:
  go_router: ^x.y.z    # check pub.dev for latest
```

Basic setup

```
final router = GoRouter(
  initialLocation: '/',
  debugLogDiagnostics: true,
  routes: [
    GoRoute(
      path: '/',
      name: 'home',
      builder: (context, state) => const HomePage(),
      routes: [                              // nested (child) routes → stacked on parent
        GoRoute(
          path: 'product/:id',               // → /product/42
          name: 'product',
          builder: (context, state) {
            final id = state.pathParameters['id']!;
            final tab = state.uri.queryParameters['tab'];   // /product/42?tab=reviews
            final extra = state.extra as Product?;            // object (NOT in URL; lost on web
refresh/deep link)
            return ProductPage(id: id, initialTab: tab, preloaded: extra);
```

```
          },
        ),
      ],
    ),
    GoRoute(path: '/login', builder: (_, __) => const LoginPage()),
  ],
  errorBuilder: (context, state) => NotFoundPage(error: state.error),
);

void main() => runApp(MaterialApp.router(routerConfig: router));
```

Navigating

```
context.go('/product/42');              // GO to location: replaces stack to match URL hierarchy
(/, /product/42)
context.push('/product/42');            // PUSH on top of current stack (returns Future for
result)
context.goNamed('product', pathParameters: {'id': '42'}, queryParameters: {'tab': 'reviews'});
context.pushNamed('product', pathParameters: {'id': '42'}, extra: product);
context.pop();                          // or context.pop(result)
context.replace('/x');  context.pushReplacement('/x');
context.canPop();
final result = await context.push<bool>('/confirm');
```

`go` vs `push` (very common interview question):

- `go` **navigates to a URL**; the page stack becomes whatever the route hierarchy for that URL defines (sub-routes stacked on parents). Previous stack outside that hierarchy is discarded. Browser URL updates.
- `push` **adds a page on top** of the current stack (like Navigator.push), regardless of hierarchy. Good for "detail on top of current context" and awaiting results.

Redirects & authentication guard

```
final router = GoRouter(
  refreshListenable: authNotifier,      // re-run redirect when auth state changes
(ChangeNotifier / Listenable)
  redirect: (context, state) {
    final loggedIn = authNotifier.isLoggedIn;
    final loggingIn = state.matchedLocation == '/login';
    if (!loggedIn && !loggingIn) {
      return '/login?from=${Uri.encodeComponent(state.uri.toString())}';  // remember where to go
back
    }
    if (loggedIn && loggingIn) return state.uri.queryParameters['from'] ?? '/';
    return null;                         // no redirect
  },
```

```
  routes: [...],
);
```

Per-route `redirect:` also available. Avoid redirect loops (go_router throws after a limit, `redirectLimit` default 5).

For Riverpod/Bloc: expose auth state as a `Listenable` (e.g., a `ValueNotifier` updated by a stream listener, or the `GoRouterRefreshStream` pattern).

ShellRoute & StatefulShellRoute (bottom navigation with nested stacks)

```
GoRouter(routes: [
  StatefulShellRoute.indexedStack(
    builder: (context, state, navigationShell) => ScaffoldWithNavBar(navigationShell:
navigationShell),
    branches: [
      StatefulShellBranch(routes: [
        GoRoute(path: '/home', builder: (_, __) => const HomeTab(), routes: [
          GoRoute(path: 'details/:id', builder: (_, s) => DetailsPage(id:
s.pathParameters['id']!)),
        ]),
      ]),
      StatefulShellBranch(routes: [
        GoRoute(path: '/profile', builder: (_, __) => const ProfileTab()),
      ]),
    ],
  ),
]);

class ScaffoldWithNavBar extends StatelessWidget {
  const ScaffoldWithNavBar({super.key, required this.navigationShell});
  final StatefulNavigationShell navigationShell;
  @override
  Widget build(BuildContext context) => Scaffold(
    body: navigationShell,                        // IndexedStack of branch navigators — each keeps
its own stack & state
    bottomNavigationBar: NavigationBar(
      selectedIndex: navigationShell.currentIndex,
      onDestinationSelected: (i) => navigationShell.goBranch(i, initialLocation: i ==
navigationShell.currentIndex), // tap active tab → pop to root
      destinations: const [
        NavigationDestination(icon: Icon(Icons.home), label: 'Home'),
        NavigationDestination(icon: Icon(Icons.person), label: 'Profile'),
      ],
    ),
  );
}
```

`ShellRoute` (non-stateful) wraps routes in a common UI (e.g., persistent bottom bar) with one nested navigator but doesn't preserve each tab's stack.

Showing a full-screen page **above** the bottom bar from inside a branch: give the route `parentNavigatorKey: rootNavigatorKey`.

Type-safe routes

`go_router_builder` generates typed route classes:

```
@TypedGoRoute<ProductRoute>(path: '/product/:id')
class ProductRoute extends GoRouteData with $ProductRoute {   // (mixin name depends on
go_router_builder version)
  const ProductRoute({required this.id});
  final String id;
  @override
  Widget build(BuildContext context, GoRouterState state) => ProductPage(id: id);
}
const ProductRoute(id: '42').go(context);
```

Custom transitions

```
GoRoute(
  path: '/modal',
  pageBuilder: (context, state) => CustomTransitionPage(
    key: state.pageKey,
    child: const ModalPage(),
    transitionsBuilder: (context, animation, secondary, child) =>
        FadeTransition(opacity: animation, child: child),
  ),
);
```

go_router gotchas

- `extra` is not serialized into the URL: lost on web refresh, deep links, and process death restoration (unless you provide an `extraCodec`). Prefer path/query params with IDs and fetch data.
- `context.go` inside `initState`/build → schedule after frame or use redirect.
- Don't create the `GoRouter` inside `build` (recreated → navigation state lost). Create once (top-level final, or in a provider).
- Using `Navigator.push` alongside go_router works but those pages aren't URL-addressable.
- Breaking changes happen in major versions — read the changelog when upgrading (e.g., `GoRouterState.location` → `uri`, `queryParams` → `uri.queryParameters`, `params` → `pathParameters`).

auto_route (alternative)

Code-generated, strongly typed routes (`context.router.push(ProductRoute(id: '42'))`), nested routers, guards (`AutoRouteGuard`), tabs (`AutoTabsRouter`). Popular in large codebases.

11.6 Deep links & App Links / Universal Links 🔗

Deep link = a URL that opens a specific screen in your app, e.g., `https://shop.example.com/product/42` or `myshop://product/42`.

Type	Example	Verified ownership?	Opens app directly?
Custom scheme	`myshop://product/42`	❌ (any app can claim)	Shows chooser / may conflict
Android App Links	`https://...`	✅ via `assetlinks.json`	✅
iOS Universal Links	`https://...`	✅ via `apple-app-site-association`	✅

Android setup

`android/app/src/main/AndroidManifest.xml` inside `<activity ...MainActivity>`:

```
<intent-filter android:autoVerify="true">
  <action android:name="android.intent.action.VIEW" />
  <category android:name="android.intent.category.DEFAULT" />
  <category android:name="android.intent.category.BROWSABLE" />
  <data android:scheme="https" android:host="shop.example.com" />
</intent-filter>
```

Host `https://shop.example.com/.well-known/assetlinks.json` with your package name and **SHA-256 signing certificate fingerprints** (both upload key and **Play App Signing key** — a common mistake is only adding the debug/upload key!).

Flutter's built-in deep link handling is controlled by `<meta-data android:name="flutter_deeplinking_enabled" android:value="true" />` (enabled by default in recent Flutter versions). If you use a third-party deep link plugin (e.g., `app_links`, Firebase Dynamic Links legacy, Branch), you may need to **disable** Flutter's handling to avoid double handling.

Test: `adb shell am start -a android.intent.action.VIEW -c android.intent.category.BROWSABLE -d "https://shop.example.com/product/42" com.example.shop`.

iOS setup

- Xcode → Signing & Capabilities → **Associated Domains** → `applinks:shop.example.com`.
- Host `https://shop.example.com/.well-known/apple-app-site-association` (JSON, no extension, served as `application/json`, HTTPS, no redirects) listing `TEAMID.bundleId` and paths.
- `Info.plist`: `FlutterDeepLinkingEnabled` = YES (for Flutter's handler; default in recent versions).

- Test: `xcrun simctl openurl booted "https://shop.example.com/product/42"`.

With go_router

Once platform setup is done, the URL path is passed to the router, which matches `/product/42` → ProductPage automatically, including on cold start. Make sure your redirect logic (auth) preserves the deep link target (`?from=`).

DevTools has a **Deep Links validator** tool to check Android/iOS config.

Note: **Firebase Dynamic Links was shut down (August 2025)** — teams migrated to App Links/Universal Links directly or to providers like Branch, AppsFlyer OneLink, Adjust.

11.7 Web URLs

- URL strategy: default is **hash** (`/#/product/42`) historically; use path URLs:

```
import 'package:flutter_web_plugins/url_strategy.dart';
void main() { usePathUrlStrategy(); runApp(...); }
```

(go_router: path strategy also via this.) Server must rewrite all routes to `index.html` (SPA fallback), or refreshes give 404.

- Browser back/forward works with Router API / go_router.

11.8 Navigation architecture best practices (org level)

1. **Single source of truth** for routes: a `router.dart` with route constants or typed routes.
2. Pass **IDs**, not big objects, between screens (deep-linkable, restoration-safe); load data in the destination via repository/cache.
3. Centralize **auth guards** in redirect; never sprinkle "if not logged in push login" in screens.
4. Keep navigation **out of business logic** where possible: logic emits state/events, UI layer navigates (or use a navigation service abstraction for testability).
5. Track screens for analytics with a `NavigatorObserver`.
6. Handle **unknown routes** (404 page).
7. Test deep links on cold start, warm start, logged-out state.
8. Restoration: `restorationScopeId` in MaterialApp and `RestorableProperty` / go_router restoration for Android process death.

11.9 Common navigation bugs

Bug	Cause	Fix
"Navigator operation requested with a context that does not include a Navigator"	context above MaterialApp	use context below MaterialApp / navigatorKey
Black screen after pop	popped the last route	check `canPop`; use `maybePop`; don't pop root
Dialog pop closes the page instead	used page context with nested navigators / popped twice	pop with dialog's context; `useRootNavigator` alignment
`setState` after navigating away	async continues	`mounted` checks
Double navigation on double tap		debounce/disable button; go_router `go` is idempotent for same location
Back button exits app from tab	nested navigators not handling back	StatefulShellRoute / `NavigatorPopHandler` / PopScope
Deep link opens app but not the page	router not matching path / redirect swallowing it	debugLogDiagnostics, preserve `from` param
App Links open browser instead of app	assetlinks fingerprint missing (Play signing key) / autoVerify failed	fix assetlinks.json; `adb shell pm get-app-links <pkg>`
Page rebuilds/resets on keyboard open	Router/GoRouter created inside a widget that rebuilds (MediaQuery dependency)	create router once outside build

---

11.10 Interview Questions — Navigation

Q1. How does the Navigator work? It manages a stack of `Route` objects (with an Overlay); push adds, pop removes; only top routes are visible.

Q2. How do you pass data between screens and return results? Constructor arguments forward; `Navigator.pop(context, result)` + `await Navigator.push` (or `context.pop(result)` with `await context.push`).

Q3. Difference between `push`, `pushReplacement`, `pushAndRemoveUntil`? Add on top; replace current; add and remove previous routes until predicate true (clear stack).

Q4. What is Navigator 2.0? Declarative Router API: pages list derived from app state, Router with RouteInformationParser/RouterDelegate syncing with platform URLs and deep links.

Q5. go vs push in go_router? go navigates to a location (stack defined by route hierarchy, URL-synced); push adds a page atop the current stack.

Q6. How do you protect routes requiring login? go_router `redirect` with `refreshListenable` bound to auth state (or auto_route guards).

Q7. How do you keep separate navigation stacks per bottom tab? `StatefulShellRoute.indexedStack` (go_router), nested Navigators with IndexedStack, or auto_route `AutoTabsRouter`.

Q8. What replaced WillPopScope and why? `PopScope` with `canPop` and `onPopInvokedWithResult`, because Android predictive back needs to know ahead of time whether pop is allowed.

Q9. How do deep links work in Flutter? Platform config (Android intent filters with App Links verification; iOS Associated Domains with AASA) delivers URL to Flutter's router which parses it into routes.

Q10. How do you navigate without context (e.g., from a service)? GlobalKey on MaterialApp, or router instance (`router.go(...)`), ideally through an abstraction.

Q11. What is `RouteAware`? Mixin to get notified when a route is pushed/popped relative to the current one (didPush, didPopNext...), via a RouteObserver.

Q12. Why avoid passing objects via `extra`? Not part of the URL — not restorable, not deep-linkable, lost on web refresh.

Chapter 12 — Animations: Implicit, Explicit, Hero, Physics, Staggered, Lottie/Rive

Goal: Make UIs feel alive — and know *which* animation approach to choose, how `AnimationController`, `Tween`, `Curve` and `Ticker` work under the hood, and how to keep animations at 60/120 fps.

12.1 5-year-old picture: a flip book 📖

An animation is a **flip book**: many slightly different pictures shown quickly. Flutter draws a new picture every frame (60–120 per second). You just say *"go from small to big in 1 second, starting slow and ending fast"* and Flutter draws all the in-between pictures (**tweening** = "in-betweening").

12.2 Decision tree: which animation tool?

```
Is it a simple property change (size, colour, opacity, padding, alignment) when state changes?
 ├─ YES → IMPLICIT animation (AnimatedContainer, AnimatedOpacity, TweenAnimationBuilder…)
 └─ NO → Does it repeat, need play/pause/reverse, sync with gestures, or chain/stagger?
          ├─ YES → EXPLICIT animation (AnimationController + Transition widgets / AnimatedBuilder)
          └─ Is it a complex designer-made animation (vector, character, illustrations)?
               ├─ YES → Lottie or Rive
               └─ Shared element between screens? → Hero
                  Custom drawing every frame? → CustomPainter + controller
                  Physics feel (spring, fling)? → physics simulations
```

12.3 Implicit animations — "just change the value"

They animate automatically from the old value to the new when you rebuild with a different value.

```dart
class _BoxState extends State<Box> {
  bool _big = false;
  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: () => setState(() => _big = !_big),
      child: AnimatedContainer(
        duration: const Duration(milliseconds: 400),
        curve: Curves.easeInOut,
        width: _big ? 200 : 100,
        height: _big ? 200 : 100,
```

```
        decoration: BoxDecoration(
          color: _big ? Colors.blue : Colors.red,
          borderRadius: BorderRadius.circular(_big ? 32 : 8),
        ),
        onEnd: () => debugPrint('done'),
      ),
    );
  }
}
```

Built-in implicit widgets (`ImplicitlyAnimatedWidget` subclasses): `AnimatedContainer`, `AnimatedOpacity`, `AnimatedPadding`, `AnimatedAlign`, `AnimatedPositioned` (in Stack), `AnimatedPositionedDirectional`, `AnimatedDefaultTextStyle`, `AnimatedPhysicalModel`, `AnimatedRotation`, `AnimatedScale`, `AnimatedSlide`, `AnimatedFractionallySizedBox`, `AnimatedTheme`.

Other "auto" animators:

- **`AnimatedSwitcher`** — cross-fades (or custom transition) when its **child changes** (different type or key!):

```
AnimatedSwitcher(
  duration: const Duration(milliseconds: 300),
  transitionBuilder: (child, anim) => ScaleTransition(scale: anim, child: child),
  child: Text('$_count', key: ValueKey(_count)),  // ❗ key needed: same type Text → otherwise no animation
);
```

- **`AnimatedCrossFade`** — between exactly two children.
- **`AnimatedSize`** — animates its size when child size changes.
- **`AnimatedList`** — insert/remove (Chapter 9).
- **`TweenAnimationBuilder`** — custom implicit animation of *any* value:

```
TweenAnimationBuilder<double>(
  tween: Tween(begin: 0, end: progress),   // animates whenever `end` changes
  duration: const Duration(milliseconds: 600),
  builder: (context, value, child) => CircularProgressIndicator(value: value),
);
```

12.4 Core animation classes (explicit)

Class	What it is (5-year-old)	Tech
`Ticker`	**A metronome ticking every frame**	**Calls a callback once per frame (vsync)**
`TickerProvider` (`vsync`)	**Gives you a metronome that stops when the screen isn't visible**	`SingleTickerProviderStateMixin` **(one controller) /** `TickerProviderStateMixin` **(many)**

Class	What it is (5-year-old)	Tech
`AnimationController`	**The remote control: play, reverse, stop, repeat; value 0.0→1.0 over duration**	`Animation<double>` **you drive**
`Tween<T>`	**Translator: "0..1 means 50px..200px"**	**maps begin→end**
`Curve` / `CurvedAnimation`	**Speed style: slow-start, bounce, elastic**	**non-linear timing**
`Animation<T>`	**A value that changes over time + listeners + status**	`value`, `status`, `addListener`, `addStatusListener`
`AnimatedBuilder` / `*Transition` **widgets**	**Redraw when animation changes**	**rebuild only the animated part**

```
class Pulse extends StatefulWidget {
  const Pulse({super.key, required this.child});
  final Widget child;
  @override
  State<Pulse> createState() => _PulseState();
}

class _PulseState extends State<Pulse> with SingleTickerProviderStateMixin {
  late final AnimationController _controller = AnimationController(
    vsync: this,                                  // ❗ prevents off-screen animations from
wasting battery
    duration: const Duration(milliseconds: 800),
  )..repeat(reverse: true);

  late final Animation<double> _scale = Tween<double>(begin: 1.0, end: 1.15)
      .chain(CurveTween(curve: Curves.easeInOut))
      .animate(_controller);

  @override
  void dispose() {
    _controller.dispose();                        // ❗ always dispose
    super.dispose();
  }

  @override
  Widget build(BuildContext context) =>
      ScaleTransition(scale: _scale, child: widget.child);
}
```

AnimationController API

```
_controller.forward();                // 0 → 1   (returns TickerFuture, awaitable)
_controller.forward(from: 0);
_controller.reverse();                // 1 → 0
_controller.repeat();                 // loop
```

```
_controller.repeat(reverse: true);    // ping-pong
_controller.stop();
_controller.reset();                  // value = lowerBound
_controller.animateTo(0.5, duration: ..., curve: ...);
_controller.fling(velocity: 2.0);     // physics-based
_controller.value = 0.3;              // jump (e.g., dragging)
_controller.isAnimating; _controller.status;  // dismissed, forward, reverse, completed
AnimationController(vsync: this, lowerBound: 0, upperBound: 360);    // custom range
AnimationController.unbounded(vsync: this);                           // for physics
```

Status listener:

```
_controller.addStatusListener((status) {
  if (status == AnimationStatus.completed) _controller.reverse();
  if (status == AnimationStatus.dismissed) _controller.forward();
});
```

Tweens

`Tween<double>`, `ColorTween`, `SizeTween`, `RectTween`, `IntTween`, `AlignmentTween`, `BorderRadiusTween`, `DecorationTween`, `EdgeInsetsTween`, `TextStyleTween`, `ConstantTween`, `TweenSequence` (multi-segment):

```
final shake = TweenSequence<double>([
  TweenSequenceItem(tween: Tween(begin: 0, end: -10), weight: 1),
  TweenSequenceItem(tween: Tween(begin: -10, end: 10), weight: 2),
  TweenSequenceItem(tween: Tween(begin: 10, end: 0), weight: 1),
]).animate(_controller);
```

Two ways to apply: `tween.animate(controller)` → `Animation<T>`; or `tween.evaluate(animation)` / `tween.transform(t)` manually.

Curves

`Curves.linear`, `ease`, `easeIn`, `easeOut`, `easeInOut`, `fastOutSlowIn` (Material standard), `easeOutCubic`, `bounceOut`, `elasticOut`, `decelerate`, `Interval(0.2, 0.6, curve: Curves.easeOut)` (sub-range — staggering), `Threshold`, `Cubic(a,b,c,d)` custom. M3 recommends `Easing.emphasizedDecelerate`, etc. (`Easing` class), and durations `Durations.medium2`.

Transition widgets (efficient, no setState)

`FadeTransition`, `ScaleTransition`, `RotationTransition` (turns), `SlideTransition` (Offset, fraction of child size), `SizeTransition`, `PositionedTransition`, `RelativePositionedTransition`, `DecoratedBoxTransition`, `AlignTransition`, `DefaultTextStyleTransition`, `MatrixTransition` (3.13+).

AnimatedBuilder for anything custom

```
AnimatedBuilder(
  animation: _controller,
  child: const FlutterLogo(size: 100),                // not rebuilt
  builder: (context, child) => Transform(
    alignment: Alignment.center,
    transform: Matrix4.identity()
      ..setEntry(3, 2, 0.001)                          // perspective
      ..rotateY(_controller.value * pi),
    child: child,
  ),
);
```

Or subclass `AnimatedWidget` for reusable animated components.

❌ Anti-pattern: setState on every tick

```
_controller.addListener(() => setState(() {}));   // rebuilds whole widget every frame — avoid in big widgets
```

Use `AnimatedBuilder`/transitions to rebuild **only** the animated subtree.

12.5 Staggered animations

One controller, many tweens with `Interval`s:

```
late final _fade = CurvedAnimation(parent: _c, curve: const Interval(0.0, 0.3, curve: Curves.easeIn));
late final _slide = Tween(begin: const Offset(0, 0.2), end: Offset.zero)
    .animate(CurvedAnimation(parent: _c, curve: const Interval(0.2, 0.6, curve: Curves.easeOut)));
late final _scale = Tween(begin: 0.8, end: 1.0)
    .animate(CurvedAnimation(parent: _c, curve: const Interval(0.5, 1.0, curve: Curves.elasticOut)));

FadeTransition(opacity: _fade, child: SlideTransition(position: _slide, child: ScaleTransition(scale: _scale, child: card)));
```

Staggered list entrance: delay each item by index (e.g., `Interval(i * 0.1, i * 0.1 + 0.5)` clamped) or use `flutter_animate` package:

```
Text('Hello').animate().fadeIn(duration: 300.ms).slideY(begin: 0.2).then(delay: 200.ms).shake();
```

12.6 Hero animations (shared element) 🦸

```
// List page
Hero(tag: 'product-${p.id}', child: Image.network(p.thumb));
// Details page
Hero(tag: 'product-${p.id}', child: Image.network(p.image));
```

- Same **unique tag** on both routes; Flutter flies the widget between positions during route transition.
- Tags must be unique per route (error: *"There are multiple heroes that share the same tag within a subtree"* — common when the same item appears twice, e.g., in two lists on one page).
- `flightShuttleBuilder` to customise the flying widget; `placeholderBuilder`; wrap text heroes in `Material(type: MaterialType.transparency)` to avoid yellow underline / text style glitches.
- `HeroMode(enabled: false)` to disable.
- Works with `Navigator` routes (and go_router pages); for nested navigators the HeroController must be present in that Navigator.

12.7 Page transitions

```
Navigator.push(context, PageRouteBuilder(
  transitionDuration: const Duration(milliseconds: 400),
  pageBuilder: (_, __, ___) => const DetailsPage(),
  transitionsBuilder: (_, animation, secondaryAnimation, child) {
    final offset = Tween(begin: const Offset(1, 0), end: Offset.zero)
        .chain(CurveTween(curve: Curves.easeOutCubic))
        .animate(animation);
    return SlideTransition(position: offset, child: child);
  },
));
```

Global: `PageTransitionsTheme` in ThemeData. The `animations` package (official) provides Material motion patterns: **Container transform** (`OpenContainer`), **Shared axis**, **Fade through**, **Fade**.

12.8 Physics-based animation

```
final simulation = SpringSimulation(
  const SpringDescription(mass: 1, stiffness: 300, damping: 15),
  _controller.value,   // start
  1.0,                 // end
  velocity,            // from drag end: details.velocity.pixelsPerSecond.dy / height
);
_controller.animateWith(simulation);
```

Also: `FrictionSimulation`, `GravitySimulation`, `ClampedSimulation`, `BouncingScrollSimulation`. Use for drag-to-dismiss sheets, card swipe, snapping. `DraggableScrollableSheet` and scroll physics use these internally.

12.9 Lottie & Rive (designer animations)

- **Lottie** (`lottie` package): plays After Effects animations exported as JSON (via Bodymovin). Great for loaders, success checkmarks, onboarding. Control with your own `AnimationController`:

```
Lottie.asset('assets/success.json', controller: _c, onLoaded: (comp) { _c..duration =
comp.duration..forward(); });
```

- **Rive** (`rive` package): interactive vector animations with **state machines** (inputs/triggers), small file sizes, runtime interactivity (e.g., animated login bear following your typing). Rive's newer runtime renders with its own renderer.
- Performance: complex Lottie files can be heavy (many layers, masks); test on low-end devices; prefer Rive for interactive ones.

12.10 CustomPainter animations

```
class WavePainter extends CustomPainter {
  WavePainter(this.animation) : super(repaint: animation);  // ⭐ repaint on tick WITHOUT
rebuilding widgets
  final Animation<double> animation;

  @override
  void paint(Canvas canvas, Size size) {
    final path = Path()..moveTo(0, size.height / 2);
    for (double x = 0; x <= size.width; x++) {
      final y = size.height / 2 + sin((x / size.width * 2 * pi) + animation.value * 2 * pi) * 20;
      path.lineTo(x, y);
    }
    canvas.drawPath(path, Paint()..color = Colors.blue..style = PaintingStyle.stroke..strokeWidth
= 3);
  }

  @override
  bool shouldRepaint(WavePainter old) => false; // repaint driven by `repaint:` listenable
}
CustomPaint(painter: WavePainter(_controller), size: const Size(double.infinity, 100));
```

Passing the animation as `repaint:` makes only the **paint** phase run each frame — cheapest possible animation.

Fragment shaders (`FragmentProgram`, GLSL `.frag` files declared under `shaders:` in pubspec) enable GPU effects (ripples, shimmer, noise).

12.11 Animation performance & best practices ⚡

1. Prefer **Transition widgets / AnimatedBuilder with `child`** over `setState` per frame.
2. Animate **transform and opacity** (cheap: paint/composite) rather than layout properties (width/height/padding cause relayout each frame) when possible: `ScaleTransition` vs `AnimatedContainer(width)`.
3. `Opacity` widget animating → use `FadeTransition` / `AnimatedOpacity` (they use an opacity layer efficiently without rebuilding).
4. Wrap frequently repainting animations in `RepaintBoundary` so the rest of the screen isn't repainted.
5. Avoid animating `BackdropFilter`/blur and `ClipPath` on big areas.
6. Always `dispose()` controllers; use `vsync`.
7. Respect **reduce motion** (`MediaQuery.disableAnimationsOf(context)`).
8. Keep durations short for UI feedback (100–300 ms), longer for large transitions (300–500 ms).
9. Test in **profile mode**; check the raster & UI thread graphs.
10. Shader compilation jank on first run was a Skia problem; Impeller removes it on iOS/Android (Chapter 20/23).

12.12 Common animation bugs

Bug	Cause	Fix
`AnimationController.dispose() called more than once` / `...used after being disposed`	double dispose / using after dispose	single owner, check lifecycle
`_MyState is a SingleTickerProviderStateMixin but multiple tickers were created`	two controllers with Single mixin	use `TickerProviderStateMixin`
Ticker leak error in tests: "A Ticker was active when the widget tree was disposed"	controller not disposed / repeat running	dispose; in tests `pumpAndSettle` won't settle with repeating animations — use `pump(duration)`
AnimatedSwitcher doesn't animate	same widget type without key	add `ValueKey`
Hero tag conflict	duplicate tags in same route	unique tags
Animation stutters on first run (older Skia builds)	shader compilation	Impeller; or SkSL warm-up (legacy)
Animation restarts on every rebuild	controller created in build / `forward()` in build	create in initState; trigger from events or didUpdateWidget

---

12.13 Interview Questions — Animations

Q1. Implicit vs explicit animations? Implicit (AnimatedFoo, TweenAnimationBuilder) animate automatically when target values change, managing their own controller. Explicit use your AnimationController for full control (repeat, reverse, sync to gestures, staggering).

Q2. What is an AnimationController? An `Animation<double>` that produces values (default 0–1) over a duration each frame using a Ticker; provides forward, reverse, repeat, stop, fling, animateTo.

Q3. What is `vsync` / TickerProvider? Supplies Tickers synchronized with the screen refresh and muted when the widget isn't visible (TickerMode), saving resources.

Q4. What is a Tween? Maps the controller's 0–1 range to a value range of type T (double, Color, Offset...).

Q5. What is a Curve? A function mapping linear time to eased progress (easeIn, bounce, elastic, Interval).

Q6. What is AnimatedBuilder and why use `child`? A widget that rebuilds its builder on each animation tick; passing an expensive static subtree as `child` avoids rebuilding it every frame.

Q7. How does Hero animation work? On route transition, Flutter finds Hero widgets with matching tags on both routes, computes their positions, and animates a "flight" in the Overlay between them.

Q8. SingleTickerProviderStateMixin vs TickerProviderStateMixin? Single supports exactly one Ticker (more efficient); the other supports many.

Q9. How to make a staggered animation? One controller with multiple tweens using `Interval` curves for different time slices.

Q10. How do you avoid jank in animations? Animate cheap properties (transform/opacity), avoid rebuilding large trees per frame, RepaintBoundary, avoid expensive effects, test in profile mode.

Q11. What is AnimatedSwitcher? Animates the transition between an old and new child when the child's type or key changes.

Q12. Lottie vs Rive? Lottie plays AE-exported JSON timeline animations; Rive supports interactive state machines with runtime inputs and typically smaller files.

Chapter 13 — Gestures, Focus & Keyboard, CustomPainter and Custom RenderObjects

Goal: Handle every kind of user input (touch, mouse, keyboard, stylus), understand the gesture arena and hit testing, draw anything with `CustomPainter`, and write your own `RenderObject` when widgets aren't enough.

13.1 Pointer events → gestures (two layers)

1. **Pointer events** (raw): `PointerDownEvent`, `PointerMoveEvent`, `PointerUpEvent`, `PointerCancelEvent`, `PointerHoverEvent`, `PointerScrollEvent`. Widget: `Listener`.
2. **Gestures** (semantic): tap, double tap, long press, drag, pan, scale. Widget: `GestureDetector` (uses gesture recognizers).

Hit testing — "who did I touch?" 🎯

When a finger goes down, Flutter performs a **hit test** from the root RenderObject down: each render box checks if the point is inside its bounds and asks its children (top-most painted first). The result is a **list of hit targets** (deepest first, then ancestors). Pointer events are delivered to all of them.

`HitTestBehavior` on `GestureDetector`/`Listener`:

- `deferToChild` (default) — only hit if a child is hit. ⚠️ A `GestureDetector` around a `Container` **without colour** or around empty space won't receive taps in the transparent areas!
- `opaque` — hit anywhere in bounds; blocks targets visually behind.
- `translucent` — hit anywhere in bounds and also lets targets behind receive events.

```
GestureDetector(
  behavior: HitTestBehavior.opaque,   // ✅ fixes "tap on empty space of Row does nothing"
  onTap: () {},
  child: Row(children: const [Icon(Icons.info), SizedBox(width: 50), Text('Info')]),
);
```

13.2 GestureDetector — all callbacks

```
GestureDetector(
  onTap: () {}, onTapDown: (d) {}, onTapUp: (d) {}, onTapCancel: () {},
  onDoubleTap: () {},
  onLongPress: () {}, onLongPressStart: (d) {}, onLongPressMoveUpdate: (d) {}, onLongPressEnd: (d)
{},
  onSecondaryTap: () {},                  // right click
```

```
  onVerticalDragUpdate: (d) {},  onHorizontalDragUpdate: (d) {},
  onPanStart: (d) {}, onPanUpdate: (d) => _offset += d.delta, onPanEnd: (d) => d.velocity,
  onScaleStart: (d) {}, onScaleUpdate: (d) => _scale = d.scale /* also d.rotation, d.focalPoint
*/, onScaleEnd: (d) {},
  onForcePressStart: (d) {},               // 3D touch devices
  child: ...,
);
```

⚠️ You can't use `onPan*` and `onScale*` together (scale is a superset of pan) → error "Having both a pan gesture recognizer and a scale gesture recognizer is redundant; scale is a superset of pan." Use scale only.

⚠️ Adding `onDoubleTap` **delays** `onTap` (Flutter waits ~300ms to see if a second tap comes).

The gesture arena ⚔️ (interview favourite)

When multiple recognizers could claim the same pointer (e.g., a horizontal drag inside a vertical scroll view, tap vs double-tap vs long press), they enter the **gesture arena**:

- Each recognizer watches pointer movement and can **accept** or **reject**.
- The first to accept **wins**; others are rejected. If all but one reject, the last one wins.
- Example: vertical `ListView` + horizontal `PageView` child: a mostly-horizontal move makes the horizontal drag recognizer win (it declares victory after moving past "touch slop", ~18 logical px); a mostly-vertical move lets the scroll win.
- Parent and child GestureDetectors with `onTap`: the **child (deepest)** wins by default because when the pointer goes up, the arena is "swept" and the first recognizer (deepest hit) wins.

Custom arena behaviour: `RawGestureDetector` with custom `GestureRecognizer` subclasses; `EagerGestureRecognizer`; for platform views, `gestureRecognizers` set.

Listener vs GestureDetector

`Listener` gets raw pointer events **without** arena (always receives events) — use for drawing apps, analytics of touches, or custom behaviour. `GestureDetector` recognizes gestures and competes in the arena.

Other gesture widgets

`InkWell` / `InkResponse` (Material ripple), `Draggable` + `DragTarget` (+ `LongPressDraggable`) for drag & drop, `Dismissible`, `InteractiveViewer` (pan/zoom images/maps), `MouseRegion` (hover, cursor: `SystemMouseCursors.click`), `AbsorbPointer` (block children, *absorbs* hit), `IgnorePointer` (children invisible to hit testing; events go to widgets behind), `TapRegion` (detect taps outside a region — used by TextField `onTapOutside`).

```
Draggable<String>(
  data: 'apple',
  feedback: const Material(child: Text('🍎', style: TextStyle(fontSize: 40))),
  childWhenDragging: const Opacity(opacity: 0.3, child: Text('🍎')),
```

```
  child: const Text('🍎'),
);
DragTarget<String>(
  onWillAcceptWithDetails: (details) => details.data == 'apple',
  onAcceptWithDetails: (details) => setState(() => basket.add(details.data)),
  builder: (context, candidates, rejected) => Container(color: candidates.isEmpty ? Colors.grey :
Colors.green, ...),
);
```

13.3 Focus & keyboard (desktop, web, TV, accessibility, hardware keyboards)

Focus tree

Parallel to widgets: `FocusNode`s organised in `FocusScopeNode`s. `Focus` widget creates a node; `FocusScope` groups (e.g., each route has a scope). `FocusManager.instance.primaryFocus` is the current focus.

```
Focus(
  autofocus: true,
  onFocusChange: (hasFocus) => setState(() => _focused = hasFocus),
  onKeyEvent: (node, event) {
    if (event is KeyDownEvent && event.logicalKey == LogicalKeyboardKey.enter) {
      _submit();
      return KeyEventResult.handled;
    }
    return KeyEventResult.ignored;
  },
  child: ...,
);
```

`RawKeyboardListener` / `RawKeyEvent` were **deprecated** (3.18) → use `KeyboardListener` / `Focus.onKeyEvent` with `KeyEvent` (`KeyDownEvent`, `KeyUpEvent`, `KeyRepeatEvent`), `HardwareKeyboard.instance`.

Shortcuts & Actions (the right way for keyboard shortcuts)

```
class SaveIntent extends Intent { const SaveIntent(); }

Shortcuts(
  shortcuts: const <ShortcutActivator, Intent>{
    SingleActivator(LogicalKeyboardKey.keyS, control: true): SaveIntent(),
    SingleActivator(LogicalKeyboardKey.keyS, meta: true): SaveIntent(),   // Cmd+S on macOS
  },
  child: Actions(
    actions: <Type, Action<Intent>>{
      SaveIntent: CallbackAction<SaveIntent>(onInvoke: (_) => _save()),
```

```
    },
    child: const Focus(autofocus: true, child: Editor()),
  ),
);
```

`CallbackShortcuts` is a simpler version. `FocusTraversalGroup` + `OrderedTraversalPolicy` control Tab order. `Actions.invoke(context, const SaveIntent())` from buttons — the same action for menu, shortcut and button.

Text input specifics

`TextInputAction`, `TextInputType`, IME composing, `SystemChannels.textInput`. Hardware keyboard Enter submits only if `onSubmitted` / `textInputAction` set. For Android TV / D-pad navigation, focus highlights matter (`FocusableActionDetector`).

13.4 CustomPainter — your own drawing canvas 🎨

5-year-old: Flutter gives you a **blank paper (Canvas)** and **crayons (Paint)**. You tell it "draw a circle here, a line there".

```
class PieChartPainter extends CustomPainter {
  PieChartPainter({required this.values, required this.colors});
  final List<double> values;
  final List<Color> colors;

  @override
  void paint(Canvas canvas, Size size) {
    final total = values.fold<double>(0, (a, b) => a + b);
    final rect = Offset.zero & size;                    // Rect from (0,0) with size
    var start = -pi / 2;                                 // start at 12 o'clock
    for (var i = 0; i < values.length; i++) {
      final sweep = values[i] / total * 2 * pi;
      canvas.drawArc(rect, start, sweep, true, Paint()..color = colors[i]);
      start += sweep;
    }
    // centre hole → donut
    canvas.drawCircle(size.center(Offset.zero), size.shortestSide * 0.25,
        Paint()..color = Colors.white);
  }

  @override
  bool shouldRepaint(PieChartPainter old) =>
      !listEquals(old.values, values) || !listEquals(old.colors, colors);
}

CustomPaint(
  size: const Size(200, 200),          // used if no child
```

```
  painter: PieChartPainter(values: [30, 50, 20], colors: [Colors.red, Colors.green, Colors.blue]),
  foregroundPainter: null,              // painted on top of child
  child: null,
);
```

Canvas API essentials

```
canvas.drawLine(p1, p2, paint);
canvas.drawRect(rect, paint);
canvas.drawRRect(RRect.fromRectAndRadius(rect, const Radius.circular(12)), paint);
canvas.drawCircle(center, radius, paint);
canvas.drawOval(rect, paint);
canvas.drawArc(rect, startAngle, sweepAngle, useCenter, paint);
canvas.drawPath(path, paint);
canvas.drawImage(image, offset, paint);           // ui.Image
canvas.drawImageRect(image, src, dst, paint);
canvas.drawPoints(PointMode.points, points, paint);
canvas.drawShadow(path, Colors.black, 4, false);
canvas.drawParagraph(paragraph, offset);
// Text with TextPainter:
final tp = TextPainter(text: const TextSpan(text: 'Hi', style: TextStyle(color: Colors.black)),
textDirection: TextDirection.ltr)..layout(maxWidth: size.width);
tp.paint(canvas, Offset(10, 10));
// Transforms & state
canvas.save(); canvas.translate(dx, dy); canvas.rotate(angle); canvas.scale(2); canvas.restore();
canvas.clipRect(rect); canvas.clipPath(path);
canvas.saveLayer(bounds, paint);   // ⚠️ expensive offscreen buffer
```

Paint:

```
final paint = Paint()
  ..color = Colors.blue
  ..style = PaintingStyle.stroke     // or fill
  ..strokeWidth = 4
  ..strokeCap = StrokeCap.round
  ..strokeJoin = StrokeJoin.round
  ..isAntiAlias = true
  ..shader = const LinearGradient(colors: [Colors.blue, Colors.purple]).createShader(rect)
  ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 4)
  ..blendMode = BlendMode.srcOver;
```

Path:

```
final path = Path()
  ..moveTo(0, 0)
  ..lineTo(100, 0)
  ..quadraticBezierTo(150, 50, 100, 100)
```

```
  ..cubicTo(80, 120, 20, 120, 0, 100)
  ..arcToPoint(const Offset(0, 0), radius: const Radius.circular(50))
  ..addOval(Rect.fromCircle(center: Offset(50, 50), radius: 10))
  ..close();
path.contains(point);                  // hit testing
path.computeMetrics();                 // animate drawing along path (extractPath)
```

Clipping widgets with a custom shape: `ClipPath(clipper: MyClipper())` with `CustomClipper<Path>`; `ShapeBorder` subclasses for custom card shapes.

Making CustomPaint interactive

Override `hitTest(Offset position)` in the painter to restrict taps to drawn areas, and wrap in `GestureDetector`. Semantics: override `semanticsBuilder` to describe drawn content for screen readers.

Performance

- `shouldRepaint` must return false when nothing changed.
- Use `repaint:` listenable for animations (Chapter 12).
- Wrap in `RepaintBoundary` if it repaints often or rarely while surroundings change.
- Cache expensive `Path`s/`Picture`s (`PictureRecorder`) between paints.

13.5 Custom RenderObjects — when widgets aren't enough 🛠️

You rarely need this, but senior interviews ask, and libraries use it. Reasons:

- A layout no combination of existing widgets can express efficiently (e.g., a "flow" text layout with inline badges, a custom chat bubble that wraps time stamps intelligently, a custom grid).
- Maximum performance (avoid many widgets).

Leaf example: a render object that paints a coloured box and sizes itself

```
class ColoredSquare extends LeafRenderObjectWidget {
  const ColoredSquare({super.key, required this.color, this.side = 50});
  final Color color;
  final double side;

  @override
  RenderColoredSquare createRenderObject(BuildContext context) =>
      RenderColoredSquare(color: color, side: side);

  @override
  void updateRenderObject(BuildContext context, RenderColoredSquare renderObject) {
    renderObject
      ..color = color
      ..side = side;
```

```
  }
}

class RenderColoredSquare extends RenderBox {
  RenderColoredSquare({required Color color, required double side})
      : _color = color, _side = side;

  Color _color;
  set color(Color value) {
    if (value == _color) return;
    _color = value;
    markNeedsPaint();          // only repaint needed
  }

  double _side;
  set side(double value) {
    if (value == _side) return;
    _side = value;
    markNeedsLayout();         // size changes → relayout
  }

  @override
  void performLayout() {
    size = constraints.constrain(Size.square(_side));   // obey constraints!
  }

  @override
  void paint(PaintingContext context, Offset offset) {
    context.canvas.drawRect(offset & size, Paint()..color = _color);
  }

  @override
  bool hitTestSelf(Offset position) => true;

  @override
  double computeMinIntrinsicWidth(double height) => _side;
  @override
  double computeMaxIntrinsicWidth(double height) => _side;
  @override
  double computeMinIntrinsicHeight(double width) => _side;
  @override
  double computeMaxIntrinsicHeight(double width) => _side;
}
```

Key RenderBox concepts

- `performLayout()` — compute `size` (and lay out children with `child.layout(constraints, parentUsesSize: true)`).

- `sizedByParent` + `computeDryLayout` — when size depends only on constraints; `computeDryLayout` used by intrinsic queries / `LayoutBuilder`.
- `paint(context, offset)` — draw self, then `context.paintChild(child, offset + childOffset)`.
- `hitTest`, `hitTestChildren`, `hitTestSelf`.
- Intrinsic dimension methods (`computeMinIntrinsicWidth` …) — used by `IntrinsicWidth`, `Table`, etc.
- `markNeedsLayout()`, `markNeedsPaint()`, `markNeedsSemanticsUpdate()`, `markNeedsCompositingBitsUpdate()`.
- **ParentData**: each child carries data used by the parent (e.g., `BoxParentData.offset`, `FlexParentData.flex`). Multi-child render objects use `ContainerRenderObjectMixin` + `RenderBoxContainerDefaultsMixin`.
- **Relayout boundary**: if a render object's size doesn't depend on its children changes (tight constraints, `sizedByParent`, or `parentUsesSize: false`), layout changes inside don't propagate upward — a key performance optimisation.
- **Repaint boundary** (`isRepaintBoundary = true`): gets its own layer, so repainting it doesn't repaint ancestors/siblings.

Simpler alternatives first: `CustomSingleChildLayout`, `CustomMultiChildLayout` (with `MultiChildLayoutDelegate` + `LayoutId`), `Flow` (with `FlowDelegate`, paint-time transforms — great for animated menus).

13.6 Interview Questions — Gestures & painting

Q1. GestureDetector vs Listener? GestureDetector recognizes high-level gestures and participates in the gesture arena; Listener gives raw pointer events without disambiguation.

Q2. What is the gesture arena? The mechanism resolving conflicts when multiple gesture recognizers track the same pointer; recognizers accept/reject, and one wins.

Q3. Why doesn't my GestureDetector receive taps on empty areas? Default `HitTestBehavior.deferToChild` only registers hits on painted children; use `HitTestBehavior.opaque` or give a background colour.

Q4. AbsorbPointer vs IgnorePointer? AbsorbPointer consumes hits itself (blocking widgets behind it and its children); IgnorePointer makes itself and children invisible to hit testing so events pass to widgets behind.

Q5. How do you implement drag and drop? `Draggable` / `LongPressDraggable` with `data` + `feedback`, and `DragTarget` with accept callbacks; or `ReorderableListView` for lists.

Q6. How do you implement keyboard shortcuts? Shortcuts + Actions with Intents (or CallbackShortcuts), within a focused subtree.

Q7. What is CustomPainter and shouldRepaint? API to draw directly on a Canvas; shouldRepaint tells Flutter whether a new painter instance needs to repaint compared to the old one.

Q8. When would you write a custom RenderObject? For layouts/painting not achievable efficiently with existing widgets or delegates, or for performance-critical components.

Q9. markNeedsLayout vs markNeedsPaint? Layout marks the render object for relayout (and repaint); paint only schedules repainting, cheaper.

Q10. What is a repaint boundary? A render object with its own compositing layer, isolating repaints so they don't spread to ancestors/siblings (`RepaintBoundary` widget).

Q11. What's a relayout boundary? A render object whose size doesn't depend on child changes, stopping layout invalidation from propagating to its parent.

Q12. How does hit testing work? From the root, render objects check if the position is inside their bounds, recursively testing children in reverse paint order; hits are added to a HitTestResult path that receives pointer events.

PART III

State Management

Chapter 14 — State Management, Part 1: Concepts, setState, InheritedWidget, ValueNotifier, ChangeNotifier & Provider

Goal: Understand *what* state is, the different kinds, *where* it should live, and master Flutter's built-in tools and Provider. This is the foundation for Riverpod and Bloc (Chapter 15).

14.1 What is "state"? 🧠

5-year-old: State is **anything that can change and that the screen needs to remember**: the number on a counter, whether you're logged in, items in your cart, which tab is selected, the text you typed.

Grown-up (Flutter docs): *"Whatever data you need in order to rebuild your UI at any moment in time."* Since **UI = f(state)**, managing state = deciding where data lives, who can change it, and how the UI learns about changes.

Two kinds of state

	Ephemeral (local/UI) state	App (shared) state
Examples	current page in PageView, animation progress, is the password visible, selected tab, text field focus	logged-in user, cart, preferences, notifications, cached server data
Used by	one widget	many widgets/screens
Survives navigation?	No need	Yes
Tool	`setState`, `ValueNotifier`	Provider, Riverpod, Bloc, ...

There's no hard rule; start local and **lift state up** when others need it.

Other useful classifications (senior-level)

- **Server state** (remote data cache: loading/error/data, refetching, staleness) vs **client state** (UI selections, form inputs). Mixing them is a common source of bugs; Riverpod's `FutureProvider` / `AsyncNotifier` or dedicated caching layers handle server state well.
- **Derived state** — computed from other state (cart total from cart items). Don't store it separately — compute or memoize, or it gets out of sync.
- **Persistent state** — must survive restarts (tokens, settings) → storage (Chapter 17).
- **Navigation state** — current route stack (router).

14.2 Principles of good state management (interview-ready)

1. **Single source of truth** — each piece of data lives in exactly one place.

2. **Unidirectional data flow** — events go up (user action → logic), state flows down (logic → UI).
3. **Immutability** — create new state objects instead of mutating (easy change detection, no hidden mutations, time-travel debugging).
4. **Separation of concerns** — UI renders; business logic lives in testable classes without `BuildContext`.
5. **Minimal rebuilds** — only widgets depending on changed data rebuild.
6. **Testability** — logic testable without widgets; dependencies injectable.
7. **Predictability** — state changes happen in known places.

14.3 `setState` — local state

Covered in Chapter 6. Use for ephemeral state inside one widget.

Lifting state up

When two siblings need the same state, move it to the **nearest common ancestor** and pass down **data + callbacks**:

```
class CartPage extends StatefulWidget { ... }
class _CartPageState extends State<CartPage> {
  final List<Item> _items = [];
  void _add(Item i) => setState(() => _items.add(i));
  @override
  Widget build(BuildContext context) => Column(children: [
    CartBadge(count: _items.length),       // data down
    ProductList(onAdd: _add),              // callbacks up
  ]);
}
```

Problem at scale: **prop drilling** — passing data through many layers that don't use it. Solutions: InheritedWidget-based tools (Provider, Riverpod, Bloc).

14.4 `ValueNotifier` & `ValueListenableBuilder` — the simplest reactive box 📦🔔

```
final counter = ValueNotifier<int>(0);   // a box that rings a bell when its value changes

ValueListenableBuilder<int>(
  valueListenable: counter,
  builder: (context, value, child) => Text('$value'),  // only this rebuilds
);
counter.value++;          // rings the bell (only if value != old value, using ==)
counter.dispose();        // when done
```

⚠️ Mutating a list inside a ValueNotifier does **not** notify:

```
final items = ValueNotifier<List<String>>([]);
items.value.add('x');                    // ❌ no notification (same list instance)
items.value = [...items.value, 'x'];     // ✅ new list → notifies
```

`ValueNotifier` + `InheritedNotifier` / `get_it` is enough for many small apps.

14.5 `ChangeNotifier` — observable class 📣

```
class CartModel extends ChangeNotifier {
  final List<Item> _items = [];
  UnmodifiableListView<Item> get items => UnmodifiableListView(_items);  // expose read-only
  int get totalPrice => _items.fold(0, (sum, i) => sum + i.price);       // derived

  void add(Item item) {
    _items.add(item);
    notifyListeners();      // 🔔 tell listeners to rebuild
  }

  void removeAll() {
    _items.clear();
    notifyListeners();
  }
}
```

- `addListener` / `removeListener`, `dispose()`, `hasListeners`.
- `notifyListeners()` after disposal → error *"A CartModel was used after being disposed."*
- `ListenableBuilder(listenable: cart, builder: ...)` rebuilds on notifications (Flutter 3.10+; `AnimatedBuilder` works too).
- Calling `notifyListeners` **during build** → "setState() or markNeedsBuild() called during build" error.
- `ChangeNotifier` is O(1) to add/remove listeners and O(N) to dispatch — fine for UI models.

Drawbacks: mutable, one big notifier rebuilds all listeners (use `select` / `Selector`), notify-forgetting bugs, async state (loading/error) must be modelled manually.

The **official Flutter architecture guide** (docs.flutter.dev/app-architecture) recommends **MVVM**: ViewModels extend `ChangeNotifier`, Views listen with `ListenableBuilder`, repositories handle data, and a "Command" pattern for actions (Chapter 21).

14.6 InheritedWidget / InheritedNotifier as DIY DI

```
class CartScope extends InheritedNotifier<CartModel> {
  const CartScope({super.key, required CartModel cart, required super.child}) : super(notifier:
cart);
```

```
  static CartModel of(BuildContext context) =>
      context.dependOnInheritedWidgetOfExactType<CartScope>()!.notifier!;
}
// Above the app:
CartScope(cart: CartModel(), child: const MyApp());
// Anywhere below:
final cart = CartScope.of(context);   // rebuilds when cart notifies
```

That's essentially what **Provider** does with nicer APIs and lifecycle handling.

14.7 Provider 📦➡️

`provider` (by Remi Rousselet) — "a wrapper around InheritedWidget to make them easier to use and more reusable". It was the Flutter team's recommended approach for years and is still extremely common in company codebases.

Setup

```
void main() {
  runApp(
    MultiProvider(
      providers: [
        Provider<ApiClient>(create: (_) => ApiClient()),                          // plain object
(DI)
        ChangeNotifierProvider(create: (_) => AuthModel()),                       // owns &
disposes it
        ChangeNotifierProxyProvider<AuthModel, CartModel>(                        // depends on
another
          create: (_) => CartModel(),
          update: (_, auth, cart) => cart!..userId = auth.userId,
        ),
        FutureProvider<Config?>(create: (_) => loadConfig(), initialData: null),
        StreamProvider<ConnectivityStatus>(create: (_) => connectivity.stream, initialData:
ConnectivityStatus.unknown),
      ],
      child: const MyApp(),
    ),
  );
}
```

Reading

```
// 1. watch: listen & rebuild when it changes (use in build)
final cart = context.watch<CartModel>();

// 2. read: get without listening (use in callbacks, NOT in build to get values for display)
```

```
onPressed: () => context.read<CartModel>().add(item),

// 3. select: rebuild only when a specific part changes
final count = context.select<CartModel, int>((c) => c.items.length);

// Older equivalents:
Provider.of<CartModel>(context);               // listen: true (watch)
Provider.of<CartModel>(context, listen: false); // read

// Widgets:
Consumer<CartModel>(
  builder: (context, cart, child) => Text('Total: ${cart.totalPrice}'),
  child: const ExpensiveStaticPart(),   // not rebuilt
);
Selector<CartModel, int>(
  selector: (_, cart) => cart.items.length,
  builder: (_, count, __) => Badge(label: Text('$count')),
);
```

`create` vs `.value` — VERY common bug & interview question

```
// ✅ Creating a NEW object → use create (Provider owns lifecycle & disposes ChangeNotifier)
ChangeNotifierProvider(create: (_) => CartModel(), child: ...);

// ✅ Providing an EXISTING object (e.g., in a list item or passing to a new route) → use .value
ChangeNotifierProvider.value(value: existingCart, child: ...);

// ❌ BAD: .value with a new instance inside build → recreated every rebuild, state lost, leaks
ChangeNotifierProvider.value(value: CartModel(), child: ...);
// ❌ BAD: create with an existing instance → Provider will DISPOSE it when removed, though you
still use it elsewhere
```

Common Provider errors

- `ProviderNotFoundException` / **"Could not find the correct Provider above this Widget"**:
  1. Provider is below the widget reading it (e.g., read in the same `build` that creates it → use `Builder` or `Consumer`).
  2. Reading in a **new route** — routes are children of `Navigator` (under MaterialApp), not of the page that pushed them; providers declared in a page aren't visible in pushed routes → move provider above MaterialApp or pass `.value` into the new route.
  3. Wrong generic type (`ChangeNotifierProvider<CartModel>` vs reading `context.watch<Cart>()`), or imported from two different paths (relative vs package import making two "different" types!).
  4. Hot reload after adding a provider → do a hot restart.
- **"Tried to listen to a value exposed with provider, from outside of the widget tree"** → used `watch`/`Provider.of` with listen:true in a callback; use `read`.

- **"setState() or markNeedsBuild() called during build"** → calling a method that `notifyListeners()` from build/initState synchronously → use `addPostFrameCallback` or `Future.microtask`.

Provider pros/cons

✅ Simple, built on Flutter primitives, official-ish, lots of examples, good for small/medium apps. ❌ Depends on `BuildContext` (can't easily access outside widgets), runtime errors (ProviderNotFound) instead of compile-time safety, combining/depending providers is clunky (ProxyProvider), same-type conflicts (two providers of the same type: the nearest wins).

The author created **Riverpod** to solve these issues (Chapter 15).

14.8 Modelling async state correctly

Every screen loading remote data has (at least) these states: **initial/idle, loading, data (success), empty, error** — and sometimes "refreshing with old data", "loading more".

❌ Fragile: separate booleans that allow impossible combinations:

```
bool isLoading; String? error; List<Item>? data;   // isLoading && error both true??
```

✅ Use a sealed class (Dart 3):

```
sealed class LoadState<T> { const LoadState(); }
final class Idle<T> extends LoadState<T> { const Idle(); }
final class Loading<T> extends LoadState<T> { const Loading({this.previous}); final T? previous; }
final class Loaded<T> extends LoadState<T> { const Loaded(this.data); final T data; }
final class Failed<T> extends LoadState<T> { const Failed(this.error, {this.previous}); final
Object error; final T? previous; }

Widget build(BuildContext context) => switch (state) {
  Idle() => const SizedBox.shrink(),
  Loading(previous: final p?) => ItemsList(p, refreshing: true),
  Loading() => const Center(child: CircularProgressIndicator()),
  Loaded(:final data) when data.isEmpty => const EmptyView(),
  Loaded(:final data) => ItemsList(data),
  Failed(:final error) => ErrorView(error: error, onRetry: reload),
};
```

Riverpod's `AsyncValue` and Bloc's state classes implement this idea.

14.9 Comparison table (overview — details in Chapter 15)

Approach	Boilerplate	Learning curve	Testability	Scales to big apps	Context needed	Notes
setState	none	trivial	low (logic in UI)	❌	yes	local state only
ValueNotifier/ChangeNotifier + InheritedNotifier/ListenableBuilder	low	low	good	medium	yes	official MVVM guide uses ChangeNotifier
Provider	low	low	good	medium–large	yes	very common legacy/current
Riverpod	low–medium	medium	excellent	✅	no (uses ref)	compile-safe, async-first, codegen optional
Bloc/Cubit	medium–high	medium	excellent	✅	yes (BlocProvider)	events/states, traceable, enterprise favourite
GetX	very low	low	weak	❌ (debated)	no	all-in-one, magic, frowned upon in many teams
MobX	medium (codegen)	medium	good	✅	no	observables/reactions
Redux	high	high	excellent	✅	yes	single store, rarely chosen for new apps
signals	low	low	good	medium	no	fine-grained reactivity (signals package)

What to answer when asked "Which state management is best?": *"There's no single best; it depends on team size, app complexity, and conventions. For small apps setState/ValueNotifier/Provider is enough. For large apps I prefer Riverpod or Bloc because they enforce separation, are testable, and handle async well. Consistency across the codebase matters more than the choice itself."* Then give concrete reasons for your preference.

14.10 Interview Questions — State management foundations

Q1. What is state in Flutter? Data needed to rebuild the UI at any moment.

Q2. Ephemeral vs app state? Ephemeral is local to one widget (setState); app state is shared/persistent across parts of the app (state management solutions).

Q3. What is "lifting state up"? Moving state to the closest common ancestor of widgets that need it, passing data down and callbacks up.

Q4. What is prop drilling and how to avoid it? Passing data through many intermediate widgets; avoid with InheritedWidget-based solutions (Provider, Riverpod, Bloc) or DI.

Q5. What is ChangeNotifier? A class providing listener registration and `notifyListeners()` to signal changes; used with ListenableBuilder/Provider.

Q6. ValueNotifier vs ChangeNotifier? ValueNotifier holds a single value and notifies when the value is replaced with a non-equal one; ChangeNotifier is a general base where you call notifyListeners manually.

Q7. context.watch vs context.read vs context.select? watch listens and rebuilds; read gets once without listening (callbacks); select listens to a derived value and rebuilds only when it changes.

Q8. Provider `create` vs `.value`? create for new instances (provider owns/disposes), .value for existing instances (no disposal) — misuse causes state loss or disposed-object errors.

Q9. Why does ProviderNotFoundException occur? The reading context isn't a descendant of the provider (same build method, different route, wrong type/import).

Q10. Why is immutable state preferred? Easy change detection (==), predictable updates, no accidental shared mutation, simpler debugging/time travel, safe across isolates.

Q11. How do you avoid unnecessary rebuilds in Provider? `select`/`Selector`, `Consumer` placed low, `child` param, splitting models, const widgets.

Q12. How do you represent loading/error/data states? Sealed classes/union types (or AsyncValue), avoiding inconsistent boolean flags.

Chapter 15 — State Management, Part 2: Riverpod, Bloc/Cubit, GetX, MobX, Redux, Signals

Goal: Become fluent in the two dominant enterprise solutions — **Riverpod** and **Bloc** — and know enough about the others to discuss trade-offs and maintain legacy code.

Versions: this chapter reflects **Riverpod 3.x** (released Sept 2025) and **flutter_bloc 9.x**. Where older APIs are still common in codebases, they're noted.

Part A — Riverpod 🌊

15.1 What and why

5-year-old: Riverpod is a **magic backpack of ingredients**. Any widget can reach into the backpack (`ref`) and grab an ingredient (`provider`). If an ingredient changes (the milk goes sour), everyone using it is told automatically. Ingredients can be made from other ingredients (pancakes need milk + eggs). And you don't need to know where in the house (widget tree) the backpack is.

Riverpod (an anagram of *Provider*), by the same author as Provider, fixes Provider's problems:

- **No dependency on BuildContext/widget tree position** to find providers → no `ProviderNotFoundException`.
- **Compile-time safety** — providers are global *declarations* (final variables), but their **state** lives in a `ProviderContainer` (inside `ProviderScope`), so they are not global singletons and are fully testable/overridable.
- **Multiple providers of the same type** — fine (they're distinct variables).
- **Async-first:** `FutureProvider`, `StreamProvider`, `AsyncNotifier`, and `AsyncValue` (loading/data/error) built in.
- **Auto-dispose**, **families** (parameterised providers), caching, **combining providers** reactively with `ref.watch`.
- Works outside Flutter (pure Dart).

15.2 Setup

```
dependencies:
  flutter_riverpod: ^3.0.0
  riverpod_annotation: ^3.0.0     # for codegen (optional)
dev_dependencies:
```

```
  riverpod_generator: ^3.0.0
  build_runner: ^2.4.0
  riverpod_lint: ^3.0.0          # (lints; works via the analyzer plugin system in recent
versions)
```

```
void main() {
  runApp(const ProviderScope(child: MyApp()));   // ❗ required at the root
}
```

15.3 Provider types

Provider	For	Example
`Provider`	**Read-only computed values / services / DI**	**repository, filtered list**
`FutureProvider`	**Async read-only value (one-shot fetch)**	**user profile**
`StreamProvider`	**Stream of values**	**auth state changes, websockets**
`NotifierProvider`	**Synchronous mutable state with methods**	**counter, filters, cart**
`AsyncNotifierProvider`	**Async state with methods (load + mutate)**	**todo list from API**
`StreamNotifierProvider`	**Stream-based state with methods**	**chat**
Legacy: `StateProvider`, `StateNotifierProvider`, `ChangeNotifierProvider`	**old code**	**In Riverpod 3 moved to** `package:flutter_riverpod/legacy.dart`

Manual (no codegen) syntax

```
// Service / DI
final apiClientProvider = Provider<ApiClient>((ref) {
  final client = ApiClient(baseUrl: ref.watch(configProvider).apiUrl);
  ref.onDispose(client.close);
  return client;
});

// Notifier (sync)
class Counter extends Notifier<int> {
  @override
  int build() => 0;                        // initial state
  void increment() => state++;             // assigning `state` notifies listeners
}
final counterProvider = NotifierProvider<Counter, int>(Counter.new);

// FutureProvider (auto-dispose when nobody listens)
final userProvider = FutureProvider.autoDispose<User>((ref) async {
  final api = ref.watch(apiClientProvider);
```

```
  return api.fetchMe();
});

// Family (parameter)
final productProvider = FutureProvider.autoDispose.family<Product, String>((ref, id) async {
  return ref.watch(apiClientProvider).fetchProduct(id);
});

// AsyncNotifier
class TodoList extends AsyncNotifier<List<Todo>> {
  @override
  Future<List<Todo>> build() => ref.watch(todoRepoProvider).fetchAll();

  Future<void> add(String title) async {
    final repo = ref.read(todoRepoProvider);
    state = const AsyncLoading();
    state = await AsyncValue.guard(() async {
      await repo.add(title);
      return repo.fetchAll();
    });
  }

  Future<void> toggle(String id) async {
    final previous = state.value ?? [];
    // Optimistic update
    state = AsyncData([
      for (final t in previous) if (t.id == id) t.copyWith(done: !t.done) else t,
    ]);
    try {
      await ref.read(todoRepoProvider).toggle(id);
    } catch (e) {
      state = AsyncData(previous);   // rollback the optimistic change
      rethrow;                       // let the UI show a snackbar (catch at call site)
    }
  }
}
final todoListProvider = AsyncNotifierProvider<TodoList, List<Todo>>(TodoList.new);
```

Tip: When a provider **rebuilds itself** (e.g., after `ref.invalidate` or a watched dependency changes), Riverpod keeps the previous value available while loading (`state.value`, `isRefreshing` / `isReloading`), so the UI can show old data with a small progress indicator. When you assign `AsyncLoading()` manually, decide explicitly whether to keep old data (e.g., keep a local copy) — helper names for this differ between Riverpod versions, so check the docs for yours.

Code generation syntax (recommended by Riverpod docs)

```
import 'package:riverpod_annotation/riverpod_annotation.dart';
part 'todo_providers.g.dart';

@riverpod                                   // auto-dispose by default
Future<User> user(Ref ref) => ref.watch(apiClientProvider).fetchMe();   // → userProvider

@riverpod
Future<Product> product(Ref ref, String id) =>                          // family automatically
    ref.watch(apiClientProvider).fetchProduct(id);                        // → productProvider(id)

@Riverpod(keepAlive: true)                  // not auto-disposed
ApiClient apiClient(Ref ref) => ApiClient();

@riverpod
class TodoList extends _$TodoList {         // → todoListProvider
  @override
  Future<List<Todo>> build() => ref.watch(todoRepoProvider).fetchAll();
  Future<void> add(String title) async { ... }
}
```

Run `dart run build_runner watch -d`.

15.4 Consuming providers in widgets

```
class TodoPage extends ConsumerWidget {           // instead of StatelessWidget
  const TodoPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final todos = ref.watch(todoListProvider);    // rebuilds on change

    // Side effects (snackbars, navigation) — NOT in build logic, use listen:
    ref.listen(todoListProvider, (previous, next) {
      if (next is AsyncError) {
        ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('${next.error}')));
      }
    });

    return switch (todos) {
      AsyncData(:final value) => TodoListView(value),
      AsyncError(:final error) => ErrorView(error, onRetry: () =>
ref.invalidate(todoListProvider)),
      _ => const Center(child: CircularProgressIndicator()),
    };
    // or: todos.when(data: ..., error: (e, st) => ..., loading: () => ...)
  }
```

```
}

// Calling methods: read the notifier (don't watch in callbacks)
onPressed: () => ref.read(todoListProvider.notifier).add('Buy milk'),

// Rebuild only on part of state
final doneCount = ref.watch(todoListProvider.select((s) => s.value?.where((t) => t.done).length ??
0));

// Stateful version
class EditPage extends ConsumerStatefulWidget { ... }
class _EditPageState extends ConsumerState<EditPage> {
  @override
  void initState() {
    super.initState();
    // ref is available in ConsumerState; use ref.read here, not watch
  }
}

// Local scope rebuild:
Consumer(builder: (context, ref, child) => Text('${ref.watch(counterProvider)}'));
```

`ref.watch` vs `ref.read` vs `ref.listen` (most asked Riverpod question)

	When	Rebuilds ?	Where
`ref.watch(p)`	**Need the value to build UI / compute a provider**	✅	`build` **of widgets and providers**
`ref.read(p)`	**One-off read, typically to call a method**	❌	**callbacks (onPressed), notifier methods**
`ref.listen(p, cb)`	**React to changes with side effects (navigate, snackbar, log)**	❌ **(calls cb)**	**build of widgets (Riverpod manages lifecycle), providers**

Rule: don't `ref.read` inside `build` to get a value for display — UI won't update.

Other ref methods: `ref.invalidate(p)` (reset & recompute lazily), `ref.refresh(p)` (invalidate + read immediately, returns new value), `ref.onDispose`, `ref.onCancel` / `onResume` (listeners leaving/coming back), `ref.keepAlive()` (prevent auto-dispose, returns a link you can `close()`), `ref.mounted` (Riverpod 3: check after `await` in notifiers, like widget `mounted`).

15.5 Combining providers (reactive dependency graph)

```
@riverpod
List<Todo> filteredTodos(Ref ref) {
  final filter = ref.watch(todoFilterProvider);
  final todos = ref.watch(todoListProvider).value ?? const [];
  return switch (filter) {
    Filter.all => todos,
    Filter.done => todos.where((t) => t.done).toList(),
```

```
    Filter.active => todos.where((t) => !t.done).toList(),
  };
}
```

When the filter or todos change, `filteredTodos` recomputes, and widgets watching it rebuild — if the result is different (`==`). Search with debounce & cancellation:

```
@riverpod
Future<List<Result>> search(Ref ref, String query) async {
  // Debounce: if the provider is disposed/rebuilt within 300ms, abort
  var cancelled = false;
  ref.onDispose(() => cancelled = true);
  await Future<void>.delayed(const Duration(milliseconds: 300));
  if (cancelled) throw Exception('cancelled');

  final cancelToken = CancelToken();          // dio
  ref.onDispose(cancelToken.cancel);
  return ref.watch(apiProvider).search(query, cancelToken: cancelToken);
}
```

15.6 Auto-dispose, keepAlive, caching

- **autoDispose**: state destroyed when no longer listened to (e.g., leaving a screen) → no memory leak, fresh data next time.
- `ref.keepAlive()` to cache e.g. after successful load; combine with a timer to expire cache:

```
final link = ref.keepAlive();
final timer = Timer(const Duration(minutes: 5), link.close);
ref.onDispose(timer.cancel);
```

15.7 Riverpod 3 highlights (know for interviews in 2026)

- **Automatic retry** of failing providers (exponential backoff by default; configurable / disable with `retry:`).
- `Ref.mounted` to check whether the notifier/provider is still alive after async gaps.
- Listeners are **paused** when widgets aren't visible (e.g., under another route / `TickerMode` off) to save resources.
- **Offline persistence** (experimental) and **mutations** (experimental) APIs.
- `AsyncValue` is sealed with `AsyncData`, `AsyncError`, `AsyncLoading`; `valueOrNull` removed — `.value` returns null during errors/loading without previous data.
- `StateProvider`, `StateNotifierProvider`, `ChangeNotifierProvider` moved to `legacy` imports.
- Family notifiers receive arguments through the **constructor** (`FamilyNotifier` removed).
- Notifiers are recreated on provider rebuild.
- Unified `Ref` (no more `FutureProviderRef` etc. subtypes in codegen).
- `ProviderContainer.test()` helper for tests.

15.8 Testing Riverpod

```
test('counter increments', () {
  final container = ProviderContainer.test();       // auto-disposed after test (Riverpod 3)
  expect(container.read(counterProvider), 0);
  container.read(counterProvider.notifier).increment();
  expect(container.read(counterProvider), 1);
});

test('todo list loads from repository', () async {
  final container = ProviderContainer.test(overrides: [
    todoRepoProvider.overrideWithValue(FakeTodoRepo([Todo(id: '1', title: 'A')])),
  ]);
  final todos = await container.read(todoListProvider.future);
  expect(todos.single.title, 'A');
});

testWidgets('shows todos', (tester) async {
  await tester.pumpWidget(ProviderScope(
    overrides: [todoListProvider.overrideWith(() => FakeTodoList())],
    child: const MaterialApp(home: TodoPage()),
  ));
  await tester.pump();
  expect(find.text('A'), findsOneWidget);
});
```

15.9 Riverpod best practices & pitfalls

- Keep providers **small and focused**; derive with `ref.watch`.
- Don't put `BuildContext` or widgets in providers.
- Use `ref.listen` for one-time side effects; never navigate inside `build` directly.
- Don't call `ref.read` on autoDispose providers in callbacks expecting state to persist — if nobody watches it, it may be disposed immediately after read. Watch it somewhere or keepAlive.
- Avoid `ref.watch` inside notifier methods (use `ref.read`); `ref.watch` belongs in `build`.
- Avoid passing complex objects as family parameters unless they implement `==` / `hashCode` (records, freezed, primitives) — otherwise each call creates a new provider → infinite loading/leaks.
- "Modifying a provider while the widget tree was building" error → don't mutate state during build/initState; use `Future.microtask` or move to event.
- `ProviderScope` overrides for flavors/environments and tests.
- Use `riverpod_lint` to catch mistakes.

Part B — Bloc & Cubit 🧱

15.10 The idea

5-year-old: A **Bloc** is a **vending machine** 🥤. You press buttons (**events**). The machine does its internal work and drops out a snack (**state**). You can't reach inside; you can only press buttons and look at what comes out. Everyone can see what was pressed and what came out — great for debugging.

A **Cubit** is a **simpler vending machine** where instead of pressing labelled buttons (events), you directly call functions (`increment()`), and it emits new states.

```
UI ──(event)──► Bloc ──(state stream)──► UI
                 │
                 ▼
            Repository → API/DB
```

- Based on **Streams** (Bloc = Business Logic Component, pattern introduced by Google in 2018; library by Felix Angelov/VGV).
- Enforces separation: UI ↔ Bloc ↔ Repository ↔ Data providers.
- Every state change is traceable (BlocObserver: onEvent, onChange, onTransition, onError) → excellent for large teams and debugging.

15.11 Cubit

```dart
class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);                // initial state
  void increment() => emit(state + 1);
  void decrement() => emit(state - 1);
}
```

15.12 Bloc with events and states (Dart 3 sealed classes)

```dart
// Events
sealed class LoginEvent {}
final class LoginEmailChanged extends LoginEvent { LoginEmailChanged(this.email); final String email; }
final class LoginPasswordChanged extends LoginEvent { LoginPasswordChanged(this.password); final String password; }
final class LoginSubmitted extends LoginEvent {}

// State (single class with status — common pattern)
enum LoginStatus { initial, submitting, success, failure }
```

```
final class LoginState extends Equatable {
  const LoginState({this.email = '', this.password = '', this.status = LoginStatus.initial,
this.error});
  final String email;
  final String password;
  final LoginStatus status;
  final String? error;

  bool get isValid => email.contains('@') && password.length >= 8;

  LoginState copyWith({String? email, String? password, LoginStatus? status, String? error}) =>
LoginState(
        email: email ?? this.email,
        password: password ?? this.password,
        status: status ?? this.status,
        error: error,
      );

  @override
  List<Object?> get props => [email, password, status, error];
}

// Bloc
class LoginBloc extends Bloc<LoginEvent, LoginState> {
  LoginBloc({required AuthRepository authRepository})
      : _auth = authRepository,
        super(const LoginState()) {
    on<LoginEmailChanged>((event, emit) => emit(state.copyWith(email: event.email)));
    on<LoginPasswordChanged>((event, emit) => emit(state.copyWith(password: event.password)));
    on<LoginSubmitted>(_onSubmitted, transformer: droppable()); // ignore extra taps while
submitting (bloc_concurrency)
  }

  final AuthRepository _auth;

  Future<void> _onSubmitted(LoginSubmitted event, Emitter<LoginState> emit) async {
    if (!state.isValid) return;
    emit(state.copyWith(status: LoginStatus.submitting));
    try {
      await _auth.logIn(email: state.email, password: state.password);
      emit(state.copyWith(status: LoginStatus.success));
    } on AuthException catch (e) {
      emit(state.copyWith(status: LoginStatus.failure, error: e.message));
    }
  }
}
```

Bloc 8+ removed `mapEventToState` (async generator style) in favour of `on<Event>` handlers. If you see `mapEventToState` in a codebase, it's Bloc ≤7.

Event transformers (`bloc_concurrency`)

By default (Bloc 8+), events are processed **concurrently**. Transformers:

- `sequential()` — one at a time in order (queue).
- `droppable()` — ignore new events while one is processing (prevent double submit).
- `restartable()` — cancel the running handler, start the new one (search-as-you-type).
- `concurrent()` — default. Debounce: custom transformer `(events, mapper) => events.debounceTime(300.ms).switchMap(mapper)` with rxdart.

Emitting from streams

```
on<TodosSubscriptionRequested>((event, emit) async {
  await emit.forEach<List<Todo>>(
    _repo.watchTodos(),
    onData: (todos) => state.copyWith(todos: todos, status: Status.success),
    onError: (_, __) => state.copyWith(status: Status.failure),
  );
});
```

⚠️ Never `emit` after the handler completed (e.g., inside a `stream.listen` callback) → error "emit was called after an event handler completed normally". Use `emit.forEach` / `emit.onEach` or add an internal event.

Bloc vs Cubit

Cubit	Bloc
Methods → emit	**Events → handlers → emit**
Less boilerplate	**More boilerplate**
Can't use event transformers	**Debounce/throttle/droppable/restartable**
Traceability: state changes only	**Full event → state transitions**
Good for simple features	**Good for complex flows, audit trails**
Start with Cubit; refactor to Bloc when you need event processing control.	

15.13 Flutter widgets (flutter_bloc)

```
// Provide (creates & closes automatically)
BlocProvider(
  create: (context) => LoginBloc(authRepository: context.read<AuthRepository>()),
  child: const LoginView(),
);
MultiBlocProvider(providers: [...], child: ...);
RepositoryProvider(create: (_) => AuthRepository(), child: ...);   // DI for repositories
BlocProvider.value(value: existingBloc, child: ...);              // existing instance (e.g., to
new route) — not closed automatically
```

```
// Build UI from state
BlocBuilder<LoginBloc, LoginState>(
  buildWhen: (prev, curr) => prev.status != curr.status,   // rebuild filter
  builder: (context, state) => state.status == LoginStatus.submitting
      ? const CircularProgressIndicator()
      : FilledButton(onPressed: () => context.read<LoginBloc>().add(LoginSubmitted()), child:
const Text('Login')),
);

// Side effects (navigation, snackbars) — called once per state change
BlocListener<LoginBloc, LoginState>(
  listenWhen: (prev, curr) => prev.status != curr.status,
  listener: (context, state) {
    if (state.status == LoginStatus.success) context.go('/home');
    if (state.status == LoginStatus.failure) {
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(state.error ?? 'Login
failed')));
    }
  },
  child: ...,
);

BlocConsumer<LoginBloc, LoginState>(listener: ..., builder: ...);   // both
BlocSelector<LoginBloc, LoginState, bool>(selector: (s) => s.isValid, builder: (_, isValid) =>
...);
MultiBlocListener(listeners: [...], child: ...);

// Extensions
context.read<LoginBloc>();                          // no listen (callbacks)
context.watch<LoginBloc>().state;                   // listen (build)
context.select((LoginBloc b) => b.state.email);     // listen to part
```

15.14 Testing Bloc (`bloc_test` + `mocktail`)

```
class MockAuthRepository extends Mock implements AuthRepository {}

void main() {
  late MockAuthRepository auth;
  setUp(() => auth = MockAuthRepository());

  blocTest<LoginBloc, LoginState>(
    'emits [submitting, success] when login succeeds',
    setUp: () => when(() => auth.logIn(email: any(named: 'email'), password: any(named:
'password')))
        .thenAnswer((_) async {}),
    build: () => LoginBloc(authRepository: auth),
    seed: () => const LoginState(email: 'a@b.com', password: '12345678'),
```

```
    act: (bloc) => bloc.add(LoginSubmitted()),
    expect: () => const [
      LoginState(email: 'a@b.com', password: '12345678', status: LoginStatus.submitting),
      LoginState(email: 'a@b.com', password: '12345678', status: LoginStatus.success),
    ],
    verify: (_) => verify(() => auth.logIn(email: 'a@b.com', password: '12345678')).called(1),
  );
}
```

Widget tests: `MockBloc` / `MockCubit` from bloc_test + `whenListen(bloc, Stream.fromIterable([...]), initialState: ...)`.

15.15 Bloc ecosystem

- `BlocObserver` — global logging/analytics/crash reporting of transitions & errors (`Bloc.observer = AppBlocObserver();`).
- `hydrated_bloc` — persists state automatically (`toJson` / `fromJson`) — great for settings, carts.
- `replay_bloc` — undo/redo.
- `bloc_lint` / `bloc_tools` (CLI) — lint rules & generators; VS Code/IntelliJ extensions for scaffolding.
- `very_good_cli` — project templates using Bloc + layered architecture.

15.16 Bloc pitfalls

- **Equality**: states must implement `==` (Equatable/freezed/records). Bloc **doesn't emit** a state equal to the current one (`emit` skips if `state == newState`). Mutating a list in place then emitting the same object → **no rebuild**! Always create new lists: `emit(state.copyWith(items: [...state.items, newItem]))`.
- Without Equatable, two identical-content states are unequal → unnecessary rebuilds.
- Forgetting to `close()` blocs you create manually (BlocProvider with `create` closes automatically).
- Using `BlocProvider(create: ...)` with an existing bloc → it gets closed when the provider is disposed; use `.value`.
- Putting navigation inside the bloc (it should be UI concern via BlocListener).
- Bloc-to-bloc direct dependencies (tight coupling) → prefer communicating via repositories (shared streams) or presentation layer (BlocListener in UI dispatches to another bloc).
- `emit` after close → `StateError: Cannot emit new states after calling close` — check `isClosed` after awaits in Cubits.

Part C — Other solutions

15.17 GetX

All-in-one: state (`.obs` + `Obx`, `GetBuilder`), DI (`Get.put`, `Get.find`, `Get.lazyPut`), routing (`Get.to(Page())`, `Get.back()`), snackbars/dialogs without context, translations.

```
class CounterController extends GetxController {
  final count = 0.obs;
  void increment() => count.value++;
}
final c = Get.put(CounterController());
Obx(() => Text('${c.count}'));
```

✅ Extremely fast to start, little boilerplate. ❌ Criticised by much of the community and many companies: lots of "magic"/global state, weaker testability, mixing concerns (routing+state+DI+utils in one package), large API surface, historically slow maintenance and breaking changes, non-idiomatic patterns (context-less everything). **Interview-safe answer:** "I can work with GetX in existing codebases, but for new large projects I prefer Riverpod/Bloc for testability and explicit architecture."

15.18 MobX

Observables, actions, computed values, reactions; uses code generation:

```
part 'counter.g.dart';
class Counter = _Counter with _$Counter;
abstract class _Counter with Store {
  @observable int value = 0;
  @computed bool get isEven => value.isEven;
  @action void increment() => value++;
}
Observer(builder: (_) => Text('${counter.value}'));
reaction((_) => counter.value, (v) => print('changed to $v'));
```

Transparent reactive programming (like MobX in JS). Good for teams from React+MobX.

15.19 Redux

Single immutable **store**, **actions**, pure **reducers** `(state, action) => newState`, **middleware** for side effects (`redux_thunk`, `redux_epics`), `StoreProvider` / `StoreConnector` (`flutter_redux`). Predictable, time travel, but verbose. Rare in new Flutter projects.

15.20 Signals

`signals` package (inspired by Preact/SolidJS signals): fine-grained reactivity.

```
final count = signal(0);
final doubled = computed(() => count.value * 2);
effect(() => print('count is ${count.value}'));
Watch((context) => Text('${doubled.value}'));
count.value++;
```

Simple and performant; growing popularity.

15.21 flutter_hooks

Not state management per se — React-style hooks to reduce StatefulWidget boilerplate:

```
class Search extends HookWidget {
  @override
  Widget build(BuildContext context) {
    final controller = useTextEditingController();          // auto-disposed
    final anim = useAnimationController(duration: const Duration(seconds: 1));
    final count = useState(0);
    useEffect(() { final sub = stream.listen(...); return sub.cancel; }, [stream]);
    return TextField(controller: controller);
  }
}
```

Often combined with Riverpod (`hooks_riverpod`, `HookConsumerWidget`). Rules: hooks must be called unconditionally in the same order in `build`.

15.22 Choosing for an organisation (decision guide)

Situation	Recommendation
Small app / prototype / 1–2 devs	**setState + ValueNotifier/ChangeNotifier, or Provider/Riverpod**
Following official Flutter architecture guide	**MVVM with ChangeNotifier ViewModels + repositories (+ Provider for DI)**
Large team, need strict conventions, audit trail, many juniors	**Bloc (with very_good structure)**
Large app, heavy async/caching, want less boilerplate, strong DI/testing	**Riverpod (with codegen)**
Existing codebase	**Stay consistent with what's there; migrate gradually only with strong reasons**

Migration strategy (e.g., Provider → Riverpod or GetX → Bloc): feature-by-feature, both can coexist; start with new features; write tests around the old behaviour first; don't do a big-bang rewrite.

15.23 Interview Questions — Riverpod, Bloc, others

Q1. What problems does Riverpod solve over Provider? No BuildContext dependency, compile-time safety (no ProviderNotFound), multiple providers of the same type, easy combination/dependency of providers, built-in async handling, auto-dispose & families, easier testing/overrides, usable outside Flutter.

Q2. ref.watch vs ref.read vs ref.listen? watch = subscribe & rebuild (build methods); read = one-time read (callbacks); listen = side effects on changes.

Q3. What is a family modifier? Creates a parameterised provider: a separate cached state per argument (e.g., productProvider(id)); args need proper equality.

Q4. What is autoDispose? Destroys provider state when it has no listeners; prevents leaks and gives fresh state; `ref.keepAlive` to retain.

Q5. What is AsyncValue? Riverpod's sealed union for async state: AsyncData, AsyncError, AsyncLoading, with helpers like `when`, `value`, `isLoading`, `hasError`, and `AsyncValue.guard`.

Q6. Notifier vs StateNotifier vs StateProvider? Notifier (Riverpod 2+) is the modern class-based mutable provider; StateNotifier and StateProvider are legacy APIs (moved to legacy in Riverpod 3).

Q7. Explain the Bloc pattern. UI dispatches events; Bloc processes them (calling repositories) and emits new immutable states via a stream; UI rebuilds from states. Separation, testability, traceability.

Q8. Bloc vs Cubit? Cubit exposes methods that emit states; Bloc receives events handled by `on<Event>` with transformers; Bloc gives more control/traceability, Cubit less boilerplate.

Q9. BlocBuilder vs BlocListener vs BlocConsumer? Builder rebuilds UI; Listener runs side effects once per change; Consumer does both.

Q10. Why use Equatable with Bloc? Value equality for states/events so identical states are deduplicated and tests can compare expected states.

Q11. Why might a Bloc not rebuild UI after emit? Emitted state is equal to the previous (same instance mutated, or Equatable props missing the changed field).

Q12. How do you prevent duplicate submissions in Bloc? `droppable()` transformer, status checks, disabling the button.

Q13. How do you implement search with debounce in Bloc? A custom transformer using `debounceTime` + `switchMap` (rxdart) or `restartable()`.

Q14. How do blocs communicate? Via shared repositories exposing streams, or in the presentation layer using BlocListener to add events to another bloc; avoid direct bloc dependencies.

Q15. What is BlocObserver? Global hook to observe all bloc creations, events, changes, transitions, errors and closes — logging/analytics/crash reporting.

Q16. What is HydratedBloc? Bloc extension that automatically persists and restores state using storage.

Q17. Why do some teams avoid GetX? Implicit global state and magic, mixing concerns, harder testing, maintenance concerns, non-standard patterns.

Q18. Which state management do you prefer and why? Give a reasoned answer with trade-offs (e.g., Riverpod for compile-safe DI + async; Bloc for strict event-driven architecture in large teams), emphasise consistency.

Q19. How would you test a Notifier/Cubit? Unit test with ProviderContainer (overrides with fakes) / blocTest with mocked repositories; assert state sequences.

Q20. What is `AsyncValue.guard`? Runs a future and converts the result to AsyncData or AsyncError, avoiding manual try/catch.

PART IV

Data & Platform

Chapter 16 — Networking: HTTP, Dio, JSON Serialization, REST, GraphQL, WebSockets, Error Handling & Offline

Goal: Talk to servers like a professional: robust API clients with interceptors, token refresh, retries, cancellation, typed models, error mapping, caching and real-time connections.

16.1 5-year-old picture: sending letters 📮

Your app **writes a letter** (request) to a server: *"Dear server, please GIVE me the list of toys"* (`GET /toys`). The server **writes back** (response) with a status ("200 OK — here are the toys" or "404 — no such toys") and a **package** (body, usually **JSON** text). Your app must **unpack** the JSON into Dart objects (deserialization).

16.2 HTTP basics you must know

Method	Meaning	Idempotent?
GET	Read	✅
POST	Create / action	❌
PUT	Replace	✅
PATCH	Partial update	❌ (usually)
DELETE	Delete	✅

Status codes: **2xx** success (200 OK, 201 Created, 204 No Content), **3xx** redirect (304 Not Modified), **4xx** client error (400 Bad Request, 401 Unauthorized = not logged in/token expired, 403 Forbidden = no permission, 404 Not Found, 409 Conflict, 422 Unprocessable Entity (validation), 429 Too Many Requests), **5xx** server error (500, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout).

Headers: `Authorization: Bearer <token>`, `Content-Type: application/json`, `Accept`, `Accept-Language`, `If-None-Match` / `ETag`, `Cache-Control`, custom `X-Request-Id`, `Idempotency-Key`.

Platform setup

- **Android:** `<uses-permission android:name="android.permission.INTERNET" />` in `src/main/AndroidManifest.xml` — **missing in release** is a classic bug: works in debug (debug manifest has it) but release can't connect! Cleartext HTTP (non-HTTPS) is blocked on Android 9+ unless `android:usesCleartextTraffic="true"` or network security config (only for dev!).
- **iOS:** App Transport Security blocks plain HTTP; exceptions via `NSAppTransportSecurity` in Info.plist (avoid in prod).

- **macOS**: enable `com.apple.security.network.client` entitlement — otherwise "Operation not permitted".
- **Web: CORS** — the server must send `Access-Control-Allow-Origin` headers; you can't fix CORS from Flutter code. During development use `web_dev_config.yaml` proxy (Flutter 3.38+) or a dev proxy.
- Android emulator → your laptop's localhost is `10.0.2.2`, not `localhost`. iOS simulator can use `localhost`. Physical device: use your machine's LAN IP or tunnelling (ngrok).

16.3 The `http` package (official, simple)

```
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<List<Post>> fetchPosts() async {
  final uri = Uri.https('jsonplaceholder.typicode.com', '/posts', {'_limit': '10'});
  final response = await http.get(uri, headers: {'Accept': 'application/json'})
      .timeout(const Duration(seconds: 15));

  if (response.statusCode == 200) {
    final list = jsonDecode(response.body) as List<dynamic>;
    return list.map((e) => Post.fromJson(e as Map<String, dynamic>)).toList();
  }
  throw HttpException('Failed: ${response.statusCode}');
}

Future<Post> createPost(Post post) async {
  final res = await http.post(
    Uri.parse('https://api.example.com/posts'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode(post.toJson()),      // ❗ encode! passing a Map sends form-encoded data
  );
  ...
}
```

- Reuse a `http.Client()` for multiple requests (keep-alive) and `close()` it.
- `response.body` decodes using charset from headers (defaults to latin1 if absent!) — for UTF-8 safety use `utf8.decode(response.bodyBytes)` when the server omits charset (common cause of garbled non-English text 🐛).
- Multipart upload: `http.MultipartRequest('POST', uri)..files.add(await http.MultipartFile.fromPath('file', path))`.
- Native-optimized clients: `cupertino_http` (NSURLSession on iOS/macOS), `cronet_http` (Cronet on Android) — HTTP/2/3, better performance, system proxy/VPN support.

16.4 Dio — the enterprise HTTP client

Dio adds **interceptors, global config, cancellation, timeouts, FormData, download/upload progress, transformers.**

```
final dio = Dio(BaseOptions(
  baseUrl: 'https://api.example.com/v1',
  connectTimeout: const Duration(seconds: 10),
  receiveTimeout: const Duration(seconds: 20),
  sendTimeout: const Duration(seconds: 20),
  headers: {'Accept': 'application/json'},
  responseType: ResponseType.json,       // auto-decodes JSON
  validateStatus: (s) => s != null && s >= 200 && s < 300,
));

final res = await dio.get<List<dynamic>>('/products', queryParameters: {'page': 1});
final products = res.data!.map((e) => Product.fromJson(e as Map<String, dynamic>)).toList();

await dio.post('/orders', data: order.toJson());

// Upload with progress
final form = FormData.fromMap({'avatar': await MultipartFile.fromFile(path, filename: 'a.jpg')});
await dio.post('/me/avatar', data: form, onSendProgress: (sent, total) => print(sent / total));

// Download
await dio.download(url, savePath, onReceiveProgress: (r, t) {});

// Cancellation
final token = CancelToken();
dio.get('/search', queryParameters: {'q': q}, cancelToken: token);
token.cancel('user typed again');
```

DioException types (Dio 5)

`connectionTimeout`, `sendTimeout`, `receiveTimeout`, `badCertificate`, `badResponse` (non-2xx; `e.response?.statusCode`), `cancel`, `connectionError` (no internet / DNS), `unknown`.

Interceptors 🔌

```
class AuthInterceptor extends QueuedInterceptor {   // Queued: requests wait while one refresh
happens
  AuthInterceptor(this._tokens, this._dio);
  final TokenStore _tokens;
  final Dio _dio;

  @override
  void onRequest(RequestOptions options, RequestInterceptorHandler handler) async {
    final token = await _tokens.accessToken();
```

```
    if (token != null) options.headers['Authorization'] = 'Bearer $token';
    handler.next(options);
  }

  @override
  void onError(DioException err, ErrorInterceptorHandler handler) async {
    final is401 = err.response?.statusCode == 401;
    final alreadyRetried = err.requestOptions.extra['retried'] == true;
    if (!is401 || alreadyRetried) return handler.next(err);

    try {
      final newToken = await _refreshToken();            // call refresh endpoint with a SEPARATE
Dio (no interceptor loop)
      final opts = err.requestOptions
        ..headers['Authorization'] = 'Bearer $newToken'
        ..extra['retried'] = true;
      final response = await _dio.fetch(opts);           // retry original request
      handler.resolve(response);
    } catch (e) {
      await _tokens.clear();                             // refresh failed → logout
      authEvents.add(AuthEvent.loggedOut);
      handler.next(err);
    }
  }

  Future<String> _refreshToken() async {
    final refresh = await _tokens.refreshToken();
    final res = await Dio(BaseOptions(baseUrl: _dio.options.baseUrl))
        .post('/auth/refresh', data: {'refresh_token': refresh});
    await _tokens.save(res.data['access_token'], res.data['refresh_token']);
    return res.data['access_token'] as String;
  }
}

dio.interceptors.addAll([
  AuthInterceptor(tokenStore, dio),
  if (kDebugMode) LogInterceptor(requestBody: true, responseBody: true), // ❗ never log
tokens/PII in production
  RetryInterceptor(dio: dio, retries: 3, retryDelays: const [Duration(seconds: 1),
Duration(seconds: 2), Duration(seconds: 4)]), // dio_smart_retry
]);
```

Token refresh race condition (very common senior question): if 5 requests fail with 401 simultaneously, a naive interceptor calls refresh 5 times (and later refreshes invalidate earlier refresh tokens → user logged out). Solutions: `QueuedInterceptor` (processes one at a time), or a shared `Completer` / `Future` lock so only one refresh runs and others await it. Also refresh **proactively** shortly before expiry (JWT `exp`).

Retrofit (typed API clients via codegen)

```
@RestApi(baseUrl: 'https://api.example.com/v1')
abstract class ProductApi {
  factory ProductApi(Dio dio, {String baseUrl}) = _ProductApi;

  @GET('/products')
  Future<List<Product>> getProducts(@Query('page') int page);

  @GET('/products/{id}')
  Future<Product> getProduct(@Path('id') String id);

  @POST('/orders')
  Future<Order> createOrder(@Body() CreateOrderRequest body);
}
```

Chopper is a similar codegen client.

16.5 JSON serialization

16.5.1 Manual

```
class User {
  const User({required this.id, required this.name, this.email, required this.createdAt, this.tags
= const []});
  final int id;
  final String name;
  final String? email;
  final DateTime createdAt;
  final List<String> tags;

  factory User.fromJson(Map<String, dynamic> json) => User(
        id: json['id'] as int,
        name: json['name'] as String,
        email: json['email'] as String?,
        createdAt: DateTime.parse(json['created_at'] as String),
        tags: (json['tags'] as List<dynamic>? ?? const []).cast<String>(),
      );

  Map<String, dynamic> toJson() => {
        'id': id,
        'name': name,
        if (email != null) 'email': email,
        'created_at': createdAt.toIso8601String(),
        'tags': tags,
```

```
      };
}
```

With Dart 3 patterns (validates shape and fails clearly):

```
factory User.fromJson(Map<String, dynamic> json) {
  if (json case {'id': int id, 'name': String name, 'created_at': String created}) {
    return User(id: id, name: name, email: json['email'] as String?, createdAt:
DateTime.parse(created));
  }
  throw FormatException('Invalid user JSON: $json');
}
```

⚠️ JSON numbers: `1` decodes as `int`, `1.0` or `1.5` as `double`. A server sending `price: 10` sometimes and `10.5` other times → `json['price'] as double` crashes for `10`. Use `(json['price'] as num).toDouble()`.

16.5.2 `json_serializable` (codegen — most common)

```
import 'package:json_annotation/json_annotation.dart';
part 'user.g.dart';

@JsonSerializable(fieldRename: FieldRename.snake, explicitToJson: true)
class User {
  User({
    required this.id,
    required this.name,
    this.email,
    required this.createdAt,
    this.avatar = '',
    this.role = Role.unknown,
    this.address,
  });

  final int id;                                    // "id"
  final String name;                               // "name"
  final String? email;                             // nullable → missing key OK
  final DateTime createdAt;                        // "created_at" (snake rename), ISO-8601 parsed
automatically
  @JsonKey(name: 'avatar_url', defaultValue: '')   // custom key + default when missing/null
  final String avatar;
  @JsonKey(unknownEnumValue: Role.unknown)         // new server enum values won't crash old apps
  final Role role;
  final Address? address;                          // nested model — explicitToJson makes toJson
recurse

  @JsonKey(includeFromJson: false, includeToJson: false)
  bool selected = false;                           // UI-only field, not serialized
```

```
  factory User.fromJson(Map<String, dynamic> json) => _$UserFromJson(json);
  Map<String, dynamic> toJson() => _$UserToJson(this);
}
```

`dart run build_runner build --delete-conflicting-outputs`. Custom `JsonConverter<T, S>` for special formats (e.g., epoch timestamps).

16.5.3 `freezed` (immutable models + unions + JSON)

```
import 'package:freezed_annotation/freezed_annotation.dart';
part 'product.freezed.dart';
part 'product.g.dart';

@freezed
abstract class Product with _$Product {          // freezed 3.x: `abstract class` (or `sealed` for unions)
  const factory Product({
    required String id,
    required String name,
    @Default(0) int stock,
    double? discount,
  }) = _Product;

  factory Product.fromJson(Map<String, dynamic> json) => _$ProductFromJson(json);
}

final p2 = p1.copyWith(stock: 5);   // copyWith, ==, hashCode, toString generated
```

Union (sealed) types with freezed:

```
@freezed
sealed class PaymentResult with _$PaymentResult {
  const factory PaymentResult.success(String txnId) = PaymentSuccess;
  const factory PaymentResult.failed(String reason) = PaymentFailed;
}
```

(In freezed 3, `when` / `map` helpers were removed in favour of Dart 3 `switch` pattern matching.) Note: freezed versions changed syntax (`class X with _$X` → `abstract class` / `sealed class`); check your version.

Other options: `dart_mappable`, `built_value` (older, Google), Dart 3 macros (`@JsonCodable`) — **cancelled** in 2025.

16.5.4 Parse big JSON off the main thread

```
final users = await Isolate.run(() =>
    (jsonDecode(body) as List).map((e) => User.fromJson(e as Map<String, dynamic>)).toList());
```

Dio: set `dio.transformer = BackgroundTransformer()` (default in Dio 5 already uses background decoding for large responses on native).

16.6 Error handling architecture (production pattern)

Never let raw `DioException` / `SocketException` reach the UI. Map to **domain failures**:

```
sealed class AppFailure implements Exception {
  const AppFailure(this.message);
  final String message;
}
final class NetworkFailure extends AppFailure { const NetworkFailure() : super('No internet
connection'); }
final class TimeoutFailure extends AppFailure { const TimeoutFailure() : super('The request timed
out'); }
final class UnauthorizedFailure extends AppFailure { const UnauthorizedFailure() : super('Please
log in again'); }
final class ServerFailure extends AppFailure { const ServerFailure(super.message,
{this.statusCode}); final int? statusCode; }
final class ValidationFailure extends AppFailure { const ValidationFailure(super.message,
this.fieldErrors); final Map<String, String> fieldErrors; }
final class UnknownFailure extends AppFailure { const UnknownFailure([super.message = 'Something
went wrong']); }

AppFailure mapDioError(DioException e) => switch (e.type) {
  DioExceptionType.connectionTimeout || DioExceptionType.sendTimeout ||
DioExceptionType.receiveTimeout => const TimeoutFailure(),
  DioExceptionType.connectionError => const NetworkFailure(),
  DioExceptionType.badResponse => switch (e.response?.statusCode) {
      401 => const UnauthorizedFailure(),
      422 => ValidationFailure('Invalid data', _fieldErrors(e.response?.data)),
      final code => ServerFailure(_serverMessage(e.response?.data) ?? 'Server error', statusCode:
code),
    },
  DioExceptionType.cancel => const UnknownFailure('Cancelled'),
  _ => const UnknownFailure(),
};
```

Repository returns a `Result<T>` (or throws AppFailure); UI shows user-friendly messages, logs technical details to crash reporting. Libraries: `fpdart` (`Either<Failure, T>`, `TaskEither`), `dartz` (older), or your own sealed `Result`.

Retry strategy

- Retry only **idempotent** requests (GET/PUT/DELETE) or POSTs with **idempotency keys**.
- Exponential backoff with jitter; cap retries (3).
- Retry on timeouts, 502/503/504, connection errors — not on 4xx.

Connectivity

`connectivity_plus` tells you the network *type* (wifi/mobile/none) — **not** whether internet actually works (captive portals!). For real reachability, use `internet_connection_checker_plus` or just attempt the request and handle failure. Listen to connectivity changes to auto-retry queued actions.

16.7 Caching & offline-first

Levels:

1. **HTTP caching**: honor `Cache-Control` / `ETag` — `dio_cache_interceptor`.
2. **Repository cache**: in-memory (Map with TTL) + persistent (database) — **stale-while-revalidate**: show cached data immediately, fetch fresh, update UI.
3. **Offline-first**: local DB is the **source of truth**; UI watches DB streams (Drift/Isar/ObjectBox/Firestore); sync engine pushes local changes (outbox/queue) and pulls server changes; conflict resolution (last-write-wins, versioning, server-authoritative merge).

```
class ProductRepository {
  ProductRepository(this._api, this._db);
  final ProductApi _api;
  final ProductDao _db;

  Stream<List<Product>> watchProducts() => _db.watchAll();   // UI listens here

  Future<void> refresh() async {
    final fresh = await _api.getProducts(1);
    await _db.replaceAll(fresh);                              // triggers stream update
  }
}
```

16.8 GraphQL

`graphql_flutter` / `graphql` (with normalized cache), `ferry` (codegen, typed, streams), `artemis` (older codegen).

```
final client = GraphQLClient(link: AuthLink(getToken: () async => 'Bearer
$t').concat(HttpLink('https://api.example.com/graphql')), cache: GraphQLCache(store:
HiveStore()));
final result = await client.query(QueryOptions(document: gql(r'''
```

```
  query Product($id: ID!) { product(id: $id) { id name price } }
'''), variables: {'id': '42'}));
if (result.hasException) { ... }
```

Know: queries/mutations/subscriptions, fragments, normalized caching, over-/under-fetching benefits, N+1 on the server side, persisted queries.

16.9 Real-time: WebSockets, SSE, gRPC, MQTT

```
import 'package:web_socket_channel/web_socket_channel.dart';
final channel = WebSocketChannel.connect(Uri.parse('wss://chat.example.com/ws'));
await channel.ready;                          // throws if connection fails
channel.stream.listen((msg) => handle(msg), onDone: reconnect, onError: (_) => reconnect());
channel.sink.add(jsonEncode({'type': 'message', 'text': 'hi'}));
await channel.sink.close();
```

Production concerns: **reconnection with backoff**, **heartbeats/pings**, re-subscribe after reconnect, message ordering & dedup (ids), app background (sockets die on mobile when backgrounded — use push notifications to wake/sync), auth on connect (token in query/header, refresh).

- Socket.IO: `socket_io_client`.
- SSE (server-sent events): one-way streaming over HTTP (used by LLM streaming APIs!) — read a streamed response line by line.
- **gRPC**: `grpc` package + protobuf codegen; HTTP/2; strongly typed; streaming; gRPC-Web on web.
- MQTT (IoT): `mqtt_client`.

16.10 Security in networking (summary — Chapter 25 for more)

- HTTPS everywhere; **certificate pinning** for high-security apps (banking): Dio `HttpClient` with `badCertificateCallback` checking fingerprints / `SecurityContext` with trusted certs; pinning on both platforms via plugins (`http_certificate_pinning`). Plan pin rotation (pin the intermediate/public key, ship backup pins), or you'll brick old app versions when certificates change.
- Store tokens in **secure storage** (Keychain/Keystore), not SharedPreferences.
- Don't log tokens/PII; strip from crash reports.
- Don't embed secret API keys in the app — proxy through your backend.

16.11 Testing networking code

- Abstract API behind an interface; inject fakes into repositories.
- Mock `http.Client` with `MockClient` from `package:http/testing.dart`:

```
final client = MockClient((request) async =>
    http.Response(jsonEncode([{'id': 1, 'title': 'x'}]), 200, headers: {'content-type':
'application/json'}));
```

- Dio: `http_mock_adapter` (`DioAdapter`).
- Widget tests: Flutter's `TestWidgetsFlutterBinding` makes **real HTTP calls return 400** by default — always mock network in tests.

16.12 Common networking bugs

Bug	Cause / Fix
Works in debug, fails in release on Android	**INTERNET permission missing in main manifest**
`SocketException: Connection refused` **on emulator**	**Using** `localhost` → `10.0.2.2`
`HandshakeException` / **CERTIFICATE_VERIFY_FAILED**	**invalid/self-signed cert, old Android root CAs (Let's Encrypt on Android < 7.1.1), wrong system time**
Garbled non-English text	**decode** `bodyBytes` **as UTF-8**
`type 'int' is not a subtype of type 'double'`	**use** `(x as num).toDouble()`
`XMLHttpRequest error` **on web**	**CORS / mixed content (http on https page)**
Infinite 401 loop	**refresh endpoint going through same auth interceptor; retry flag missing**
Multiple refreshes	**QueuedInterceptor / lock**
UI freezes on large responses	**parse in isolate**
Responses arrive out of order in search	**cancel previous request (CancelToken / switchMap / restartable)**
Memory leak / setState after dispose	**cancel requests in dispose; check** `mounted`
Request body empty on server	**forgot** `jsonEncode` / `Content-Type` **header**

16.13 Interview Questions — Networking

Q1. http vs Dio? http is minimal and official; Dio adds interceptors, base options, cancellation, FormData, progress, retries ecosystem — better for large apps.

Q2. How do you handle token refresh? Auth interceptor adds token; on 401, refresh once (queued/locked), retry original request, logout on refresh failure; refresh endpoint uses a separate client to avoid loops.

Q3. How do you parse JSON? jsonDecode → Map → fromJson factories (manual or json_serializable/freezed); heavy payloads in an isolate.

Q4. Why use code generation for JSON? Less boilerplate, fewer typos, consistent null handling, nested models, maintainability.

Q5. How do you handle errors from APIs? Map transport and HTTP errors into domain failure types; show friendly messages; log details; retry where appropriate.

Q6. How do you cancel requests? Dio CancelToken (or closing http client); cancel in dispose or when a newer request supersedes.

Q7. What is CORS and why does it affect Flutter web? Browser security policy requiring the server to allow cross-origin requests; it's fixed on the server/proxy, not in Flutter.

Q8. How do you build an offline-first app? Local DB as single source of truth, UI observes DB, sync layer with outbox queue and conflict resolution, connectivity awareness, background sync.

Q9. What is certificate pinning? Validating the server certificate/public key against known values to prevent MITM even with compromised CAs; requires rotation planning.

Q10. How do you implement retries correctly? Exponential backoff with jitter for transient failures, only idempotent requests or with idempotency keys, max attempts.

Q11. What is an interceptor? Middleware hooking into request/response/error to modify, log, retry or resolve them.

Q12. REST vs GraphQL? REST: resource endpoints, simple caching; GraphQL: single endpoint, clients specify fields, reduces over/under-fetching, needs more tooling and caching strategy.

Chapter 17 — Local Storage: Preferences, Secure Storage, Files, SQLite (sqflite/Drift), NoSQL & Caching

Goal: Choose the right storage for each kind of data, use it correctly, migrate schemas safely, and avoid data-loss and security bugs.

17.1 5-year-old picture: drawers, safes and filing cabinets 🗄️

- **Sticky notes on the fridge** (SharedPreferences) — tiny things like "dark mode on".
- **A locked safe** (secure storage) — secret keys, passwords, tokens.
- **A box of papers** (files) — photos, PDFs, downloads.
- **A big filing cabinet with an index** (databases) — thousands of organised records you can search.

17.2 Decision table

Data	Store in	Why
Small settings/flags (theme, onboarding seen, last tab)	`shared_preferences`	Simple key-value, not secure
Tokens, passwords, encryption keys, PII secrets	`flutter_secure_storage`	Keychain (iOS) / Keystore-backed encryption (Android)
Structured relational data, queries, joins, offline-first	Drift (SQLite, typed, reactive) or sqflite	SQL power, migrations
Fast object storage / NoSQL	ObjectBox, Isar (community fork), Hive CE, Sembast	Simple objects, speed
Large binary (images, PDFs, videos)	Files via `path_provider`	Don't put big blobs in prefs/DB
Cached HTTP responses/images	`dio_cache_interceptor`, `cached_network_image` (uses `flutter_cache_manager`)	Automatic expiry
Cloud-synced data	Firestore (offline cache), Supabase, Realm-like solutions	Sync handled

⚠️ Ecosystem note (2024–2026): the original authors stopped maintaining **Hive** and **Isar**. Community forks `hive_ce` and `isar_community` continue them. For new projects, many teams choose **Drift** (actively maintained, SQL, web support) or **ObjectBox**. Interviewers may ask "Hive vs Isar vs Drift" — mention maintenance status.

17.3 shared_preferences

```dart
// Classic API (cached, synchronous reads after getInstance)
final prefs = await SharedPreferences.getInstance();
await prefs.setBool('onboarding_done', true);
await prefs.setString('locale', 'hi');
await prefs.setInt('launch_count', (prefs.getInt('launch_count') ?? 0) + 1);
await prefs.setStringList('recent_searches', ['a', 'b']);
final done = prefs.getBool('onboarding_done') ?? false;
await prefs.remove('locale');
await prefs.clear();
```

Newer APIs (shared_preferences 2.3+):

- `SharedPreferencesAsync` — always reads from the platform (no stale cache across isolates/processes).
- `SharedPreferencesWithCache` — explicit cache with an `allowList` of keys. The legacy `SharedPreferences` class is planned for eventual deprecation — new code should prefer the new APIs.

Facts:

- Android: stored in XML (`SharedPreferences`) or **DataStore** (newer backend option); iOS/macOS: `NSUserDefaults` ; web: `localStorage` ; Windows/Linux: JSON file.
- **Not encrypted** — readable on rooted/jailbroken devices and in backups. Never store tokens/passwords.
- Data **isn't guaranteed to be persisted** synchronously across crashes (write is async).
- Not for large data (it's loaded into memory).
- Only primitive types — store complex objects as JSON strings (or use a DB).
- Tests: `SharedPreferences.setMockInitialValues({'key': value})` .

Best practice: wrap in a typed `SettingsStore` class with keys as constants — avoid string typos scattered everywhere.

17.4 flutter_secure_storage 🔐

```dart
const storage = FlutterSecureStorage(
  aOptions: AndroidOptions(encryptedSharedPreferences: true), // option depends on plugin version; newer versions use their own cipher by default
  iOptions: IOSOptions(accessibility: KeychainAccessibility.first_unlock_this_device),
);
await storage.write(key: 'refresh_token', value: token);
final token = await storage.read(key: 'refresh_token');
await storage.delete(key: 'refresh_token');
await storage.deleteAll();
```

- iOS: **Keychain**. ⚠️ Keychain items **survive app uninstall** on iOS! If a user reinstalls, old tokens may still exist → on first launch after install (detect using a SharedPreferences flag, which *is* wiped on uninstall), clear secure storage.
- Android: encrypts values using keys stored in the **Android Keystore**. Android **Auto Backup** can restore encrypted data to a new device where the Keystore key doesn't exist → decryption crashes (`BadPaddingException`). Exclude secure storage files from backup (`android:allowBackup="false"` or `fullBackupContent` / `dataExtractionRules` rules) or handle errors by clearing.
- `KeychainAccessibility` choices matter for background access (e.g., `first_unlock` to read in background fetch).
- Biometric-protected secrets: `local_auth` + secure storage (or platform APIs binding keys to biometrics).

17.5 Files with path_provider

```
import 'dart:io';
import 'package:path_provider/path_provider.dart';
import 'package:path/path.dart' as p;

final docs = await getApplicationDocumentsDirectory();   // user data, backed up (iOS)
final support = await getApplicationSupportDirectory();  // app data not user-visible
final temp = await getTemporaryDirectory();              // cache; OS may delete anytime
final cache = await getApplicationCacheDirectory();
// Android only: getExternalStorageDirectory(), getExternalCacheDirectories(); iOS:
getLibraryDirectory()
// Desktop: getDownloadsDirectory()

final file = File(p.join(docs.path, 'notes.json'));
await file.writeAsString(jsonEncode(notes), flush: true);
final content = await file.readAsString();
if (await file.exists()) await file.delete();
```

- `dart:io` isn't available on **web** — use conditional imports or IndexedDB-based storage.
- Always build paths with `package:path` (`p.join`) — never string-concatenate with `/`.
- Write atomically for important data: write to a temp file, then `rename` (prevents corruption on crash).
- Large file copies/encryption → isolate.
- User-visible files (save to Downloads/Gallery): Android scoped storage rules (Android 10+), `MediaStore`; packages `gal` (save images/videos to gallery), `file_picker`, `share_plus`, `open_filex`. iOS needs `UIFileSharingEnabled` / `LSSupportsOpeningDocumentsInPlace` to expose files in the Files app.

17.6 SQLite with sqflite

```dart
import 'package:sqflite/sqflite.dart';

class NotesDb {
  static const _version = 2;
  Database? _db;

  Future<Database> get db async => _db ??= await openDatabase(
        p.join(await getDatabasesPath(), 'notes.db'),
        version: _version,
        onConfigure: (db) async => db.execute('PRAGMA foreign_keys = ON'),
        onCreate: (db, version) async {
          await db.execute('''
            CREATE TABLE notes(
              id INTEGER PRIMARY KEY AUTOINCREMENT,
              title TEXT NOT NULL,
              body TEXT,
              pinned INTEGER NOT NULL DEFAULT 0,
              updated_at INTEGER NOT NULL
            )''');
          await db.execute('CREATE INDEX idx_notes_updated ON notes(updated_at)');
        },
        onUpgrade: (db, oldVersion, newVersion) async {
          if (oldVersion < 2) {
            await db.execute('ALTER TABLE notes ADD COLUMN pinned INTEGER NOT NULL DEFAULT 0');
          }
          // if (oldVersion < 3) { ... } — cumulative, never edit old steps
        },
      );

  Future<int> insert(Note n) async =>
      (await db).insert('notes', n.toMap(), conflictAlgorithm: ConflictAlgorithm.replace);

  Future<List<Note>> search(String q) async {
    final rows = await (await db).query('notes',
        where: 'title LIKE ?', whereArgs: ['%$q%'],          // ❗ parameterised — prevents SQL injection
        orderBy: 'updated_at DESC', limit: 50);
    return rows.map(Note.fromMap).toList();
  }

  Future<void> bulkInsert(List<Note> notes) async {
    final database = await db;
    await database.transaction((txn) async {                  // ❗ transactions: atomic + MUCH faster
      final batch = txn.batch();
      for (final n in notes) batch.insert('notes', n.toMap());
      await batch.commit(noResult: true);
```

```
    });
  }
}
```

Notes:

- SQLite has no boolean/date types → store `INTEGER` 0/1 and epoch millis (or ISO text).
- Inside a `transaction`, use `txn`, **not** `db` → using `db` inside deadlocks!
- `sqflite` works on Android/iOS/macOS; desktop via `sqflite_common_ffi`; web via `sqflite_common_ffi_web` (experimental).
- Keep a single DB instance (opening multiple connections → "database is locked").
- Migrations: increase `version`; `onUpgrade` must handle **every** older version path; test upgrades from old app versions with real data!

17.7 Drift (type-safe, reactive SQLite) ⭐

```dart
import 'package:drift/drift.dart';
part 'app_database.g.dart';

class Todos extends Table {
  IntColumn get id => integer().autoIncrement()();
  TextColumn get title => text().withLength(min: 1, max: 200)();
  BoolColumn get done => boolean().withDefault(const Constant(false))();
  DateTimeColumn get createdAt => dateTime().withDefault(currentDateAndTime)();
  IntColumn get categoryId => integer().nullable().references(Categories, #id)();
}

class Categories extends Table {
  IntColumn get id => integer().autoIncrement()();
  TextColumn get name => text()();
}

@DriftDatabase(tables: [Todos, Categories])
class AppDatabase extends _$AppDatabase {
  AppDatabase(super.e);

  @override
  int get schemaVersion => 2;

  @override
  MigrationStrategy get migration => MigrationStrategy(
        onCreate: (m) => m.createAll(),
        onUpgrade: (m, from, to) async {
          if (from < 2) await m.addColumn(todos, todos.categoryId);
        },
        beforeOpen: (details) async => customStatement('PRAGMA foreign_keys = ON'),
      );
```

```
  Stream<List<Todo>> watchActive() =>
      (select(todos)..where((t) => t.done.equals(false))..orderBy([(t) =>
OrderingTerm.desc(t.createdAt)])).watch();

  Future<int> addTodo(String title) => into(todos).insert(TodosCompanion.insert(title: title));
  Future<void> toggle(Todo t) => update(todos).replace(t.copyWith(done: !t.done));
}
// Open with drift_flutter: AppDatabase(driftDatabase(name: 'app'))
```

Why teams love Drift: compile-time checked queries (Dart API or `.drift` SQL files), **streams that auto-update** UI when tables change, migrations tooling (`drift_dev schema dump` + generated step-by-step migrations + migration tests), joins, transactions, isolates support, web support (WASM SQLite), encryption via SQLCipher setup.

17.8 NoSQL options (brief)

Hive (CE)

```
await Hive.initFlutter();
final box = await Hive.openBox<String>('settings');
await box.put('theme', 'dark');
box.get('theme');
// Typed objects need TypeAdapters (codegen with hive_ce_generator)
ValueListenableBuilder(valueListenable: box.listenable(), builder: ...);
```

Pure Dart, fast for small/medium data, encryption via `HiveAesCipher` (store key in secure storage). Boxes are loaded into memory (non-lazy boxes). Not for complex queries.

Isar (community)

Fast, indexed queries, links, full-text-ish search, multi-isolate. Original maintainer left; use `isar_community` fork.

ObjectBox

Very fast embedded object DB with relations, queries, vector search (on-device AI), and a commercial sync product.

Sembast

Pure-Dart NoSQL JSON store; works on web (IndexedDB) — simple.

17.9 Web storage

- `shared_preferences` → localStorage.
- IndexedDB via `idb_shim`, Drift (WASM + OPFS/IndexedDB), Sembast web, Hive CE (IndexedDB).
- Remember: browser storage can be cleared by the user/browser at any time; quota limits.

17.10 Data security at rest

- Encrypt sensitive databases: **SQLCipher** (`sqlcipher_flutter_libs` with Drift/sqflite), Hive encryption; key in secure storage.
- Don't store PII unnecessarily (privacy laws: GDPR, India's DPDP Act 2023, CCPA).
- Clear user data on logout (DB, files, secure storage, caches, image cache).
- Exclude sensitive files from cloud backups when required.

17.11 Common storage bugs

Bug	Cause / Fix
`MissingPluginException(No implementation found for method getAll on channel plugins.flutter.io/shared_preferences)`	Plugin added without full restart/rebuild; in tests not mocked; called in a background isolate without setup → full rebuild, `setMockInitialValues`, BackgroundIsolateBinaryMessenger
`Binding has not yet been initialized`	Using storage before `runApp` without `WidgetsFlutterBinding.ensureInitialized()`
Data lost after app update	Changed key names/DB schema without migration
`DatabaseException(no such column)` after update	Missing `onUpgrade` step / didn't bump version
Database locked / deadlock	Using `db` inside `transaction` instead of `txn`; multiple instances
Tokens survive reinstall on iOS	Keychain persists — clear on first run
Crash decrypting secure storage after device restore	Backup restored without Keystore key — exclude from backup, catch & reset
Slow app start	Opening big Hive boxes / heavy DB work before first frame → lazy open, show UI first
Web: `Unsupported operation: Platform._operatingSystem` / dart:io errors	dart:io used on web → conditional imports

---

17.12 Interview Questions — Storage

Q1. Where would you store an auth token? flutter_secure_storage (Keychain/Keystore), never SharedPreferences; consider short-lived access tokens + refresh tokens.

Q2. SharedPreferences limitations? Small primitive data only, unencrypted, async writes, loaded into memory, not for complex/large data.

Q3. sqflite vs Drift vs Hive/Isar/ObjectBox? sqflite: raw SQLite with manual SQL; Drift: typed, reactive SQLite with migrations tooling; Hive: lightweight key-value boxes; Isar/ObjectBox: fast NoSQL object DBs with indexes/queries; consider maintenance status (Hive/Isar community forks).

Q4. How do you handle DB migrations? Versioned schema; cumulative onUpgrade steps for every old version; never modify past migrations; test upgrade paths; Drift's schema tools.

Q5. How do you make DB writes fast? Batch inserts in transactions, indexes on queried columns, avoid N+1 queries, run heavy work in isolates.

Q6. Which directory for what? Documents for user data, Support for app internal data, Temporary/Cache for re-creatable caches.

Q7. How do you clear data on logout? Delete tokens from secure storage, clear DB tables/user-specific files, caches (image cache, HTTP cache), reset state providers/blocs.

Q8. What is an offline-first approach? Local DB as source of truth, UI observes it, background sync with server and conflict resolution.

Chapter 18 — Firebase, Backend Services & Common Integrations (Auth, Push, Analytics, Crashlytics, Payments, Maps, Media, Permissions, Background Work)

Goal: Integrate the services every real company app uses — correctly, securely, and with knowledge of the platform gotchas that cause most production incidents.

18.1 Firebase setup (FlutterFire)

```
dart pub global activate flutterfire_cli
firebase login
flutterfire configure            # creates lib/firebase_options.dart, registers apps, downloads config
flutter pub add firebase_core
```

```dart
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  runApp(const MyApp());
}
```

- Multiple environments: separate Firebase projects (dev/staging/prod) — `flutterfire configure --project=myapp-dev --out=lib/firebase_options_dev.dart` per flavor; plus `google-services.json` per Android flavor (`android/app/src/dev/`) and `GoogleService-Info.plist` per iOS scheme (copy script in build phase).
- Firebase **client config/API keys are not secrets** — security is enforced by **Security Rules** and **App Check**, not by hiding keys.

18.2 Firebase Authentication

```dart
final auth = FirebaseAuth.instance;
await auth.createUserWithEmailAndPassword(email: e, password: p);
await auth.signInWithEmailAndPassword(email: e, password: p);
await auth.signOut();
auth.authStateChanges().listen((user) { /* null = signed out */ });   // drive your router
```

```
redirect with this
auth.idTokenChanges(); auth.userChanges();
final idToken = await auth.currentUser?.getIdToken();                    // send to your backend;
verify server-side

// Google Sign-In (google_sign_in package + credential)
// Sign in with Apple (sign_in_with_apple) — REQUIRED by App Store if you offer other third-party
logins (App Store guideline 4.8, with some exceptions)
// Phone auth: verifyPhoneNumber(codeSent:, verificationCompleted:, verificationFailed:,
codeAutoRetrievalTimeout:)
try { ... } on FirebaseAuthException catch (e) {
  switch (e.code) {
    case 'user-not-found' || 'wrong-password' || 'invalid-credential': ...
    case 'email-already-in-use': ...
    case 'too-many-requests': ...
    case 'network-request-failed': ...
  }
}
```

Note: Firebase has moved toward `invalid-credential` for wrong email/password (email enumeration protection), so don't rely on `user-not-found` / `wrong-password` alone.

Other auth providers/backends: Supabase Auth, Auth0 (`auth0_flutter`), AWS Cognito/Amplify, Keycloak/OAuth2+PKCE with `flutter_appauth`, custom JWT backend.

Auth architecture pattern: `AuthRepository` (wraps SDK) → exposes `Stream<AuthUser?>` → app-level auth state (Bloc/Riverpod) → router `redirect` + `refreshListenable` → on logout clear all user data.

18.3 Cloud Firestore

```
final db = FirebaseFirestore.instance;

// Typed collection with converter
final todosRef = db.collection('users').doc(uid).collection('todos').withConverter<Todo>(
  fromFirestore: (snap, _) => Todo.fromJson({...snap.data()!, 'id': snap.id}),
  toFirestore: (todo, _) => todo.toJson()..remove('id'),
);

await todosRef.add(Todo(title: 'Buy milk'));
await todosRef.doc(id).update({'done': true});   // update on raw ref for partial fields
await todosRef.doc(id).delete();

// Real-time stream
Stream<List<Todo>> watch() => todosRef
    .where('done', isEqualTo: false)
    .orderBy('createdAt', descending: true)
    .limit(20)
```

```
    .snapshots()
    .map((s) => s.docs.map((d) => d.data()).toList());

// Pagination with cursors
final next = await todosRef.orderBy('createdAt').startAfterDocument(lastDoc).limit(20).get();

// Transactions & batched writes
await db.runTransaction((tx) async {
  final snap = await tx.get(accountRef);
  tx.update(accountRef, {'balance': (snap['balance'] as num) - 100});
});
final batch = db.batch()..set(a, dataA)..delete(b);
await batch.commit();

FieldValue.serverTimestamp(); FieldValue.increment(1); FieldValue.arrayUnion(['x']);
```

Key knowledge:

- Offline persistence enabled by default on mobile (cache); writes are queued.
- **Composite indexes** required for multi-field queries → error message includes a link to create it.
- Billing is per document read → avoid reading whole collections; paginate; denormalize data for reads.
- Listeners must be cancelled (StreamBuilder handles it; manual `listen` → cancel in dispose).
- **Security Rules** are mandatory (never ship `allow read, write: if true;`):

```
rules_version = '2';
service cloud.firestore {
  match /databases/{db}/documents {
    match /users/{uid}/todos/{todoId} {
      allow read, write: if request.auth != null && request.auth.uid == uid;
    }
  }
}
```

- Realtime Database (older JSON tree DB), Cloud Storage (`firebase_storage` for file uploads with `putFile`, progress via `snapshotEvents`), Cloud Functions (`cloud_functions` `httpsCallable('fn').call(data)`), **App Check** (`firebase_app_check` — attestation with Play Integrity / App Attest so only your genuine app can call your backend).

18.4 Push notifications (FCM) 🔔

Concepts

- **FCM** (Firebase Cloud Messaging) delivers to Android directly and to iOS via **APNs** (upload APNs auth key `.p8` to Firebase).

- Message types:
  - **Notification message** (`notification` payload): OS shows it automatically when app is in background/terminated.
  - **Data message** (`data` only): delivered to your app code; you decide what to show (on iOS, data-only messages are delivered as "silent/background" pushes and are **throttled & not guaranteed**).
- App states: **foreground**, **background**, **terminated** — each handled differently.

Code

```
@pragma('vm:entry-point')     // ❗ needed so the function isn't tree-shaken in release
Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage message) async {
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform); // runs in its
own isolate
  // light work only: e.g., update badge/local DB
}

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler); // top-level
function
  runApp(const MyApp());
}

Future<void> setupPush() async {
  final fm = FirebaseMessaging.instance;
  final settings = await fm.requestPermission(alert: true, badge: true, sound: true); // iOS +
Android 13+
  if (settings.authorizationStatus == AuthorizationStatus.denied) return;

  final token = await fm.getToken();          // send to your backend (associate with user)
  fm.onTokenRefresh.listen(sendTokenToServer);  // tokens change! (reinstall, restore, clear data)

  // Foreground messages: NOT shown automatically on Android → show with
flutter_local_notifications
  FirebaseMessaging.onMessage.listen((m) => localNotifications.show(m));
  // iOS foreground presentation:
  await fm.setForegroundNotificationPresentationOptions(alert: true, badge: true, sound: true);

  // User tapped notification while app in background:
  FirebaseMessaging.onMessageOpenedApp.listen((m) => router.go(m.data['route'] ?? '/'));
  // App launched from terminated state by tapping a notification:
  final initial = await fm.getInitialMessage();
  if (initial != null) router.go(initial.data['route'] ?? '/');

  await fm.subscribeToTopic('promotions');
}
```

Platform setup checklist

- **Android 13+:** `POST_NOTIFICATIONS` runtime permission (requestPermission handles it). Create **notification channels** (Android 8+) — with `flutter_local_notifications` (importance levels, sound).
- Default notification icon must be **white-on-transparent** (else shows a grey square) — set `com.google.firebase.messaging.default_notification_icon` meta-data.
- **iOS:** enable Push Notifications + Background Modes (Remote notifications) capabilities; APNs key in Firebase; test on a **real device** (simulators support push in recent Xcode versions via `.apns` files / drag & drop, but end-to-end FCM delivery is best tested on device).
- iOS rich notifications (images) need a **Notification Service Extension**.

Common push bugs

Bug	Fix
No notifications on iOS	**APNs key missing/wrong team, capability missing, testing on simulator, provisioning profile without push**
Foreground notification doesn't appear (Android)	**Expected — show via flutter_local_notifications**
Background handler not called in release	**Missing `@pragma('vm:entry-point')`, handler not top-level**
Tap doesn't navigate on cold start	**Handle `getInitialMessage` after router is ready**
Duplicate notifications	**Both FCM notification payload and your local notification shown — use data messages or don't re-show**
Notifications stopped for some users	**Stale tokens: update on `onTokenRefresh`; remove invalid tokens server-side (`UNREGISTERED` errors)**
Xiaomi/Oppo/etc. kill background app	**OEM battery optimizations; ask users to whitelist; high-priority messages**

Local notifications & scheduling: `flutter_local_notifications` (with `timezone` for zoned schedules; Android 12+ exact alarms permission `SCHEDULE_EXACT_ALARM` / `USE_EXACT_ALARM` restrictions). Alternatives: OneSignal, Airship, AWS SNS/Pinpoint.

18.5 Crash reporting & analytics

Crashlytics

```
FlutterError.onError = FirebaseCrashlytics.instance.recordFlutterFatalError;
PlatformDispatcher.instance.onError = (error, stack) {
  FirebaseCrashlytics.instance.recordError(error, stack, fatal: true);
  return true;
};
await FirebaseCrashlytics.instance.setUserIdentifier(hashedUserId);  // no raw PII
FirebaseCrashlytics.instance.log('Checkout started');                // breadcrumbs
await FirebaseCrashlytics.instance.setCrashlyticsCollectionEnabled(!kDebugMode);
```

- **Obfuscated builds** (`--obfuscate --split-debug-info=build/symbols`) → upload symbols (`firebase crashlytics:symbols:upload --app=<id> build/symbols`) or stack traces are unreadable.
- iOS: dSYM upload (Crashlytics run script). Android NDK: native symbols if needed.
- Alternatives: **Sentry** (`sentry_flutter` — performance tracing, release health, breadcrumbs, session replay), Bugsnag, Datadog, Instabug.

Analytics

```
await FirebaseAnalytics.instance.logEvent(name: 'add_to_cart', parameters: {'item_id': id,
'value': 499});
await FirebaseAnalytics.instance.logScreenView(screenName: 'Checkout');
// GoRouter/Navigator: FirebaseAnalyticsObserver(analytics: FirebaseAnalytics.instance)
```

Org practice: an `AnalyticsService` interface (so you can swap/multiplex providers: Firebase, Mixpanel, Amplitude, Segment), an **event taxonomy document** (naming conventions, required params), **consent** management (GDPR/ATT — iOS App Tracking Transparency prompt via `app_tracking_transparency` when tracking across apps).

Remote Config & A/B testing & feature flags

```
final rc = FirebaseRemoteConfig.instance;
await rc.setConfigSettings(RemoteConfigSettings(fetchTimeout: const Duration(seconds: 10),
minimumFetchInterval: const Duration(hours: 1)));
await rc.setDefaults({'new_checkout_enabled': false, 'min_supported_version': '1.0.0'});
await rc.fetchAndActivate();
final enabled = rc.getBool('new_checkout_enabled');
rc.onConfigUpdated.listen((_) => rc.activate());   // real-time updates
```

Uses: kill switches, gradual rollouts, **force update** (compare `min_supported_version` with `package_info_plus` version → show blocking update dialog / `in_app_update` on Android / App Store link on iOS). Alternatives: LaunchDarkly, Statsig, ConfigCat, Unleash.

Firebase Performance Monitoring (`firebase_performance`), In-App Messaging, App Distribution (beta builds to testers), Test Lab (device farm).

18.6 Supabase & other BaaS

`supabase_flutter`: Postgres + Row Level Security, Auth, Storage, Realtime, Edge Functions.

```
await Supabase.initialize(url: url, anonKey: anonKey);   // anon key is public; RLS protects data
final data = await Supabase.instance.client.from('todos').select().eq('done',
false).order('created_at');
Supabase.instance.client.from('todos').stream(primaryKey: ['id']).listen(...);
```

Others: AWS Amplify, Appwrite, PocketBase, Parse, Back4App; Serverpod / Dart Frog (Dart backends).

18.7 Permissions (permission_handler)

```
final status = await Permission.camera.status;
if (status.isDenied) {
  final result = await Permission.camera.request();
  if (result.isPermanentlyDenied) {
    // Explain & send to settings
    await openAppSettings();
  }
}
// Others: Permission.location, locationAlways, photos, storage (legacy), notification,
microphone, contacts, bluetoothScan…
```

Rules:

- **Declare** in AndroidManifest (`<uses-permission>`) and Info.plist usage descriptions — else instant crash (iOS) or silent denial.
- Ask **in context** (when the user taps "Scan"), with a **pre-permission explanation** screen — better grant rates and required by store policies.
- Handle all states: granted, denied, permanently denied (Android "don't ask again" / iOS after first denial), restricted (parental controls), limited (iOS limited photo access), provisional (iOS notifications).
- Android 13+: granular media permissions (`READ_MEDIA_IMAGES`, `READ_MEDIA_VIDEO`, ...); Android 14+ "selected photos" partial access; prefer the **Photo Picker** (`image_picker` uses it) which needs **no permission**.
- Background location requires a separate permission + strong justification for Play/App Store review.
- Remove unused permissions (plugins can merge permissions into your manifest — check the merged manifest; remove with `tools:node="remove"`).

18.8 Device features & popular plugins (know what exists)

Need	Package(s)
Camera / photos	`image_picker`, `camera`, `image_cropper`, `photo_manager`
Video	`video_player` **(official)**, `chewie`, `media_kit`, `better_player_plus`
Audio	`just_audio`, `audioplayers`, `audio_service` **(background)**, `record`
Location / geofencing	`geolocator`, `location`, `geocoding`, `flutter_background_geolocation` **(commercial)**
Maps	`google_maps_flutter`, `flutter_map` **(OpenStreetMap)**, `mapbox_maps_flutter`, **Apple Maps** (`apple_maps_flutter`)

Need	Package(s)
QR/Barcode	`mobile_scanner`
Biometrics	`local_auth`
In-app web	`webview_flutter` (official), `flutter_inappwebview`
URLs / phone / email	`url_launcher` (add `<queries>` in Android manifest for intent visibility on Android 11+; `LSApplicationQueriesSchemes` on iOS for canLaunch)
Share	`share_plus`
Device info / app version	`device_info_plus`, `package_info_plus`
Connectivity	`connectivity_plus`
Sensors	`sensors_plus`
Bluetooth	`flutter_blue_plus`, `flutter_reactive_ble`
NFC	`nfc_manager`
PDF	`pdf` + `printing`, `syncfusion_flutter_pdfviewer`, `pdfrx`
Charts	`fl_chart`, `syncfusion_flutter_charts`, `graphic`
Payments	`in_app_purchase` (official), RevenueCat `purchases_flutter`, Stripe `flutter_stripe`, Razorpay `razorpay_flutter`, `pay` (Google Pay/Apple Pay)
Ads	`google_mobile_ads`
Contacts / calendar	`flutter_contacts`, `device_calendar`
Home widgets	`home_widget` (Android/iOS home screen widgets — native UI)
On-device ML / AI	`google_mlkit_*`, `tflite_flutter`, Firebase AI Logic (`firebase_ai`), `google_generative_ai` (deprecated in favour of Firebase AI Logic)
App review / update	`in_app_review`, `in_app_update`, `upgrader`

Payments: policy knowledge (interviews ask!)

- **Digital goods/subscriptions** consumed in the app (premium features, coins) → must use **Apple In-App Purchase / Google Play Billing** (store policies), with exceptions/regional changes (e.g., US court rulings in 2025 allowing external purchase links on iOS in the US, EU DMA alternatives, Google Play alternative billing in some regions). Know that rules changed and vary by region.
- **Physical goods/services** (e-commerce, food delivery, rides) → normal payment gateways (Stripe, Razorpay, PayPal).
- Always **verify purchases server-side** (receipts / Google Play Developer API / App Store Server API & notifications), handle pending/restored purchases, and `completePurchase` to avoid automatic refunds on Android (3 days un-acknowledged).

18.9 Background work

Flutter apps can't run arbitrary Dart forever in the background on mobile; the OS controls it.

Need	Tool
Periodic/deferred work (sync, uploads)	`workmanager` (Android WorkManager; iOS BGTaskScheduler — iOS runs it opportunistically, not exact)
Background fetch	`background_fetch`
Long-running (music, navigation, tracking)	Foreground service (Android, with notification and foreground service type declarations required on Android 14+) — `flutter_foreground_task`; iOS background modes (audio, location)
Large downloads/uploads	`background_downloader` (uses URLSession / WorkManager)
Alarms	`android_alarm_manager_plus` (Android only)
Push-triggered	FCM data/silent push → background handler

Background Dart code runs in a **separate isolate** with a fresh memory space — re-initialize Firebase/DI there; annotate entry points with `@pragma('vm:entry-point')`.

18.10 Interview Questions — Integrations

Q1. How do push notifications work in Flutter? FCM (via APNs on iOS) delivers messages; app requests permission, registers a token with backend; handle foreground (onMessage + local notification), background tap (onMessageOpenedApp), terminated tap (getInitialMessage), and background data handler (top-level, vm:entry-point).

Q2. Notification vs data messages? Notification messages are displayed by the OS when app isn't in foreground; data messages are handled by app code (and are throttled on iOS when silent).

Q3. How do you secure Firestore? Security Rules validating auth and data, App Check, least-privilege structure; client API keys aren't secrets.

Q4. How do you implement force update? Remote config / backend minimum version vs package_info version → blocking dialog to store (in_app_update on Android).

Q5. How do you handle permissions properly? Declare in manifests, request contextually with rationale, handle denied/permanently denied (open settings), use permissionless pickers when possible.

Q6. How do you report crashes in release? Crashlytics/Sentry with FlutterError.onError + PlatformDispatcher.onError, upload symbols for obfuscated builds, breadcrumbs, user IDs (hashed).

Q7. What are the rules for in-app payments? Digital content → store billing (IAP/Play Billing) with server-side verification; physical goods → regular payment gateways; regional rule variations exist.

Q8. How do you run tasks in the background? workmanager/BGTaskScheduler for deferred tasks, foreground services for long-running work, push-triggered handlers; separate isolate with re-initialization.

Q9. Why does my background message handler not run in release? It must be a top-level/static function annotated with `@pragma('vm:entry-point')` to survive tree shaking, and registered before runApp.

Q10. Firebase vs Supabase? Firebase: NoSQL (Firestore), mature mobile SDKs, FCM, Crashlytics; Supabase: Postgres SQL with RLS, open-source, relational queries, realtime; choose based on data model and vendor needs.

Chapter 19 — Talking to Native: Platform Channels, Pigeon, FFI, Plugins, Platform Views, Add-to-App & Web Interop

Goal: Call Kotlin/Swift/C/JS from Dart and back, write your own plugin, embed native views, and add Flutter into existing native apps. Senior interviews almost always include "how would you access a native SDK that has no Flutter plugin?"

19.1 5-year-old picture: walkie-talkies 📻

Dart lives in one house; Android (Kotlin) and iOS (Swift) live in neighbouring houses. They speak different languages. A **platform channel** is a **walkie-talkie** on a named frequency (`'com.myapp/battery'`). Dart says "getBatteryLevel, over!" and Kotlin answers "87, over!". Messages are translated into a common "radio language" by a **codec**.

19.2 Channel types

Channel	Direction	Use
`MethodChannel`	Dart ↔ Native, request/response (async)	Call a function and get a result
`EventChannel`	Native → Dart stream	Sensors, location updates, connectivity changes
`BasicMessageChannel`	Bidirectional messages with custom codec	Custom protocols, strings/JSON

All channel calls are **asynchronous** and messages are serialized with a codec: `StandardMessageCodec` supports `null`, `bool`, `int`, `double`, `String`, `Uint8List` / `Int32List` / `Int64List` / `Float64List`, `List`, `Map` (nested). Custom objects must be converted (Map) or use Pigeon.

Dart	Kotlin	Swift
null	null	nil (NSNull when nested)
bool	Boolean	NSNumber(value: Bool)
int (≤32 bit)	Int	NSNumber(value: Int32)
int (>32 bit)	Long	NSNumber(value: Int)
double	Double	NSNumber(value: Double)
String	String	String
Uint8List	ByteArray	FlutterStandardTypedData(bytes:)
List	List	Array
Map	HashMap	Dictionary

19.3 MethodChannel example: battery level

Dart

```
class BatteryService {
  static const _channel = MethodChannel('com.mycompany.app/battery');

  Future<int?> getBatteryLevel() async {
    try {
      return await _channel.invokeMethod<int>('getBatteryLevel');
    } on PlatformException catch (e) {     // native returned an error
      debugPrint('Failed: ${e.code} ${e.message}');
      return null;
    } on MissingPluginException {          // no handler registered on this platform (e.g., web,
or forgot registration)
      return null;
    }
  }
}
```

Android (Kotlin) — `MainActivity.kt`

```
class MainActivity : FlutterActivity() {
    private val CHANNEL = "com.mycompany.app/battery"

    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)
        MethodChannel(flutterEngine.dartExecutor.binaryMessenger, CHANNEL).setMethodCallHandler {
call, result ->
            when (call.method) {
                "getBatteryLevel" -> {
                    val level = getBatteryLevel()
                    if (level != -1) result.success(level)
                    else result.error("UNAVAILABLE", "Battery level not available.", null)
                }
                else -> result.notImplemented()
            }
        }
    }

    private fun getBatteryLevel(): Int {
        val bm = getSystemService(BATTERY_SERVICE) as BatteryManager
        return bm.getIntProperty(BatteryManager.BATTERY_PROPERTY_CAPACITY)
    }
}
```

iOS (Swift) — `AppDelegate.swift`

```
@main
@objc class AppDelegate: FlutterAppDelegate {
  override func application(_ application: UIApplication,
                           didFinishLaunchingWithOptions launchOptions:
[UIApplication.LaunchOptionsKey: Any]?) -> Bool {
    let controller = window?.rootViewController as! FlutterViewController
    let channel = FlutterMethodChannel(name: "com.mycompany.app/battery",
                                       binaryMessenger: controller.binaryMessenger)
    channel.setMethodCallHandler { call, result in
      guard call.method == "getBatteryLevel" else { result(FlutterMethodNotImplemented); return }
      UIDevice.current.isBatteryMonitoringEnabled = true
      let level = UIDevice.current.batteryLevel
      if level < 0 {
        result(FlutterError(code: "UNAVAILABLE", message: "Battery level not available.", details:
nil))
      } else {
        result(Int(level * 100))
      }
    }
    GeneratedPluginRegistrant.register(with: self)
    return super.application(application, didFinishLaunchingWithOptions: launchOptions)
  }
}
```

Note: newer Flutter iOS templates adopt the **UIScene** lifecycle and implicit engine registration (`FlutterImplicitEngineDelegate` / registering in `didInitializeImplicitFlutterEngine`) — follow the migration guide for your Flutter version when the template differs. The channel concepts are identical.

Rules

- **Always** reply exactly once: `result.success`, `result.error`, or `result.notImplemented` — never twice (crash: "Reply already submitted"), never zero (Dart future hangs forever).
- Native handlers run on the **platform main thread** by default — don't block it; dispatch heavy work to a background thread and reply on the main thread (iOS: `DispatchQueue.main.async { result(...) }`; Android: post to main looper). (Android/iOS also support background **task queues** for channels: `binaryMessenger.makeBackgroundTaskQueue()`.)
- Dart → native calls must originate from the root isolate unless using `BackgroundIsolateBinaryMessenger`.
- Native → Dart calls: `channel.invokeMethod("onEvent", args)` on native; Dart side `channel.setMethodCallHandler((call) async { ... })`.

19.4 EventChannel example (native → Dart stream)

Dart

```
const _events = EventChannel('com.mycompany.app/charging');
Stream<String> get chargingStatus => _events.receiveBroadcastStream().map((e) => e as String);
```

Kotlin

```
EventChannel(flutterEngine.dartExecutor.binaryMessenger, "com.mycompany.app/charging")
  .setStreamHandler(object : EventChannel.StreamHandler {
      private var receiver: BroadcastReceiver? = null
      override fun onListen(arguments: Any?, events: EventChannel.EventSink) {
          receiver = object : BroadcastReceiver() {
              override fun onReceive(context: Context, intent: Intent) {
                  val status = intent.getIntExtra(BatteryManager.EXTRA_STATUS, -1)
                  events.success(if (status == BatteryManager.BATTERY_STATUS_CHARGING) "charging"
else "discharging")
              }
          }
          registerReceiver(receiver, IntentFilter(Intent.ACTION_BATTERY_CHANGED))
      }
      override fun onCancel(arguments: Any?) {   // Dart cancelled subscription → clean up!
          unregisterReceiver(receiver); receiver = null
      }
  })
```

19.5 Pigeon — type-safe channels (recommended) 🕊️

Hand-written channels use stringly-typed method names and maps → runtime errors. **Pigeon** (official) generates type-safe Dart, Kotlin/Java, Swift/ObjC, C++ (Windows), GObject (Linux) code from a Dart interface definition.

`pigeons/messages.dart`:

```
import 'package:pigeon/pigeon.dart';

@ConfigurePigeon(PigeonOptions(
  dartOut: 'lib/src/messages.g.dart',
  kotlinOut: 'android/app/src/main/kotlin/com/mycompany/app/Messages.g.kt',
  kotlinOptions: KotlinOptions(package: 'com.mycompany.app'),
  swiftOut: 'ios/Runner/Messages.g.swift',
))

class DeviceInfo {
  DeviceInfo({required this.model, required this.osVersion, this.batteryLevel});
  String model;
  String osVersion;
  int? batteryLevel;
}
```

```
@HostApi()                          // Dart calls native
abstract class DeviceApi {
  DeviceInfo getDeviceInfo();
  @async
  bool requestPermission(String name);
}

@FlutterApi()                       // native calls Dart
abstract class DeviceEventsApi {
  void onLowBattery(int level);
}
```

Run: `dart run pigeon --input pigeons/messages.dart`. Then implement the generated `DeviceApi` interface in Kotlin/Swift and register with `DeviceApi.setUp(binaryMessenger, impl)`. In Dart: `final info = await DeviceApi().getDeviceInfo();` — fully typed. Pigeon also supports event channels (`@EventChannelApi`) in newer versions.

19.6 Dart FFI — calling C directly ⚡

FFI (Foreign Function Interface) calls C-ABI functions **synchronously**, with no message serialization — much faster than channels for compute-heavy native libraries (image processing, crypto, SQLite, ML runtimes, audio DSP, Rust libs).

```
// native/add.c
#include <stdint.h>
int32_t add(int32_t a, int32_t b) { return a + b; }
```

```
import 'dart:ffi';
import 'dart:io';

typedef AddNative = Int32 Function(Int32 a, Int32 b);
typedef AddDart = int Function(int a, int b);

final DynamicLibrary lib = Platform.isAndroid
    ? DynamicLibrary.open('libadd.so')
    : DynamicLibrary.process();          // iOS: statically linked

final add = lib.lookupFunction<AddNative, AddDart>('add');
print(add(2, 3)); // 5
```

- Memory: `package:ffi` → `calloc`/`malloc`, `toNativeUtf8()`, **free manually** (`calloc.free(ptr)`) or use `Arena`/`using`. Native finalizers (`NativeFinalizer`) for cleanup.

- `@Native` annotations (Dart 3.x) for static binding resolution with **native assets / build hooks** (`hook/build.dart`, stable in Dart 3.10 / Flutter 3.38) — packages can compile and bundle native code automatically (C via `native_toolchain_c`, Rust via community tooling).
- **ffigen** generates bindings from C headers; **jnigen** generates Dart bindings for Java/Kotlin (Android) via JNI; **Objective-C/Swift** interop via ffigen (Objective-C bindings; Swift via `@objc` / swiftgen tooling). This "direct native interop" is the Flutter team's long-term direction to reduce channel boilerplate; combined with the merged UI/platform thread it enables synchronous calls to platform APIs.
- `flutter create --template=plugin_ffi` / `package_ffi` templates.
- Long-running native calls block the calling isolate → call from a helper isolate.
- Rust: `flutter_rust_bridge` generates high-level bindings (async, structs, streams) — popular for crypto/heavy logic.

Channels vs FFI

	Platform channels / Pigeon	FFI
Target	**Platform APIs in Kotlin/Java/Swift/ObjC**	**C ABI (C, C++ via extern "C", Rust, Go via cgo) — and via jnigen/ffigen, Java/ObjC**
Sync?	**Async only**	**Sync (and async callbacks via NativeCallable)**
Overhead	**Serialization + thread hop**	**Very low**
Use	**Camera SDKs, payment SDKs, OS services**	**Compute libraries, DBs, codecs**

19.7 Writing a plugin 🔌

```
flutter create --template=plugin --platforms=android,ios,web --org com.mycompany my_scanner
```

Structure:

```
my_scanner/
├── lib/my_scanner.dart                      # public Dart API
├── lib/my_scanner_platform_interface.dart    # abstract PlatformInterface (federated style)
├── lib/my_scanner_method_channel.dart        # default MethodChannel implementation
├── android/src/main/kotlin/.../MyScannerPlugin.kt  # implements FlutterPlugin, MethodCallHandler,
(ActivityAware)
├── ios/Classes/MyScannerPlugin.swift         # (or ios/my_scanner/Sources/... for SwiftPM)
├── example/                                  # example app (also used for integration tests)
└── pubspec.yaml  →  flutter: plugin: platforms: android: package/pluginClass, ios: pluginClass
...
```

Android plugin essentials:

- `FlutterPlugin.onAttachedToEngine` / `onDetachedFromEngine` (register/unregister channels).

- Need an `Activity` (permissions, startActivityForResult, UI)? Implement `ActivityAware` (`onAttachedToActivity`, `onDetachedFromActivityForConfigChanges`, ...) and `PluginRegistry.ActivityResultListener` / `RequestPermissionsResultListener`.
- Android V1 embedding (`Registrar`) is **removed** — old plugins using it don't compile with current Flutter ("No registrar"/"v1 embedding" errors). Update or fork.

iOS plugin: `FlutterPlugin` protocol, `register(with registrar:)`, `addApplicationDelegate` for app lifecycle events, Swift Package Manager support (`Package.swift`) + CocoaPods `.podspec`.

Federated plugins

Split one plugin into packages: `my_scanner` (app-facing), `my_scanner_platform_interface` (abstract interface using `plugin_platform_interface` with token verification), `my_scanner_android`, `my_scanner_ios`, `my_scanner_web`, ... Benefits: different teams/maintainers per platform, add platforms without touching the core, endorsed implementations (`default_package` in pubspec). Used by official plugins (url_launcher, shared_preferences, camera...).

Testing plugins

- Dart side: mock the channel:

```
TestDefaultBinaryMessengerBinding.instance.defaultBinaryMessenger
    .setMockMethodCallHandler(const MethodChannel('com.mycompany.app/battery'), (call) async {
  if (call.method == 'getBatteryLevel') return 42;
  return null;
});
```

- Or set `MyScannerPlatform.instance = FakePlatform();` (platform interface pattern).
- Native unit tests (JUnit / XCTest) in the example app; integration tests in `example/integration_test`.

19.8 Platform views — embedding native UI 🪟

When you need a **native view** inside Flutter (Google Maps, WebView, native ad views, a native video player/AR SDK):

- Android: `AndroidView` / `PlatformViewLink` with modes:
  - **Hybrid Composition (HC)** — native view added to the Android view hierarchy; best fidelity (accessibility, text input), but historically reduced Flutter performance / thread synchronization costs.
  - **Texture Layer Hybrid Composition (TLHC)** — renders native view into a texture; better Flutter performance; some views (SurfaceView-based) have issues.
  - **Virtual Display** (legacy fallback).
  - Newer Flutter versions (3.44) introduced an improved **"Hybrid Composition++"** mode for Android (requires Vulkan-capable devices) — check docs for status.

- iOS: `UiKitView` (native views composited with Flutter; costs extra layers — avoid many overlapping).
- macOS: `AppKitView`; Web: `HtmlElementView` (with `ui_web.platformViewRegistry.registerViewFactory`).

Costs: more memory, possible jank when scrolling many platform views, gesture conflicts (`gestureRecognizers: {Factory(() => EagerGestureRecognizer())}` to give the native view gestures inside scrollables), keyboard issues. Keep their number small; avoid platform views in long lists.

19.9 Add-to-App (Flutter inside an existing native app)

Large companies (e.g., banks, e-commerce) often adopt Flutter **incrementally**:

```
flutter create -t module my_flutter_module
```

- Android: include the module as a source dependency in `settings.gradle` or as AAR (`flutter build aar`); start with `FlutterActivity.withCachedEngine("engine_id")`, `FlutterFragment`, or `FlutterView`.
- iOS: embed via CocoaPods (`install_all_flutter_pods`) or frameworks (`flutter build ios-framework`); present `FlutterViewController(engine:)`.
- **Pre-warm & cache** `FlutterEngine` to avoid startup delay (engine startup takes time and memory). Use `FlutterEngineGroup` to create multiple lightweight engines sharing resources (multiple Flutter screens).
- Entry points: run a specific Dart function (`@pragma('vm:entry-point') void featureMain()`) and initial route.
- Communication with host via platform channels/Pigeon.
- Limitations/gotchas: multiple Flutter instances memory, plugin registration per engine, hot reload via `flutter attach`, consistent theming, back navigation between native/Flutter stacks.

19.10 Web interop (JS)

Modern way: `dart:js_interop` + `package:web` (replacing old `dart:html`, `dart:js`, `package:js`, which are deprecated and **don't work with Wasm**).

```
import 'dart:js_interop';
import 'package:web/web.dart' as web;

@JS('alert')
external void jsAlert(JSString message);

@JS('JSON.stringify')
external JSString stringify(JSAny? obj);

extension type Analytics._(JSObject _) implements JSObject {
```

```
  external void track(JSString event);
}
@JS('window.analytics')
external Analytics get analytics;

void demo() {
  jsAlert('Hello from Dart'.toJS);
  analytics.track('signup'.toJS);
  web.window.localStorage.setItem('k', 'v');
}
```

Conditional imports keep mobile code free of web libraries (Chapter 3).

19.11 Platform detection

```
import 'package:flutter/foundation.dart';
if (kIsWeb) { ... }
switch (defaultTargetPlatform) {          // works on web too (reports underlying OS)
  case TargetPlatform.android: ...
  case TargetPlatform.iOS: ...
  default: ...
}
// dart:io Platform.isAndroid — ❌ crashes on web; guard with kIsWeb first
Theme.of(context).platform                // overridable in tests/theme
```

19.12 Common native integration bugs

Error	Cause / Fix
`MissingPluginException(No implementation found for method X on channel Y)`	Hot reload after adding plugin (do a full rebuild); channel name mismatch; plugin doesn't support platform; native registration missing (custom engine/add-to-app without `GeneratedPluginRegistrant`); background isolate
`PlatformException(...)`	Native returned error — read code/message; handle in Dart
Future never completes	Native handler never called `result`
`Reply already submitted` crash	result called twice
App crashes calling UI API from background thread	Reply/UI on main thread
`Unhandled Exception: type '_Map<Object?, Object?>' is not a subtype of type 'Map<String, dynamic>'`	Channel maps are `Map<Object?, Object?>` → `Map<String, dynamic>.from(result)` or `.cast<String, dynamic>()` (nested maps too!)
Release build: native class not found	R8/ProGuard stripped it → add `-keep` rules
iOS: plugin not found	`pod install` not run / SwiftPM issue / min iOS version
Plugin uses v1 embedding	Upgrade plugin/fork

Error	Cause / Fix
FFI `Failed to lookup symbol`	Wrong lib name, symbol mangled (C++ needs `extern "C"`), library not bundled for ABI, iOS symbol stripped (`-Wl,-u` / mark as used)

19.13 Interview Questions — Platform integration

Q1. What are platform channels? Named asynchronous message channels between Dart and host platform code using codecs; MethodChannel (RPC), EventChannel (streams), BasicMessageChannel (messages).

Q2. How would you use a native SDK without a Flutter plugin? Write a plugin or in-app channel code (preferably Pigeon) wrapping the SDK in Kotlin/Swift, expose a Dart API, handle lifecycle/threading/errors, test on both platforms; or FFI if it's a C library; or jnigen/ffigen for direct bindings.

Q3. What is Pigeon? A code generator producing type-safe channel code for Dart and native languages from a Dart interface definition.

Q4. What is Dart FFI and when would you use it? Direct synchronous calls into C-ABI libraries with minimal overhead — compute-heavy or existing native libraries.

Q5. MethodChannel vs EventChannel? Request/response vs continuous native→Dart event stream.

Q6. What is a federated plugin? A plugin split into app-facing, platform-interface and per-platform implementation packages.

Q7. What are platform views and their costs? Embedding native views in the Flutter tree (AndroidView/UiKitView/HtmlElementView); costs in performance, memory, composition and gestures.

Q8. What is add-to-app? Integrating Flutter modules into existing native apps; uses cached/pre-warmed FlutterEngines and channels for communication.

Q9. Which thread do platform channel handlers run on? The platform main thread by default (background task queues are available); responses should be sent on the main thread.

Q10. How do you call JS from Flutter web? `dart:js_interop` with `@JS` external declarations and extension types; `package:web` for DOM APIs.

Q11. Why does `MissingPluginException` happen? No registered native implementation for that channel on the current engine/platform — commonly after hot reload of a newly added plugin, in tests, unsupported platform, or wrong channel name.

PART V

Expert Engineering

Chapter 20 — Flutter Internals: Bindings, the Frame Pipeline, Build/Layout/Paint/Composite, Layers, Impeller, the Dart VM & Hot Reload

Goal: Know how Flutter works *under the hood*. This is what distinguishes a senior/lead candidate, and it's what lets you debug "impossible" bugs and performance problems.

20.1 The bindings — glue between framework and engine

`WidgetsFlutterBinding` is a singleton mixing in several bindings:

Binding	Responsibility
`BindingBase`	Base, service extensions
`GestureBinding`	Receives pointer events from the engine, hit tests, dispatches, runs gesture arena
`SchedulerBinding`	Frame scheduling: transient callbacks (animations/Tickers), persistent callbacks (rendering pipeline), post-frame callbacks; task priorities
`ServicesBinding`	Platform messages (channels), lifecycle, keyboard, system chrome
`PaintingBinding`	Image cache, shader warm-up (legacy), font loading
`SemanticsBinding`	Accessibility
`RendererBinding`	Owns the `PipelineOwner` & render views (root of RenderObject tree), drives layout/paint/composite
`WidgetsBinding`	Owns the `BuildOwner` (element tree builds), lifecycle observers, routes from platform

`runApp(widget)` → `WidgetsFlutterBinding.ensureInitialized()` → attaches root widget (`View` → `RawView` ... with the `RenderView` root) → `scheduleWarmUpFrame()`.

Multi-view: since Flutter 3.x the binding supports **multiple views** (`PlatformDispatcher.views`, `View` widget, `runWidget`) — used for multi-window desktop (in progress) and web embedding into multiple DOM elements.

20.2 The life of a frame 🎞️

```
VSync signal from OS (e.g., every 16.67ms @60Hz / 8.33ms @120Hz)
        |
        ▼
Engine calls PlatformDispatcher.onBeginFrame ──► SchedulerBinding.handleBeginFrame
        |   ① Transient callbacks: Tickers → AnimationControllers update values
        |      (listeners mark widgets dirty / render objects needsPaint)
        ▼
Microtasks run (between beginFrame and drawFrame)
        ▼
Engine calls onDrawFrame ──► SchedulerBinding.handleDrawFrame
        |   ② Persistent callbacks → WidgetsBinding.drawFrame():
        |        a. BUILD   — buildOwner.buildScope(): rebuild dirty elements
        |        b. LAYOUT  — pipelineOwner.flushLayout()
        |        c. COMPOSITING BITS — pipelineOwner.flushCompositingBits()
        |        d. PAINT   — pipelineOwner.flushPaint()  (records into layers/pictures)
        |        e. COMPOSITE — renderView.compositeFrame(): build Scene from layer tree → engine
        |        f. SEMANTICS — pipelineOwner.flushSemantics()
        |        g. FINALIZE TREE — buildOwner.finalizeTree(): unmount inactive elements (dispose)
        |   ③ Post-frame callbacks (addPostFrameCallback)
        ▼
UI thread done (must be < frame budget)
        ▼
RASTER THREAD: Scene/LayerTree → Impeller (or Skia) → GPU commands → present
(must ALSO be < frame budget; runs in parallel with the next frame's UI work)
```

Key insights:

- **Two budgets**: UI thread work (build/layout/paint recording) and raster thread work (GPU rasterization) each must fit in ~16ms (60 fps) or ~8ms (120 fps). DevTools shows both bars. Red UI bar → expensive Dart code; red raster bar → expensive drawing (saveLayer, blur, clipping, huge images, too many layers).
- Flutter doesn't draw continuously — **frames are only scheduled when something changes** (`scheduleFrame` from setState, animations, etc.). Idle apps use no CPU.
- `addPostFrameCallback` runs after the frame is laid out & painted — safe to measure sizes or show dialogs after first build.
- `SchedulerBinding.instance.schedulerPhase` tells you where you are; calling `setState` during the build phase (except for descendants being built later) triggers the "setState during build" assertion.

20.3 Build phase in detail

- `BuildOwner` maintains a **dirty list** of elements (from `markNeedsBuild`).
- `buildScope` sorts dirty elements by **depth** (parents first) and calls `rebuild()` on each → `performRebuild()` → `build()` → `updateChild` for its children (reconciliation, Chapter 6).

- An element rebuilt by its parent in this pass is removed from the dirty list (no double build).
- Inactive elements (removed this frame) are kept in `_InactiveElements` and either **reactivated** (GlobalKey reparenting) or **unmounted** at `finalizeTree` (→ `State.dispose`).
- **GlobalKey lookups** use a registry in BuildOwner.
- `InheritedElement` keeps a set of dependents; when updated and `updateShouldNotify` is true → `didChangeDependencies` + mark dependents dirty.
- Each element has `_inheritedElements` (a persistent hash map) → `dependOnInheritedWidgetOfExactType` is O(1).

Element types

- `ComponentElement` (`StatelessElement`, `StatefulElement`, `ProxyElement` → `InheritedElement`, `ParentDataElement`).
- `RenderObjectElement` (`LeafRenderObjectElement`, `SingleChildRenderObjectElement`, `MultiChildRenderObjectElement`, `RootRenderObjectElement`), slivers too.
- `RenderObjectElement.mount` → `widget.createRenderObject(this)` and attaches to the nearest ancestor RenderObjectElement's render object (`insertRenderObjectChild`).
- On update → `widget.updateRenderObject(this, renderObject)` (setters call markNeedsLayout/Paint only if values changed).

20.4 Layout phase in detail

- `PipelineOwner._nodesNeedingLayout`: render objects marked by `markNeedsLayout()`.
- `markNeedsLayout` walks **up** to the nearest **relayout boundary** and marks it; only boundaries are queued.
- A render object is a **relayout boundary** if: `!parentUsesSize` OR `sizedByParent` OR constraints are **tight** OR parent isn't a RenderObject. Then its size can't change due to child changes from the parent's point of view.
- `flushLayout` sorts by depth and calls `_layoutWithoutResize` → `performLayout()`.
- `layout(constraints, parentUsesSize:)` skips work if constraints unchanged and not dirty (**caching**).
- **Single-pass layout**: each render object is laid out at most once per frame in normal cases → O(N). Exceptions: **intrinsic** queries (`getMinIntrinsicWidth`, used by `IntrinsicHeight/Width`, `Table`) do speculative extra passes → potentially $O(N^2)$; `LayoutBuilder` delays building children to layout time; **dry layout** (`computeDryLayout`) computes size without side effects.
- Box protocol: `BoxConstraints` → `Size`. Sliver protocol: `SliverConstraints` (scrollOffset, remainingPaintExtent, overlap, axis direction, cacheOrigin…) → `SliverGeometry` (scrollExtent, paintExtent, layoutExtent, maxPaintExtent, hitTestExtent, hasVisualOverflow…).
- Debug flags: `debugPrintMarkNeedsLayoutStacks`, `debugProfileLayoutsEnabled` (shows layout events in timeline).

20.5 Paint & compositing

- `markNeedsPaint()` walks up to the nearest **repaint boundary** (`isRepaintBoundary == true`) and schedules it.
- Each repaint boundary has its own `OffsetLayer`; painting records drawing commands into a `PictureRecorder` → `Picture` stored in a `PictureLayer`.
- Painting doesn't rasterize! It **records** commands (display list). Rasterization happens on the raster thread.
- **Layer tree**: `ContainerLayer`, `OffsetLayer`, `PictureLayer`, `OpacityLayer`, `ClipRectLayer` / `ClipRRectLayer` / `ClipPathLayer`, `TransformLayer`, `ColorFilterLayer`, `ImageFilterLayer`, `BackdropFilterLayer`, `ShaderMaskLayer`, `TextureLayer` (video/camera), `PlatformViewLayer`, `LeaderLayer` / `FollowerLayer`, `AnnotatedRegionLayer`.
- `needsCompositing` bits: some effects (opacity on a repaint boundary subtree, platform views, textures) need separate layers; `flushCompositingBits` computes them.
- `compositeFrame` → `SceneBuilder` builds a `Scene` from the layer tree → `FlutterView.render(scene)` hands it to the engine.
- Retained layers: a repaint boundary that didn't change **reuses its previous layer** (no re-recording) — this is why `RepaintBoundary` helps for frequently-changing subtrees (e.g., an animated spinner over a static page) and for static complex subtrees next to changing ones.
- The engine may also **cache rasterized layers** (raster cache in Skia; Impeller has different caching strategies).
- `saveLayer` forces an offscreen buffer — expensive (Opacity on complex children historically, `ShaderMask`, `ColorFilter`, some clips with antiAliasWithSaveLayer, `BackdropFilter`). DevTools "Highlight oversized images" / "checkerboard offscreen layers" (`checkerboardOffscreenLayers: true`) help find them.

RepaintBoundary trade-off

Each boundary = extra layer (memory, compositing). Use when a subtree repaints often and independently. Flutter already adds them for list items (`addRepaintBoundaries`), routes, scroll views etc. DevTools "Highlight repaints" (rainbow borders) shows what repaints.

20.6 The rendering backends: Skia & Impeller 🖌️

Skia (historic default)

- Google's 2D graphics library (also used by Chrome, Android).
- Problem: **runtime shader compilation**. The first time an effect (e.g., a particular blur/gradient/clip combo) is drawn, Skia generates & compiles GPU shaders on the raster thread → long first-frame stutter = **"shader compilation jank"**, notably on iOS/Metal.
- Mitigation was **SkSL warm-up**: run app with `--cache-sksl`, press `M` to save shaders, bundle with `--bundle-sksl-path` — device/driver-specific & brittle. (No longer relevant with Impeller.)

Impeller (current default on mobile)

- Built by the Flutter team specifically for Flutter.
- **All shaders compiled ahead of time** at engine build time (a fixed, smaller set of shader variants) → **no runtime shader compilation jank**.
- Uses modern APIs: **Metal** (iOS/macOS), **Vulkan** (Android; falls back to **OpenGL ES** on devices without good Vulkan support).
- Predictable performance, tessellation of paths on GPU/CPU, designed for concurrency, easier to profile.
- Timeline:
  - iOS: default since **Flutter 3.10** (May 2023); Skia opt-out **removed in 3.29**.
  - Android: default on API 29+ since **Flutter 3.27**; older devices/drivers fall back (to OpenGL ES Impeller backend or Skia depending on version/device).
  - macOS/desktop: preview, becoming default progressively (Flutter 3.47 makes Impeller the desktop default per release notes).
  - Web: uses CanvasKit (Skia compiled to WebAssembly) or **skwasm** (with Wasm builds); Impeller-on-web work is ongoing.
- Opt-out on Android (while allowed): `flutter run --no-enable-impeller` or manifest meta-data `io.flutter.embedding.android.EnableImpeller` = false. (Some SDKs/plugins historically forced this off — and newer Flutter versions are removing Skia on modern Android entirely, so check your version.)

20.7 Threads & runners (recap with depth)

- **Platform runner** — main OS thread.
- **UI runner** — root isolate Dart code. Since Flutter 3.29 (Android/iOS) the UI and platform threads are **merged** by default (Dart runs on the platform main thread), enabling synchronous native interop. Conceptually, "UI thread" work is still separate from raster work.
- **Raster runner** — rasterization via Impeller/Skia.
- **IO runner** — image decoding, asset loading, GPU uploads.
- Dart `Isolate`s you spawn get their own OS threads from a thread pool.

20.8 The Dart VM, compilation & memory

Compilation pipeline

- **Debug (native)**: Dart source → **kernel** (`.dill`, intermediate representation produced by the CFE — Common Front End) → run in VM with **JIT** compiler; incremental compilation enables hot reload.
- **Profile/Release (native)**: kernel → **AOT compiler** (gen_snapshot) → native machine code snapshot (`libapp.so` on Android, `App.framework` on iOS). The release engine (`libflutter.so` / `Flutter.framework`) contains a precompiled runtime (no JIT — iOS forbids JIT anyway).

- **Web:** `dartdevc` (debug, modular JS), `dart2js` (release, optimized JS), `dart2wasm` (WasmGC).
- **Tree shaking:** removes unused classes/methods in AOT and dart2js; also icon font subsetting.
- **Obfuscation** (`--obfuscate --split-debug-info=<dir>`): renames symbols, reducing reverse engineering ease; symbol map needed to de-obfuscate stack traces (`flutter symbolize`).

Memory management & garbage collection

- Dart uses a **generational GC**:
  - **New space (young generation):** a **semi-space scavenger** (copying collector) — very fast for short-lived objects (most widgets!). Live objects are copied between two halves; dead ones are just abandoned.
  - **Old space: concurrent mark-sweep** (with compaction when needed) for long-lived objects (elements, render objects, state, caches).
- GC runs per isolate group; Flutter engine can schedule GCs at idle times between frames (`notifyIdle`) to avoid jank.
- **Memory leaks in Dart** = objects still reachable unintentionally: listeners not removed, stream subscriptions not cancelled, controllers not disposed, static/global caches growing, closures capturing `BuildContext`/State, timers. GC can't collect reachable objects.
- Tools: DevTools Memory view (heap snapshots, diff, allocation tracing), `leak_tracker` package (Flutter's own tests use it; can be enabled in widget tests via `LeakTesting`).
- Images are the biggest memory consumers — decoded bitmaps live in native memory (ImageCache holds `ui.Image` handles).

Snapshot & startup

App startup = native launch → engine init → Dart VM/isolate start → load AOT snapshot → `main()` → first frame. Measure with `flutter run --trace-startup --profile` (writes `start_up_info.json`: time to first frame). Optimize: defer heavy init, lazy-load features, avoid big synchronous work before `runApp`, use deferred components (Android dynamic feature modules) / deferred imports (web).

20.9 How hot reload actually works 🔥

1. You save a file; the Flutter tool's **frontend server** incrementally recompiles **changed libraries** into a kernel diff.
2. The diff is sent to the running Dart VM via the **VM Service** protocol.
3. The VM **reloads** the libraries: replaces class/function definitions **in place**, keeping the heap (all existing objects/state stay alive). Field layouts are updated; existing instances keep values.
4. Flutter calls `reassemble()` on the tree and schedules a full rebuild of all widgets (`build` methods rerun with new code).
5. Because `main()` / `initState` aren't re-run and static fields keep values, those changes aren't reflected (Chapter 1 list).

Hot reload is only possible with JIT (debug). On web, hot reload became supported (Flutter 3.32 experimental, stable/default in 3.35–3.38) via DDC's new module system. Hot restart: recompiles and restarts the root isolate (Dart state lost) without rebuilding native code.

20.10 Text rendering pipeline (brief)

`Text` → `RichText` → `RenderParagraph` → `TextPainter` → `dart:ui ParagraphBuilder` → engine (SkParagraph/libtxt, HarfBuzz shaping, ICU line breaking, font fallback) → glyph runs drawn via Impeller/Skia text renderer. Text layout can be expensive: avoid re-laying out large text unnecessarily; `TextPainter` should be cached in custom painters.

20.11 Platform channels internals

`MethodChannel.invokeMethod` → codec encodes to `ByteData` → `BinaryMessenger.send` → `PlatformDispatcher.sendPlatformMessage` → engine → platform runner → embedder's `BinaryMessenger` → handler → reply bytes back through the same path → completes Dart Future. Thread hops are why channels are slower than FFI.

20.12 Web rendering

- **CanvasKit** renderer: Skia compiled to WebAssembly drawing into `<canvas>` (WebGL) — consistent with mobile, ~1.5MB+ download.
- **skwasm**: used when building with `--wasm` (requires WasmGC support — Chromium-based browsers 119+ and Firefox 120+; Safari/WebKit support has lagged because of engine bugs, so check the current Flutter web docs); faster startup & multi-threaded rendering where cross-origin isolation headers allow; falls back to JS + CanvasKit when Wasm isn't supported.
- The old **HTML renderer** (DOM/CSS based) was **deprecated and removed** (2024–2025). `--web-renderer` flag removed accordingly.
- Semantics/accessibility tree mapped to DOM ARIA nodes; text input uses hidden DOM input elements.
- Implications: canvas apps aren't SEO-friendly; first load size; use Flutter web for **apps** (dashboards, tools, PWAs) rather than content sites.

20.13 Interview Questions — Internals

Q1. Describe what happens when setState is called until pixels change. markNeedsBuild → scheduleFrame → vsync → beginFrame (animations) → drawFrame: build dirty elements (diff widgets, update render objects) → layout dirty render objects (to relayout boundaries) → paint dirty repaint boundaries into layers → composite scene → raster thread rasterizes with Impeller/Skia → GPU presents.

Q2. What are relayout and repaint boundaries? Points in the render tree that stop layout/paint invalidation from propagating upward: relayout boundary when parent doesn't use child size/tight constraints/sizedByParent; repaint boundary has its own layer.

Q3. What is the difference between the UI thread and raster thread? UI runs Dart build/layout/paint recording; raster converts the layer tree into GPU commands. Either exceeding the frame budget causes jank.

Q4. What is shader compilation jank and how does Impeller fix it? Skia compiled shaders at first use causing stutters; Impeller precompiles all shaders at build time.

Q5. How does Dart's garbage collector work? Generational: fast copying scavenger for young objects, concurrent mark-sweep(-compact) for old; Flutter schedules GC during idle time.

Q6. JIT vs AOT in Flutter? JIT in debug for hot reload & fast iteration; AOT in profile/release for speed, startup and iOS compliance.

Q7. How does hot reload work? Incremental kernel compile of changed libraries, VM reloads code while preserving heap state, framework reassembles and rebuilds the widget tree.

Q8. What is the layer tree? Output of painting: a tree of layers (picture, transform, opacity, clip, etc.) which the engine composites and rasterizes.

Q9. What is saveLayer and why is it expensive? It renders content into an offscreen buffer and then composites it — costs memory bandwidth and GPU time, and on some GPUs forces render pass switches.

Q10. What is tree shaking? Removing unreachable code at compile time (AOT/dart2js) to reduce binary size.

Q11. What renderers exist for Flutter web? CanvasKit (default for JS builds) and skwasm (for Wasm builds); the HTML renderer was removed.

Q12. What is a Ticker and how does it relate to vsync? An object that calls a callback once per frame (driven by SchedulerBinding's transient callbacks on vsync), muted via TickerMode when not visible.

Chapter 21 — App Architecture for Large Projects: MVVM, Clean Architecture, Repositories, DI, Modularization & Design Patterns

Goal: Structure a codebase that 5–50 developers can work on for years without it collapsing. This chapter is crucial for senior interviews ("How would you architect X?") and for real company work.

21.1 Why architecture? (5-year-old) 🏠

If you build a house by throwing bricks anywhere, it falls when you add a second floor. **Architecture** is the **blueprint**: kitchen here, plumbing there, walls with clear jobs. Then many builders can work at once, and you can replace the fridge without breaking the walls.

Goals of good app architecture:

1. **Separation of concerns** — UI, business logic and data access are separate.
2. **Testability** — logic testable without UI or real network.
3. **Scalability** — add features without touching everything.
4. **Maintainability/readability** — newcomers find things quickly.
5. **Replaceability** — swap REST for GraphQL, Hive for Drift, Firebase for your own backend with limited changes.
6. **Team parallelism** — features developed independently.

21.2 The official Flutter architecture recommendation (docs.flutter.dev/app-architecture)

Flutter's team published an architecture guide (2024–2025) recommending **MVVM with a layered approach**:



```
└──────────────────────────────┬──────────────────────────────┘
                               │
┌──────────────────── Data layer ────────────────────┐
│  Repositories (source of truth for app data,       │
│   caching, error handling, polling, domain models)│
│        | use                                       │
│  Services (wrap ONE external API: HTTP client,     │
│   platform plugin, local storage, Firebase)        │
└────────────────────────────────────────────────────┘
```

Key recommendations:

- **Views** contain minimal logic (only UI logic: animations, simple if/else for layout, routing calls).
- **ViewModels** convert repository data into UI state, hold UI state, expose **Commands** (an object wrapping an action with `running`, `error`, `completed` states — prevents double execution and lets UI show loading/errors).
- **Repositories** are the single source of truth for each data type; they transform raw service data into **domain models**; handle caching, retries, errors (return a `Result` type).
- **Services** are stateless wrappers around APIs.
- **Unidirectional data flow**: data flows Data → UI; events flow UI → Data.
- **Dependency injection** via `provider` package in the official examples.
- Use **immutable data models** (freezed recommended), `Result<T>` (Ok/Error) for errors.
- **Feature/layer folder structure** (UI organized by feature, data by type).

Command and Result (simplified from the official `compass_app` sample):

```dart
// A tiny Result type (the official sample defines a similar sealed class)
sealed class Result<T> {
  const Result();
  const factory Result.ok(T value) = Ok<T>;
  const factory Result.error(Exception error) = Error<T>;
}
final class Ok<T> extends Result<T> { const Ok(this.value); final T value; }
final class Error<T> extends Result<T> { const Error(this.error); final Exception error; }

class Command0<T> extends ChangeNotifier {
  Command0(this._action);
  final Future<Result<T>> Function() _action;

  bool _running = false;
  bool get running => _running;
  Result<T>? _result;
  bool get error => _result is Error;
  bool get completed => _result is Ok;

  Future<void> execute() async {
    if (_running) return;           // prevents double taps / duplicate requests
```

```
    _running = true;
    _result = null;
    notifyListeners();
    try {
      _result = await _action();
    } finally {
      _running = false;
      notifyListeners();
    }
  }
}

class BookingViewModel extends ChangeNotifier {
  BookingViewModel({required BookingRepository repo}) : _repo = repo {
    load = Command0<void>(_load)..execute();
  }
  final BookingRepository _repo;
  late final Command0<void> load;

  List<Booking> _bookings = [];
  List<Booking> get bookings => List.unmodifiable(_bookings);

  Future<Result<void>> _load() async {
    final result = await _repo.getBookings();     // Future<Result<List<Booking>>>
    switch (result) {
      case Ok(:final value):
        _bookings = value;
        notifyListeners();
        return const Result.ok(null);
      case Error(:final error):
        return Result.error(error);
    }
  }
}

// View
ListenableBuilder(
  listenable: viewModel.load,
  builder: (context, _) {
    if (viewModel.load.running) return const Center(child: CircularProgressIndicator());
    if (viewModel.load.error) return ErrorRetry(onRetry: viewModel.load.execute);
    return ListenableBuilder(
      listenable: viewModel,
      builder: (context, _) => BookingList(viewModel.bookings),
    );
  },
);
```

Note: naming a class `Error` shadows `dart:core`'s `Error` inside that library — the official sample does this deliberately; in your own code you may prefer `Failure` / `Err`.

21.3 Clean Architecture (very common in companies/interviews)

Based on Robert C. Martin's Clean Architecture — **dependencies point inward**: outer layers depend on inner layers, never the reverse.

```
lib/features/auth/
├── presentation/          ← widgets, pages, state (Bloc/Cubit/Notifier/ViewModel)
│   ├── pages/login_page.dart
│   ├── widgets/
│   └── bloc/login_bloc.dart
├── domain/                ← PURE DART: no Flutter, no JSON, no Dio
│   ├── entities/user.dart
│   ├── repositories/auth_repository.dart   ← abstract interface
│   └── usecases/login_user.dart
└── data/
    ├── models/user_dto.dart                ← JSON (de)serialization, mapping to entities
    ├── datasources/auth_remote_data_source.dart
    ├── datasources/auth_local_data_source.dart
    └── repositories/auth_repository_impl.dart  ← implements domain interface
```

Flow: `LoginPage` → `LoginBloc` → `LoginUser` use case → `AuthRepository` (interface) ← `AuthRepositoryImpl` → remote/local data sources → API/DB.

```dart
// domain
abstract interface class AuthRepository {
  Future<Result<User>> login(String email, String password);
}
class LoginUser {
  const LoginUser(this._repo);
  final AuthRepository _repo;
  Future<Result<User>> call(String email, String password) {
    if (!email.contains('@')) return Future.value(const Err(InvalidEmailFailure()));
    return _repo.login(email, password);
  }
}

// data
class AuthRepositoryImpl implements AuthRepository {
  AuthRepositoryImpl(this._remote, this._local);
  final AuthRemoteDataSource _remote;
  final AuthLocalDataSource _local;
  @override
```

```
  Future<Result<User>> login(String email, String password) async {
    try {
      final dto = await _remote.login(email, password);
      await _local.saveToken(dto.token);
      return Ok(dto.toEntity());
    } on DioException catch (e) {
      return Err(mapDioError(e));
    }
  }
}
```

Entities vs DTOs/models: entities are business objects used everywhere in the app; DTOs mirror the API JSON. Mapping isolates the app from API changes.

Criticism / pragmatism (senior insight)

Full Clean Architecture can be **over-engineering** for small features: use cases that just forward a call (`GetUser` → `repo.getUser()`), triple models per entity, lots of boilerplate. Pragmatic teams:

- Make the **domain layer optional** (the official Flutter guide does too) — add use cases only when logic is complex or shared.
- Keep repositories + clear state management as the minimum.
- Be consistent across the codebase.

21.4 Folder structure: feature-first vs layer-first

Layer-first (`lib/models`, `lib/screens`, `lib/services`, `lib/widgets`): simple for tiny apps; becomes a mess — a feature is scattered across folders.

Feature-first (recommended for medium/large):

```
lib/
├── main.dart / main_dev.dart / main_prod.dart
├── app/                        ← App widget, router, theme, DI bootstrap
├── core/                       ← shared: network client, errors, utils, extensions, constants,
design system
│   ├── network/
│   ├── error/
│   ├── storage/
│   └── widgets/
├── features/
│   ├── auth/ (data/ domain/ presentation/)
│   ├── cart/
│   ├── checkout/
│   └── profile/
└── l10n/
```

Rules: features don't import each other's internals (only public APIs / shared core); shared code goes to `core` (but keep `core` from becoming a junk drawer).

21.5 Modularization & monorepos 🧩

For big apps: split into **packages** (local path dependencies):

```
repo/
├── apps/
│   ├── customer_app/
│   └── merchant_app/          ← multiple apps sharing packages
├── packages/
│   ├── design_system/
│   ├── api_client/            ← generated OpenAPI client
│   ├── core/
│   ├── feature_auth/
│   ├── feature_payments/
│   └── analytics/
└── pubspec.yaml               ← Dart pub workspace root (Dart 3.6+)
```

Benefits: enforced boundaries (a package can't import what it doesn't depend on), faster incremental builds & tests, team ownership (CODEOWNERS), reuse across apps (white-label apps).

Tools:

- **Pub workspaces** (Dart 3.6+): root `pubspec.yaml` with `workspace: [apps/customer_app, packages/core, ...]`, members with `resolution: workspace` → single shared resolution & lock file, `dart pub get` once.
- **Melos** — scripts across packages (`melos bootstrap`, `melos run test`, versioning/publishing, filtering by changed packages). Melos 7 works on top of pub workspaces.
- OpenAPI generators (`openapi_generator`, `swagger_parser`) to produce typed API clients from backend specs.

21.6 Dependency Injection (DI) 💉

5-year-old: Instead of a toy making its own batteries, you **hand** it batteries. Then you can hand it **pretend batteries** in tests.

Constructor injection (the core idea):

```dart
class CheckoutViewModel {
  CheckoutViewModel({required this.cartRepository, required this.paymentService});
  final CartRepository cartRepository;
  final PaymentService paymentService;
}
```

How to wire up instances:

1. **Provider / Riverpod** — providers as DI containers (Riverpod providers are ideal: lazy, overridable in tests, scoped).
2. **get_it** — service locator:

```
final getIt = GetIt.instance;
void setupDi(Env env) {
  getIt
    ..registerLazySingleton<Dio>(() => buildDio(env))
    ..registerLazySingleton<AuthApi>(() => AuthApi(getIt()))
    ..registerLazySingleton<AuthRepository>(() => AuthRepositoryImpl(getIt(), getIt()))
    ..registerFactory<LoginBloc>(() => LoginBloc(authRepository: getIt()));   // new each time
}
final repo = getIt<AuthRepository>();
// tests: getIt.reset(); register fakes; or allowReassignment
```

- `registerSingleton` (eager), `registerLazySingleton` (on first use), `registerFactory` (new instance each time), `registerSingletonAsync` (async init) + `allReady()`, **scopes** (`pushNewScope` on login / `popScope` on logout — great for user sessions).

3. **injectable** — annotations (`@injectable`, `@lazySingleton`, `@module`, `@Environment('dev')`) generating get_it registration code.
4. **BlocProvider/RepositoryProvider** for Bloc apps.

Service locator criticism: dependencies hidden (calls to `getIt()` inside classes) → prefer injecting via constructors and only use the locator at composition roots (where objects are created).

21.7 SOLID in Flutter (with examples)

- **S**ingle Responsibility — a widget renders; a repository fetches; a validator validates. A 1,500-line `HomePage` violates it.
- **O**pen/Closed — extend behaviour via new classes (new payment method class implementing `PaymentMethod`) instead of editing a giant `switch`.
- **L**iskov Substitution — any `AuthRepository` implementation (real/fake) must behave correctly where the interface is expected.
- **I**nterface Segregation — small interfaces (`Readable`, `Writable`), not one god interface.
- **D**ependency Inversion — high-level modules (blocs) depend on abstractions (`AuthRepository`), not concretions (`FirebaseAuthRepository`).

Other principles: DRY (but avoid premature abstraction — "rule of three"), KISS, YAGNI, composition over inheritance, Law of Demeter.

21.8 Design patterns used in Flutter (interview list)

Pattern	Where in Flutter
Composite	Widget tree itself
Observer	ChangeNotifier, Streams, Listenable, InheritedWidget dependents
Builder	`ListView.builder`, `LayoutBuilder`, builder callbacks
Factory	Factory constructors, `fromJson`
Singleton	`WidgetsBinding.instance`, `FirebaseAuth.instance`, get_it singletons
Strategy	`ScrollPhysics`, `Curve`, `SliverGridDelegate`, `PageTransitionsBuilder`
Decorator	Wrapping widgets (`Padding(child:)`), Dio interceptors
Adapter	Repository mapping DTOs → entities; platform interface implementations
Facade	Repository hiding multiple data sources
Repository	Data layer
Command	Official MVVM Commands, `Intent` / `Action`
State	Bloc states / sealed classes
Proxy	`ProxyWidget`, `ProxyProvider`
Delegate	`SliverChildBuilderDelegate`, `CustomPainter`, `LocalizationsDelegate`
Template method	`State` lifecycle hooks, `RenderBox.performLayout`
Visitor	`visitChildElements`, `Element.visitChildren`
Mediator	Bloc between UI and repositories
Dependency Injection	Providers, get_it

21.9 Error handling architecture

- **Layers translate errors**: data layer catches technical exceptions and maps to domain failures; presentation maps failures to user messages (localized).
- **Result type** vs throwing: Result makes errors explicit in signatures (`Future<Result<User>>`); exceptions keep happy paths clean. Pick one convention per codebase.
- Global handlers for unexpected errors (FlutterError.onError, PlatformDispatcher.onError) + crash reporting.
- UI patterns: inline field errors, snackbar for transient, full-screen error with retry for page loads, offline banner.

21.10 Environment & configuration architecture

- **Flavors**: dev / staging / prod (Chapter 26) with different app IDs, names, icons, Firebase projects, API base URLs.
- `AppConfig` object built from `--dart-define-from-file` and injected at startup.
- Feature flags via remote config with local defaults.
- Separate `main_dev.dart`, `main_prod.dart` calling a shared `bootstrap(AppConfig)`:

```
Future<void> bootstrap(AppConfig config) async {
  WidgetsFlutterBinding.ensureInitialized();
  await setupErrorHandling(config);
  await setupDi(config);
  runApp(ProviderScope(overrides: [appConfigProvider.overrideWithValue(config)], child: const
App()));
}
```

21.11 Architecture case study: an e-commerce app (system design answer template)

Requirements: browse catalog (offline cache), search, product details, cart (sync across devices), checkout with payment gateway, order tracking (real-time), push notifications, auth, analytics, dark mode, i18n (EN/HI/AR).

Answer structure (use in interviews):

1. **Clarify** scope, scale (DAU), platforms, offline needs, team size.
2. **High-level modules**: auth, catalog, search, cart, checkout, orders, profile, notifications; core: network, storage, design system, analytics, config.
3. **Layers**: UI (Riverpod/Bloc), domain (cart rules, pricing), data (repositories with remote + local sources).
4. **Data flow**: catalog repository → stale-while-revalidate (Drift cache + API); search with debounce + cancellation; cart as local source of truth synced to server (optimistic updates, conflict resolution: server wins on price/stock); orders via WebSocket/FCM updates.
5. **Navigation**: go_router with StatefulShellRoute tabs, deep links (`/product/:id`), auth redirect preserving target.
6. **Networking**: Dio + interceptors (auth refresh with queue, retry, logging), OpenAPI generated client, error mapping, certificate pinning if needed.
7. **Payments**: gateway SDK via plugin; server creates order/payment intent; app never trusts client-side success — verify via backend webhook; idempotency keys.
8. **Performance**: image CDN with sizes, cached_network_image, list virtualization, pagination (cursor), isolates for parsing, startup optimization.
9. **Security**: secure storage for tokens, obfuscation, root/jailbreak detection (optional), no secrets in app, App Check/Play Integrity.
10. **Quality**: unit/widget/golden/integration tests, CI with lint/test/build, feature flags, staged rollouts, crash monitoring, analytics funnels.
11. **Scalability of team**: monorepo packages per feature, CODEOWNERS, design system package, ADRs (architecture decision records).

21.12 Interview Questions — Architecture

Q1. How do you structure a large Flutter app? Feature-first folders/packages, layered architecture (presentation/domain/data), repositories as single source of truth, DI, consistent state management, shared core & design system, tests per layer.

Q2. What is the repository pattern? An abstraction over data sources providing a clean API for domain data, handling caching, mapping and errors so the rest of the app doesn't know where data comes from.

Q3. Explain Clean Architecture in Flutter. Presentation, domain (entities, use cases, repository interfaces — pure Dart), data (models, data sources, repository implementations); dependency rule points inward; benefits testability & independence; trade-off boilerplate.

Q4. What does the official Flutter architecture guide recommend? MVVM: Views + ViewModels in UI layer, Repositories + Services in data layer, optional domain use cases, unidirectional data flow, Commands, Result type, immutable models, DI with provider.

Q5. What is dependency injection and how do you do it in Flutter? Supplying dependencies from outside rather than creating them inside; constructor injection wired via Riverpod/Provider/get_it(+injectable)/RepositoryProvider.

Q6. Service locator vs DI? Locator pulls dependencies from a global registry (hidden dependencies); DI pushes them via constructors (explicit). Use the locator only at composition roots.

Q7. How do you share code between multiple apps? Monorepo with packages (pub workspaces/melos), design system package, shared core/network/analytics packages.

Q8. Entities vs models/DTOs? Entities: domain objects independent of external formats; DTOs: data transfer shapes matching APIs/DB, mapped to entities.

Q9. What design patterns does Flutter use? Composite (widgets), observer (listenables/streams), builder, factory, singleton, strategy, delegate, decorator, repository, etc.

Q10. How do you avoid over-engineering? Start simple, add layers when complexity demands, avoid pass-through use cases, keep consistent conventions, document decisions (ADRs).

Chapter 22 — Testing: Unit, Widget, Golden, Integration, Mocking, Coverage & TDD

Goal: Write tests that catch real bugs, run fast in CI, and give the team confidence to refactor. Interviewers ask about test types, mocking, `pump` vs `pumpAndSettle`, testing async/streams/blocs/providers, and golden tests.

22.1 The testing pyramid 🔺

```
          ▲  Integration / E2E (few, slow, real device/emulator)
         ▲▲▲  Widget tests (many, fast, no device)
       ▲▲▲▲▲▲  Unit tests (most, fastest, pure Dart)
```

Type	Tests	Speed	Confidence	Tooling
Unit	**A function/class (logic, repos, blocs)**	**ms**	**narrow**	`test` / `flutter_test`
Widget	**A widget/screen in a fake environment**	**ms–s**	**UI behaviour**	`flutter_test` (`testWidgets`)
Golden	**Pixel output of a widget vs saved image**	**s**	**visual regressions**	`matchesGoldenFile`, `alchemist`
Integration	**Full app on device/emulator/browser**	**s–min**	**end-to-end flows**	`integration_test`, **Patrol, Maestro, Appium**

5-year-old: Unit test = check **one LEGO brick** isn't broken. Widget test = check **a small LEGO car** rolls. Integration test = check the **whole LEGO city** works together.

22.2 Unit tests

```dart
// lib/cart.dart
class Cart {
  final _items = <String, int>{};
  void add(String id, {int qty = 1}) {
    if (qty <= 0) throw ArgumentError.value(qty, 'qty');
    _items.update(id, (q) => q + qty, ifAbsent: () => qty);
  }
  int get totalItems => _items.values.fold(0, (a, b) => a + b);
}

// test/cart_test.dart
import 'package:flutter_test/flutter_test.dart';   // or package:test/test.dart for pure Dart packages
```

```dart
void main() {
  group('Cart', () {
    late Cart cart;
    setUp(() => cart = Cart());          // runs before each test
    tearDown(() {});                     // after each
    // setUpAll / tearDownAll — once per group/file

    test('starts empty', () {
      expect(cart.totalItems, 0);
    });

    test('adds quantities for same item', () {
      cart..add('a')..add('a', qty: 2);
      expect(cart.totalItems, equals(3));
    });

    test('throws on non-positive qty', () {
      expect(() => cart.add('a', qty: 0), throwsArgumentError);
    });
  });
}
```

Matchers: `equals`, `isTrue/isFalse`, `isNull/isNotNull`, `isA<Type>()`, `throwsA(isA<ApiException>())`, `contains`, `containsAll`, `hasLength`, `isEmpty`, `greaterThan`, `closeTo(3.14, 0.01)`, `orderedEquals`, `unorderedEquals`, `predicate`, `allOf/anyOf`, `isA<User>().having((u) => u.name, 'name', 'Asha')`.

Async:

```dart
test('fetches user', () async {
  final user = await repo.getUser('1');
  expect(user.name, 'Asha');
});
expect(future, completion(equals(42)));
expect(future, throwsA(isA<TimeoutException>()));
expect(stream, emitsInOrder([1, 2, emitsDone]));
expect(stream, emitsThrough(3));
expect(stream, neverEmits(0));
```

Time-dependent code: `fake_async` / `FakeAsync().run((async) { ...; async.elapse(Duration(seconds: 5)); })`; inject a `Clock` (package `clock`) instead of calling `DateTime.now()` directly.

Run: `flutter test`, `flutter test test/cart_test.dart`, `--name "adds"`, `--tags`, `--coverage`, `-j 4` (concurrency), `--reporter expanded`.

22.3 Mocking & fakes

- **Fake** — a simple working implementation (in-memory repository). Preferred for many cases; readable.
- **Mock** — records calls and returns programmed responses; verify interactions.
- **Stub** — returns canned data.

mocktail (no codegen — most popular)

```
import 'package:mocktail/mocktail.dart';

class MockAuthApi extends Mock implements AuthApi {}
class FakeLoginRequest extends Fake implements LoginRequest {}

void main() {
  setUpAll(() => registerFallbackValue(FakeLoginRequest())); // needed for any() with custom types

  test('login saves token', () async {
    final api = MockAuthApi();
    final store = InMemoryTokenStore();                      // a fake
    when(() => api.login(any())).thenAnswer((_) async => const LoginResponse(token: 't1'));

    final repo = AuthRepositoryImpl(api, store);
    await repo.login('a@b.com', 'pw');

    expect(await store.read(), 't1');
    verify(() => api.login(any())).called(1);
    verifyNoMoreInteractions(api);
  });

  test('throws on network error', () {
    final api = MockAuthApi();
    when(() => api.login(any())).thenThrow(const SocketException('no net'));
    expect(() => AuthRepositoryImpl(api, InMemoryTokenStore()).login('a', 'b'),
throwsA(isA<NetworkFailure>()));
  });
}
```

- `thenReturn` for sync values; `thenAnswer((_) async => ...)` for Futures (using `thenReturn(Future.value(x))` is an error in mockito/mocktail).
- `captureAny()` + `verify(...).captured`.
- **mockito** — uses codegen `@GenerateNiceMocks([MockSpec<AuthApi>()])` + build_runner (null-safety era).

Rule: **mock at the boundaries** (API client, platform services), not your own domain logic. Too many mocks = brittle tests.

22.4 Widget tests 🧪

```
void main() {
  testWidgets('counter increments when FAB tapped', (WidgetTester tester) async {
    await tester.pumpWidget(const MaterialApp(home: CounterPage()));   // ❗ wrap in MaterialApp
for Theme/Directionality/Navigator

    expect(find.text('0'), findsOneWidget);
    expect(find.text('1'), findsNothing);

    await tester.tap(find.byIcon(Icons.add));
    await tester.pump();                      // rebuild one frame

    expect(find.text('1'), findsOneWidget);
  });

  testWidgets('login shows error on invalid email', (tester) async {
    await tester.pumpWidget(MaterialApp(home: LoginPage(authRepository: FakeAuthRepository())));
    await tester.enterText(find.byKey(const Key('login_email')), 'bad-email');
    await tester.tap(find.widgetWithText(FilledButton, 'Login'));
    await tester.pumpAndSettle();
    expect(find.text('Enter a valid email'), findsOneWidget);
  });
}
```

Finders

`find.text('x')`, `find.textContaining('x')`, `find.byType(ListTile)`, `find.byKey(const Key('k'))`, `find.byIcon(Icons.add)`, `find.byWidget(w)`, `find.byWidgetPredicate((w) => w is Text && w.data!.startsWith('A'))`, `find.byTooltip('Delete')`, `find.bySemanticsLabel('...')`, `find.descendant(of: ..., matching: ...)`, `find.ancestor(...)`, `find.widgetWithText(ElevatedButton, 'Save')`, `.first`, `.last`, `.at(i)`. Matchers: `findsOneWidget`, `findsNothing`, `findsWidgets`, `findsNWidgets(n)`, `findsAtLeastNWidgets(n)`, `findsAny`.

Interactions

`tap`, `longPress`, `enterText`, `drag(finder, Offset(0, -300))`, `fling`, `scrollUntilVisible(finder, 200, scrollable: find.byType(Scrollable).first)`, `ensureVisible`, `sendKeyEvent(LogicalKeyboardKey.enter)`, `pageBack()`, `binding.setSurfaceSize(Size(400, 800))` / `tester.view.physicalSize = ...` (+ `addTearDown(tester.view.reset)`) to test responsive layouts.

`pump` vs `pumpAndSettle` vs `pump(duration)` (MOST ASKED)

- `pump()` — triggers **one frame** (rebuilds dirty widgets). Time doesn't advance (unless you pass a duration).
- `pump(Duration(ms: 500))` — advances the fake clock by that duration and pumps a frame — use to step through animations/timers.

- `pumpAndSettle()` — keeps pumping frames (100ms steps by default) **until no more frames are scheduled** (animations finished). ⚠️ **Times out** (error "pumpAndSettle timed out") with infinite animations (`CircularProgressIndicator`, repeating controllers) → use `pump(duration)` instead.
- Widget tests run in a **FakeAsync zone**: timers and `Future.delayed` only progress when you pump with durations. Real async I/O (HTTP, files) won't work naturally → use `tester.runAsync(() async {...})` for real async, or (better) fakes.
- By default, **HTTP requests in widget tests return 400** (`HttpOverrides` set by the test binding) — mock network.

Providing dependencies in widget tests

```
await tester.pumpWidget(
  ProviderScope(
    overrides: [productRepoProvider.overrideWithValue(FakeProductRepo())],
    child: const MaterialApp(home: ProductListPage()),
  ),
);
// Bloc:
await tester.pumpWidget(BlocProvider<LoginBloc>.value(value: mockLoginBloc, child: const
MaterialApp(home: LoginView())));
whenListen(mockLoginBloc, Stream.fromIterable([const LoginState(status: LoginStatus.failure,
error: 'x')]), initialState: const LoginState());
```

Navigation: provide a `MockNavigatorObserver` or a test `GoRouter`, then verify current route/widgets.

Images: `Image.network` fails in tests → use `mocktail_image_network` (`mockNetworkImagesFor(() => tester.pumpWidget(...))`) or inject image providers.

22.5 Golden tests 🖼️

```
testWidgets('ProductCard golden', (tester) async {
  await tester.pumpWidget(MaterialApp(
    theme: AppTheme.light,
    home: Scaffold(body: Center(child: ProductCard(product: sampleProduct))),
  ));
  await expectLater(find.byType(ProductCard),
matchesGoldenFile('goldens/product_card_light.png'));
});
```

- Generate/update: `flutter test --update-goldens`.
- ⚠️ **Fonts**: tests use the "Ahem" test font by default (text renders as boxes) unless you load real fonts (`loadAppFonts()` from `golden_toolkit` (discontinued), or `alchemist`, or load fonts manually with `FontLoader`).

- ⚠️ **Platform differences**: golden pixels differ between macOS/Linux/Windows rendering → generate goldens in the same environment as CI (e.g., Docker/Linux) or use tolerant comparators; `alchemist` provides CI-friendly goldens (with obscured text) + platform goldens.
- Test multiple themes (light/dark), text scales, locales (RTL), and screen sizes.
- Great for design systems (catch unintended visual changes in PRs).

22.6 Integration tests (E2E)

```
dev_dependencies:
  integration_test:
    sdk: flutter
  flutter_test:
    sdk: flutter
```

`integration_test/app_test.dart`:

```
import 'package:integration_test/integration_test.dart';
import 'package:my_app/main.dart' as app;

void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets('user can log in and see home', (tester) async {
    app.main();                          // or pump your App with test config
    await tester.pumpAndSettle();
    await tester.enterText(find.byKey(const Key('login_email')), 'test@example.com');
    await tester.enterText(find.byKey(const Key('login_password')), 'password123');
    await tester.tap(find.byKey(const Key('login_submit')));
    await tester.pumpAndSettle(const Duration(seconds: 5));
    expect(find.text('Welcome'), findsOneWidget);
  });
}
```

Run: `flutter test integration_test/app_test.dart -d <device>`; for web: `flutter drive --driver=test_driver/integration_test.dart --target=integration_test/app_test.dart -d chrome`. Firebase Test Lab / AWS Device Farm / BrowserStack for device matrices.

Limitations of `integration_test`: can't interact with **native UI** (permission dialogs, notifications, system settings, WebViews' native content).

- **Patrol** (by LeanCode) — extends Flutter tests with native automation (grant permissions, tap notifications, toggle Wi-Fi), nicer finders (`$('Login').tap()`), test isolation, sharding.
- **Maestro** — YAML-based black-box UI flows (works with Flutter via semantics/text), popular for quick E2E.
- **Appium Flutter driver** — for QA teams using Appium.

- `flutter_driver` — the **old** API (deprecated in favour of integration_test).

Tips: use **stable keys** (`Key('login_submit')`) or semantics identifiers (`Semantics(identifier: ...)` for automation tools) instead of texts that change with localization; seed test data via a test backend/mock server; make tests independent; avoid fixed sleeps.

22.7 Testing specific things

- **Blocs** — `bloc_test` (Chapter 15).
- **Riverpod** — `ProviderContainer.test` + overrides (Chapter 15).
- **ChangeNotifier** — call methods, assert state, count `notifyListeners` with a listener counter.
- **Streams** — `emitsInOrder`, `StreamQueue` (package `async`).
- **Platform channels** — `setMockMethodCallHandler` (Chapter 19).
- **SharedPreferences** — `SharedPreferences.setMockInitialValues({})`.
- **Navigation with go_router** — pump a `MaterialApp.router` with the real router and fake dependencies; assert `find.byType(DetailsPage)`.
- **Accessibility** — `meetsGuideline` (Chapter 10).
- **Memory leaks** — `leak_tracker_flutter_testing` (`testWidgets(..., experimentalLeakTesting: LeakTesting.settings.withTrackedAll())`).
- **Performance** — `integration_test` with `binding.traceAction(...)` / `watchPerformance` to record frame timings in CI (`flutter drive --profile`).

22.8 Coverage

```
flutter test --coverage                      # coverage/lcov.info
genhtml coverage/lcov.info -o coverage/html   # (lcov tool) HTML report
lcov --remove coverage/lcov.info '**/*.g.dart' '**/*.freezed.dart' -o coverage/lcov.info  #
exclude generated
```

Upload to Codecov/Coveralls/SonarQube in CI. Coverage is a **signal, not a goal** — 100% coverage with weak assertions is worthless; aim for meaningful coverage of business logic (e.g., 80%+ on domain/data layers).

22.9 TDD & testing best practices

- **Arrange–Act–Assert** structure; one behaviour per test; descriptive names ("emits failure when API returns 401").
- Test **behaviour**, not implementation details.
- Tests must be **deterministic** (no real time/network/random without seeds) and **independent** (no order dependence; reset singletons like get_it between tests).
- Keep test helpers: `pumpApp(tester, widget, {overrides})` extension wrapping MaterialApp, theme, localizations, providers.

- Write a failing test reproducing every production bug before fixing it (**regression tests**).
- TDD: Red → Green → Refactor.
- Run tests in CI on every PR; block merges on failures.
- Flaky test policy: fix or quarantine with a ticket — never ignore silently.

```
extension PumpApp on WidgetTester {
  Future<void> pumpApp(Widget widget, {List<Override> overrides = const []}) {
    return pumpWidget(ProviderScope(
      overrides: overrides,
      child: MaterialApp(
        localizationsDelegates: AppLocalizations.localizationsDelegates,
        supportedLocales: AppLocalizations.supportedLocales,
        theme: AppTheme.light,
        home: widget,
      ),
    ));
  }
}
```

22.10 Common test failures & fixes

Failure	Fix
`No MediaQuery widget ancestor found` / `No Directionality widget found` / `No Material widget found`	Wrap in `MaterialApp` (or `Directionality` / `Material`)
`pumpAndSettle timed out`	Infinite animation → `pump(duration)`; replace spinners in tests
`A Timer is still pending even after the widget tree was disposed`	Cancel timers in dispose; pump enough time; avoid real timers
`Bad state: No element` from finder `.first`	Widget not found — pump/settle first, check finder
`The finder ... could not be tapped because it's offscreen` / hit test warning	`ensureVisible` / `scrollUntilVisible`, larger surface size, widget covered by another
Real HTTP returns 400	Mock network
`MissingPluginException`	Mock channels / platform interface
Golden mismatch on CI only	Fonts/platform rendering differences → consistent environment
`LateInitializationError` in tests	setUp not initializing, test order dependence
`type 'Null' is not a subtype of type 'Future<...>'` (mocktail)	Missing `when(...).thenAnswer` stub for that call

22.11 Interview Questions — Testing

Q1. Types of tests in Flutter? Unit, widget, integration (plus golden visual tests).

Q2. pump vs pumpAndSettle? pump renders a single frame (optionally advancing time); pumpAndSettle repeatedly pumps until no frames are scheduled — fails with endless animations.

Q3. How do you mock dependencies? Inject interfaces; use mocktail/mockito mocks or hand-written fakes; override providers or use BlocProvider.value with mock blocs.

Q4. What are golden tests? Snapshot tests comparing rendered widget pixels against reference images to catch visual regressions.

Q5. How do you test a Bloc? bloc_test's `blocTest` with build/act/expect/verify, mocking repositories.

Q6. Why wrap widgets in MaterialApp during tests? Many widgets need ancestors provided by MaterialApp (MediaQuery, Directionality, Theme, Navigator, Localizations).

Q7. How do you test code that uses DateTime.now or timers? Inject a clock; use fake_async or widget tester's fake time with pump(duration).

Q8. integration_test vs flutter_driver vs Patrol? integration_test runs widget-test-style tests on devices (replaces deprecated flutter_driver); Patrol adds native interactions and better ergonomics.

Q9. What is code coverage and is 100% necessary? Percentage of lines executed by tests; useful signal but not a goal — focus on critical logic and meaningful assertions.

Q10. How do you make widget tests robust? Use keys/semantics finders, avoid real network/time, test behaviour, helper pump methods, deterministic data.

Q11. How do you test platform channel code? Mock MethodChannel handlers with TestDefaultBinaryMessengerBinding, or swap platform interface instances with fakes.

Q12. What's the difference between a mock and a fake? A fake is a lightweight working implementation; a mock is programmed with expectations and verifies interactions.

Chapter 23 — Performance Optimization & DevTools: Jank, Rebuilds, Memory Leaks, Startup Time & App Size

Goal: Diagnose and fix every common performance problem with evidence (DevTools), not guesses. Senior interviews: "Your app is janky on scroll — how do you investigate?", "How do you reduce app size?", "How do you find memory leaks?"

23.1 Performance mindset 🧭

1. **Measure first** — in **profile mode**, on a **real, low/mid-range device** (not the emulator, not a flagship only).
2. Find the **bottleneck** (UI thread? raster thread? network? startup?).
3. Fix the **biggest** problem; re-measure.
4. Prevent regressions (performance tests, benchmarks in CI, code review checklists).

Frame budgets: 60 Hz → **16.67 ms**; 90 Hz → 11.1 ms; 120 Hz → **8.33 ms**. Both the UI thread and the raster thread must finish within the budget.

23.2 DevTools tour 🧰

Open via `flutter run --profile` then press `v`, or from the IDE.

Tool	Use
Flutter Inspector	Widget tree, layout explorer (constraints, flex), select widget on device, "Highlight repaints", "Highlight oversized images", "Show guidelines", slow animations
Performance	Frame chart (UI & raster bars per frame; jank frames in red), timeline events (Build, Layout, Paint, Raster), "Track widget builds" / "Track layouts" / "Track paints" (shows which widgets rebuilt), rebuild stats, shader compilation flags (Skia)
CPU Profiler	Sample Dart CPU usage: bottom-up/top-down/call tree, flame chart — find expensive functions
Memory	Heap usage chart, GC events, heap snapshots & diff (find leaking classes), allocation tracing, "Profile memory" by class
Network	HTTP requests (dart:io HttpClient-based — includes `http`/Dio on native), timing, headers, bodies
Logging	`dart:developer` `log()` output, framework events
App Size	Analyze `--analyze-size` JSON: treemap of code/assets by package; diff two builds
Deep Links	Validate Android/iOS deep link setup
DevTools extensions	Provider, Riverpod, Bloc, Drift, Patrol... package-provided tabs
Debugger	Breakpoints, stepping, evaluate expressions

Also: **Performance overlay** (`showPerformanceOverlay: true` in MaterialApp, or `P` key) — two graphs (top = raster, bottom = UI); red bars = slow frames.

Custom timeline events for your own code:

```
import 'dart:developer';
Timeline.startSync('parseCatalog');
parseCatalog();
Timeline.finishSync();
// or: Timeline.timeSync('parseCatalog', parseCatalog);
```

23.3 Diagnosing jank: the decision tree 🌳

```
Jank frame in DevTools →
 ├─ UI bar high (Dart/main thread)?
 │    ├─ Many widgets rebuilding?       → reduce rebuild scope (const, split widgets,
select/BlocSelector, ValueListenableBuilder)
 │    ├─ Expensive build() logic?        → move computation out (memoize, precompute in state
layer, isolates)
 │    ├─ Layout expensive?               → avoid IntrinsicHeight/Width, nested shrinkWrap lists,
deep trees; use itemExtent
 │    ├─ Synchronous heavy work (JSON, sorting, crypto, image processing)? → Isolate.run / compute
 │    └─ GC pauses from huge allocations? → reduce allocations in hot paths
 └─ Raster bar high (GPU)?
      ├─ saveLayer-heavy effects: Opacity on complex subtree, ShaderMask, ColorFilter,
BackdropFilter, ClipPath with antiAlias → simplify/avoid, use FadeTransition, pre-blurred images
      ├─ Huge images decoded at full res → cacheWidth/ResizeImage, proper image sizes
      ├─ Too many layers / platform views → reduce
      ├─ Complex paths/shadows (many BoxShadows, large blur radius) → cheaper shadows, cache
      └─ (Skia only) First-run shader compilation → Impeller
```

23.4 Reducing rebuilds (UI thread) — techniques with examples

1. `const` everywhere possible

```
const SizedBox(height: 16);
const Text('Static label');
const MyStaticHeader();      // requires const constructor
```

Identical instances → Flutter skips rebuilding them.

2. Push state down / split widgets

```
// ❌ Entire page rebuilds every second because the clock is stored in the page state
class _PageState extends State<Page> { DateTime now = DateTime.now(); ... Timer → setState }

// ✅ Only ClockText rebuilds
class ClockText extends StatefulWidget { ... }  // owns its own timer & setState
```

3. Rebuild only what listens

- Provider: `Selector`, `context.select`, `Consumer` low in the tree.
- Riverpod: `ref.watch(provider.select((s) => s.count))`.
- Bloc: `BlocSelector`, `buildWhen`.
- `ValueListenableBuilder` / `ListenableBuilder` for tiny reactive parts.

4. Use the `child` parameter in builders

```
AnimatedBuilder(animation: c, child: const HeavyStatic(), builder: (_, child) =>
Transform.rotate(angle: c.value, child: child));
```

5. Avoid rebuilding because of MediaQuery

`MediaQuery.sizeOf(context)` instead of `MediaQuery.of(context)` (keyboard animation would otherwise rebuild everything every frame!).

6. Don't do work in build

❌ Sorting/filtering a 5,000-item list in `build`, `DateFormat(...)` creation per item, regex compilation per build, `jsonDecode` in build. ✅ Precompute in state layer; cache formatters (`static final _fmt = DateFormat.yMMMd();`).

7. Keys & list item stability

Stable `ValueKey`s keep elements/state when lists change; avoid `UniqueKey()` in build (forces rebuild & state loss each frame).

8. Avoid `setState` on huge widgets for tiny changes (e.g., text field typing rebuilding a whole form/page).

23.5 Rendering (raster) optimizations

- **Opacity**: animating `Opacity` widget → use `FadeTransition` / `AnimatedOpacity`. For a static semi-transparent image use `color.withValues(alpha:)` on the color or `Image(color:, colorBlendMode:)` / `opacity:` param instead of an Opacity widget (`Image` has an `opacity` parameter taking an `Animation<double>`).

- **Clipping**: `ClipRRect` per list item is costly; prefer `BoxDecoration(borderRadius:)` with `clipBehavior: Clip.antiAlias` only where needed. Avoid `Clip.antiAliasWithSaveLayer`.
- **Shadows/blur**: big `BoxShadow` blur radii and `BackdropFilter` (frosted glass) are expensive, especially over scrolling content; limit area (wrap with `ClipRect`), use `BackdropGroup` / `BackdropFilter.grouped` (newer Flutter APIs to share backdrop work) where available.
- **RepaintBoundary** around frequently repainting parts (animations, video, tickers, charts) to avoid repainting static content; verify with "Highlight repaints".
- **Images**: right size from CDN; `cacheWidth`; avoid `Image.network` with 4000×3000 photos in thumbnails; precache hero images.
- **Text**: avoid huge `Text` reflows each frame; cache `TextPainter`s in custom painters.
- **Platform views**: few, not in lists.

23.6 Lists & scrolling performance

(See Chapter 9 checklist) — builder constructors, itemExtent/prototypeItem, avoid shrinkWrap, keep item builds cheap, paginate, cached images, avoid nested scroll views.

23.7 Heavy computation → isolates

```
final result = await Isolate.run(() => expensiveSearchIndex(data));
```

Typical candidates: JSON > ~100KB, image filters/compression, PDF generation, encryption, diffing big lists, Markdown parsing of large docs. Note the message copying cost (send minimal data; use `TransferableTypedData` for bytes).

23.8 Memory: leaks & bloat 🧠

Common leak sources

1. Not disposing: `AnimationController`, `TextEditingController`, `ScrollController`, `PageController`, `TabController`, `FocusNode`, `ChangeNotifier`s you create, `StreamController`s.
2. Not cancelling: `StreamSubscription`, `Timer`, listeners added to global objects (`WidgetsBinding.addObserver` without remove), `ValueNotifier.addListener` on long-lived objects.
3. Global/static collections growing forever (caches without limits).
4. Closures capturing `BuildContext`/State stored in long-lived objects (singletons, streams).
5. Images: big decoded images in `ImageCache`; videos/camera controllers not disposed.
6. Riverpod providers that should be autoDispose; blocs not closed; get_it singletons holding user data after logout.

Finding leaks

- DevTools Memory → take snapshot → perform action N times (open/close screen) → snapshot → **diff**: classes whose instance counts keep growing (e.g., `_DetailsPageState` 10 instances after closing 10 times) = leak. Inspect **retaining path** (what holds it).
- `leak_tracker` in tests detects not-disposed and not-GCed objects.
- Android Studio Profiler / Xcode Instruments for native memory (images, platform views).

Reducing memory

- `ImageCache` limits: `PaintingBinding.instance.imageCache.maximumSizeBytes = 100 << 20;` clear on memory pressure (`didHaveMemoryPressure`).
- Decode at display size (`cacheWidth`).
- Paginate data; don't keep entire datasets in memory.
- Dispose heavy resources when screens close.

23.9 Startup time 🚀

Measure: `flutter run --profile --trace-startup` → `build/start_up_info.json` (engineEnterTimestampMicros, timeToFirstFrameMicros...). Also Android vitals / Xcode Organizer launch metrics.

Improve:

- Do the **minimum before** `runApp` — show UI fast (native splash → first frame), initialize non-critical SDKs later (analytics, remote config fetch, ads) in parallel (`Future.wait`) or after first frame.
- Avoid synchronous heavy work in `main()`/global initializers.
- Lazy-load features; deferred imports (web code splitting; Android deferred components with Play Feature Delivery).
- Reduce initial asset decoding (big images/fonts/Lottie on the first screen).
- Pre-warm engine in add-to-app scenarios.
- Keep the first screen's widget tree lean.

23.10 App size 📦

Measure: `flutter build apk --analyze-size --target-platform android-arm64` (or `appbundle`/`ios`) → open the generated JSON in DevTools App Size tool. Real download size: Play Console's App size report / App Store Connect's App Thinning size report.

A Flutter "hello world" release is a few MB (engine + framework). Reduce:

1. **App Bundles (AAB)** — Play delivers per-ABI/density splits automatically. For APK distribution: `--split-per-abi`.

2. **Obfuscation + split debug info:** `--obfuscate --split-debug-info=build/symbols` (removes symbol names, smaller snapshot).
3. **Tree-shake icons** (default in release); avoid dynamic `IconData` construction.
4. **Assets:** compress images (WebP/AVIF), remove unused assets, use vector graphics (SVG→ `vector_graphics`), download large assets on demand (CDN), subset fonts, avoid shipping many font weights.
5. **Dependencies:** audit heavy packages (some pull big native SDKs: ML Kit models, maps, video, ffmpeg_kit). Use `--analyze-size` to see per-package size.
6. **Android:** `isMinifyEnabled` / `isShrinkResources` (R8) for Java/Kotlin code; `resConfigs` / `localeFilters` to keep only needed languages from libraries.
7. **iOS:** bitcode is deprecated (Xcode 14+); App Thinning delivers device-specific slices; check embedded frameworks.
8. **Deferred components** (Android) / deferred libraries (web).

23.11 Network performance

Batch/paginate requests, compress (gzip — automatic with Dio/http on native when server supports), cache (ETag/Cache-Control, local DB), avoid waterfall requests (parallelize with `Future.wait`), prefetch next pages, cancel unneeded requests, use HTTP/2/3 native clients (`cupertino_http`, `cronet_http`), CDN for images.

23.12 Battery & background

Avoid continuous timers/polling (use push or longer intervals), stop animations offscreen (vsync does it for Tickers), pause location/sensors when not needed, batch work with WorkManager, avoid wake locks.

23.13 Performance testing & monitoring in organisations

- **Benchmarks** in CI using `integration_test` + `traceAction` (frame build/raster time percentiles, e.g., `average_frame_build_time_millis`, `90th_percentile_frame_rasterizer_time_millis`, `missed_frame_build_budget_count`).
- **Production monitoring:** Firebase Performance (app start, screen rendering: slow/frozen frames, network traces), Sentry performance, Android vitals (ANR rate, slow rendering), Xcode Organizer (hangs, launch time).
- `FrameTiming` API: `SchedulerBinding.instance.addTimingsCallback((timings) { ... })` to report jank metrics from real users.
- Performance budget per screen; PR checklist.

23.14 Performance checklist (print this) ✅

- [ ] Measured in profile mode on low-end device
- [ ] `const` constructors used; lint `prefer_const_constructors` on
- [ ] Stateful widgets small; state scoped; selective rebuilds
- [ ] No heavy work in `build`; formatters cached
- [ ] Lists use builders/slivers; no `shrinkWrap` on long lists; `itemExtent` when possible
- [ ] Images sized/cached; `cacheWidth` for thumbnails
- [ ] No unnecessary `Opacity` / `ClipPath` / `BackdropFilter` / `saveLayer`
- [ ] Animations use transitions/AnimatedBuilder with `child`; RepaintBoundary where useful
- [ ] Heavy CPU work in isolates
- [ ] All controllers/subscriptions/timers disposed
- [ ] Startup: minimal pre-`runApp` work, deferred SDK init
- [ ] App size analyzed; assets optimized; R8 enabled; obfuscation on
- [ ] Performance monitored in production

23.15 Interview Questions — Performance

Q1. How do you investigate jank? Reproduce in profile mode on a real device; DevTools Performance to see whether UI or raster thread exceeds budget; track widget rebuilds/layouts/paints; CPU profiler for Dart hotspots; fix and re-measure.

Q2. How do you reduce unnecessary rebuilds? const widgets, splitting widgets, localized state, selective listening (select/Selector/BlocSelector/buildWhen), builder `child` param, avoiding MediaQuery.of, stable keys.

Q3. What is RepaintBoundary? Creates a separate layer so repaints inside don't trigger repaints outside (and vice versa); useful for frequently changing subtrees; costs memory.

Q4. Why is Opacity expensive and alternatives? It may require an offscreen saveLayer for children; use FadeTransition/AnimatedOpacity for animation, alpha colours, or Image opacity.

Q5. How do you handle heavy computation? Move to background isolates (Isolate.run/compute), chunk work, or move server-side.

Q6. How do you find memory leaks? DevTools memory snapshots & diffs, retaining paths, leak_tracker in tests; check disposals and subscriptions.

Q7. How do you reduce app size? AAB/split per ABI, obfuscation + split debug info, asset optimization, remove heavy deps, R8 shrinking, deferred components, analyze-size.

Q8. How do you improve startup time? Minimize work before first frame, defer/parallelize initialization, lazy loading, lightweight first screen, native splash, trace-startup measurement.

Q9. Why test performance in profile mode? Debug mode uses JIT with assertions and debugging overhead; profile is AOT like release but keeps profiling hooks.

Q10. What are the UI and raster thread bars in the performance overlay? Bottom graph = UI (Dart) thread frame times; top = raster (GPU) thread; red bars exceed the frame budget.

Q11. What is the frame budget at 120Hz? ~8.33 ms per frame for each of UI and raster work.

Q12. How do images affect performance? Decoding large images costs CPU/memory and raster time; downscale at decode (cacheWidth), cache appropriately, use correct formats/sizes.

Chapter 24 — Debugging Mastery & the Flutter Error Encyclopedia

Goal: A systematic debugging method plus a lookup table of the errors you will meet in real projects — what they mean, why they happen, and exactly how to fix them. Keep this chapter open while working.

24.1 The debugging method (works for every bug) 🕵️

1. **Read the FIRST error** in the console, completely. Flutter error messages are detailed and often contain the fix. Later errors are often consequences.
2. **Reproduce** reliably. Note platform, device, OS version, build mode, Flutter version, app version, user state (logged in?), network state.
3. **Isolate**: minimal reproduction (new screen/DartPad/`flutter create` sample). Binary search with git (`git bisect`) to find the commit that introduced it.
4. **Inspect**: breakpoints, `debugPrint`, DevTools inspector, logs (`flutter logs`, `adb logcat`, Xcode console / Console.app).
5. **Hypothesize → test → confirm**. Change one thing at a time.
6. **Fix the root cause**, not the symptom (e.g., don't wrap everything in try/catch or add `!`).
7. **Add a regression test.**
8. **Document** (PR description, postmortem for incidents).

Debugging tools & tricks

- Breakpoints & conditional breakpoints in VS Code/Android Studio; "Break on exceptions" (uncaught/all).
- `debugger();` from `dart:developer` — programmatic breakpoint.
- `debugPrint`, `log('msg', name: 'Cart', error: e, stackTrace: st)` (DevTools Logging view).
- `debugDumpApp()`, `debugDumpRenderTree()`, `debugDumpLayerTree()`, `debugDumpSemanticsTree()`, `debugDumpFocusTree()`.
- `debugPaintSizeEnabled`, `debugPaintBaselinesEnabled`, `debugPaintPointersEnabled`, `debugRepaintRainbowEnabled`, `debugPrintRebuildDirtyWidgets`, `debugPrintBeginFrameBanner`, `timeDilation = 5.0` (slow animations).
- `toString()` / `toStringShort()` / `debugFillProperties` for readable diagnostics in your classes.
- `FlutterError.demangleStackTrace`, `StackTrace.current`, `Chain.capture` (package `stack_trace`) for async stack chains.
- Network: DevTools Network tab, Charles/Proxyman/mitmproxy (needs trusting user CAs on Android: network security config for debug builds), `LogInterceptor`.
- Native: Android Studio (attach debugger to Android process, Logcat filters), Xcode (Debug → Attach to process; exception breakpoints; Instruments).

- Release-only bugs: `flutter run --release` locally; check R8/ProGuard, obfuscation, missing manifest permissions, `kDebugMode`-only code, asserts that hid issues, tree-shaken entry points (`@pragma('vm:entry-point')`).
- Crash reports: Crashlytics/Sentry with breadcrumbs; symbolize obfuscated traces with `flutter symbolize`.

24.2 Error encyclopedia — Layout & rendering

#	Error	Meaning / Cause	Fix
1	A RenderFlex overflowed by N pixels on the right/bottom	Row/Column children exceed available space	`Expanded` / `Flexible`, text `overflow`, `Wrap`, scroll view, smaller content
2	Vertical viewport was given unbounded height	Scrollable inside an unbounded-height parent (Column, another scrollable)	`Expanded`, fixed `SizedBox` height, slivers; last resort `shrinkWrap`
3	Horizontal viewport was given unbounded width	Horizontal list inside Row	`Expanded` / fixed width
4	RenderBox was not laid out: RenderXxx	Consequence of earlier constraint error	Fix the first error
5	BoxConstraints forces an infinite width/height	Widget wants infinite size (e.g., `SizedBox.expand`, `double.infinity` width) inside unbounded parent	Provide bounds
6	An InputDecorator, which is typically created by a TextField, cannot have an unbounded width	TextField in Row	`Expanded(child: TextField())`
7	RenderFlex children have non-zero flex but incoming height constraints are unbounded	`Expanded` / `Spacer` inside Column inside scroll view	Remove Expanded or use `SliverFillRemaining(hasScrollBody: false)` / IntrinsicHeight pattern
8	Incorrect use of ParentDataWidget	Expanded/Flexible not direct child of Flex; Positioned not direct child of Stack	Restructure
9	Cannot hit test a render box with no size / 'hasSize': is not true	Querying size before layout / invisible widget	Use post-frame callback; ensure layout
10	A RenderViewport expected a child of type RenderSliver but received RenderBox	Box widget in `slivers:`	`SliverToBoxAdapter`, `SliverPadding`
11	'Cannot provide both a color and a decoration'	Container with both	Move color into BoxDecoration
12	Text has yellow double underline	Text without a `Material` / `DefaultTextStyle` ancestor (e.g., in Hero flight, Overlay, or no Scaffold)	Wrap in `Material` or `Scaffold`, set style
13	Black/blank screen on launch	Exception before first frame; missing `runApp`; awaiting a never-completing future in main; iOS: missing LaunchScreen	Check logs; wrap init in try/catch; ensure `runApp`

#	Error	Meaning / Cause	Fix
14	Red screen "Exception caught by widgets library" (debug) / grey box (release)	Exception thrown in `build`	Fix the exception; customize `ErrorWidget.builder`
15	LayoutBuilder does not support returning intrinsic dimensions	LayoutBuilder inside IntrinsicHeight/Width or something asking intrinsics	Avoid intrinsics around LayoutBuilder
16	Image appears blurry	Low-res asset / wrong devicePixelRatio variant / `cacheWidth` too small	Provide 2x/3x assets; correct cache size
17	Unable to load asset: assets/...	Not declared in pubspec, wrong path/case (case-sensitive on Linux/Android!), subfolder not listed, needs full restart	Fix pubspec & path; `flutter clean`
18	Status bar icons invisible	Light icons on light background	`AnnotatedRegion<SystemUiOverlayStyle>` / AppBar `systemOverlayStyle`
19	Content behind navigation bar / notch (Android 15 edge-to-edge)	Not handling insets	`SafeArea`, `MediaQuery.paddingOf`

24.3 Error encyclopedia — State & lifecycle

#	Error	Cause	Fix
20	setState() called after dispose()	Async result/timer/stream after widget removed	`if (!mounted) return;`, cancel subscriptions/timers in dispose
21	setState() or markNeedsBuild() called during build	Triggering rebuilds (setState/notifyListeners/provider update) while building	Move to callbacks; `WidgetsBinding.instance.addPostFrameCallback`
22	dependOnInheritedWidgetOfExactType() ... was called before initState() completed	`Theme.of` / `MediaQuery.of` / `Provider.of(listen:true)` in initState	Use `didChangeDependencies` or `context.read` (no listen)
23	Looking up a deactivated widget's ancestor is unsafe	Using context of a disposed widget (after await, in dispose, in dialog after page popped)	Capture `Navigator.of(context)` / `ScaffoldMessenger.of(context)` before await; check `context.mounted`
24	This widget has been unmounted, so the State no longer has a context	Same as above	Same
25	Multiple widgets used the same GlobalKey	GlobalKey reused (static key, key created once but widget shown twice, two routes)	One key per widget instance
26	Duplicate keys found	Same ValueKey among siblings	Unique ids
27	A dismissed Dismissible widget is still part of the tree	Didn't remove item from list in onDismissed	Remove from data + setState
28	LateInitializationError: Field 'x' has not been initialized	late read before assignment	Initialize earlier / nullable

#	Error	Cause	Fix
29	Null check operator used on a null value	`!` on null (common: `_formKey.currentState!` when Form not built, `snapshot.data!` when not loaded, `widget.x!`)	Check nulls; correct lifecycle
30	A XYZ was used after being disposed (ChangeNotifier/AnimationController/TextEditingController)	Using after dispose, double dispose, Provider `create` disposing shared instance	Single owner; `.value` providers for existing objects
31	Bad state: Stream has already been listened to	Second listener on single-subscription stream (e.g., StreamBuilder rebuilt with same stream after `.listen`, or two StreamBuilders)	`asBroadcastStream()` /broadcast controller; don't listen twice
32	Bad state: Cannot add new events after calling close	Adding to closed StreamController (often in dispose race)	Check `isClosed`; order of disposal
33	StateError: Cannot emit new states after calling close (Bloc)	Emitting after bloc closed (await gaps)	Check `isClosed`; cancel work on close
34	emit was called after an event handler completed normally (Bloc)	emit inside stream.listen callback / unawaited future	`await emit.forEach` / `onEach`; await all futures
35	Could not find the correct Provider above this Widget	Provider not ancestor (same build, new route, wrong type/import)	Builder/Consumer, move provider up, `.value` in new routes, consistent imports
36	Tried to listen to a value exposed with provider, from outside of the widget tree	`watch` / `Provider.of` in callbacks	`read`
37	Bad state: No ProviderScope found (Riverpod)	Missing root `ProviderScope`	Wrap app
38	Tried to modify a provider while the widget tree was building (Riverpod)	State change during build/initState/didChangeDependencies	Delay with `Future(() => ...)` /post-frame or move to event
39	State not updating when parent changes value	Copied widget property into state only in initState	`didUpdateWidget` or read `widget.x` directly
40	UI doesn't update after modifying list	Mutated same list instance (ValueNotifier/Bloc/Riverpod equality)	Create new list (`[...old, item]`)
41	Infinite rebuild loop / repeated API calls	Future/stream created in build; setState in build; provider watching something it changes	Create in initState/state layer
42	TextField loses text/focus	Controller or key recreated on rebuild	Create controller once; avoid UniqueKey

24.4 Error encyclopedia — Navigation & context

#	Error	Fix
43	Navigator operation requested with a context that does not include a Navigator	Use context under MaterialApp (Builder) or a navigatorKey
44	Scaffold.of() called with a context that does not contain a Scaffold	Builder / separate widget / GlobalKey

#	Error	Fix
45	No MaterialLocalizations found	Widget (e.g., showDatePicker, TextField context menu) needs MaterialApp & localizations delegates; in tests wrap with MaterialApp
46	No Material widget found (e.g., InkWell/TextField/ListTile outside Material)	Wrap with `Material` / `Scaffold`
47	No MediaQuery widget ancestor found	Use under MaterialApp/WidgetsApp or `MediaQuery` in tests
48	No Overlay widget found (tooltip/dropdown above Navigator)	Use under MaterialApp
49	There are multiple heroes that share the same tag within a subtree	Unique hero tags per route
50	'!_debugLocked': is not true	Navigator push/pop during another navigation or during build
51	GoException: no routes for location	Wrong path / missing route / leading slash
52	Black screen after `pop`	Popped last route

24.5 Error encyclopedia — Async, types & Dart

#	Error	Fix
53	type 'Null' is not a subtype of type 'String' in type cast	JSON key missing/null → nullable fields & defaults; validate data
54	type 'List' is not a subtype of type 'List'	`.cast<String>()` / `List<String>.from(...)`
55	type '_Map<String, dynamic>' is not a subtype of type 'Map<String, String>'	`Map<String, String>.from(...)`
56	type 'int' is not a subtype of type 'double'	`(x as num).toDouble()`
57	_TypeError ... '_InternalLinkedHashMap<Object?, Object?>' (platform channel results)	`Map<String, dynamic>.from(result as Map)` (recursively)
58	Unhandled Exception: ... (async)	Await futures inside try/catch; handle stream errors; global handlers
59	FormatException: Unexpected character (at character 1)	Server returned HTML (error page/login redirect/wrong URL) instead of JSON → check status codes & URL before decoding
60	RangeError (index): Invalid value: Not in inclusive range 0..N	Index bounds; list changed during async gap
61	Concurrent modification during iteration	Iterate over a copy
62	Stack Overflow	Infinite recursion (getter calling itself, `==` calling itself, build recursion)
63	NoSuchMethodError: The method 'x' was called on null / Class 'X' has no instance method 'y'	dynamic misuse → add types
64	TimeoutException after 0:00:10	Slow network/server; tune timeouts; retry
65	Isolate: Illegal argument in isolate message: (object is a ReceivePort / closure capturing unsendable)	Send only sendable data; don't capture `this` with sockets/streams

#	Error	Fix
66	Binding has not yet been initialized	`WidgetsFlutterBinding.ensureInitialized()` before using plugins; in tests `TestWidgetsFlutterBinding.ensureInitialized()`
67	Zone mismatch	ensureInitialized and runApp in different zones → same zone

24.6 Error encyclopedia — Build, Gradle, CocoaPods, Xcode

#	Error	Fix
68	Gradle task assembleDebug failed with exit code 1	Scroll up for the real error; `cd android && ./gradlew assembleDebug --stacktrace --info`
69	Unsupported class file major version 61/65/67	JDK too new for Gradle → upgrade Gradle wrapper/AGP, or `flutter config --jdk-dir` to JDK 17
70	Minimum supported Gradle version is X. Current version is Y	Update `gradle-wrapper.properties` `distributionUrl`
71	Your project's Android Gradle Plugin version is lower than Flutter's minimum supported version	Update AGP in `settings.gradle(.kts)`
72	Manifest merger failed: minSdkVersion X cannot be smaller than version Y declared in library	Raise `minSdk`
73	Namespace not specified (AGP 8)	Add `namespace` in module build.gradle; update old plugins
74	Duplicate class kotlin.collections.jdk8...	Kotlin stdlib version conflict → align Kotlin version / Kotlin BOM
75	Execution failed for task ':app:mergeDexRelease' / 64K method limit	minSdk ≥ 21 (multidex native)
76	Execution failed for task ':app:minifyReleaseWithR8' / Missing classes detected	Add ProGuard `-keep` / `-dontwarn` rules from plugin docs (`missing_rules.txt` generated)
77	Inconsistent JVM-target compatibility detected for tasks 'compileDebugJavaWithJavac' (1.8) and 'compileDebugKotlin' (17)	Align `compileOptions` & `kotlinOptions.jvmTarget` (or `kotlin { jvmToolchain(17) }`)
78	SDK location not found	`local.properties` `sdk.dir` / `ANDROID_HOME`
79	Build failed due to use of deleted Android v1 embedding	Migrate app/plugin to v2 embedding (FlutterActivity from `io.flutter.embedding.android`)
80	Keystore file not found for signing config 'release' / Failed to read key from store: Keystore was tampered with, or password was incorrect	Check `key.properties` paths/passwords
81	CocoaPods could not find compatible versions for pod "X"	`pod repo update`, raise iOS deployment target in Podfile, delete Podfile.lock & Pods
82	The iOS deployment target 'IPHONEOS_DEPLOYMENT_TARGET' is set to 9.0, but the range of supported deployment target versions is 12.0 to 18.x	Post-install hook in Podfile to set min target for pods / update pods
83	No such module 'Flutter' / 'Flutter/Flutter.h' file not found	Open `.xcworkspace`; `flutter clean && flutter pub get && cd ios && pod install`; build from Flutter first
84	Signing for "Runner" requires a development team	Set Team in Xcode Signing & Capabilities

#	Error	Fix
85	Provisioning profile doesn't include the X capability / entitlement	Regenerate profile with capability (Push, Associated Domains)
86	Module compiled with Swift X cannot be imported by Swift Y	Clean derived data; update pods/Xcode
87	Sandbox: rsync.samba deny file-write-create (Xcode 15+)	Set `ENABLE_USER_SCRIPT_SANDBOXING = NO` for Runner build settings (common with some pod scripts)
88	Command PhaseScriptExecution failed with a nonzero exit code	Generic; read the script log; often FLUTTER_ROOT, CocoaPods, or flutter build errors
89	Error (Xcode): ... Flutter.framework ... does not contain bitcode	Disable bitcode (deprecated)
90	Invalid Bundle / ITMS-90xxx on upload	Read Apple's message: missing icons, privacy manifest (PrivacyInfo.xcprivacy required for certain APIs since 2024), invalid architectures, missing usage descriptions
91	ITMS-91053: Missing API declaration	Add privacy manifest declaring required-reason APIs (update plugins that include their own manifests)
92	Target of URI doesn't exist: 'package:x/x.dart'	`flutter pub get`; check package name & path; IDE restart (analysis server)
93	The getter 'x' isn't defined for the type ... after codegen change	Run build_runner again; delete conflicting outputs
94	Conflicting outputs were detected and the build is unable to prompt	`--delete-conflicting-outputs`
95	Because X depends on Y ... version solving failed	Chapter 2 §2.5.2

24.7 Error encyclopedia — Platform & plugins

#	Error	Fix
96	MissingPluginException(No implementation found for method ...)	Full rebuild (not hot reload) after adding plugin; platform supported? channel name? background isolate? add-to-app registration? tests mocked?
97	PlatformException(sign_in_failed, ApiException: 10) (Google Sign-In Android)	SHA-1/SHA-256 of signing keys (debug, upload, Play signing) not registered in Firebase/Google Cloud
98	PlatformException(already_active, Image picker is already active)	Prevent double taps
99	Permission denied crash on iOS when requesting camera	Info.plist usage description missing
100	CERTIFICATE_VERIFY_FAILED	Invalid cert / missing intermediate / device date wrong / outdated root store
101	SocketException: Failed host lookup	No internet/DNS/emulator network; Android INTERNET permission in release
102	XMLHttpRequest error (web)	CORS, mixed content, server down
103	Unsupported operation: Platform._operatingSystem (web)	`dart:io` Platform on web → `kIsWeb` / `defaultTargetPlatform`
104	WebView blank on Android	Missing internet permission / cleartext / JS disabled (`setJavaScriptMode(JavaScriptMode.unrestricted)`)

#	Error	Fix
105	Google Maps blank/grey	API key missing/restricted wrongly, Maps SDK not enabled, billing not enabled
106	Push not received	Chapter 18 table
107	Deep link opens browser	Verification failure — Chapter 11
108	Release crash only, ClassNotFoundException	R8 removed class → keep rules
109	`flutter_secure_storage` crash after restore	Backup excluded / reset on error
110	Background handler not running in release	`@pragma('vm:entry-point')`

24.8 "What would you do if..." — situational playbook 📘

...the app crashes for 2% of users after a release?

1. Check Crashlytics/Sentry: group by issue, versions, devices, OS.
2. If severe: **halt staged rollout** (Play Console) / pause phased release (App Store), toggle a **feature flag/kill switch** if the feature is flagged.
3. Reproduce with the same OS/device (Firebase Test Lab).
4. Hotfix → test → expedited release; consider code push (Shorebird) for Dart-only fixes if your org uses it (respect store policies).
5. Postmortem: root cause, why tests didn't catch it, add regression test & monitoring.

...the list screen is janky on low-end Android? Profile mode → DevTools → identify UI vs raster → apply Chapter 23 fixes (builders, images, const, isolates, effects) → verify on the same device.

...a screen is slow to load data? Check network waterfall (sequential calls?), payload size, server latency, parsing on main thread, caching (show cached first), pagination, prefetch.

...users report being logged out randomly? Token refresh race (parallel 401s), refresh token rotation conflicts, secure storage read failures (Keychain accessibility when locked/background), clock skew with JWT expiry, server revoking sessions → add logging around auth, use queued refresh.

...the app works in debug but not in release? Missing INTERNET permission, R8 stripping, obfuscation breaking reflection/`runtimeType.toString()` logic, `kDebugMode`-guarded init, tree-shaken entry points, env/config differences, asserts masking bugs.

...build fails after upgrading Flutter? Read the breaking changes page for that version; `dart fix --apply`; update Gradle/AGP/Kotlin per Flutter's warnings; update plugins (`flutter pub upgrade --major-versions` carefully); `flutter clean`; pod repo update. Upgrade on a branch with CI; never on release day.

...two features conflict in state (e.g., cart badge stale)? Ensure single source of truth (cart repository stream), derived values computed not duplicated, invalidate/refresh on mutations.

...memory grows until crash on image-heavy screens? `cacheWidth`, limit ImageCache, dispose controllers, check leaks via snapshots.

...a plugin is abandoned and broken on new Android/iOS? Check forks/community alternatives; fork & patch (git dependency pinned to commit) and upstream PR; or write your own minimal plugin with Pigeon.

...a production bug you can't reproduce? Add breadcrumbs/logging (non-PII) around the flow, remote config–controlled verbose logging, collect device/OS/version patterns, try same conditions (slow network via network link conditioner, low memory, different locale/timezone/text scale, RTL, dark mode, accessibility settings, process death via "Don't keep activities").

24.9 Interview Questions — Debugging

Q1. How do you debug a layout overflow? Read error (which RenderFlex, by how much), use Inspector layout explorer / debugPaintSizeEnabled, apply Expanded/Flexible/scroll/overflow handling.

Q2. How do you debug a release-only crash? Reproduce with `--release`, check logs (logcat/Xcode), symbolicate obfuscated traces, check R8 rules/permissions/entry points/config differences.

Q3. What does "setState called after dispose" mean and how to prevent it? Async callback updating a disposed State; check `mounted`, cancel subscriptions/timers/requests in dispose.

Q4. How do you handle "Null check operator used on a null value"? Find the `!` in the stack trace, understand why null occurs (lifecycle/data), replace with proper null handling/state modelling.

Q5. What tools do you use for debugging Flutter apps? IDE debugger, DevTools (inspector, performance, CPU, memory, network, logging), logs, crash reporting, proxies, native debuggers.

Q6. How do you debug network issues? DevTools Network tab, interceptor logs (debug only), proxy tools, verifying status codes/headers/body, testing with curl/Postman, checking platform configs (permissions, ATS, CORS).

Chapter 25 — Mobile App Security for Flutter (OWASP MASVS in Practice)

Goal: Build apps that protect user data and company systems. Banks, fintech, health and enterprise clients will ask about these topics, and security interviews for senior roles use OWASP terminology.

25.1 The golden rule 🔑

The app runs on a device you don't control. Anything inside the app (code, strings, assets, API keys) can be extracted, and any check in the app can be bypassed.

5-year-old: Your app is a **letter you mail to a stranger**. You can put it in an envelope (obfuscation), but they can still open it. So **never write your secret password inside the letter** — keep secrets in your own house (the server).

Therefore: real security decisions (authorization, prices, payment success, feature entitlements) happen on the **server**. The client adds **defense in depth**.

25.2 OWASP Mobile Top 10 (2024) → Flutter mapping

Risk	Flutter-specific practice
M1 Improper Credential Usage	No hard-coded secrets; tokens in secure storage; short-lived tokens
M2 Inadequate Supply Chain Security	Vet packages, pin versions (lock file), dependency scanning, verified publishers, review native code in plugins
M3 Insecure Authentication/Authorization	Server-side authz; OAuth2 + PKCE; biometrics as convenience only; session expiry
M4 Insufficient Input/Output Validation	Validate input; parameterized SQL; sanitize WebView content; deep link parameter validation
M5 Insecure Communication	HTTPS/TLS only; no cleartext; certificate pinning for high-risk apps
M6 Inadequate Privacy Controls	Minimize PII, consent, privacy manifests, data deletion
M7 Insufficient Binary Protections	Obfuscation, integrity checks, root/jailbreak detection, anti-tamper (RASP) where justified
M8 Security Misconfiguration	Disable debug flags in release, `android:debuggable=false`, `allowBackup` rules, exported components
M9 Insecure Data Storage	Secure storage, encrypted DB, no sensitive data in logs/prefs/caches/screenshots
M10 Insufficient Cryptography	Use vetted libraries/platform crypto; no custom crypto; proper key storage

OWASP **MASVS** (verification standard) and **MASTG** (testing guide) are the references auditors use.

25.3 Secrets & API keys

- ❌ Hard-coding `const stripeSecretKey = 'sk_live_...'` — extractable from the binary (`strings libapp.so`) in minutes.
- ❌ `.env` assets — also bundled & readable.
- ✅ Keep secrets on the **backend**; the app calls your backend, which calls third parties.
- ✅ Public keys meant for clients (Firebase config, Stripe *publishable* key, Maps keys) are fine — **restrict them** (Android package + SHA-1, iOS bundle ID, HTTP referrers, API restrictions, quotas).
- ✅ `--dart-define` values are *not* secret either; `envied` obfuscation only slows attackers.
- Rotate keys if leaked; add secret scanning in CI (gitleaks, GitHub secret scanning); never commit keystores/`key.properties`/service account JSON.

25.4 Authentication & sessions

- OAuth 2.0 / OIDC with **Authorization Code + PKCE** (`flutter_appauth`) for third-party identity providers; system browser (ASWebAuthenticationSession / Custom Tabs), never embedded WebView for credentials.
- Access token short-lived (5–15 min), refresh token longer & rotated; store in secure storage; refresh with locking (Chapter 16).
- Logout: revoke tokens server-side, clear secure storage, DB, caches, in-memory state.
- **Biometrics** (`local_auth`) unlock locally-stored credentials/keys; never treat "biometric success" as server authentication by itself. For strong binding, use keys in Keystore/Secure Enclave that require biometric auth (`CryptoObject`), via plugins or native code.
- Re-authenticate for sensitive actions (change password, payments).
- Device binding / attestation: **Play Integrity API**, **App Attest/DeviceCheck** (iOS), Firebase App Check — backend verifies the request comes from a genuine app on a genuine device.
- Rate limiting & lockouts on the server; CAPTCHA/bot protection where needed.

25.5 Secure data storage

- Tokens/keys → `flutter_secure_storage` (Chapter 17 gotchas: iOS keychain persists after uninstall; Android backup restore).
- Sensitive DBs → SQLCipher / encrypted Hive with key in secure storage.
- Don't log sensitive data (`print(token)`!); strip logs in release; scrub crash reports.
- Caches: HTTP caches may store sensitive responses — disable caching for sensitive endpoints (`Cache-Control: no-store`).
- Clipboard: avoid copying secrets; clear after timeout; Android 13+ shows clipboard preview — mark sensitive (`ClipDescription.EXTRA_IS_SENSITIVE` via native).
- **Screenshots / app switcher snapshots**: hide sensitive screens:
  - Android: `FLAG_SECURE` (`WindowManager.LayoutParams.FLAG_SECURE`) via a plugin (`flutter_windowmanager_plus`, `no_screenshot`, `screen_protector`) or native code — blocks screenshots/recording and blanks recents thumbnail.

    - iOS: can't fully block screenshots; detect via `UIApplication.userDidTakeScreenshotNotification`; obscure content when going to background (overlay on `inactive` lifecycle state) and detect screen recording (`UIScreen.isCaptured`).
- Keyboard: `enableSuggestions: false`, `autocorrect: false` for sensitive fields; `obscureText` for passwords (prevents keyboard learning on most IMEs).

25.6 Network security

- HTTPS only (Android blocks cleartext by default since API 28; iOS ATS).
- **Certificate / public key pinning** for high-risk apps:

```
// Dio with a custom HttpClient (native platforms)
(dio.httpClientAdapter as IOHttpClientAdapter).createHttpClient = () {
  final client = HttpClient(context: SecurityContext(withTrustedRoots: false)
    ..setTrustedCertificatesBytes(pinnedCertPemBytes));   // trust ONLY your cert/CA
  return client;
};
// Alternative: validateCertificate callback in IOHttpClientAdapter checking SHA-256 fingerprint
of cert.der
```

Plan **rotation** (pin multiple keys/CA, remote-config fallback, expiry monitoring) — pinning failures brick older app versions.

- Android `network_security_config.xml` can declare pins and debug-only user CA trust.
- Don't disable certificate validation (`badCertificateCallback = (_, __, ___) => true`) except in local debug with explicit guards.
- Validate server responses (types/sizes); limit redirects.

25.7 Code protection (binary hardening)

- **Obfuscation:** `flutter build apk --obfuscate --split-debug-info=build/symbols` — renames Dart symbols in the AOT snapshot. Keep symbols to de-obfuscate crashes. Note: obfuscation doesn't hide strings or logic flow; it raises the bar.
- Release builds are AOT machine code (harder to reverse than JS/Java bytecode) but tools (e.g., reFlutter, Blutter) can analyze Flutter snapshots — assume determined attackers can read your logic.
- Android R8 for Java/Kotlin code.
- **Root/jailbreak detection** (`flutter_jailbreak_detection`, `safe_device`, freeRASP/`freerasp` by Talsec): detect rooted devices, emulators, debuggers, hooking frameworks (Frida), tampering (signature mismatch), and react (warn, limit features, report to backend). All bypassable by skilled attackers → combine with server-side attestation (Play Integrity / App Attest).
- Don't rely on client checks for business rules (e.g., "isPremium" flag in prefs → server must enforce).

25.8 Input validation & injection

- SQL: always parameterized (`whereArgs`), Drift handles this.
- WebView: don't load untrusted URLs with JS bridges enabled; validate `JavascriptChannel` messages; restrict navigation (`NavigationDelegate`); don't expose sensitive native functions to web content.
- Deep links: treat parameters as untrusted — validate IDs, don't auto-execute actions (e.g., `myapp://transfer?to=x&amount=1000` must require user confirmation & server checks); prevent open redirects.
- File paths: avoid path traversal when saving/downloading (`../`).
- Rendering HTML/Markdown from users: sanitize.

25.9 Platform configuration hardening

Android

- `android:debuggable` is false in release automatically — don't override.
- Exported components: `android:exported` explicit (required Android 12+); only export what's needed; protect with permissions.
- `android:allowBackup` / `dataExtractionRules` to exclude sensitive data.
- `usesCleartextTraffic=false`.
- Intent filters & PendingIntents with `FLAG_IMMUTABLE`.
- Keep `targetSdk` up to date (security behaviour changes).
- Tapjacking: `filterTouchesWhenObscured` for sensitive buttons (native).

iOS

- ATS enabled; keychain accessibility classes; `NSFaceIDUsageDescription` for Face ID; Privacy manifest (`PrivacyInfo.xcprivacy`); data protection entitlement (`NSFileProtectionComplete`) for files.
- Universal Links instead of custom schemes for sensitive flows (custom schemes can be hijacked).

Web

- XSS considerations (HtmlElementView, `dart:js_interop`), CSP headers, don't store tokens in localStorage if avoidable (prefer httpOnly cookies with backend), CORS correctly configured server-side.

25.10 Supply chain & dependencies

- Review packages before adding (Chapter 2 checklist), especially plugins with native code and broad permissions.
- Commit `pubspec.lock`; pin git dependencies to commit SHAs.

- Dependency update cadence with changelog review; Dependabot/Renovate for pub (supported).
- OSV scanner (`osv-scanner --lockfile=pubspec.lock`) for known vulnerabilities.
- Protect CI secrets (signing keys in secret storage, least privilege), signed commits/tags if required, SBOM generation for regulated industries.

25.11 Privacy & compliance

- Collect minimum data; get consent (GDPR/DPDP/CCPA); ATT prompt on iOS for tracking; Google Play **Data safety** form & App Store **privacy nutrition labels** must match reality (SDKs included!).
- Account deletion in-app is **required** by both Apple (since 2022) and Google Play (since 2023/2024) if you allow account creation.
- Children's apps: COPPA / Families policy restrictions on ads/analytics SDKs.
- Health/finance: HIPAA, PCI-DSS (never store card data; use gateway SDKs/tokenization), RBI guidelines (India) for payments & data localization.

25.12 Security testing

- Static analysis: MobSF (Mobile Security Framework) on APK/IPA; lints; secret scanning.
- Dynamic: proxy traffic (Burp/mitmproxy) to verify TLS/pinning and no sensitive data leakage; Frida-based tests in pentests.
- Third-party penetration testing before major releases for regulated apps.
- Threat modeling (STRIDE) for new features handling sensitive data.

25.13 Interview Questions — Security

Q1. How do you store sensitive data in Flutter? flutter_secure_storage (Keychain/Keystore) for small secrets; encrypted DB for larger data; never SharedPreferences/plain files; clear on logout.

Q2. How do you protect API keys? Don't ship secrets; proxy via backend; restrict public client keys by app signature/bundle/API; use App Check/attestation.

Q3. What is certificate pinning and its risk? Trusting only specific certs/keys for your domains to block MITM; risk of breaking the app on certificate rotation — pin multiple keys, plan rotation.

Q4. Does obfuscation make the app secure? No — it raises reverse-engineering cost; secrets and critical logic must stay on the server.

Q5. How do you prevent screenshots of sensitive screens? Android FLAG_SECURE; on iOS obscure content when backgrounded and detect screen capture/screenshot.

Q6. What is OWASP MASVS? The OWASP Mobile Application Security Verification Standard defining security requirements (storage, crypto, auth, network, platform, code quality, resilience, privacy).

Q7. How do you implement secure authentication? OAuth2/OIDC with PKCE via system browser, short-lived tokens, refresh rotation, secure storage, server-side authorization, MFA/biometric re-auth for sensitive actions.

Q8. What is root/jailbreak detection and its limitations? Detects compromised devices/hooking; bypassable — combine with server-side integrity attestation and risk-based responses.

Q9. How do you secure deep links? Use verified App Links/Universal Links, validate parameters, require auth/confirmation for actions, avoid sensitive data in URLs.

Q10. How do you handle third-party package risk? Vetting, version pinning, vulnerability scanning, minimal permissions, code review of native parts, update policies.

Chapter 26 — Flavors, Signing, Building, Store Release, CI/CD & Code Push

Goal: Ship confidently: multiple environments, correct signing, automated pipelines, store submissions, staged rollouts and hotfix strategies. Every company job includes this; many juniors have never done it — knowing it makes you stand out.

26.1 Flavors (build variants) 🍦

5-year-old: Same ice cream machine, different flavours: **dev** (testing server, orange icon), **staging** (pre-production), **prod** (real users). Each can be installed side-by-side on the same phone.

Android (productFlavors) — `android/app/build.gradle.kts`

```
android {
    flavorDimensions += "env"
    productFlavors {
        create("dev") {
            dimension = "env"
            applicationIdSuffix = ".dev"
            versionNameSuffix = "-dev"
            resValue("string", "app_name", "MyApp Dev")
        }
        create("staging") {
            dimension = "env"
            applicationIdSuffix = ".stg"
            resValue("string", "app_name", "MyApp Staging")
        }
        create("prod") {
            dimension = "env"
            resValue("string", "app_name", "MyApp")
        }
    }
}
```

AndroidManifest: `android:label="@string/app_name"`. Per-flavor resources/configs: `android/app/src/dev/res/...`, `android/app/src/dev/google-services.json`.

iOS (schemes + build configurations)

- Duplicate build configurations: `Debug-dev`, `Release-dev`, `Debug-prod`, `Release-prod`, (+ `Profile-*`).
- Create schemes named exactly like the flavor (`dev`, `prod`) using those configurations.

- Per-config settings: `PRODUCT_BUNDLE_IDENTIFIER`, display name (`FLUTTER_APP_NAME` user-defined setting referenced in Info.plist `CFBundleDisplayName`), app icon set, `GoogleService-Info.plist` copied by a Run Script phase based on `${CONFIGURATION}`.
- Recent Flutter versions can also read flavor-specific settings and support `flavors` declared in pubspec for assets (`flavors:` under an asset entry → asset only bundled in that flavor).

Dart side

```
flutter run --flavor dev -t lib/main_dev.dart --dart-define-from-file=env/dev.json
flutter build appbundle --flavor prod -t lib/main_prod.dart --dart-define-from-file=env/prod.json
```

```
// lib/main_dev.dart
void main() => bootstrap(AppConfig.dev());
// Or read flavor at runtime: appFlavor (from package:flutter/services.dart, Flutter 3.13+)
import 'package:flutter/services.dart';
final flavor = appFlavor; // 'dev' | 'staging' | 'prod' | null
```

Tools: `flutter_flavorizr` generates all this native setup; `very_good_cli` templates include flavors.

26.2 Versioning

`pubspec.yaml`: `version: 2.4.1+241` → versionName/CFBundleShortVersionString `2.4.1`, versionCode/CFBundleVersion `241`.

- Build number must **increase** for every store upload (Play rejects duplicates; App Store requires higher build per version).
- Override in CI: `flutter build appbundle --build-name=2.4.1 --build-number=$GITHUB_RUN_NUMBER`.
- Semantic versioning for user-facing versions; tag releases in git (`v2.4.1`); CHANGELOG.
- Force-update policy: minimum supported version via remote config (Chapter 18).

26.3 Android signing & release

1. Create an **upload keystore** (once; back it up securely — losing it is painful, though Play App Signing allows upload key reset):

```
keytool -genkey -v -keystore ~/upload-keystore.jks -keyalg RSA -keysize 2048 -validity 10000 -
alias upload
```

2. `android/key.properties` (**never commit**; add to .gitignore):

```
storePassword=...
keyPassword=...
```

```
keyAlias=upload
storeFile=/Users/me/upload-keystore.jks
```

3. `build.gradle.kts`:

```
import java.util.Properties
import java.io.FileInputStream

val keystoreProperties = Properties().apply {
    val f = rootProject.file("key.properties")
    if (f.exists()) load(FileInputStream(f))
}
android {
    signingConfigs {
        create("release") {
            keyAlias = keystoreProperties["keyAlias"] as String?
            keyPassword = keystoreProperties["keyPassword"] as String?
            storeFile = (keystoreProperties["storeFile"] as String?)?.let { file(it) }
            storePassword = keystoreProperties["storePassword"] as String?
        }
    }
    buildTypes {
        getByName("release") { signingConfig = signingConfigs.getByName("release") }
    }
}
```

4. Build: `flutter build appbundle --release --obfuscate --split-debug-info=build/symbols`.
5. **Play App Signing** (mandatory for new apps with AAB): Google holds the **app signing key**; you sign uploads with your **upload key**. ⚠️ SHA fingerprints for Firebase/Google Sign-In/App Links must include the **app signing key** fingerprint from Play Console (common bug: works from local build, fails from Play Store).
6. Play Console: internal testing → closed → open → production; **staged rollout** (e.g., 5% → 20% → 50% → 100%), halt rollout if crash spikes. New personal developer accounts must run a closed test with testers for a period before production access (Play policy introduced in 2023 — check current requirements).
7. Requirements: target API level policy (recent), Data safety form, content rating, privacy policy URL, app access instructions for reviewers (test account), 64-bit (Flutter provides arm64-v8a), 16 KB memory page size support for apps targeting Android 15+ (Flutter engine supports it in recent versions; check native libs of plugins).

26.4 iOS signing & release

- Apple Developer Program membership.

- **Certificates** (Apple Development / Apple Distribution), **App IDs** (bundle identifier + capabilities), **Provisioning profiles** (Development, Ad Hoc, App Store), devices (for dev/ad hoc).
- Xcode "Automatically manage signing" for small teams; **fastlane match** (certs/profiles in encrypted git/S3) for teams & CI.
- Build: `flutter build ipa --release --obfuscate --split-debug-info=build/symbols --export-options-plist=ios/ExportOptions.plist` → `.ipa` in `build/ios/ipa`.
- Upload: Xcode Organizer, **Transporter** app, `xcrun altool` / `notarytool` (altool upload deprecated → use Transporter / App Store Connect API via fastlane `pilot` / `deliver`).
- **TestFlight** for internal (no review) and external testers (beta review).
- App Store review guidelines pitfalls: crashes, incomplete features/placeholder content, missing account deletion, login required without demo account, Sign in with Apple requirement (4.8), IAP rules (3.1.1), misleading metadata, missing privacy usage strings, private API use, privacy manifest/required reason API declarations, minimum functionality (4.2) — "app is just a website".
- **Phased release** (7-day automatic rollout, pausable), expedited review for critical fixes.
- Export compliance (encryption): `ITSAppUsesNonExemptEncryption` = NO in Info.plist if you only use standard HTTPS.

26.5 Web & desktop distribution

- Web: `flutter build web --release` (add `--wasm` for WebAssembly) → deploy `build/web` to Firebase Hosting, Netlify, Vercel, S3+CloudFront, GitHub Pages; set `--base-href` for subpaths; configure SPA rewrites; caching headers (service worker behaviour: Flutter's default service worker was deprecated — use standard HTTP caching / your own PWA strategy); PWA manifest.
- Windows: `flutter build windows` → MSIX (`msix` package) for Microsoft Store or installer (Inno Setup).
- macOS: `flutter build macos` → sign + **notarize** (Developer ID) for outside-App-Store distribution, or Mac App Store with sandbox entitlements.
- Linux: Snap (`snapcraft`), Flatpak, AppImage, .deb.

26.6 CI/CD 🤖

Pipeline stages (typical)

1. **Setup**: checkout, Flutter version (from `.fvmrc`/pinned), cache pub & Gradle & CocoaPods.
2. **Static checks**: `dart format --output=none --set-exit-if-changed .`, `flutter analyze`, custom lints, codegen check (`build_runner` → `git diff --exit-code` to ensure generated files committed if your team commits them).
3. **Tests**: `flutter test --coverage` (+ goldens), upload coverage.
4. **Build**: debug/profile APKs for QA on PRs; release builds on main/tags.
5. **Integration tests**: emulators in CI or Firebase Test Lab/BrowserStack.

6. **Distribution**: Firebase App Distribution / TestFlight / Play internal track.
7. **Release**: tag-triggered production builds, upload to stores (fastlane), changelog, symbols upload (Crashlytics/Sentry), notify Slack.

GitHub Actions example

```
name: CI
on:
  pull_request:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with:
          channel: stable
          flutter-version-file: pubspec.yaml   # or a fixed version / .fvmrc
          cache: true
      - run: flutter pub get
      - run: dart format --output=none --set-exit-if-changed .
      - run: flutter analyze
      - run: flutter test --coverage
      - uses: codecov/codecov-action@v4
        with: { files: coverage/lcov.info }

  build-android:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17' }
      - uses: subosito/flutter-action@v2
        with: { channel: stable, cache: true }
      - name: Decode keystore
        run: echo "${{ secrets.ANDROID_KEYSTORE_BASE64 }}" | base64 -d > android/app/upload-
keystore.jks
      - name: key.properties
        run: |
          cat > android/key.properties <<EOF
          storePassword=${{ secrets.KEYSTORE_PASSWORD }}
          keyPassword=${{ secrets.KEY_PASSWORD }}
          keyAlias=upload
          storeFile=upload-keystore.jks
```

```
          EOF
      - run: flutter build appbundle --flavor prod -t lib/main_prod.dart --build-number=${{
github.run_number }} --obfuscate --split-debug-info=build/symbols
      - uses: actions/upload-artifact@v4
        with: { name: aab, path: build/app/outputs/bundle/prodRelease/app-prod-release.aab }
```

(`storeFile` in key.properties is resolved relative to the `android/app` module when using `file(...)` in the app's build script.)

iOS builds need **macOS runners** (`runs-on: macos-latest`), certificates/profiles via fastlane match or App Store Connect API keys, `flutter build ipa`.

Other CI/CD platforms

- **Codemagic** (Flutter-focused, by Nevercode) — easy iOS signing, workflows in `codemagic.yaml`, M-series Macs.
- **Bitrise**, **GitLab CI**, **Azure DevOps**, **CircleCI**, **Jenkins**, **Xcode Cloud** (iOS), **Appcircle**.
- **fastlane** — automation lanes: `match` (signing), `gym` (build), `pilot` (TestFlight), `deliver` (App Store metadata/screenshots), `supply` (Play upload), `increment_build_number`.

26.7 Code push / over-the-air (OTA) updates

- **Shorebird** — code push for Flutter (by former Flutter team lead Eric Seidel's company): `shorebird release android`, then `shorebird patch android` ships **Dart-only** changes directly to users without store review. Native code/assets changes still need a store release. Patches are applied on next app launch. Store policy: Apple allows interpreted code updates that don't change the app's primary purpose (guideline 3.3.2/2.5.2 nuance) — Shorebird positions itself as compliant; follow their guidance.
- Server-driven UI (SDUI) is an alternative for dynamic screens (JSON-defined layouts: `rfw` package from Flutter team — Remote Flutter Widgets, `json_dynamic_widget`, `mirai` / `stac`).

26.8 Release management best practices (organisational)

- **Release train**: regular cadence (e.g., every 2 weeks) with code freeze, release branch, QA regression, staged rollout.
- **Git flow / trunk-based** with short-lived feature branches + feature flags (preferred for fast teams).
- Release checklist: version bump, changelog, translations complete, analytics events verified, crash-free rate of beta, performance checks, store metadata/screenshots, privacy forms updated for new SDKs, backend compatibility (old app versions must keep working with new APIs — **API versioning**!).
- Monitor after release: crash-free users (target ≥ 99.5–99.9%), ANRs, key funnels, reviews.
- Rollback strategy: mobile can't roll back binaries → halt rollout, kill switches, hotfix, code push.

- Keep old versions working: backward-compatible APIs, minimum version enforcement as last resort.

26.9 Common release problems

Problem	Fix
Release build crashes on startup	Check logs; R8 rules; missing permissions; obfuscation-sensitive code
Google Sign-In fails only from Play Store	Add Play App Signing SHA-1/SHA-256 to Firebase/Google Cloud
App Links don't verify after Play release	assetlinks.json needs Play signing key fingerprint
"You uploaded an APK or Android App Bundle that was signed in debug mode"	Configure release signing
"Version code X has already been used"	Increase build number
iOS upload: "Missing required icon / Invalid bundle"	Provide all icon sizes (1024×1024 marketing icon without alpha), correct Info.plist
ITMS-90683 Missing purpose string	Add usage description for each permission API referenced (even by SDKs)
Crash reports unreadable	Upload obfuscation symbols & dSYMs
Large AAB/IPA	Chapter 23 app size
Users stuck on old version with broken API	Force update mechanism, API versioning

---

26.10 Interview Questions — Release & CI/CD

Q1. What are flavors and how do you set them up? Build variants for environments: Android productFlavors, iOS schemes/configurations, Dart entrypoints/dart-defines; separate app IDs, names, icons, Firebase configs.

Q2. How do you sign an Android app? Generate an upload keystore, configure key.properties and signingConfigs (kept secret), build an AAB; Play App Signing re-signs with the app signing key.

Q3. APK vs AAB? AAB is a publishing format; Play generates optimized APKs per device (ABI, density, language) → smaller downloads; required for new Play apps.

Q4. How do you release to the App Store? Configure signing (certificates/profiles), build ipa, upload via Transporter/fastlane, TestFlight, submit for review with metadata, phased release.

Q5. What does your CI pipeline look like? Format, analyze, test (+coverage/goldens), build, integration tests, distribute to testers, release on tags with symbol upload; secrets stored securely; pinned Flutter version.

Q6. What is staged rollout and why use it? Releasing to a percentage of users first to catch issues; halt if crash metrics spike.

Q7. How do you handle a critical production bug quickly? Halt rollout, kill switch/feature flag, hotfix with expedited review, code push for Dart-only fixes where allowed, communicate, postmortem.

Q8. What is obfuscation's impact on crash reporting? Stack traces are obfuscated; upload split debug info symbols to symbolize.

Q9. What is Shorebird? A code push solution for Flutter allowing Dart code patches to be delivered without store releases.

Q10. How do you manage different API endpoints per environment? Flavors + dart-define-from-file/AppConfig injected at bootstrap; never hard-code per build manually.

Chapter 27 — Flutter Web, Desktop, Embedded, TV, Wearables, Games & AI Features

Goal: Know what's different when targeting platforms beyond phones, so you can make good decisions and answer "Should we use Flutter for our web app/desktop app?".

27.1 Flutter Web 🌐

What it's good for

✅ Web **apps**: dashboards, admin panels, internal tools, PWAs, interactive editors, companion web versions of mobile apps, embedding Flutter widgets in existing sites. ❌ Content-heavy/SEO-critical sites (blogs, marketing pages, e-commerce landing pages) — Flutter renders to canvas; use HTML-first frameworks (or a hybrid: marketing site in HTML + app in Flutter).

Renderers & builds (recap)

- `flutter build web` → JS (dart2js) + **CanvasKit** (Skia in Wasm).
- `flutter build web --wasm` → **dart2wasm** (WasmGC) + **skwasm**, with automatic fallback to JS/CanvasKit in browsers without WasmGC support. Packages must use `dart:js_interop` / `package:web` (not `dart:html`) to be Wasm-compatible.
- The HTML renderer has been removed.

Key web topics

- **URL strategy** (`usePathUrlStrategy()`), server rewrites to index.html.
- **Initial load size** (main.dart.js / wasm + CanvasKit): use deferred imports (`deferred as`) for code splitting, compress assets, CDN, caching headers; show an HTML loading indicator in `index.html` while Flutter boots (customize via `flutter_bootstrap.js` and `_flutter.loader.load({ onEntrypointLoaded })`).
- **SEO**: limited; set `<title>`/meta in index.html; semantics tree can expose text; consider prerendered static pages for marketing.
- **Text selection & copy**: `SelectionArea`.
- **Browser features**: right-click context menus (`BrowserContextMenu.disableContextMenu()` if you implement your own), keyboard shortcuts, mouse hover (`MouseRegion`), scrollbars, tab focus traversal.
- **CORS** (Chapter 16), cookies/auth (httpOnly cookies with backend sessions are often safer than tokens in localStorage).
- **Platform channels** don't exist on web in the same way; plugins provide web implementations in Dart/JS.

- `dart:io` unavailable → conditional imports.
- **Embedding**: Flutter can render into a specific `<div>` (multi-view embedding: `hostElement`) inside an existing React/Angular app.
- **Hot reload on web**: supported in recent versions (stable/default around 3.35+).
- Accessibility: semantics must be enabled (button or `SemanticsBinding.instance.ensureSemantics()`).
- Testing: `flutter test --platform chrome`, integration tests with `flutter drive -d chrome`.

27.2 Desktop (Windows, macOS, Linux) 🖥️

Desktop UX differences (adaptive design!)

- Mouse + keyboard: hover states, cursors, right-click menus (`ContextMenuController` / `MenuAnchor`), keyboard shortcuts (`Shortcuts` / `Actions`), Tab focus traversal, scroll wheel/trackpad.
- Window management: resizing (responsive layouts!), min sizes, title bars, multiple windows (multi-window support is in development — previewed around Flutter 3.44 with Canonical leading; community `desktop_multi_window`), `window_manager` package (size, position, always on top, hide on close), system tray (`tray_manager` / `system_tray`), native menus (`PlatformMenuBar` on macOS).
- Native file dialogs: `file_selector` (official), `file_picker`.
- Denser layouts (`VisualDensity.compact`), larger screens → master-detail.
- Desktop distribution & signing (Chapter 26), auto-update (Sparkle on macOS via `auto_updater`, MSIX updates).
- macOS sandbox entitlements: network client/server, file access (`com.apple.security.files.user-selected.read-write`).
- Exit handling: `AppLifecycleListener.onExitRequested` to confirm unsaved work.
- Rendering: Impeller becoming default on desktop in recent releases (3.47 per release notes).

27.3 Embedded, automotive & TV

- **Custom embedders** using the Flutter Embedder API (C): Toyota (infotainment), BMW (iX app UI), Sony, LG webOS TVs (LG announced Flutter support), Raspberry Pi kiosks (`flutter-pi`), **Sony's/Toyota's `flutter-elinux` /ivi-homescreen** (embedded Linux).
- **Android TV / Google TV**: D-pad focus navigation (`Focus`, `FocusTraversalGroup`, visible focus highlights), leanback launcher intent-filter, 10-foot UI.
- **Apple TV (tvOS)**: not officially supported (community efforts).

27.4 Wearables

- **Wear OS**: Flutter apps run (round screens: `wear_plus` / `wearable_rotary`, ambient mode); resource constraints.

- **watchOS**: not supported natively — build a native SwiftUI watch app and communicate via `WatchConnectivity` (plugins like `watch_connectivity`).

27.5 Home screen widgets, App Clips, Live Activities, extensions

- Home screen widgets (Android AppWidgets / iOS WidgetKit) must be **native UI**; share data via `home_widget` (App Groups on iOS, SharedPreferences on Android); can render Flutter widgets to images for display.
- iOS **Live Activities / Dynamic Island**: native (ActivityKit) with data from Flutter via plugins (`live_activities`).
- **App Clips** (iOS) / **Instant Apps** (Android) have size limits; Flutter is possible but challenging (size).
- Share extensions, notification service extensions, keyboard extensions — native targets; sharing code/data with App Groups.

27.6 Games 🎮

- **Flame** engine: game loop, sprites, animations, collision detection, input, effects, Forge2D (Box2D physics), Tiled maps, audio (`flame_audio`). Example: Google's I/O Pinball, "Super Dash" demo.
- Flutter **Casual Games Toolkit** (ads, IAP, games services, Firebase).
- `flutter_gpu` (experimental low-level GPU API) and fragment shaders for custom rendering.
- For 3D/heavy games, Unity/Unreal/Godot remain better choices.

27.7 AI features in Flutter apps (2025–2026 landscape) 🤖

- **Firebase AI Logic** (`firebase_ai`, formerly Vertex AI in Firebase) — call Gemini models from the app with App Check protection (don't embed raw API keys).
- `google_generative_ai` Dart SDK was deprecated in favour of Firebase AI Logic.
- **Flutter AI Toolkit** (`flutter_ai_toolkit`) — ready-made chat UI widgets (LlmChatView) with pluggable providers.
- On-device ML: ML Kit (`google_mlkit_*` — text recognition, barcode, face detection, translation), TensorFlow Lite / LiteRT (`tflite_flutter`), MediaPipe; on-device LLMs (Gemini Nano via Android AICore — platform-specific; `flutter_gemma` community package).
- Streaming responses (SSE/streams) → `StreamBuilder`/incremental text rendering; markdown rendering (`flutter_markdown_plus` / `gpt_markdown`).
- Developer tooling: **Dart & Flutter MCP server** (lets AI coding assistants introspect your running app, analyze errors, search pub.dev), Gemini in Android Studio, AI rules files for Flutter.
- Responsible AI: rate limit, moderate content, don't send PII without consent, cost control via backend proxies.

27.8 Interview Questions — Beyond mobile

Q1. Is Flutter web good for SEO? Not by default — canvas rendering; use for apps, not content sites; add metadata/prerendering if needed.

Q2. CanvasKit vs skwasm? CanvasKit: Skia compiled to Wasm used with JS builds; skwasm: used with Wasm (dart2wasm) builds — faster and can use multiple threads; falls back to CanvasKit when unsupported.

Q3. How do you reduce Flutter web load time? Deferred loading, Wasm build, asset optimization, caching/CDN, loading indicator, tree shaking, avoid huge packages.

Q4. What changes when targeting desktop? Mouse/keyboard interactions, window management, menus, file dialogs, responsive dense layouts, platform signing/distribution.

Q5. Can Flutter build watchOS/home screen widgets? Not directly; use native code with data sharing (WatchConnectivity, App Groups, home_widget).

Q6. How would you add AI chat to a Flutter app? Backend-proxied or Firebase AI Logic (App Check) LLM calls, streaming responses into UI, flutter_ai_toolkit or custom chat UI, safety & cost controls.

Chapter 28 — Working on a Big Flutter Project in a Company: Standards, Code Review, Git, Upgrades, Collaboration & Playbooks

Goal: Behave like an experienced engineer on day one: follow conventions, write reviewable code, handle upgrades and dependencies safely, collaborate with design/backend/QA/product, and know what to do in common team situations.

28.1 Your first weeks on an existing Flutter codebase 🗺️

1. **Run it**: follow README; note every undocumented step and fix the README (great first PR).
2. **Identify the stack**: Flutter version (`.fvmrc` /CI), state management, router, DI, networking, storage, architecture style, codegen tools, flavors, CI.
3. **Read the core**: `main*.dart` /bootstrap, router, DI setup, network client & interceptors, error handling, theme/design system, one complete feature end to end (UI → state → repository → API).
4. **Read the conventions**: `analysis_options.yaml` , CONTRIBUTING, PR template, ADRs, folder structure.
5. **Run the tests**; look at CI config.
6. **Ask** about release process, environments, test accounts, feature flags, monitoring dashboards.
7. **Ship something small** (bug fix) to learn the full workflow.

Golden rule: *Consistency beats personal preference.* Follow the existing patterns even if you'd choose differently; propose changes through discussion/ADR, not by mixing styles.

28.2 Coding standards checklist ✅

- `dart format` (CI enforced); lints (flutter_lints / very_good_analysis) with zero warnings.
- Naming: files `snake_case.dart` ; classes `UpperCamelCase` ; private widgets `_Foo` in same file when small.
- One public widget per file (generally); files < ~300–400 lines; extract widgets.
- `const` constructors; `final` fields; immutable models (freezed/equatable/records).
- No hard-coded strings (l10n), colours (theme), magic numbers (spacing tokens), URLs (config).
- No `print` (logger); no `dynamic` unless JSON/interop; no `!` unless proven safe; no empty `catch` .
- Dispose everything you create; cancel subscriptions; check `mounted` / `context.mounted` after awaits.
- Business logic out of widgets; widgets small & declarative.
- Imports: package imports (or relative — be consistent); no importing other features' internals.

- Public APIs documented with `///`.
- Tests for logic and critical UI; regression test for each bug fix.
- Accessibility: semantic labels, tap targets, text scaling.
- Feature flags for risky changes.

28.3 Git workflow & pull requests

- Branch naming: `feature/ABC-123-cart-coupons`, `fix/ABC-456-crash-on-logout`.
- Commits: small, meaningful; Conventional Commits (`feat:`, `fix:`, `refactor:`, `test:`, `chore:`) if the team uses them (enables automated changelogs).
- **PR size**: small (< ~400 lines changed of real code); split refactors from features.
- PR description: what & why, screenshots/videos for UI (light/dark, small/large screens), test steps, risks, linked ticket, migration notes.
- Generated files: follow team policy (commit `*.g.dart` or generate in CI) — consistency matters.
- Rebase/merge per team policy; keep main green; protected branches; required checks & reviews; CODEOWNERS for modules.

Code review checklist (as reviewer and author) 👀

Correctness: Does it meet requirements? Edge cases (empty, error, loading, offline, long text, RTL, large fonts, slow network)? Null safety? Async races (double taps, out-of-order responses, setState after dispose)? **Architecture**: Logic in right layer? Follows existing patterns? No new state management style sneaking in? Dependencies injected? **Performance**: Unnecessary rebuilds? Heavy work in build? Lists lazy? Images sized? Leaks (dispose/cancel)? **UI/UX**: Matches design (spacing tokens, typography, colours), responsive, accessible, dark mode, localization. **Security/Privacy**: No secrets, no PII in logs, secure storage, input validation. **Tests**: Present & meaningful; not brittle. **Readability**: Names, small functions/widgets, comments for "why". **Release**: Feature flagged? Analytics events? Backward compatible with APIs/older app versions? Migration for stored data? Tone: be kind, specific, explain why, distinguish blocking issues vs nits ("nit:"), praise good work.

28.4 Dependency management policy

- Approval process for new dependencies (Chapter 2 checklist), documented in an ADR for big ones.
- Wrap third-party SDKs behind your own interfaces (analytics, crash reporting, payments, maps) → swap/mocking easy.
- Regular update cadence (e.g., monthly `flutter pub outdated` review; Renovate/Dependabot PRs); read changelogs for breaking changes; update one major at a time with tests.
- Avoid `dependency_overrides` in main branch without ticket & comment.
- Monitor abandoned packages; plan replacements early.
- License compliance (`flutter pub deps`, license checker; `LicenseRegistry` / `showLicensePage` for in-app attributions).

28.5 Flutter SDK upgrade playbook ⬆️

Flutter stable ships roughly quarterly; staying ≤ 1–2 versions behind keeps upgrades small.

1. Read the release notes & **breaking changes** page for every version you skip.
2. Create an upgrade branch; bump `.fvmrc` /CI version.
3. `flutter pub upgrade` (and needed major bumps); `dart fix --apply`.
4. Update Android toolchain if warned (Gradle, AGP, Kotlin, compileSdk/targetSdk, Java version); iOS (min deployment target, CocoaPods/SwiftPM, Xcode version).
5. Fix analyzer deprecations (e.g., `withOpacity` → `withValues`, `WillPopScope` → `PopScope`, `MaterialState` → `WidgetState`, `textScaleFactor` → `textScaler`, Material/Cupertino package migration).
6. Run full tests, goldens (expect diffs from framework visual changes — review carefully), integration tests on real devices.
7. Smoke test critical flows on Android/iOS (and web/desktop if shipped); release via internal track first.
8. Document notable changes for the team.

28.6 Collaborating with other roles 🤝

Designers

- Use Figma tokens → theme/ThemeExtension; agree on spacing scale, typography, components; request states (loading/empty/error/disabled/focus), dark mode, RTL, large text, tablet layouts.
- Clarify platform adaptation (Material vs Cupertino behaviours).
- Widgetbook/golden tests as shared component catalog.

Backend

- API contracts first (OpenAPI/GraphQL schema), generated clients, mock servers for parallel development.
- Agree on error format (codes + messages), pagination style (cursor), date formats (ISO 8601 UTC), versioning, idempotency keys, rate limits, auth flows (token lifetimes, refresh).
- Backward compatibility: old app versions in the wild for months — never break existing endpoints without versioning; feature flags server-side.

QA

- Testable builds per PR (Firebase App Distribution), test accounts, feature flag toggles in debug menus, environment switcher (dev builds only), stable keys/semantics for automation, reproducible bug reports (steps, device, logs, video).

Product/PM

- Estimate with buffers for platform specifics (permissions, store review, native SDKs), clarify acceptance criteria incl. edge cases & analytics; release timelines vs store review times; communicate risk early.

28.7 Debug menu (hidden developer tools) — common in companies

Include in non-prod builds: environment switcher, feature flag overrides, network logs viewer (e.g., `chucker_flutter`, `alice`), crash test button, clear data/reset onboarding, view device info/FCM token, mock locations, slow animations toggle, accessibility debugging.

28.8 Documentation

- README (setup, run, flavors, test, release), CONTRIBUTING (conventions), ARCHITECTURE.md (layers, folders, diagrams), ADRs (`docs/adr/0001-use-riverpod.md`: context, decision, consequences), API docs (`dart doc`), runbooks (release, incident response), onboarding guide.

28.9 Technical debt management

- Track debt explicitly (tickets with impact), allocate % of each sprint, boy-scout rule (leave code better), refactor behind tests, strangler pattern for legacy modules (e.g., migrate GetX features to Bloc one by one), deprecate internal APIs with `@Deprecated('Use X')`.

28.10 Situational playbook (what to do when...) 📘

Situation	Action
You disagree with the architecture	Raise with data in an RFC/ADR; propose incremental path; respect team decision; don't fork patterns silently
Deadline is impossible	Communicate early with options (scope cut, phased release, feature flag), not silent overtime with hacks
A teammate's PR introduces a bug in prod	Blameless: fix forward, add tests/process improvements, postmortem focusing on systems
You must use an unfamiliar native SDK	Spike/prototype in a separate branch; Pigeon-based plugin wrapper; document; estimate with buffer
Store rejection	Read the guideline cited; reply in Resolution Center with clarifications or fix; keep demo accounts ready; appeal if wrong
Huge widget file (2,000 lines)	Refactor incrementally: extract widgets, move logic into state layer, add tests first
Flaky CI tests	Quarantine with ticket, fix root cause (timing, shared state), never just retry forever
Release day crash spike	Halt rollout, flag off, hotfix, communicate (Chapter 24 §24.8)
Inconsistent state management across modules	Pick standard via ADR, document migration path, migrate opportunistically
Junior asks for help	Pair program, explain why, point to docs/chapters, review kindly

Situation	Action
Need to estimate a feature	Break down: UI screens, states, API integration, edge cases, tests, platform specifics, analytics, l10n, accessibility, review/QA/store time
Security issue found	Follow responsible disclosure internally, hotfix, rotate secrets, notify per policy

---

28.11 Agile & delivery basics (interview behavioural readiness)

- Scrum (sprints, standups, planning, review, retro) / Kanban; story points; Definition of Done (code + tests + review + docs + analytics + accessibility + l10n + QA).
- Communicate blockers early; demo progress; write clear tickets (context, acceptance criteria, designs, API).

28.12 Interview Questions — Professional practice

Q1. How do you ensure code quality in a team? Lints & formatting in CI, code reviews with checklist, tests, architecture conventions & ADRs, pair programming, static analysis, definition of done.

Q2. How do you handle Flutter upgrades in a large project? Scheduled upgrades, read breaking changes, branch, dart fix, toolchain updates, full test suites & device testing, staged rollout.

Q3. How do you decide whether to add a package? Maintenance, popularity, publisher, platform support, license, size, native permissions, alternatives, ability to wrap/replace.

Q4. How do you work with backend teams? Contract-first APIs, generated clients, mock servers, agreed error/pagination/versioning formats, backward compatibility.

Q5. Tell me about a challenging bug you fixed. Use STAR (Situation, Task, Action, Result): describe symptoms, investigation tools (DevTools, logs), root cause, fix, prevention (tests/monitoring).

Q6. How do you manage technical debt? Make it visible, prioritize by impact, allocate capacity, refactor behind tests incrementally, avoid big-bang rewrites.

Q7. How do you mentor juniors? Pairing, constructive reviews, documentation, gradual ownership, psychological safety.

PART VI

Interview Preparation

Chapter 29 — Interview Question Bank, Part 1: Freshers & Junior (0–2 years)

How to use: read the question, answer **out loud** in your own words, then compare. Answers are written the way a strong candidate would say them — short first, then detail. Questions come from commonly reported company interviews (service companies, product startups, MNCs) and community question banks, updated for Dart 3 / Flutter 3.x.

A. Flutter basics

1. What is Flutter? Google's open-source UI toolkit for building natively compiled apps for mobile, web, desktop and embedded from a single Dart codebase. It uses its own rendering engine (Impeller/Skia) to draw every pixel, so UI is consistent across platforms.

2. What language does Flutter use and why? Dart — supports JIT (hot reload) and AOT (fast release code), has a GC optimised for short-lived objects, sound null safety, isolates for concurrency, and is controlled by Google, evolving with Flutter.

3. What are the advantages of Flutter? Single codebase, fast development with hot reload, expressive custom UI, near-native performance (AOT), consistent look across platforms, rich widget catalog, strong tooling (DevTools), growing ecosystem, multi-platform (mobile/web/desktop).

4. Disadvantages? Larger app size, non-native widgets need emulation of platform look, web not SEO-friendly, some platform features need native code/plugins, plugin quality varies.

5. What is a widget? An immutable description of part of the UI. Everything in Flutter UI is built from widgets — layout, styling, gestures, even the app.

6. Stateless vs Stateful widget? Stateless: no mutable state, rebuilds only when parent/inherited data changes. Stateful: has a `State` object that persists across rebuilds and can call `setState` to update UI.

7. When would you use a StatelessWidget? When the UI depends only on constructor parameters/inherited widgets — e.g., a label, an icon button that receives a callback, a product card receiving a product.

8. What is `setState`? A method on `State` that runs a callback and marks the widget dirty so it rebuilds in the next frame.

9. What is the `build` method? Returns the widget tree describing the UI for the current state; called often, so it must be fast and side-effect free.

10. What is `BuildContext`? A reference to the widget's location in the tree (the Element), used to find ancestors like Theme, MediaQuery, Navigator.

11. What is hot reload? Hot restart? Hot reload injects code changes into the running app and rebuilds UI while preserving state. Hot restart restarts the app's Dart code from `main()`, losing state.

12. What is `main()` and `runApp()`? `main()` is the Dart entry point; `runApp()` inflates the root widget and attaches it to the screen.

13. What is `MaterialApp`? A convenience widget that sets up Material design: theme, navigation (Navigator), localization, MediaQuery, ScaffoldMessenger, etc.

14. What is `Scaffold`? Basic Material page layout: app bar, body, floating action button, drawer, bottom navigation, snackbars/bottom sheets.

15. What is `Container`? A convenience widget combining padding, margin, decoration, constraints, alignment and transform.

16. Container vs SizedBox? SizedBox is lightweight, for fixed size or spacing, and const-friendly; Container is heavier and used when you need decoration/padding/etc.

17. Row vs Column? Row lays children horizontally, Column vertically. Both are Flex widgets with main/cross axis alignment.

18. What is `mainAxisAlignment` and `crossAxisAlignment`? Alignment along the main axis (Row: horizontal) and the perpendicular cross axis.

19. What is Expanded? Flexible? Expanded makes a child fill remaining space in a Row/Column (tight). Flexible lets a child take up to its share of space but be smaller (loose).

20. What is a Stack? A widget layering children on top of each other; `Positioned` places children relative to the edges.

21. What is `pubspec.yaml`? Project manifest: name, version, SDK constraints, dependencies, assets, fonts.

22. How do you add a package? `flutter pub add name` or add to `dependencies` in pubspec then `flutter pub get`, then import it.

23. How do you add images? Put them in `assets/`, declare in pubspec under `flutter: assets:`, and use `Image.asset('assets/x.png')`.

24. How do you load a network image? `Image.network(url)` (or `cached_network_image` for caching), with loading and error builders.

25. What is `ListView`? A scrollable list of widgets; `ListView.builder` builds items lazily for long lists.

26. Difference between `ListView` and `ListView.builder`? Default constructor builds all children immediately; builder creates items on demand as they scroll into view.

27. What is `GridView`? A scrollable 2D grid; `GridView.builder` with a grid delegate (`crossAxisCount` or `maxCrossAxisExtent`).

28. What is `SingleChildScrollView`? Makes a single child scrollable (e.g., a form) — builds the whole child at once.

29. What is `TextField`? How do you read its value? An input widget; read via a `TextEditingController` (`controller.text`) or the `onChanged` callback.

30. How do you validate a form? Use `Form` with a `GlobalKey<FormState>`, `TextFormField` validators, and call `formKey.currentState!.validate()`.

31. How do you navigate between screens? `Navigator.push(context, MaterialPageRoute(builder: (_) => NextPage()))` and `Navigator.pop(context)`; or named routes / go_router.

32. How do you pass data to another screen? Through the constructor of the next screen's widget.

33. How do you get data back from a screen? `final result = await Navigator.push(...)`, and in the second screen `Navigator.pop(context, value)`.

34. What is a SnackBar and how do you show it? A brief message at the bottom: `ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Hi')))`.

35. How do you show a dialog? `showDialog(context: context, builder: (_) => AlertDialog(...))`.

36. What is `GestureDetector`? A widget that detects gestures (tap, double tap, long press, drag, scale) on its child.

37. InkWell vs GestureDetector? InkWell shows Material ripple feedback (needs a Material ancestor); GestureDetector has no visual feedback but supports more gestures.

38. What is `Theme`? Defines colours, typography and component styles for the app; read via `Theme.of(context)`.

39. How do you implement dark mode? Provide `theme` and `darkTheme` to MaterialApp with `themeMode: ThemeMode.system` (or user choice).

40. What is `MediaQuery`? Provides screen/window info: size, padding (notch), text scale, orientation, brightness.

41. What is `SafeArea`? Adds padding to avoid notches, status bars and system UI.

42. What is `Padding` and `EdgeInsets`? Padding adds space around a child; EdgeInsets defines amounts (`all`, `symmetric`, `only`).

43. What is a `Key`? An identifier to preserve state of widgets when they move in the tree, e.g., list items being reordered.

44. What is the `const` keyword used for in Flutter? Creates compile-time constant widgets/objects, reused across rebuilds, improving performance.

45. What is `final` vs `const`? final: assigned once at runtime; const: compile-time constant, deeply immutable.

46. What is null safety? Types are non-nullable by default; nullable types use `?`; prevents null reference errors at compile time.

47. What are `?`, `!`, `??`, `?.` operators? Nullable type marker, null assertion, if-null default, null-aware member access.

48. What is `async` / `await`? Syntax to write asynchronous code that waits for Futures without blocking the thread.

49. What is a Future? An object representing a value (or error) that will be available later.

50. What is a Stream? A sequence of asynchronous events/values over time.

51. What is FutureBuilder? A widget that builds UI based on the latest snapshot of a Future (waiting, data, error).

52. What is StreamBuilder? Builds UI from the latest value of a Stream.

53. What are packages vs plugins? Packages are Dart code; plugins include platform-specific native code.

54. What is `flutter doctor`? A command that checks your environment setup and lists missing dependencies.

55. What are debug, profile, release modes? Debug (JIT, hot reload, slower), profile (AOT with profiling), release (AOT, optimised for users).

56. How do you build an APK / app bundle? `flutter build apk --release` / `flutter build appbundle`.

57. What is the `assets` folder used for? Bundled files like images, fonts, JSON config.

58. What is `Material Design` and `Cupertino`? Google's and Apple's design languages; Flutter provides widgets for both.

59. What is `AppBar`? Top toolbar with title, leading and action widgets.

60. What is `BottomNavigationBar` / `NavigationBar`? Tab bars at the bottom for top-level navigation (NavigationBar is the Material 3 version).

61. What is `Drawer`? A side panel menu opened from the edge or via the hamburger icon.

62. What is `TabBar` and `TabBarView`? Tabs with swipeable pages, coordinated by a TabController (often DefaultTabController).

63. What is `Icon` and where are icons from? Widget for vector icons from fonts like `Icons` (Material) or `CupertinoIcons`.

64. What is `Text.rich` / `RichText`? Text with multiple styles via TextSpans.

65. What is `Wrap`? Lays children in a row/column and wraps to the next line when out of space.

66. What is `Align` / `Center`? Position a child within itself; Center is Align with center alignment.

67. What is `AspectRatio`? Sizes a child to a specific width:height ratio.

68. How do you make a widget clickable? Wrap with GestureDetector/InkWell or use a button widget.

69. How do you disable a button? Pass `onPressed: null`.

70. What is `CircularProgressIndicator`? A spinner indicating loading.

71. What is `initState` used for? One-time initialization when the State is created: controllers, subscriptions, initial loads.

72. What is `dispose` used for? Cleanup: disposing controllers, cancelling subscriptions/timers.

73. What is `didChangeDependencies`? Called after initState and whenever inherited dependencies change.

74. What is `didUpdateWidget`? Called when the parent rebuilds with a new widget configuration.

75. What is the widget tree? The hierarchical structure of widgets forming the UI.

76. What is `debugShowCheckedModeBanner`? MaterialApp flag to hide the "DEBUG" ribbon.

77. What is the difference between `Navigator.push` and `Navigator.pushReplacement`? push adds on top (can go back); pushReplacement replaces current screen (can't go back to it).

78. How do you clear the navigation stack (e.g., after logout)?
`Navigator.pushAndRemoveUntil(context, route, (r) => false)`.

79. What is `SharedPreferences`? Key-value persistent storage for small simple data like settings.

80. How do you make an HTTP request? With `http` or `dio` packages: `await http.get(Uri.parse(url))`, then `jsonDecode(response.body)`.

81. How do you parse JSON? `jsonDecode` to a Map/List, then `Model.fromJson(map)` factories (manual or json_serializable).

82. What is `Visibility` widget? Shows/hides a child, optionally maintaining state/size.

83. What is `Opacity`? Makes a child partially transparent.

84. What is `Hero` widget? Animates a shared widget between two screens.

85. What is `AnimatedContainer`? A Container that animates changes to its properties automatically.

86. What is `Spacer`? An empty Expanded widget to push siblings apart in Row/Column.

87. What is `Divider`? A thin horizontal line separator.

88. What is `Card`? A Material panel with rounded corners and elevation.

89. What is `ListTile`? A fixed-height row with leading, title, subtitle, trailing — used in lists.

90. What is `Checkbox`, `Switch`, `Radio`, `Slider`? Selection controls, each with a value and `onChanged` callback.

91. What's the difference between `Image.asset` and `AssetImage`? Image.asset is a widget; AssetImage is an ImageProvider (used with `Image(image:)`, `DecorationImage`, CircleAvatar).

92. What is `ClipRRect`? Clips a child with rounded corners.

93. What is `BoxDecoration`? Defines background colour, gradient, image, border, radius and shadows for a box.

94. What is the `Flutter Inspector`? DevTools tool to view the widget tree, layout, and debug UI.

95. What file is the entry point of a Flutter app? `lib/main.dart` by default (configurable with `-t`).

96. How do you change the app name and icon? Android: `android:label` in AndroidManifest + mipmap icons; iOS: `CFBundleDisplayName` in Info.plist + AppIcon; or packages like flutter_launcher_icons.

97. What is `Orientation` / how to lock orientation? `SystemChrome.setPreferredOrientations([DeviceOrientation.portraitUp])` after ensureInitialized.

98. How do you get the screen width? `MediaQuery.sizeOf(context).width`.

99. What is `LayoutBuilder`? Builds widgets based on parent constraints (for responsive designs).

100. What is `State management`? How app data that changes is stored, updated and reflected in the UI (setState, Provider, Riverpod, Bloc...).

B. Dart basics (fresher level)

101. What are Dart's basic data types? int, double, num, String, bool, List, Set, Map, Runes, Symbol, Null, Record.

102. `var` vs `dynamic`? var infers a fixed type; dynamic disables type checking.

103. What is string interpolation? `'Hello $name'`, `'${user.name}'`.

104. What is a List / Set / Map? Ordered collection with duplicates / unique items / key-value pairs.

105. What are named parameters? Parameters passed by name in `{}`; can be `required` or have defaults.

106. What are optional positional parameters? In `[]`, optional by position.

107. What is an arrow function? `=> expr` shorthand for a function returning one expression.

108. What is an anonymous function? A function without a name, e.g., `(x) => x * 2`.

109. What is a class and object? A blueprint and its instance.

110. What is a constructor? Named constructor? Special method to create objects; named constructors give alternative creation paths (`Point.origin()`).

111. What is inheritance? A class extending another to reuse/override behavior (`extends`).

112. What is an abstract class? Cannot be instantiated; can declare abstract methods for subclasses.

113. What is an interface in Dart? Every class implicitly defines an interface; use `implements`.

114. What is a mixin? A reusable set of members added to a class with `with`.

115. What is `static`? Class-level members not tied to instances.

116. What is an enum? A fixed set of named values; enhanced enums can have fields and methods.

117. What is `try/catch/finally`? Exception handling blocks.

118. What is a getter/setter? Computed property accessors (`get` / `set`).

119. What does `_` prefix mean? Private to the library (file).

120. What is `late`? Non-nullable variable initialized later or lazily.

121. What is the cascade operator `..`? Chain multiple operations on the same object.

122. What is the spread operator `...`? Inserts elements of a collection into another.

123. What is `typedef`? A type alias.

124. What is `assert`? Debug-only check that throws if false.

125. What is `is` and `as`? Type test and type cast.

126. What's `map`, `where`, `reduce`, `fold` on lists? Transform, filter, combine (non-empty), combine with initial value.

127. What is a generic? Type parameter allowing reusable type-safe code (`List<int>`).

128. What is `toString()` override for? Readable string representation for debugging.

129. What is `==` and `hashCode`? Equality operator and hash used in sets/maps; override both for value equality.

130. What is `required` keyword? Marks named parameters mandatory.

C. Short practical tasks freshers are asked

- Build a counter app (Chapter 1).
- Build a login form with validation (Chapter 8).

- Fetch and show a list of users from `https://jsonplaceholder.typicode.com/users` with loading/error states (Chapter 32 has a full solution).
- Build a to-do list with add/delete/complete and persistence (Chapter 32).
- Explain the output of a given layout (why overflow happens).
- Swap two variables, reverse a string, check palindrome, find duplicates in a list, FizzBuzz in Dart (Chapter 32).

D. Tips for fresher interviews

- Explain with small examples; draw the widget tree.
- Mention `const`, keys, disposing controllers and `mounted` checks — shows maturity.
- Have 1–2 personal projects on GitHub with clean code, README, screenshots, and tests.
- Be honest about what you don't know; describe how you'd find out (docs, DevTools, experiments).

Chapter 30 — Interview Question Bank, Part 2: Intermediate (2–5 years)

These are the questions that decide most mid-level interviews. Answers include the "why" and the trade-offs interviewers want to hear.

A. Widgets, rendering & lifecycle

1. Explain the widget, element and render object trees. Widgets are immutable configs rebuilt often; elements are long-lived instances that hold position, lifecycle and State and link widgets to render objects; render objects do layout, paint and hit testing. Flutter diffs new widgets against existing elements (`canUpdate`: same runtimeType & key) to reuse expensive objects.

2. What happens when you call setState? The callback runs synchronously; the element is marked dirty and a frame scheduled; on the next frame, dirty elements rebuild in depth order, then layout/paint/composite only what changed.

3. Full StatefulWidget lifecycle? createState → initState → didChangeDependencies → build → (didUpdateWidget/setState/didChangeDependencies → build)* → deactivate → (activate) → dispose; reassemble on hot reload.

4. When is `didChangeDependencies` called? Immediately after initState and whenever an InheritedWidget the State depends on changes (e.g., Theme, MediaQuery, Locale, Provider).

5. Why can't we call `Theme.of(context)` in initState? Dependencies on inherited widgets must be registered when the element is fully mounted; Flutter asserts in debug. Use didChangeDependencies.

6. When do you need keys? Give an example. When stateful widgets of the same type are reordered/removed in a collection — e.g., a reorderable todo list with checkboxes holding local state: without keys, state stays at positions; with `ValueKey(todo.id)` it follows the item.

7. ValueKey vs ObjectKey vs UniqueKey vs GlobalKey? ValueKey compares value equality; ObjectKey compares identity of an object; UniqueKey is unique each instance (forces new element); GlobalKey is unique app-wide and provides access to State/context and reparenting.

8. What is InheritedWidget and how does Provider use it? A widget that exposes data to descendants with O(1) lookup and notifies dependents when `updateShouldNotify` returns true. Provider wraps InheritedWidget (InheritedProvider) to simplify exposing/listening and to handle disposal.

9. Difference between `dependOnInheritedWidgetOfExactType` and `getInheritedWidgetOfExactType`? The first registers a dependency (rebuild on change); the second just reads.

10. Why are widgets immutable? Simplifies diffing and reasoning; allows const canonicalization; changes are expressed by creating new widgets; mutable state lives in State/elsewhere.

11. What does the `const` constructor do at runtime for widgets? Instances are canonicalized at compile time; identical instance on rebuild lets the element skip updating that subtree.

12. Explain the "constraints go down, sizes go up" rule with an example of an overflow fix. Row gives unbounded width to non-flex children; a long Text tries to be its full width and overflows. Wrapping it in Expanded gives it a bounded width (remaining space), and `overflow: ellipsis` handles truncation.

13. What causes "Vertical viewport was given unbounded height"? A ListView (wants infinite height) inside a Column/other scrollable (unbounded height). Fix with Expanded/SizedBox or slivers.

14. What is the difference between `MediaQuery.of(context).size` and `MediaQuery.sizeOf(context)`? sizeOf subscribes only to size changes (InheritedModel aspect); `of` rebuilds on any MediaQuery change (e.g., keyboard insets).

15. What is LayoutBuilder vs MediaQuery for responsiveness? LayoutBuilder gives parent constraints (component-level), MediaQuery gives window info (app-level).

16. What is a `RepaintBoundary`? Isolates painting into its own layer so repaints don't propagate; useful for frequently changing subtrees.

17. What are slivers and when do you use CustomScrollView? Lazy scrollable building blocks with sliver protocol; CustomScrollView combines multiple slivers (app bars, lists, grids) into one scroll with effects.

18. How do you preserve state in tabs/PageView? AutomaticKeepAliveClientMixin with `wantKeepAlive`, PageStorageKey, IndexedStack, or state outside widgets.

19. What is `addPostFrameCallback` and when do you use it? Runs a callback after the current frame is rendered; used to show dialogs after build, measure sizes, or trigger navigation from initState safely.

20. What is the difference between `Navigator 1.0` and `2.0`? Imperative push/pop vs declarative page lists synced with app state & URLs (Router API).

B. State management

21. How do you decide between setState and a state management library? setState for local ephemeral UI state; libraries for shared app state, business logic separation, testability, async/caching needs.

22. Explain Provider's `watch`, `read`, `select`. watch: listen & rebuild; read: one-time access in callbacks; select: rebuild only when a derived value changes.

23. What is `ChangeNotifierProvider.value` vs `create`? create instantiates and disposes the notifier; value provides an existing instance without disposing it.

24. Riverpod vs Provider? Riverpod is compile-safe, context-independent, supports multiple same-type providers, autoDispose/family, built-in async (AsyncValue), easy overrides for tests.

25. Explain ref.watch vs ref.read vs ref.listen. watch rebuilds/recomputes on change (in build); read is a one-time read (callbacks); listen triggers side effects.

26. What is AsyncValue and how do you render it? Union of loading/data/error; render via `when` or pattern matching on AsyncData/AsyncError/AsyncLoading.

27. Explain the Bloc pattern and its components. Events in, states out via streams; `on<Event>` handlers call repositories and emit immutable states; UI uses BlocBuilder/BlocListener; BlocProvider for DI.

28. Bloc vs Cubit? Cubit: functions emitting states; Bloc: events processed by handlers with transformers (debounce, droppable, restartable) and full traceability.

29. Why might BlocBuilder not rebuild after emit? New state equals old state (mutated list or Equatable props missing) — Bloc skips equal states.

30. How do you handle one-time events like navigation or snackbars? BlocListener / ref.listen / listening to a stream of effects; not inside build logic. Avoid storing "show snackbar" as permanent state without consuming it.

31. What is `Equatable` / why do we need value equality? Provides `==`/hashCode based on props → states compare by value → proper deduplication and testing.

32. How do you combine multiple states (e.g., filters + data)? Riverpod: provider that watches both. Bloc: combine streams in a repository or listen to one bloc and add events to another in UI; or a single bloc owning both.

33. What is the difference between ephemeral and app state? Local vs shared/persistent; examples: current tab vs logged-in user.

34. Describe how you'd structure state for a paginated list with pull-to-refresh and errors. State: items, page/cursor, status (initial/loading/success/failure), hasMore, isLoadingMore, error; events/methods: fetchFirst, fetchMore (guarded, droppable), refresh (reset); UI: footer loader, retry, empty view.

35. What is GetX and its criticisms? All-in-one state/DI/routing; criticized for magic/global state, testability, coupling and maintenance concerns.

C. Async & Dart depth

36. Explain the event loop and microtasks. After synchronous code, Dart drains the microtask queue fully, then processes one event, then microtasks again. Futures from timers/IO go to event queue; completion callbacks and scheduleMicrotask go to microtask queue.

37. Predict the output:

```
void main() {
  print('A');
  Future(() => print('B'));
  Future.microtask(() => print('C'));
  print('D');
}
```

`A D C B`.

38. How do you run network calls in parallel? Start futures then `await Future.wait([...])` or records `(f1, f2).wait`.

39. What is an isolate and when do you use it? Separate memory/thread for CPU-heavy work (JSON parsing, image processing) to keep UI smooth; `Isolate.run` / `compute`.

40. Why does `items.forEach((i) async => await upload(i))` not wait? forEach ignores returned futures; use `for` loop with await or `Future.wait`.

41. Single-subscription vs broadcast stream? One listener with buffering vs many listeners without buffering.

42. How do you debounce user input? Timer cancel/restart, rxdart `debounceTime`, Bloc transformers, Riverpod delay + dispose check.

43. What are sealed classes and pattern matching used for in Flutter apps? Modelling finite states/results with exhaustive switches (e.g., UI states, API results), with destructuring.

44. What is a factory constructor? Where used? Constructor that can return existing/subtype instances: fromJson, singletons, caches.

45. What are extension methods? Example in Flutter? Add methods to existing types: `context.theme`, `String.capitalized`.

46. What is `late final` useful for in State? Non-nullable fields initialized in initState (controllers) or lazily.

47. What's the difference between `Iterable` and `List`? Iterable is lazy & sequential access; List is indexed and materialized.

48. Mixins: what does `on` do? Restricts which classes can use the mixin (must extend/implement the `on` type), allowing the mixin to call its members.

49. Explain records and give a use case. Anonymous immutable tuples with structural equality; return multiple values like `(User, List<Order>)`.

50. What are class modifiers (`base`, `interface`, `final`, `sealed`)? Control extension/implementation outside the library; sealed enables exhaustiveness.

D. Networking, storage & data

51. How do you structure networking in an app? API client (Dio) with interceptors → remote data sources/services → repositories mapping DTOs to domain models & errors → state layer.

52. How do you implement token refresh? Interceptor catching 401, single refresh (QueuedInterceptor/lock), retry original request, logout on failure, separate client for refresh.

53. How do you serialize JSON at scale? json_serializable/freezed codegen; parse large payloads in isolates; handle nulls/unknown enum values.

54. Where do you store tokens and why not SharedPreferences? Secure storage (Keychain/Keystore) because prefs are unencrypted.

55. sqflite vs Drift vs Hive? Raw SQL vs typed reactive SQL with migrations vs simple key-value boxes (maintenance status: community fork).

56. How do you design offline support? Local DB as source of truth, sync queue, conflict resolution, connectivity awareness, optimistic UI.

57. How do you cancel an in-flight request when the user leaves a screen? Dio CancelToken cancelled in dispose / provider onDispose / bloc close.

58. How do you handle pagination? Offset or cursor, guarded loadMore, hasMore flag, merging results, dedup, refresh reset.

59. How do you cache API responses? HTTP cache interceptor, in-memory with TTL in repository, persistent DB cache with stale-while-revalidate.

60. How do you handle different environments' base URLs? Flavors + dart-define(-from-file) → AppConfig injected into Dio.

E. Navigation, platform & UI features

61. How do you implement authentication-aware routing? go_router redirect + refreshListenable bound to auth state; preserve original location.

62. go vs push in go_router? go: navigate to URL with stack built from route hierarchy; push: add on top of current stack.

63. How do you handle Android back button / predictive back? PopScope (`canPop`, `onPopInvokedWithResult`); avoid deprecated WillPopScope.

64. How do deep links work? App Links/Universal Links configuration + domain verification files; router maps URL to screens.

65. What are platform channels? Give a real example. MethodChannel to call native APIs not covered by plugins (e.g., reading a vendor SDK's device ID); EventChannel for native streams (sensor updates).

66. What is Pigeon and why use it? Generates type-safe channel code, removing stringly-typed errors.

67. How do you request runtime permissions? permission_handler with manifest/Info.plist declarations; contextual requests; handle permanently denied.

68. How do push notifications work in different app states? onMessage (foreground → show local notification), onMessageOpenedApp (background tap), getInitialMessage (terminated tap), onBackgroundMessage handler (top-level, vm:entry-point).

69. How do you implement localization? flutter_localizations + gen-l10n ARB files, ICU plurals; RTL support with directional widgets.

70. How do you support tablets/large screens? Breakpoints with LayoutBuilder, adaptive navigation (bar/rail/drawer), master-detail, constrained content widths.

71. How do you implement custom themes/design tokens? ColorScheme.fromSeed + component themes + ThemeExtension for custom tokens; design system package.

72. Implicit vs explicit animations? Implicit animate property changes automatically; explicit use AnimationController for full control.

73. How do you create a shimmer/skeleton loader? Shimmer package or custom ShaderMask with LinearGradient animated by a controller; skeleton widgets with fixed sizes to avoid layout shift.

74. How do you build a custom chart or shape? CustomPainter with Canvas; shouldRepaint; repaint listenable for animation.

75. How do you make a widget accessible? Semantics labels, tooltips, contrast, tap targets, text scaling, focus order.

F. Testing, performance & release

76. Unit vs widget vs integration tests? Pure logic vs UI component in test environment vs full app on device.

77. pump vs pumpAndSettle? Single frame vs pump until no scheduled frames (fails on infinite animations).

78. How do you mock an API in tests? Inject repository interfaces; mocktail mocks or fakes; MockClient for http; http_mock_adapter for Dio.

79. What are golden tests and their pitfalls? Visual snapshots; fonts and platform rendering differences, update with --update-goldens.

80. How do you find out why a screen is janky? Profile mode DevTools: UI vs raster thread; track rebuilds; CPU profiler; fix heavy builds/effects/images; re-measure.

81. How do you reduce rebuilds? const, smaller widgets, select/BlocSelector, builder child, MediaQuery aspects.

82. How do you reduce app size? AAB, split-per-abi, obfuscate+split-debug-info, asset compression, remove heavy deps, R8.

83. How do you detect memory leaks? DevTools memory snapshot diff, leak_tracker; check disposal of controllers/subscriptions.

84. What are flavors? Environment-specific builds with distinct IDs/configs.

85. How do you sign and publish an Android app? Upload keystore + key.properties + signingConfig; AAB; Play App Signing; tracks & staged rollout.

86. What is obfuscation and how do you read crash stack traces afterwards? Symbol renaming; keep split-debug-info symbols and symbolize (`flutter symbolize`, Crashlytics/Sentry upload).

87. What CI steps do you set up? Format, analyze, test, build, distribute; pinned Flutter version; secrets management.

88. What is the difference between `flutter build apk` and `appbundle`? APK is installable directly; AAB is for Play distribution which generates optimized APKs.

89. How do you handle a production crash spike? Halt rollout, feature flag off, hotfix, monitor, postmortem.

90. What's your approach to code review? Correctness, architecture consistency, performance, accessibility, tests, readability; kind & specific feedback.

G. Scenario-based (mid-level favourites)

91. A user taps "Pay" twice and is charged twice. How do you prevent this? Disable button during request/droppable transformer, command running-state guard, backend idempotency key per order, server verifies payment status; show progress & handle retries safely.

92. Search results flicker/out of order when typing fast. Fix? Debounce input, cancel previous requests (CancelToken/restartable/switchMap), ignore stale responses by query token, show loading state.

93. App shows stale profile data after editing on another screen. Fix? Single source of truth (repository stream/provider), invalidate/refresh after mutation, avoid copying data into local widget state.

94. App freezes for 2 seconds when opening a big report. Fix? Parse/compute in isolate, paginate/virtualize rendering, show progress, cache results.

95. The list loses scroll position when switching tabs. Fix? PageStorageKey or keep-alive/IndexedStack/StatefulShellRoute.

96. Push notification tap does nothing when app is closed. Fix? Handle getInitialMessage after router initialization; ensure payload route mapping.

97. Login works on debug build, fails on Play Store build (Google Sign-In). Fix? Add Play App Signing SHA-1/SHA-256 to Firebase/Google Cloud OAuth client.

98. Images make the app crash on low-end devices. Fix? Decode at display size (cacheWidth), smaller CDN images, limit image cache, dispose, pagination.

99. A form screen is slow while typing. Fix? Avoid setState on the whole page per keystroke; isolate field widgets; ValueListenableBuilder on controllers; debounce validations.

100. Designers want the same app to feel native on iOS and Android. Approach? Adaptive widgets (`.adaptive` constructors, Cupertino where needed), platform-aware navigation transitions/back gestures, platform icons, consistent brand theme; test on both.

Chapter 31 — Interview Question Bank, Part 3: Senior, Lead & Architect (5+ years) + Mobile System Design

Senior interviews test **depth** (internals), **judgment** (trade-offs), **scale** (architecture, teams, releases) and **leadership**. Each answer below is a model answer you can adapt with your own experiences.

A. Deep internals

1. Walk me through a frame from vsync to pixels. Vsync → `handleBeginFrame` runs transient callbacks (Tickers advance animations) → microtasks → `handleDrawFrame` runs persistent callbacks: `buildScope` rebuilds dirty elements (depth-ordered), `flushLayout` lays out dirty relayout boundaries, `flushCompositingBits`, `flushPaint` records display lists into layers for dirty repaint boundaries, `compositeFrame` builds a Scene from the layer tree and submits it, `flushSemantics`, `finalizeTree` unmounts inactive elements → post-frame callbacks. The raster thread then rasterizes the layer tree with Impeller (Metal/Vulkan/GLES) and presents. Two separate budgets (UI & raster) must be met.

2. How does Flutter achieve O(N) layout? Single-pass constraint propagation with caching: each render object is laid out once per frame given constraints; relayout boundaries stop propagation; layout is skipped if constraints are unchanged and not dirty. Intrinsic sizing breaks this (speculative passes), hence its cost.

3. What is a relayout boundary exactly? A RenderObject whose size can't affect its parent due to: parent not using its size (`parentUsesSize: false`), tight constraints, `sizedByParent`, or it's the root. markNeedsLayout stops there.

4. How are InheritedWidget lookups O(1)? Each Element holds a persistent hash map of inherited elements by type copied from parent at mount; lookup is a map access; dependents are tracked in the InheritedElement.

5. Explain how keys influence `updateChildren` for multi-child widgets. Algorithm walks from the top and bottom matching by `canUpdate`; for the middle section, old keyed children are put in a map and new children with keys are matched there; unkeyed are matched positionally; unmatched old ones are deactivated. This is why keys preserve state across reorders.

6. What's the difference between Skia and Impeller, and what trade-offs remain with Impeller? Skia compiled shaders at runtime → first-use jank; Impeller precompiles a fixed shader set, uses Metal/Vulkan, offers predictable performance. Trade-offs: some effects had fidelity/perf differences during migration, device/driver fallbacks on Android (GLES), larger engine size slightly; team evaluates per-release issues.

7. What happens during hot reload at the VM level, and why can't it update `initState` changes? Kernel deltas for changed libraries are loaded into the running VM, replacing function bodies/class definitions while preserving heap objects; framework calls reassemble and rebuilds. Already-executed code (initState, main, static initializers) isn't re-run, so their effects remain.

8. How do platform channels and FFI differ in threading/performance? Channels serialize messages via codecs and hop to the platform thread asynchronously; FFI calls native functions synchronously on the calling thread with near-zero overhead. Since UI/platform thread merging (3.29+ on mobile), synchronous native interop (FFI-based jnigen/ffigen bindings) becomes practical.

9. How does Dart's GC interact with Flutter's frame scheduling? Generational GC: fast scavenges for young objects; Flutter engine notifies the VM of idle time between frames (`notifyIdle`) so GC work can be scheduled without janking. Excessive allocation of long-lived objects increases old-gen GC pressure.

10. How would you diagnose a raster-thread bottleneck? DevTools frame chart shows raster time high; check for saveLayer (opacity, shader mask, color filters), BackdropFilter, clips, large images, many platform views; use "highlight oversized images", Impeller-specific tracing, simplify effects, add RepaintBoundaries to cache static layers.

11. Explain element reparenting with GlobalKey. When a widget with a GlobalKey moves to a new parent in the same frame, its element is found in the inactive list (or detached) and re-inserted, preserving State and render objects instead of recreating.

12. How do Tickers know to stop when a route is not visible? `TickerMode` widget (set false by routes when fully obscured / in some cases) mutes tickers created from TickerProviderStateMixin; muted tickers don't schedule frames.

13. How does `const` canonicalization interact with `Widget.==`? Widget's `==` is identity by default (not overridden); const instances are identical, so `Element.updateChild` short-circuits when `child.widget == newWidget` (identity).

14. What is `BuildOwner` and `PipelineOwner`? BuildOwner manages element rebuilds (dirty list, global keys, inactive elements); PipelineOwner manages render objects needing layout/paint/compositing/semantics updates.

15. How does the semantics tree get built and what's its cost? Render objects describe semantics via `describeSemanticsConfiguration`; PipelineOwner flushes semantics into SemanticsNodes sent to the platform when accessibility is enabled — adds cost proportional to the tree; merge/exclude reduces nodes.

B. Architecture & scale

16. Design the architecture for a 30-developer Flutter app. Monorepo with pub workspaces/melos; packages per feature with clear public APIs, shared core (network, storage, analytics abstractions), design system package with widgetbook & goldens; layered architecture (presentation/domain-

optional/data) with one state management standard (Riverpod or Bloc) and DI; go_router with typed routes and deep links; feature flags; CODEOWNERS; CI with affected-package testing; ADRs; strict lints; contract-first APIs with generated clients; observability (Crashlytics/Sentry, analytics taxonomy, performance monitoring); release train with staged rollouts.

17. Riverpod or Bloc for a large enterprise app — defend your choice. (Example) Bloc when the org values explicit event-driven flows, traceability (BlocObserver audit trails), strict separation and many juniors following a rigid pattern; Riverpod when you want less boilerplate, compile-time-safe DI, built-in async caching/invalidation and composable providers. Either works; consistency, testing culture and team familiarity outweigh library choice. Mention migration costs and that both can coexist during transitions.

18. How do you design offline-first sync with conflict resolution? Local DB (Drift) as source of truth; mutations recorded in an outbox with operation type, payload, idempotency key, timestamps/version; background sync (WorkManager/BGTask + on connectivity regain) pushes outbox then pulls deltas (updatedAt cursor); conflicts resolved by strategy per entity: server-authoritative, last-write-wins with vector/version numbers, or field-level merge; UI shows sync status; retries with backoff; handle deletes with tombstones; tests for sync edge cases.

19. How do you handle API versioning and old app versions? Backend keeps backward-compatible endpoints or versioned routes (v1/v2); app sends version header; minimum supported version enforced via remote config/force update; feature flags gate new APIs; monitor version adoption.

20. How would you modularize a monolithic Flutter app incrementally? Identify boundaries (features, core), extract shared core first (network/storage/design system) into packages, then features one at a time behind public interfaces; enforce dependencies with package structure & lints (e.g., import rules); maintain build green; measure build/test times.

21. How do you design a design system in Flutter? Tokens (color, typography, spacing, radius, elevation, motion) → ThemeData + ThemeExtensions; component library wrapping Material widgets with brand APIs (AppButton variants), documentation in widgetbook, golden tests per component/state/theme, accessibility built in, versioned package, Figma token sync.

22. How do you structure error handling across layers? Data layer maps exceptions to typed failures (sealed), repositories return Result or throw domain failures consistently, state layer exposes error states, UI maps to localized messages; global handlers capture unexpected errors to crash reporting; error boundaries via ErrorWidget.builder.

23. How do you implement feature flags properly? Remote config/LaunchDarkly with local defaults, typed flag accessors, flag evaluation cached per session, kill switches, cleanup policy for stale flags, debug overrides in dev builds, analytics for experiment variants.

24. How do you approach dependency injection & scoping (e.g., per-user session)? Composition root at bootstrap; app-scoped singletons (HTTP client), session-scoped dependencies created on login and disposed on logout (get_it scopes, Riverpod providers watching auth state, or nested ProviderScopes/BlocProviders), feature-scoped dependencies via route-level providers.

25. What metrics define app health and how do you monitor them? Crash-free users/sessions, ANR rate, startup time (cold/warm), slow/frozen frames, network error rates/latency, memory, app size, store rating, funnel conversion; tools: Crashlytics/Sentry, Firebase Performance, Play vitals, Xcode Organizer, analytics dashboards, alerts.

C. Performance & reliability at senior level

26. Startup takes 4s on low-end Android. Plan? Measure (`--trace-startup` , Android vitals, systrace/Perfetto); split pre-runApp work; defer SDK inits (analytics, remote config, ads) after first frame or lazily; parallelize independent inits; avoid sync disk IO & large asset decoding on startup; lighter first screen; check native Application.onCreate work from plugins; consider baseline profiles for Android native code; deferred components; verify Impeller/GL backend behaviour on device.

27. Memory grows steadily during a long session. Approach? Reproduce with automated navigation loop; DevTools snapshots & diff; find growing classes and retaining paths; check listeners/subscriptions/static caches/image cache; use leak_tracker in tests; fix and add regression tests.

28. How do you guarantee 120fps scrolling of a complex feed? Lazy slivers with known extents where possible, cheap item builds (precomputed view models), const subtrees, image sizing/caching & prefetch, avoid saveLayer effects, RepaintBoundaries for animated items, isolate parsing, pagination, profile on target devices, performance tests in CI.

29. How would you build a performance regression gate in CI? integration_test with traceAction on scripted scenarios in profile mode on fixed device(s)/Firebase Test Lab; extract frame build/raster percentiles; compare with baseline thresholds; fail PR or alert on regression; track trends.

30. How do you handle large lists of heterogeneous items with dynamic heights efficiently? SliverList with builder delegates, `findChildIndexCallback` for keyed reorders, avoid intrinsic measurements, cache measurements in models if needed, `super_sliver_list` for better extent estimation and jumping, chunked rendering.

D. Platform & native expertise

31. You must integrate a native payment SDK (Android AAR + iOS XCFramework) with callbacks and an activity result. Design it. Write a plugin with Pigeon: HostApi for `startPayment(PaymentRequest)` (async), FlutterApi/EventChannel for status updates; Android implements ActivityAware and ActivityResultListener, handles process death; iOS presents SDK UI from the root view controller; map errors to typed results; threading to main; ProGuard rules; integrate via federated structure if reused; example app + integration tests; server verifies payment.

32. How would you add Flutter to an existing large native app? Add-to-app module, pre-warmed cached engines (FlutterEngineGroup for multiple instances), routes/entrypoints per feature, Pigeon bridge for shared services (auth token, analytics), consistent navigation/back behaviour, theming parity, CI integration (AAR/frameworks), memory considerations, gradual rollout with feature flags.

33. When would you use platform views and what problems do you anticipate? For native-only components (maps, webviews, ads, AR). Problems: performance (composition), gesture conflicts, keyboard/focus issues, accessibility, z-order with overlays, testing; mitigate by limiting count/size, choosing composition modes, handling gestures explicitly.

34. How would you build background location tracking reliably on both platforms? Android foreground service with location type & notification, permissions (fine + background), battery optimizations; iOS background mode location, significant-change/region monitoring, "Always" permission flow; persist locally and batch upload; handle OS kills; consider commercial SDK (flutter_background_geolocation); comply with store policies.

35. Explain how you'd implement secure biometric login. Generate/store a key in Keystore/Secure Enclave requiring biometric auth; on login, biometric prompt unlocks key to sign a server challenge or decrypt a refresh token; fallback to password; handle enrollment changes (key invalidation); never just "biometric success → logged in" client-side.

E. Security & compliance (senior)

36. How do you secure an app handling financial transactions? TLS + pinning with rotation plan, secure storage & encrypted DB, short-lived tokens with refresh rotation, device attestation (Play Integrity/App Attest), RASP/root detection with server-side risk scoring, FLAG_SECURE/snapshot hiding, no secrets in app, obfuscation, server-side authorization & idempotency, audit logs, MASVS-L2 compliance, pentests, compliance (PCI DSS scope reduction with tokenization).

37. A leaked API key was found in your app binary. What now? Rotate/revoke immediately, restrict new key, move secret server-side, assess abuse via logs/billing, release update, add secret scanning to CI, postmortem.

F. Leadership & process

38. How do you lead a migration (e.g., Provider → Riverpod) across a big codebase? RFC with goals/costs, pilot on one feature, guidelines & codemods/snippets, train team, incremental migration with coexistence, tracking dashboard, definition of done, avoid freezing feature work, measure outcomes.

39. How do you handle conflicting opinions in architecture discussions? Clarify requirements and constraints, compare options with data/prototypes, document trade-offs in ADR, decide (with owner), commit as a team, revisit with new evidence.

40. How do you mentor and grow a team? Pairing, code review culture, learning sessions, ownership areas, clear career expectations, psychological safety, feedback loops.

41. How do you estimate a large feature? Break into tasks (UI, state, data, native, tests, analytics, l10n, a11y, QA, release), identify unknowns (spikes), use ranges, add risk buffers, revisit after spikes, communicate assumptions.

42. How do you ensure quality while shipping fast? Trunk-based development with feature flags, automated tests at the right levels, CI gates, staged rollouts, monitoring & alerting, fast rollback strategies (flags/code push), postmortems.

G. Mobile system design — the framework 🏗️

Use this structure for any "Design X app" question (45–60 min):

1. **Clarify requirements** (5 min)
    - Functional: core user flows.
    - Non-functional: scale (DAU), offline, latency, real-time, security, platforms, accessibility, i18n, battery/data constraints.
    - Out of scope.
2. **High-level architecture** (10 min) — client modules, layers, backend touchpoints (REST/GraphQL/WebSocket/push), CDN.
3. **Data model & API** (10 min) — entities, endpoints/events, pagination, caching headers.
4. **Client deep dive** (15 min) — state management, local storage & sync, networking (retries, auth), navigation/deep links, image handling, background work, notifications.
5. **Non-functional deep dive** (10 min) — performance, offline, security, observability, testing, release strategy.
6. **Trade-offs & extensions** (5 min).

Example 1: Design a chat app (WhatsApp-like) 💬

- **Requirements**: 1:1 & group chats, text/images, delivery/read receipts, offline sending, push notifications, typing indicators, message history sync, E2E encryption (optional discussion).
- **Client modules**: auth, conversations list, chat thread, media, contacts, notifications, settings; core: socket client, DB, sync engine, media cache.
- **Transport**: WebSocket for real-time (reconnect with backoff, heartbeats); REST for history/pagination & media upload (pre-signed URLs to object storage); FCM/APNs for background delivery/wake-up.
- **Local storage**: Drift tables `conversations`, `messages(id, clientId, conversationId, senderId, body, status, createdAt, serverSeq)`, `outbox`; UI observes DB streams; reverse-ordered ListView (`reverse: true`) with pagination by `serverSeq` cursor.
- **Sending flow**: create message with `clientId` (UUID) status `pending` → insert DB (instant UI) → outbox → socket send → server ack with server id/seq → update status `sent`; delivery/read receipts update status; retries on reconnect; idempotency via clientId to avoid duplicates.
- **Ordering**: server sequence numbers per conversation; client merges by seq; handle gaps by fetching missing range.
- **Media**: compress image (isolate), upload with progress (background_downloader), thumbnail placeholder/blurhash, cache with size limits.
- **Notifications**: data push → update DB / show local notification grouped per conversation; tap → deep link to chat; foreground suppression when chat open.
- **Typing indicators/presence**: ephemeral socket events, throttled.

- **E2E encryption**: Signal protocol (keys per device in secure storage), server sees ciphertext only — complexity trade-off.
- **Performance**: pagination, message bubble widget const-ness, avoid rebuilding whole list on status change (per-message selectors), image sizing.
- **Testing & observability**: sync engine unit tests, socket reconnect tests, metrics for send latency & failures.

Example 2: Design an e-commerce app 🛒

(see Chapter 21 §21.11 for a full template) — emphasise catalog caching (stale-while-revalidate), search (debounce/cancel), cart sync & optimistic updates, checkout reliability (idempotency, server verification, payment SDK integration), deep links for products/campaigns, analytics funnels, feature flags for promotions, A/B testing, accessibility & localization (currency formatting).

Example 3: Design a news feed (Instagram/Twitter-like) 📰

- Cursor-based infinite feed, prefetch next page at 70% scroll, image/video prefetch & caching with memory limits, video autoplay only for visible item (`visibility_detector`), pause offscreen, like with optimistic update & rollback, offline cached first page, pull-to-refresh with "new posts" banner instead of jumping, feed item view models precomputed, impression analytics batched, A/B ranking flags.

Example 4: Design a ride-hailing app (Uber-like) 🚗

- Map (google_maps_flutter) with driver markers updated via WebSocket (throttled, interpolated animations), location tracking (foreground service during trip), trip state machine (sealed classes: idle → searching → driverAssigned → arriving → inTrip → completed/cancelled), push for state changes, payment flow, ETA polling fallback, battery optimization (adaptive GPS accuracy), offline resilience (reconnect, state restoration after process death), safety features (share trip deep link).

Example 5: Design an offline-first field-data collection app 📋

- Forms defined by server schema (dynamic form rendering), local Drift DB, attachments stored as files, outbox sync with retries & conflict resolution, background sync via WorkManager, auth with long offline sessions (refresh tokens + device binding), encryption at rest, data export, multi-tenant configs, admin feature flags.

Example 6: Design a video streaming app 🎬

- Player (`video_player`/media_kit with HLS/DASH), adaptive bitrate, DRM (Widevine/FairPlay via native SDK/platform views or specialized plugins), download for offline with encryption & expiry, continue watching sync, Chromecast/AirPlay, PiP, background audio, analytics (QoE: startup time, buffering ratio), CDN.

System design trade-off phrases to use

- "For the **source of truth**, I'd make the local database authoritative for the UI and treat the network as a sync mechanism."
- "I'd prefer **cursor pagination** here because the feed changes frequently."
- "To keep the **UI thread free**, parsing and compression run in isolates."
- "We need **idempotency keys** so retries don't duplicate orders."
- "I'd gate this behind a **feature flag** and roll out gradually with monitoring."
- "Old app versions will exist for months, so the API must remain **backward compatible**."

H. Rapid-fire senior questions (answer in one or two sentences)

1. *What's* `TickerMode`*?* Disables tickers in a subtree (e.g., offscreen routes).
2. *What's* `Overlay`*?* A Stack of entries above routes used by Navigator, tooltips, dialogs, drag feedback.
3. *What's* `OverlayPortal`*?* Declarative way to render a child into the nearest Overlay while keeping its position in the widget tree for inherited data.
4. *What's* `CompositedTransformFollower`*?* Positions a widget (often in an overlay) relative to a `CompositedTransformTarget` using leader/follower layers.
5. *What is* `Restoration` *API?* Saves/restores UI state across process death via RestorationMixin/RestorableProperties and restorationScopeId.
6. *What does* `debugNeedsPaint`*/* `debugNeedsLayout` *tell you?* Whether a render object is marked dirty — useful in debugging custom render objects.
7. *Why does* `Opacity(0)` *still receive taps?* RenderOpacity doesn't block hit testing; use IgnorePointer/Visibility.
8. *What is* `ParentData`*?* Per-child data stored by parent render objects for layout (offsets, flex factors, positions).
9. *What's the difference between* `ScrollPosition` *and* `ScrollController`*?* Controller creates/holds positions for attached scrollables; position holds pixels/extents & drives physics.
10. *What is* `PrimaryScrollController`*?* Inherited default controller used by scroll views with `primary: true` (e.g., iOS status bar tap to top).
11. *What is a* `Viewport`*?* Render object showing a portion of slivers based on offset.
12. *What is* `SchedulerBinding.scheduleTask` *priority?* Runs tasks between frames by priority (animation, touch, idle).
13. *What's* `compute` *limitation on web?* Runs on the main thread (no isolates).
14. *What is* `dart:ui`*'s* `PlatformDispatcher`*?* Central interface to engine: views, locales, accessibility features, platform messages, frame callbacks, error handler.
15. *What is* `FlutterView`*?* A surface the engine renders into; apps can have multiple views (multi-window/multi-embedding).
16. *What is deferred loading?* Loading libraries lazily (`deferred as`) for web code splitting / Android deferred components.
17. *What is tree shaking of icons?* Removes unused glyphs from icon fonts in release; dynamic IconData needs `--no-tree-shake-icons` .
18. *What's AOT snapshot?* Precompiled machine code + data loaded at startup in release/profile.

19. *What is `kReleaseMode` used for?* Compile-time constant to exclude debug code from release.
20. *What is `@pragma('vm:entry-point')`?* Prevents tree shaking/obfuscation of functions/classes invoked from native or by name (background handlers, add-to-app entrypoints).

Chapter 32 — Coding Rounds: Dart Problems, Output Puzzles, Machine-Coding Apps & Behavioral Questions

Flutter interviews usually include: (1) Dart/DSA problems (easy–medium), (2) "predict the output" puzzles, (3) a **machine coding** round (build a small app in 60–120 min), and (4) behavioral questions. Every solution below is complete and idiomatic Dart 3.

Part A — Dart problem solving

A1. Reverse a string (Unicode-safe)

```dart
import 'package:characters/characters.dart';

String reverse(String s) => s.characters.toList().reversed.join();
// Simple ASCII version: s.split('').reversed.join();
void main() => print(reverse('Flutter 💙')); // 💙 rettulF
```

Why `characters`? `split('')` splits UTF-16 code units, breaking emoji/grapheme clusters.

A2. Palindrome (ignore case & non-alphanumerics)

```dart
bool isPalindrome(String s) {
  final clean = s.toLowerCase().replaceAll(RegExp(r'[^a-z0-9]'), '');
  var i = 0, j = clean.length - 1;
  while (i < j) {
    if (clean[i] != clean[j]) return false;
    i++; j--;
  }
  return true;
}
// isPalindrome('A man, a plan, a canal: Panama') == true
```

A3. FizzBuzz

```dart
void fizzBuzz(int n) {
  for (var i = 1; i <= n; i++) {
    print(switch ((i % 3, i % 5)) {
      (0, 0) => 'FizzBuzz',
      (0, _) => 'Fizz',
      (_, 0) => 'Buzz',
```

```
      _ => '$i',
    });
  }
}
```

A4. Character frequency & first non-repeating character

```dart
String? firstUnique(String s) {
  final freq = <String, int>{};
  for (final c in s.split('')) {
    freq[c] = (freq[c] ?? 0) + 1;
  }
  for (final c in s.split('')) {
    if (freq[c] == 1) return c;
  }
  return null;
}
// firstUnique('swiss') == 'w'
```

A5. Anagram check

```dart
bool isAnagram(String a, String b) {
  if (a.length != b.length) return false;
  final counts = <int, int>{};
  for (final c in a.codeUnits) counts[c] = (counts[c] ?? 0) + 1;
  for (final c in b.codeUnits) {
    final v = (counts[c] ?? 0) - 1;
    if (v < 0) return false;
    counts[c] = v;
  }
  return true;
}
```

A6. Two Sum (O(n))

```dart
(int, int)? twoSum(List<int> nums, int target) {
  final seen = <int, int>{}; // value -> index
  for (var i = 0; i < nums.length; i++) {
    final need = target - nums[i];
    final j = seen[need];
    if (j != null) return (j, i);
    seen[nums[i]] = i;
  }
  return null;
}
// twoSum([2, 7, 11, 15], 9) == (0, 1)
```

A7. Find duplicates

```
Set<T> duplicates<T>(Iterable<T> items) {
  final seen = <T>{};
  final dups = <T>{};
  for (final x in items) {
    if (!seen.add(x)) dups.add(x);   // Set.add returns false if already present
  }
  return dups;
}
```

A8. Remove duplicates keeping order

```
List<T> unique<T>(List<T> items) => items.toSet().toList(); // LinkedHashSet keeps insertion order
```

A9. Fibonacci (iterative + memoized recursive)

```
int fibIter(int n) {
  if (n < 2) return n;
  var a = 0, b = 1;
  for (var i = 2; i <= n; i++) {
    final t = a + b;
    a = b;
    b = t;
  }
  return b;
}

final _memo = <int, int>{};
int fibMemo(int n) => n < 2 ? n : _memo[n] ??= fibMemo(n - 1) + fibMemo(n - 2);
```

A10. Valid parentheses

```
bool isValid(String s) {
  const pairs = {')': '(', ']': '[', '}': '{'};
  final stack = <String>[];
  for (final c in s.split('')) {
    if (pairs.containsValue(c)) {
      stack.add(c);
    } else if (pairs.containsKey(c)) {
      if (stack.isEmpty || stack.removeLast() != pairs[c]) return false;
    }
  }
  return stack.isEmpty;
}
```

A11. Flatten nested list

```
List<Object?> flatten(List<Object?> input) => [
      for (final e in input)
        if (e is List<Object?>) ...flatten(e) else e,
    ];
// flatten([1, [2, [3, 4]], 5]) == [1, 2, 3, 4, 5]
```

A12. Group by

```
Map<K, List<T>> groupBy<T, K>(Iterable<T> items, K Function(T) key) {
  final map = <K, List<T>>{};
  for (final item in items) {
    map.putIfAbsent(key(item), () => []).add(item);
  }
  return map;
}
// groupBy(['apple', 'avocado', 'banana'], (s) => s[0]) => {a: [apple, avocado], b: [banana]}
```

A13. Binary search

```
int binarySearch(List<int> sorted, int target) {
  var lo = 0, hi = sorted.length - 1;
  while (lo <= hi) {
    final mid = lo + ((hi - lo) >> 1);
    if (sorted[mid] == target) return mid;
    if (sorted[mid] < target) {
      lo = mid + 1;
    } else {
      hi = mid - 1;
    }
  }
  return -1;
}
```

A14. Merge overlapping intervals

```
List<(int, int)> merge(List<(int, int)> intervals) {
  if (intervals.isEmpty) return [];
  final sorted = [...intervals]..sort((a, b) => a.$1.compareTo(b.$1));
  final result = <(int, int)>[sorted.first];
  for (final (start, end) in sorted.skip(1)) {
    final (lastStart, lastEnd) = result.last;
    if (start <= lastEnd) {
      result[result.length - 1] = (lastStart, end > lastEnd ? end : lastEnd);
    } else {
      result.add((start, end));
```

```
    }
  }
  return result;
}
// merge([(1,3),(2,6),(8,10),(15,18)]) == [(1,6),(8,10),(15,18)]
```

A15. LRU cache (very common!)

```
class LruCache<K, V> {
  LruCache(this.capacity) : assert(capacity > 0);
  final int capacity;
  final _map = <K, V>{};   // LinkedHashMap: iteration order = insertion order

  V? get(K key) {
    final value = _map.remove(key);
    if (value == null) return null;   // (assumes non-null values)
    _map[key] = value;                // move to most-recently-used end
    return value;
  }

  void put(K key, V value) {
    _map.remove(key);
    _map[key] = value;
    if (_map.length > capacity) {
      _map.remove(_map.keys.first);   // evict least recently used
    }
  }
}
```

A16. Debounce function (generic utility)

```
import 'dart:async';

class Debouncer {
  Debouncer(this.delay);
  final Duration delay;
  Timer? _timer;

  void run(void Function() action) {
    _timer?.cancel();
    _timer = Timer(delay, action);
  }

  void dispose() => _timer?.cancel();
}
```

A17. Throttle

```
class Throttler {
  Throttler(this.interval);
  final Duration interval;
  DateTime? _last;

  void run(void Function() action) {
    final now = DateTime.now();
    if (_last == null || now.difference(_last!) >= interval) {
      _last = now;
      action();
    }
  }
}
```

A18. Retry with exponential backoff

```
Future<T> retry<T>(
  Future<T> Function() task, {
  int maxAttempts = 3,
  Duration initialDelay = const Duration(milliseconds: 500),
  bool Function(Object error)? retryIf,
}) async {
  var attempt = 0;
  var delay = initialDelay;
  while (true) {
    attempt++;
    try {
      return await task();
    } catch (e) {
      final shouldRetry = attempt < maxAttempts && (retryIf?.call(e) ?? true);
      if (!shouldRetry) rethrow;
      await Future<void>.delayed(delay);
      delay *= 2;
    }
  }
}
```

A19. Run futures with limited concurrency

```
Future<List<R>> mapConcurrent<T, R>(List<T> items, int limit, Future<R> Function(T) f) async {
  final results = List<R?>.filled(items.length, null);
  var next = 0;
  Future<void> worker() async {
    while (true) {
      final i = next++;              // safe: single-threaded event loop
```

```
      if (i >= items.length) return;
      results[i] = await f(items[i]);
    }
  }
  await Future.wait(List.generate(limit, (_) => worker()));
  return results.cast<R>();
}
```

A20. Countdown stream

```
Stream<int> countdown(int from) async* {
  for (var i = from; i >= 0; i--) {
    yield i;
    if (i > 0) await Future<void>.delayed(const Duration(seconds: 1));
  }
}
```

A21. Implement `map` for a custom iterable using `sync*`

```
Iterable<R> myMap<T, R>(Iterable<T> source, R Function(T) f) sync* {
  for (final x in source) {
    yield f(x);
  }
}
```

A22. Deep equality of JSON-like structures

```
import 'package:collection/collection.dart';
const deepEq = DeepCollectionEquality();
deepEq.equals({'a': [1, 2]}, {'a': [1, 2]}); // true
```

A23. Count words, top-K frequent

```
List<MapEntry<String, int>> topK(String text, int k) {
  final counts = <String, int>{};
  for (final w in text.toLowerCase().split(RegExp(r'\W+')).where((w) => w.isNotEmpty)) {
    counts[w] = (counts[w] ?? 0) + 1;
  }
  final entries = counts.entries.toList()
    ..sort((a, b) => b.value != a.value ? b.value.compareTo(a.value) : a.key.compareTo(b.key));
  return entries.take(k).toList();
}
```

A24. Simple event bus with streams

```
class EventBus {
  final _controller = StreamController<Object>.broadcast();
  Stream<T> on<T>() => _controller.stream.where((e) => e is T).cast<T>();
  void fire(Object event) => _controller.add(event);
  Future<void> dispose() => _controller.close();
}
```

Part B — Predict the output (with explanations) 🧩

B1.

```
void main() async {
  print(1);
  Future.delayed(Duration.zero, () => print(2));
  scheduleMicrotask(() => print(3));
  await Future.value();
  print(4);
}
```

Output: `1 3 4 2`. Sync `1`; microtask 3 queued; `await Future.value()` schedules continuation as a microtask after 3 → `3`, `4`; the timer event `2` last.

B2.

```
void main() {
  const a = [1, 2];
  const b = [1, 2];
  final c = [1, 2];
  print(identical(a, b)); // ?
  print(a == c);          // ?
}
```

`true` (const canonicalized), `false` (List `==` is identity).

B3.

```
mixin A { String who() => 'A'; }
mixin B { String who() => 'B'; }
class C with A, B {}
void main() => print(C().who());
```

`B` — last mixin wins.

B4.

```
void main() {
  final fns = <Function>[];
  for (var i = 0; i < 3; i++) {
    fns.add(() => print(i));
  }
  for (final f in fns) f();
}
```

`0 1 2` — each loop iteration has a fresh `i`.

B5.

```
void main() {
  var list = [1, 2, 3];
  final mapped = list.map((e) { print('map $e'); return e * 2; });
  print('before');
  print(mapped.first);
}
```

`before`, `map 1`, `2` — lazy; `first` evaluates only the first element.

B6.

```
Future<void> f() async {
  print('f start');
  await Future.delayed(Duration.zero);
  print('f end');
}
void main() {
  f();
  print('main');
}
```

`f start`, `main`, `f end`.

B7.

```
void main() {
  String? name;
  print(name?.length ?? -1);
  name ??= 'Flutter';
  name ??= 'Dart';
```

```
  print(name);
}
```

`-1`, `Flutter`.

B8.

```
class P { P() { print('P'); } }
class Q extends P { Q() : super() { print('Q'); } }
void main() => Q();
```

`P` then `Q`.

B9.

```
void main() {
  try {
    Future.error('boom');
    print('after');
  } catch (e) {
    print('caught $e');
  }
}
```

Prints `after`; the error is **not** caught (not awaited) → reported as an unhandled async error.

B10.

```
void main() {
  final s = {1, 2, 3};
  final m = {};
  print(s is Set<int>);  // true — a set literal (implementation is a LinkedHashSet)
  print(m is Map);       // true — `{}` is a Map literal
}
```

B11. Flutter: what's printed when the button is tapped?

```
class Demo extends StatefulWidget { const Demo({super.key}); @override State<Demo> createState()
=> _DemoState(); }
class _DemoState extends State<Demo> {
  int n = 0;
  @override
  Widget build(BuildContext context) {
    print('build $n');
    return TextButton(onPressed: () { setState(() => n++); setState(() => n++); }, child:
Text('$n'));
```

```
  }
}
```

After a tap: `build 2` once — multiple setState calls in the same frame are batched into one rebuild.

B12. Why doesn't the colour swap?

```
Row(children: swapped ? [Tile(), Tile2()] : [Tile2(), Tile()])
```

If `Tile` / `Tile2` are *different types*, Flutter replaces elements (state lost but no "wrong state" issue). If both are the same StatefulWidget type without keys, state stays by position → colours don't swap. Fix with keys.

Part C — Machine coding rounds 🛠️

Typical prompt: *"Build an app that fetches users from an API, shows them in a list with loading/error states, supports pull-to-refresh and search, and a details screen. Use any state management. 90 minutes."*

Evaluation criteria: works end-to-end, clean architecture (separation of UI/state/data), error/loading/empty states, code readability, naming, null safety, no obvious bugs (dispose, mounted), small widgets, maybe tests, git commits.

C1. Users directory (Riverpod 3 + http) — complete solution

`pubspec.yaml` deps: `flutter_riverpod`, `http`.

```
// lib/main.dart
import 'dart:async';
import 'dart:convert';

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:http/http.dart' as http;

void main() => runApp(const ProviderScope(child: App()));

class App extends StatelessWidget {
  const App({super.key});
  @override
  Widget build(BuildContext context) => MaterialApp(
        title: 'Users',
        theme: ThemeData(colorSchemeSeed: Colors.indigo),
        home: const UsersPage(),
      );
}
```

```dart
// ---------- Model ----------
class User {
  const User({required this.id, required this.name, required this.email, required this.company});
  final int id;
  final String name;
  final String email;
  final String company;

  factory User.fromJson(Map<String, dynamic> json) => User(
        id: json['id'] as int,
        name: json['name'] as String,
        email: json['email'] as String,
        company: (json['company'] as Map<String, dynamic>)['name'] as String,
      );
}

// ---------- Data ----------
class UserRepository {
  UserRepository(this._client);
  final http.Client _client;

  Future<List<User>> fetchUsers() async {
    final res = await _client
        .get(Uri.parse('https://jsonplaceholder.typicode.com/users'))
        .timeout(const Duration(seconds: 15));
    if (res.statusCode != 200) {
      throw Exception('Server error (${res.statusCode})');
    }
    final list = jsonDecode(res.body) as List<dynamic>;
    return list.map((e) => User.fromJson(e as Map<String, dynamic>)).toList();
  }
}

final httpClientProvider = Provider<http.Client>((ref) {
  final client = http.Client();
  ref.onDispose(client.close);
  return client;
});

final userRepositoryProvider =
    Provider<UserRepository>((ref) => UserRepository(ref.watch(httpClientProvider)));

final usersProvider = FutureProvider<List<User>>(
  (ref) => ref.watch(userRepositoryProvider).fetchUsers(),
);

class SearchQuery extends Notifier<String> {
  @override
  String build() => '';
```

```
  void set(String value) => state = value;
}

final searchQueryProvider = NotifierProvider<SearchQuery, String>(SearchQuery.new);

final filteredUsersProvider = Provider<AsyncValue<List<User>>>((ref) {
  final query = ref.watch(searchQueryProvider).trim().toLowerCase();
  return ref.watch(usersProvider).whenData(
        (users) => query.isEmpty
            ? users
            : users
                .where((u) => u.name.toLowerCase().contains(query) ||
u.email.toLowerCase().contains(query))
                .toList(),
      );
});

// ---------- UI ----------
class UsersPage extends ConsumerWidget {
  const UsersPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final users = ref.watch(filteredUsersProvider);
    return Scaffold(
      appBar: AppBar(
        title: const Text('Users'),
        bottom: const PreferredSize(
          preferredSize: Size.fromHeight(64),
          child: Padding(padding: EdgeInsets.fromLTRB(16, 0, 16, 8), child: _SearchField()),
        ),
      ),
      body: RefreshIndicator(
        onRefresh: () => ref.refresh(usersProvider.future),
        child: switch (users) {
          AsyncData(:final value) when value.isEmpty => const _CenteredScrollable(child: Text('No
users found')),
          AsyncData(:final value) => ListView.separated(
              physics: const AlwaysScrollableScrollPhysics(),
              itemCount: value.length,
              separatorBuilder: (_, __) => const Divider(height: 1),
              itemBuilder: (context, i) => _UserTile(user: value[i]),
            ),
          AsyncError(:final error) => _CenteredScrollable(
              child: Column(mainAxisSize: MainAxisSize.min, children: [
                Text('Something went wrong\n$error', textAlign: TextAlign.center),
                const SizedBox(height: 12),
                FilledButton(onPressed: () => ref.invalidate(usersProvider), child: const
Text('Retry')),
```

```
              ]),
            ),
          _ => const Center(child: CircularProgressIndicator()),
        },
      ),
    );
  }
}

class _SearchField extends ConsumerStatefulWidget {
  const _SearchField();
  @override
  ConsumerState<_SearchField> createState() => _SearchFieldState();
}

class _SearchFieldState extends ConsumerState<_SearchField> {
  final _controller = TextEditingController();
  Timer? _debounce;

  @override
  void dispose() {
    _debounce?.cancel();
    _controller.dispose();
    super.dispose();
  }

  void _onChanged(String value) {
    _debounce?.cancel();
    _debounce = Timer(const Duration(milliseconds: 300), () {
      ref.read(searchQueryProvider.notifier).set(value);
    });
  }

  @override
  Widget build(BuildContext context) => TextField(
        controller: _controller,
        onChanged: _onChanged,
        textInputAction: TextInputAction.search,
        decoration: const InputDecoration(
          hintText: 'Search by name or email',
          prefixIcon: Icon(Icons.search),
          border: OutlineInputBorder(),
          isDense: true,
        ),
      );
}

class _UserTile extends StatelessWidget {
  const _UserTile({required this.user});
```

```
  final User user;

  @override
  Widget build(BuildContext context) => ListTile(
        leading: CircleAvatar(child: Text(user.name[0])),
        title: Text(user.name),
        subtitle: Text(user.email),
        trailing: const Icon(Icons.chevron_right),
        onTap: () => Navigator.of(context).push(
          MaterialPageRoute<void>(builder: (_) => UserDetailsPage(user: user)),
        ),
      );
}

class UserDetailsPage extends StatelessWidget {
  const UserDetailsPage({super.key, required this.user});
  final User user;

  @override
  Widget build(BuildContext context) {
    final t = Theme.of(context).textTheme;
    return Scaffold(
      appBar: AppBar(title: Text(user.name)),
      body: ListView(padding: const EdgeInsets.all(16), children: [
        Text(user.name, style: t.headlineSmall),
        const SizedBox(height: 8),
        Text(user.email, style: t.bodyLarge),
        const SizedBox(height: 8),
        Text('Works at ${user.company}', style: t.bodyMedium),
      ]),
    );
  }
}

class _CenteredScrollable extends StatelessWidget {
  const _CenteredScrollable({required this.child});
  final Widget child;
  @override
  Widget build(BuildContext context) => LayoutBuilder(
        builder: (context, c) => SingleChildScrollView(
          physics: const AlwaysScrollableScrollPhysics(),   // keeps pull-to-refresh working
          child: SizedBox(height: c.maxHeight, child: Center(child: child)),
        ),
      );
}
```

Talking points: repository abstraction (swap for fakes in tests), derived filtered provider, debounce, refresh via `ref.refresh(provider.future)`, error retry with `invalidate`, disposal of timers/controllers, AlwaysScrollableScrollPhysics for refresh on empty/error.

Unit test for the repository

```
import 'package:http/testing.dart';
test('fetchUsers parses users', () async {
  final client = MockClient((_) async => http.Response(
      '[{"id":1,"name":"Leanne","email":"l@x.com","company":{"name":"Acme"}}]', 200));
  final users = await UserRepository(client).fetchUsers();
  expect(users.single.company, 'Acme');
});
```

C2. Todo app with Cubit + persistence (shared_preferences)

```
// todo.dart
import 'dart:convert';
import 'package:equatable/equatable.dart';

class Todo extends Equatable {
  const Todo({required this.id, required this.title, this.done = false});
  final String id;
  final String title;
  final bool done;

  Todo copyWith({String? title, bool? done}) => Todo(id: id, title: title ?? this.title, done:
done ?? this.done);
  Map<String, dynamic> toJson() => {'id': id, 'title': title, 'done': done};
  factory Todo.fromJson(Map<String, dynamic> j) =>
      Todo(id: j['id'] as String, title: j['title'] as String, done: j['done'] as bool? ?? false);

  @override
  List<Object?> get props => [id, title, done];
}

// todo_repository.dart
import 'package:shared_preferences/shared_preferences.dart';

class TodoRepository {
  TodoRepository(this._prefs);
  final SharedPreferences _prefs;
  static const _key = 'todos_v1';

  List<Todo> load() {
    final raw = _prefs.getString(_key);
    if (raw == null) return const [];
    return (jsonDecode(raw) as List<dynamic>)
        .map((e) => Todo.fromJson(e as Map<String, dynamic>))
        .toList();
  }
```

```
  Future<void> save(List<Todo> todos) =>
      _prefs.setString(_key, jsonEncode(todos.map((t) => t.toJson()).toList()));
}

// todo_cubit.dart
import 'package:flutter_bloc/flutter_bloc.dart';

enum TodoFilter { all, active, done }

class TodoState extends Equatable {
  const TodoState({this.todos = const [], this.filter = TodoFilter.all});
  final List<Todo> todos;
  final TodoFilter filter;

  List<Todo> get visible => switch (filter) {
        TodoFilter.all => todos,
        TodoFilter.active => todos.where((t) => !t.done).toList(),
        TodoFilter.done => todos.where((t) => t.done).toList(),
      };
  int get remaining => todos.where((t) => !t.done).length;

  TodoState copyWith({List<Todo>? todos, TodoFilter? filter}) =>
      TodoState(todos: todos ?? this.todos, filter: filter ?? this.filter);

  @override
  List<Object?> get props => [todos, filter];
}

class TodoCubit extends Cubit<TodoState> {
  TodoCubit(this._repo) : super(TodoState(todos: _repo.load()));
  final TodoRepository _repo;

  void add(String title) {
    final trimmed = title.trim();
    if (trimmed.isEmpty) return;
    final todo = Todo(id: DateTime.now().microsecondsSinceEpoch.toString(), title: trimmed);
    _update([...state.todos, todo]);                    // new list → new state
  }

  void toggle(String id) => _update([
        for (final t in state.todos) t.id == id ? t.copyWith(done: !t.done) : t,
      ]);

  void remove(String id) => _update(state.todos.where((t) => t.id != id).toList());

  void setFilter(TodoFilter f) => emit(state.copyWith(filter: f));

  void _update(List<Todo> todos) {
    emit(state.copyWith(todos: todos));
```

```
    _repo.save(todos);     // fire-and-forget persistence (could await & handle errors)
  }
}

// main.dart
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final prefs = await SharedPreferences.getInstance();
  runApp(BlocProvider(create: (_) => TodoCubit(TodoRepository(prefs)), child: const
MaterialApp(home: TodoPage())));
}

class TodoPage extends StatefulWidget {
  const TodoPage({super.key});
  @override
  State<TodoPage> createState() => _TodoPageState();
}

class _TodoPageState extends State<TodoPage> {
  final _controller = TextEditingController();

  @override
  void dispose() { _controller.dispose(); super.dispose(); }

  void _submit() {
    context.read<TodoCubit>().add(_controller.text);
    _controller.clear();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: BlocSelector<TodoCubit, TodoState, int>(
          selector: (s) => s.remaining,
          builder: (_, remaining) => Text('Todos ($remaining left)'),
        ),
        actions: [
          PopupMenuButton<TodoFilter>(
            onSelected: context.read<TodoCubit>().setFilter,
            itemBuilder: (_) => [for (final f in TodoFilter.values) PopupMenuItem(value: f, child:
Text(f.name))],
          ),
        ],
      ),
      body: Column(children: [
        Padding(
          padding: const EdgeInsets.all(12),
          child: Row(children: [
```

```
            Expanded(
              child: TextField(
                controller: _controller,
                onSubmitted: (_) => _submit(),
                decoration: const InputDecoration(hintText: 'What needs to be done?'),
              ),
            ),
            IconButton(onPressed: _submit, icon: const Icon(Icons.add)),
          ]),
        ),
        Expanded(
          child: BlocBuilder<TodoCubit, TodoState>(
            builder: (context, state) {
              final items = state.visible;
              if (items.isEmpty) return const Center(child: Text('Nothing here 🎉'));
              return ListView.builder(
                itemCount: items.length,
                itemBuilder: (context, i) {
                  final todo = items[i];
                  return Dismissible(
                    key: ValueKey(todo.id),
                    background: Container(color: Colors.red),
                    onDismissed: (_) => context.read<TodoCubit>().remove(todo.id),
                    child: CheckboxListTile(
                      value: todo.done,
                      onChanged: (_) => context.read<TodoCubit>().toggle(todo.id),
                      title: Text(
                        todo.title,
                        style: todo.done ? const TextStyle(decoration: TextDecoration.lineThrough)
: null,
                      ),
                    ),
                  );
                },
              );
            },
          ),
        ),
      ]),
    );
  }
}
```

(Imports for Flutter/Equatable/Bloc/SharedPreferences omitted between "files" for brevity — in a real project each file has its own imports.)

bloc_test for TodoCubit

```dart
class MockRepo extends Mock implements TodoRepository {}
void main() {
  late MockRepo repo;
  setUp(() {
    repo = MockRepo();
    when(() => repo.load()).thenReturn(const []);
    when(() => repo.save(any())).thenAnswer((_) async {});
  });
  blocTest<TodoCubit, TodoState>(
    'add creates a todo',
    build: () => TodoCubit(repo),
    act: (c) => c.add('Milk'),
    verify: (c) {
      expect(c.state.todos.single.title, 'Milk');
      verify(() => repo.save(any())).called(1);
    },
  );
}
```

(`registerFallbackValue(<Todo>[])` may be needed for `any()` with typed lists in mocktail.)

C3. Stopwatch (tests lifecycle & timers)

```dart
class StopwatchPage extends StatefulWidget {
  const StopwatchPage({super.key});
  @override
  State<StopwatchPage> createState() => _StopwatchPageState();
}

class _StopwatchPageState extends State<StopwatchPage> with SingleTickerProviderStateMixin {
  final _stopwatch = Stopwatch();
  late final Ticker _ticker = createTicker((_) => setState(() {}));   // rebuild each frame while
running

  @override
  void dispose() {
    _ticker.dispose();
    super.dispose();
  }

  void _toggle() {
    setState(() {
      if (_stopwatch.isRunning) {
        _stopwatch.stop();
        _ticker.stop();
      } else {
        _stopwatch.start();
        _ticker.start();
```

```
      }
    });
  }

  void _reset() => setState(() { _stopwatch.reset(); });

  String _format(Duration d) {
    String two(int n) => n.toString().padLeft(2, '0');
    return
'${two(d.inMinutes.remainder(60))}:${two(d.inSeconds.remainder(60))}.${two(d.inMilliseconds.remain
der(1000) ~/ 10)}';
  }

  @override
  Widget build(BuildContext context) => Scaffold(
        body: Center(
          child: Column(mainAxisSize: MainAxisSize.min, children: [
            Text(_format(_stopwatch.elapsed),
                style: Theme.of(context).textTheme.displayMedium?.copyWith(fontFeatures: const
[FontFeature.tabularFigures()])),
            const SizedBox(height: 24),
            Row(mainAxisSize: MainAxisSize.min, children: [
              FilledButton(onPressed: _toggle, child: Text(_stopwatch.isRunning ? 'Stop' :
'Start')),
              const SizedBox(width: 12),
              OutlinedButton(onPressed: _stopwatch.isRunning ? null : _reset, child: const
Text('Reset')),
            ]),
          ]),
        ),
      );
}
```

Talking points: Ticker vs Timer.periodic (frame-synced, paused offscreen), tabular figures to avoid jitter, dispose, only the text needs rebuilding (could extract to its own widget for efficiency). (Requires `import 'package:flutter/scheduler.dart';` for `Ticker` and `dart:ui` for `FontFeature`.)

C4. Custom reusable widget: star rating

```
class StarRating extends StatelessWidget {
  const StarRating({super.key, required this.value, this.max = 5, this.onChanged, this.size =
28});
  final int value;
  final int max;
  final ValueChanged<int>? onChanged;
  final double size;

  @override
```

```
  Widget build(BuildContext context) {
    final color = Theme.of(context).colorScheme.primary;
    return Semantics(
      label: 'Rating $value of $max',
      child: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          for (var i = 1; i <= max; i++)
            IconButton(
              visualDensity: VisualDensity.compact,
              iconSize: size,
              tooltip: '$i star${i > 1 ? 's' : ''}',
              onPressed: onChanged == null ? null : () => onChanged!(i),
              icon: Icon(i <= value ? Icons.star : Icons.star_border, color: color),
            ),
        ],
      ),
    );
  }
}
```

Talking points: stateless & controlled (parent owns value), accessibility, theming, null callback = read-only.

C5. Other common machine-coding prompts (practice list)

1. Weather app (API, search city, current + forecast, error handling, units toggle, caching last result).
2. Product catalog with cart (Riverpod/Bloc), badge count, quantity changes, totals, checkout summary.
3. Infinite scroll list with pagination & pull-to-refresh (Chapter 9 pattern).
4. Login flow with validation, loading state, mocked API, token storage, auto-login & logout.
5. Image gallery grid with Hero detail view and pinch-to-zoom (InteractiveViewer).
6. Chat UI (reverse list, input bar, message bubbles, timestamps grouping).
7. Expense tracker with charts (fl_chart) and local DB (Drift/sqflite).
8. Quiz app with timer per question and results screen.
9. Kanban board with drag & drop between columns.
10. Countdown/OTP input with auto-focus between fields & resend timer.
11. Movie search with debounce, cancellation, and details (OMDb/TMDb).
12. Settings screen with dark mode & locale switching persisted.

Machine coding strategy (90 minutes):

- 5 min: clarify requirements, list screens/states.
- 10 min: skeleton — models, repository interface, state, routing.
- 45 min: core flows end-to-end (happy path first).
- 15 min: loading/error/empty states, edge cases, polish.

- 10 min: one or two tests, README with decisions & next steps.
- 5 min: walkthrough prep: explain architecture & trade-offs.
- Commit in small steps; keep it running at all times.

Part D — Behavioral & HR questions (with STAR answers) 🗣️

Use **STAR**: **S**ituation, **T**ask, **A**ction, **R**esult (+ what you learned).

D1. Tell me about yourself. 60–90s: current role & domain, Flutter experience (apps shipped, scale), strengths (architecture, performance, mentoring), a highlight achievement with numbers, why this role.

D2. Describe a challenging bug you solved. *Example:* "Users reported random logouts (S). I owned the investigation (T). I added breadcrumbs around token refresh, discovered parallel 401s triggered multiple refresh calls invalidating each other; I replaced the interceptor with a queued single-flight refresh and added tests simulating concurrent requests (A). Logout complaints dropped to zero, crash-free sessions improved, and we documented the pattern (R)."

D3. A time you improved performance. Measure → fix → numbers: "Reduced feed jank: 30% janky frames → 3% by moving JSON parsing to an isolate, adding image cacheWidth, and splitting rebuild scopes."

D4. A disagreement with a teammate/lead. Show respect, data-driven discussion, prototype comparison, compromise/ADR, outcome.

D5. A failure or mistake. Own it, explain fix and prevention (e.g., shipped crash due to missing null handling → added tests, lint rules, staged rollout).

D6. How do you handle tight deadlines? Prioritize must-haves, communicate early, feature flags, cut scope not quality on critical paths.

D7. How do you keep up with Flutter changes? Release notes & breaking changes pages, Flutter/Dart blogs, GitHub issues, Flutter YouTube (Widget of the Week, Observable Flutter), community newsletters, conferences, experimenting with beta in side projects.

D8. Why Flutter (over native/React Native)? Productivity, consistent UI, performance, multi-platform reach, team's experience; acknowledge cases where native is better.

D9. How do you mentor juniors? Pairing, constructive reviews, documentation, gradually increasing ownership.

D10. Where do you see yourself in 3–5 years? Growth aligned with company: technical leadership, architecture, mentoring, cross-platform strategy.

D11. Questions to ask the interviewer: Team size & structure, state management/architecture choices, release cadence & CI, testing culture, how tech debt is handled, on-call/incident process, design system maturity, biggest technical challenge in the next 6 months.

Chapter 33 — Cheat Sheets, Deprecated-API Migration Table, Glossary & Learning Roadmap

33.1 One-page Flutter cheat sheet 📄

Widget basics

```
class X extends StatelessWidget { const X({super.key}); @override Widget build(BuildContext c) =>
...; }
class Y extends StatefulWidget { const Y({super.key}); @override State<Y> createState() =>
_YState(); }
class _YState extends State<Y> { @override void initState(){super.initState();} @override void
dispose(){super.dispose();} @override Widget build(BuildContext c) => ...; }
```

Layout quick picks

Need	Use
Horizontal / vertical	Row / Column (+ Expanded, Flexible, Spacer, `spacing:`)
Overlap	Stack + Positioned
Gap	SizedBox(height/width)
Padding	Padding(EdgeInsets…)
Center/align	Center / Align
Wrap lines	Wrap
Scroll a page	SingleChildScrollView / ListView
Long list	ListView.builder / SliverList
Grid	GridView.builder
Mixed scrolling	CustomScrollView + slivers
Responsive	LayoutBuilder / MediaQuery.sizeOf
Ratio	AspectRatio
Scale to fit	FittedBox
Safe area	SafeArea

State quick picks

Scope	Tool
One widget	setState / ValueNotifier
Small app	ChangeNotifier + Provider / ListenableBuilder
Medium–large	Riverpod / Bloc

Navigation

```
Navigator.push(context, MaterialPageRoute(builder: (_) => Page()));
final r = await Navigator.push<T>(...); Navigator.pop(context, value);
context.go('/path'); context.push('/path'); context.pop();
PopScope(canPop: false, onPopInvokedWithResult: (didPop, result) {...}, child: ...)
```

Async

```
final (a, b) = await (fa(), fb()).wait;
await Future.wait([f1, f2]);
Isolate.run(() => heavy());
if (!mounted) return;  if (!context.mounted) return;
```

Must-dispose list

AnimationController · TextEditingController · ScrollController · PageController · TabController · FocusNode · StreamSubscription (cancel) · Timer (cancel) · StreamController (close) · ChangeNotifier you own · VideoPlayerController · CameraController · WebSocket channels · Tickers · Observers (removeObserver) · GestureRecognizers you create (e.g., TapGestureRecognizer in TextSpan)

CLI

```
flutter create --org com.x app · flutter run [-d id] [--release|--profile] [--flavor f -t
lib/main_f.dart]
flutter pub get|add|upgrade|outdated · dart run build_runner build -d · flutter gen-l10n
flutter analyze · dart format . · dart fix --apply · flutter test [--coverage] [--update-goldens]
flutter build apk|appbundle|ipa|web [--wasm]|windows|macos|linux [--obfuscate --split-debug-
info=dir]
flutter clean · flutter doctor -v · flutter upgrade · flutter symbolize
```

33.2 Deprecated / removed APIs → modern replacements (interview gold) 🔄

Old	New	Since (approx.)
`RaisedButton`, `FlatButton`, `OutlineButton`	`ElevatedButton`, `TextButton`, `OutlinedButton` (+ `FilledButton`)	Deprecated 2.0, removed 3.x
`Scaffold.of(context).showSnackBar`	`ScaffoldMessenger.of(context).showSnackBar`	2.0
`WillPopScope`	`PopScope` (`canPop`, `onPopInvokedWithResult`)	3.12 (→ result variant 3.24)
`MaterialStateProperty`, `MaterialState`	`WidgetStateProperty`, `WidgetState`	3.19
`textScaleFactor`	`TextScaler` / `textScaler`	3.16

Old	New	Since (approx.)
`Color.withOpacity()`, `.value`, `.red/.green/.blue/.opacity`	`withValues(alpha:)`, `toARGB32()`, `.r/.g/.b/.a`	3.27
`ColorScheme.background/onBackground/surfaceVariant`	`surface/onSurface/surfaceContainerHighest`	3.22
`useMaterial3: false` default	Material 3 default	3.16
`headline1`, `bodyText1`, …	`displayLarge`, `bodyLarge`, …	3.x
`RawKeyboardListener`, `RawKeyEvent`	`KeyboardListener`, `Focus.onKeyEvent`, `KeyEvent`	3.18
`Navigator.onPopPage`	`Navigator.onDidRemovePage`	3.24
`ThemeData.cardTheme: CardTheme` etc.	`CardThemeData`, `DialogThemeData`, `TabBarThemeData`	3.27+
`dart:html`, `dart:js`, `package:js`	`dart:js_interop`, `package:web`	Dart 3.3+ (required for Wasm)
HTML web renderer (`--web-renderer html`)	CanvasKit / skwasm	removed 2025
Android v1 embedding (`Registrar`, `io.flutter.app.FlutterActivity`)	v2 embedding (`FlutterPlugin`, `io.flutter.embedding.android.FlutterActivity`)	removed 3.x
Gradle imperative `apply plugin` / `apply from: flutter.gradle`	Declarative `plugins { id("dev.flutter.flutter-gradle-plugin") }`	3.16+
`flutter_driver`	`integration_test` (+ Patrol)	2.x
`mapEventToState` (Bloc ≤7)	`on<Event>` handlers	Bloc 8
Riverpod `StateNotifierProvider`, `StateProvider`, `ChangeNotifierProvider`	`NotifierProvider`, `AsyncNotifierProvider` (legacy imports)	Riverpod 2 → 3
`AsyncValue.valueOrNull`	`.value`	Riverpod 3
`package:flutter_gen/gen_l10n/...` synthetic package	generated files in source tree (`output-dir`)	~3.29+
Firebase Dynamic Links	App Links / Universal Links / third-party (Branch etc.)	shut down Aug 2025
`google_generative_ai` Dart SDK	Firebase AI Logic (`firebase_ai`)	2025
`MediaQuery.of(context).size` (not deprecated, but)	`MediaQuery.sizeOf(context)` (fewer rebuilds)	3.10
`import 'package:flutter/material.dart'` (framework)	`material_ui` / `cupertino_ui` packages (opt-in; framework copies to be deprecated)	3.47
Dart macros (`@JsonCodable`) — experimental	cancelled → use build_runner codegen	Jan 2025

33.3 Dart & Flutter version timeline (high-level) 🗓️

Version	Highlights
Flutter 1.0 (Dec 2018)	First stable
Flutter 2.0 (Mar 2021) / Dart 2.12	Web stable, sound null safety
Flutter 2.10 (Feb 2022)	Windows stable
Flutter 3.0 (May 2022) / Dart 2.17	macOS & Linux stable, enhanced enums, super params
Flutter 3.7 (Jan 2023)	Background isolate channels, `BuildContext.mounted`
Flutter 3.10 (May 2023) / Dart 3.0	Records, patterns, sealed & class modifiers, 100% null safety, Impeller default on iOS
Flutter 3.13 (Aug 2023)	AppLifecycleListener, new slivers groups
Flutter 3.16 (Nov 2023) / Dart 3.2	Material 3 default, private field promotion
Flutter 3.19 / 3.22 (2024) / Dart 3.3–3.4	Extension types; Wasm web stable (3.22)
Flutter 3.24 (Aug 2024) / Dart 3.5	Sliver headers (Pinned/Floating), CarouselView, SwiftPM early support
Flutter 3.27 (Dec 2024) / Dart 3.6	Impeller default Android API 29+, Row/Column spacing, Color wide-gamut APIs, pub workspaces
Flutter 3.29 (Feb 2025) / Dart 3.7	Merged UI/platform threads (mobile), wildcard variables, new formatter style, Skia removed on iOS
Flutter 3.32 (May 2025) / Dart 3.8	Web hot reload (experimental), null-aware collection elements
Flutter 3.35 (Aug 2025) / Dart 3.9	Widget previews (experimental), web hot reload on by default
Flutter 3.38 (Nov 2025) / Dart 3.10	Dot shorthands, widget previews in IDEs, build hooks stable, web dev config
Flutter 3.41 (Feb 2026) / Dart 3.11	Stability/performance release; Dart 3.11 has no new language features
Flutter 3.44 (May 2026) / Dart 3.12	SwiftPM default (iOS/macOS), Android Hybrid Composition++, Material/Cupertino frozen in framework, multi-window preview
Flutter 3.47 (Aug 2026) / Dart 3.13	Standalone `material_ui` / `cupertino_ui` packages, Impeller desktop default, primary constructors

> Treat minor details as approximate — always confirm with the official release notes for the exact version your project uses.

33.4 Glossary (A–Z) 📚

- **AAB** — Android App Bundle; publishing format for Play.
- **ADR** — Architecture Decision Record.
- **AOT** — Ahead-of-time compilation to machine code (release/profile).
- **Arena (gesture)** — Mechanism resolving competing gesture recognizers.

- **AsyncValue** — Riverpod's loading/data/error union.
- **Binding** — Singleton glue between framework and engine (WidgetsFlutterBinding).
- **Bloc/Cubit** — Stream-based state management components.
- **BoxConstraints** — min/max width/height passed from parent to child.
- **BuildContext** — Handle to an element's location in the tree.
- **CanvasKit** — Skia compiled to WebAssembly for Flutter web.
- **Codec (channel)** — Serializer for platform channel messages.
- **Composition (widgets)** — Building UIs from small widgets rather than inheritance.
- **Cubit** — Simplified Bloc without events.
- **Dart VM** — Runtime executing Dart (JIT/AOT).
- **Deferred loading** — Lazy loading of libraries.
- **DevTools** — Flutter's debugging/profiling suite.
- **DTO** — Data Transfer Object, mirrors API data.
- **Element** — Instantiation of a widget at a location in the tree.
- **Embedder** — Platform host for the engine.
- **Engine** — C++ core: rendering, Dart runtime, text, platform messaging.
- **Ephemeral state** — Local UI state.
- **Extension type** — Zero-cost static wrapper type (Dart 3.3).
- **FFI** — Foreign Function Interface to C.
- **Flavor** — Build variant (dev/staging/prod).
- **Frame budget** — Time per frame (16.67ms @60Hz).
- **Golden test** — Pixel snapshot test.
- **GlobalKey** — App-unique key for state access/reparenting.
- **Hot reload / restart** — Inject code keeping state / restart Dart state.
- **Hit test** — Finding which render objects are under a pointer.
- **Impeller** — Flutter's modern renderer with precompiled shaders.
- **InheritedWidget** — Efficient data propagation down the tree.
- **Isolate** — Independent Dart worker with own memory/event loop.
- **JIT** — Just-in-time compilation (debug).
- **Jank** — Visible stutter from missed frames.
- **Key** — Identity for widgets across rebuilds.
- **Layer tree** — Composited output of paint phase.
- **Microtask** — High-priority async task run before next event.
- **Mixin** — Reusable class members applied with `with`.
- **Null safety** — Non-nullable by default type system.
- **Pigeon** — Type-safe platform channel code generator.
- **Platform channel** — Async message bridge Dart ↔ native.
- **Platform view** — Native view embedded in Flutter.
- **Provider** — InheritedWidget-based DI/state package.
- **Raster thread** — Thread that converts layer tree to GPU commands.
- **Record** — Anonymous immutable tuple type.
- **Relayout/Repaint boundary** — Points stopping layout/paint invalidation propagation.
- **RenderObject** — Object performing layout/paint/hit testing.
- **Repository** — Data abstraction layer.

- **Riverpod** — Compile-safe reactive state/DI framework.
- **Sealed class** — Closed type hierarchy enabling exhaustive switches.
- **Semantics** — Accessibility information tree.
- **Skia** — 2D graphics library previously default renderer.
- **skwasm** — Wasm-based renderer for Flutter web Wasm builds.
- **Sliver** — Scrollable layout piece using sliver protocol.
- **Stream** — Async sequence of events.
- **Ticker** — Per-frame callback source for animations.
- **Tree shaking** — Removing unused code in compilation.
- **Tween** — Mapping 0..1 to a value range.
- **Viewport** — Visible window of slivers.
- **Vsync** — Display refresh signal driving frames.
- **Wasm (WasmGC)** — WebAssembly with garbage collection used by dart2wasm.
- **Widget** — Immutable UI configuration.
- **Zone** — Async execution context for error handling/locals.

33.5 Learning roadmap (beginner → expert) 🧭

Stage 1 — Foundations (2–3 weeks)

Chapters 1–5. Build: counter, BMI calculator, unit converter, quiz app. Practice Dart problems (Chapter 32 Part A).

Stage 2 — UI mastery (3–4 weeks)

Chapters 6–13. Build: profile screen clone (responsive), e-commerce product page, animated onboarding, custom chart with CustomPainter, form-heavy signup with validation.

Stage 3 — App development (4–6 weeks)

Chapters 14–19. Build: users directory (API + Riverpod), todo with Bloc + Drift, chat UI with Firebase, notes app with offline sync, a small plugin with Pigeon.

Stage 4 — Professional engineering (4–6 weeks)

Chapters 20–28. Refactor one project into clean feature-first architecture with DI, add unit/widget/golden/integration tests, set up CI (GitHub Actions), flavors, crash reporting, analytics, performance profiling, release to internal testing track/TestFlight.

Stage 5 — Interview sprint (2 weeks)

Chapters 29–32: answer every question aloud; do 3 timed machine-coding rounds; prepare 6 STAR stories (bug, performance win, conflict, failure, leadership, learning); mock system design (chat, e-commerce, feed).

Daily habit

- 1 Dart puzzle, 1 widget from the Widget of the Week series, read 1 Flutter source file (e.g., `framework.dart` element lifecycle), review 1 open-source Flutter app's architecture.

Projects that impress interviewers

1. A production-like app with: feature-first architecture, Riverpod/Bloc, go_router + deep links, offline cache, l10n (incl. RTL), dark mode, accessibility, tests (≥60% on logic), CI badge, screenshots, README with architecture diagram.
2. A published package/plugin on pub.dev (even small) with tests & docs.
3. Contributions/issues on Flutter or popular packages.

33.6 Final interview-day checklist ✅

- ☐ Can explain widget/element/render trees and setState flow
- ☐ Can explain constraints & fix overflow/unbounded errors live
- ☐ Can explain keys with an example
- ☐ Can compare Provider/Riverpod/Bloc with trade-offs
- ☐ Can explain event loop, Future vs Stream, isolates
- ☐ Can design app architecture (layers, DI, repository, error handling)
- ☐ Can discuss performance investigation with DevTools
- ☐ Can describe testing strategy (unit/widget/golden/integration) and pump vs pumpAndSettle
- ☐ Knows platform channels/Pigeon/FFI basics
- ☐ Knows release process: flavors, signing, CI/CD, staged rollout
- ☐ Knows security basics: secure storage, no secrets, pinning trade-offs
- ☐ Knows recent changes (Impeller, Dart 3 features, Material 3, PopScope, dot shorthands, material_ui decoupling, Riverpod 3)
- ☐ Has STAR stories ready and questions for the interviewer

PART VII

Deep Dives with Full Code

Chapter 34 — Camera, Image Picking, Video & Audio (Full Code)

Goal: Build media features properly: pick and compress photos, show a live camera preview, record and play video, play audio (including in the background), and record your voice. Each section covers setup, complete code, lifecycle rules and the bugs companies hit most.

Package APIs change between major versions. The code below follows each package's current major version as of 2026. Check the package's README and CHANGELOG when you upgrade.

34.1 5-year-old picture 📸

Your phone has **eyes** (camera), a **photo album** (gallery), a **TV** (video player), a **radio** (audio player) and **ears** (microphone). Flutter can't use them by itself. It asks the phone through **plugins**, and the phone only says yes if you asked **politely** (permissions with a reason).

34.2 Picking photos & videos: `image_picker`

Use this when the user only needs to **choose or take one photo/video**. It opens the system UI, so you don't build a camera screen yourself.

Setup

- iOS `Info.plist`:
  - `NSPhotoLibraryUsageDescription`
  - `NSCameraUsageDescription`
  - `NSMicrophoneUsageDescription` (only if you record video with sound)
- **Android:** no permission is needed for the gallery on Android 13+, because the plugin uses the **Photo Picker** (Android 11–12 also get a backported picker through Google Play services). Taking a photo with the camera uses the system camera app.

Code

```
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';

class AvatarPicker extends StatefulWidget {
  const AvatarPicker({super.key, required this.onPicked});
  final ValueChanged<File> onPicked;
```

```
  @override
  State<AvatarPicker> createState() => _AvatarPickerState();
}

class _AvatarPickerState extends State<AvatarPicker> {
  final _picker = ImagePicker();
  File? _file;
  bool _busy = false;                    // prevents "already_active" errors from double taps

  Future<void> _pick(ImageSource source) async {
    if (_busy) return;
    setState(() => _busy = true);
    try {
      final XFile? picked = await _picker.pickImage(
        source: source,
        maxWidth: 1080,                  // ⭐ resize on the device → smaller uploads & less
memory
        maxHeight: 1080,
        imageQuality: 85,                // JPEG quality 0-100
        preferredCameraDevice: CameraDevice.front,
      );
      if (picked == null || !mounted) return;   // user cancelled
      final file = File(picked.path);
      setState(() => _file = file);
      widget.onPicked(file);
    } on Exception catch (e) {
      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text('Could not pick image:
$e')));
    } finally {
      if (mounted) setState(() => _busy = false);
    }
  }

  void _showSourceSheet() {
    showModalBottomSheet<void>(
      context: context,
      showDragHandle: true,
      builder: (ctx) => SafeArea(
        child: Wrap(children: [
          ListTile(
            leading: const Icon(Icons.photo_library),
            title: const Text('Choose from gallery'),
            onTap: () { Navigator.pop(ctx); _pick(ImageSource.gallery); },
          ),
          ListTile(
            leading: const Icon(Icons.photo_camera),
            title: const Text('Take a photo'),
            onTap: () { Navigator.pop(ctx); _pick(ImageSource.camera); },
```

```
          ),
        ]),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: _showSourceSheet,
      child: CircleAvatar(
        radius: 48,
        backgroundImage: _file != null ? FileImage(_file!) : null,
        child: _file == null ? const Icon(Icons.add_a_photo, size: 32) : null,
      ),
    );
  }
}
```

Other APIs:

```
final List<XFile> many = await _picker.pickMultiImage(limit: 5, imageQuality: 80);
final XFile? video = await _picker.pickVideo(source: ImageSource.gallery, maxDuration: const
Duration(minutes: 1));
final List<XFile> media = await _picker.pickMultipleMedia();   // photos + videos together
```

⚠️ Android "lost data" (activity killed while the camera is open)

On low-memory Android devices, the OS may **kill your app's activity** while the camera app is open. When the user comes back, `pickImage` never returns a value. You can recover the result at startup:

```
Future<void> retrieveLostData() async {
  final LostDataResponse response = await _picker.retrieveLostData();
  if (response.isEmpty) return;
  final files = response.files;
  if (files != null) { /* handle recovered images */ } else { /* response.exception */ }
}
```

You can reproduce this with **Developer options → Don't keep activities.**

Web

`XFile.path` is a **blob URL** on the web, and `dart:io` `File` doesn't work there. Use `await xfile.readAsBytes()` and `Image.memory(bytes)` to show it.

34.3 Cropping & compressing

- `image_cropper` opens a native crop UI (uCrop on Android, TOCropViewController on iOS). On Android you must add `UCropActivity` to the manifest, as the plugin's README explains.
- `flutter_image_compress` does native compression and format conversion:

```
final XFile? out = await FlutterImageCompress.compressAndGetFile(
  input.absolute.path, '${tempDir.path}/out.jpg',
  quality: 80, minWidth: 1080, minHeight: 1080, format: CompressFormat.jpeg,
);
```

- **Pure Dart:** the `image` package can resize and apply filters. It is CPU-heavy, so always run it inside `Isolate.run`.
- **EXIF:** photos from phones may be **rotated** in their EXIF metadata. Either the server reads the EXIF orientation, or you "bake in" the rotation before uploading (compress libraries can do this, e.g. `autoCorrectionAngle`). Remove GPS EXIF data for privacy when needed.

34.4 Custom camera screen: the `camera` plugin

Use this when you need your **own** camera UI: a scanner overlay, KYC selfie frames, custom buttons, filters, or frame-by-frame image streams for ML.

Setup

- **iOS `Info.plist`:**
  - `NSCameraUsageDescription`
  - `NSMicrophoneUsageDescription` (if `enableAudio` is true)
- **Android:** follow the README's `minSdk` requirement. The plugin requests the camera permission at runtime when you initialize the controller.

Complete camera page (with correct lifecycle handling)

```
import 'package:camera/camera.dart';
import 'package:flutter/material.dart';

class CameraPage extends StatefulWidget {
  const CameraPage({super.key});
  @override
  State<CameraPage> createState() => _CameraPageState();
}

class _CameraPageState extends State<CameraPage> with WidgetsBindingObserver {
  List<CameraDescription> _cameras = const [];
  CameraController? _controller;
  int _cameraIndex = 0;
  bool _recording = false;
```

```dart
  String? _error;

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addObserver(this);
    _setup();
  }

  Future<void> _setup() async {
    try {
      _cameras = await availableCameras();
      if (_cameras.isEmpty) {
        setState(() => _error = 'No camera found');
        return;
      }
      await _initController(_cameras[_cameraIndex]);
    } on CameraException catch (e) {
      setState(() => _error = _describe(e));
    }
  }

  Future<void> _initController(CameraDescription description) async {
    final old = _controller;
    _controller = null;
    if (mounted) setState(() {});
    await old?.dispose();                          // ❗ always dispose the previous controller

    final controller = CameraController(
      description,
      ResolutionPreset.high,                       // low/medium/high/veryHigh/ultraHigh/max
      enableAudio: true,
      imageFormatGroup: ImageFormatGroup.jpeg,
    );
    try {
      await controller.initialize();
      await controller.setFlashMode(FlashMode.auto);
      if (!mounted) {
        await controller.dispose();
        return;
      }
      setState(() => _controller = controller);
    } on CameraException catch (e) {
      await controller.dispose();
      if (mounted) setState(() => _error = _describe(e));
    }
  }

  String _describe(CameraException e) => switch (e.code) {
```

```
        'CameraAccessDenied' => 'Camera permission denied. Enable it in Settings.',
        'CameraAccessDeniedWithoutPrompt' => 'Camera permission denied permanently (iOS). Open
Settings.',
        'CameraAccessRestricted' => 'Camera access restricted (parental controls).',
        'AudioAccessDenied' => 'Microphone permission denied.',
        _ => 'Camera error: ${e.code} ${e.description}',
      };

  // ❗ The camera is a shared OS resource. Release it when the app goes to the background
  // and set it up again when the app comes back. Otherwise other apps can't use it,
  // and your preview may freeze or crash when the app resumes.
  @override
  void didChangeAppLifecycleState(AppLifecycleState state) {
    final controller = _controller;
    if (controller == null || !controller.value.isInitialized) return;
    if (state == AppLifecycleState.inactive) {
      _controller = null;
      controller.dispose();
    } else if (state == AppLifecycleState.resumed) {
      _initController(controller.description);
    }
  }

  @override
  void dispose() {
    WidgetsBinding.instance.removeObserver(this);
    _controller?.dispose();
    super.dispose();
  }

  Future<void> _takePicture() async {
    final c = _controller;
    if (c == null || !c.value.isInitialized || c.value.isTakingPicture) return;
    try {
      final XFile photo = await c.takePicture();
      if (!mounted) return;
      Navigator.of(context).pop(photo);           // return the file to the caller
    } on CameraException catch (e) {
      setState(() => _error = _describe(e));
    }
  }

  Future<void> _toggleRecording() async {
    final c = _controller;
    if (c == null) return;
    if (_recording) {
      final XFile video = await c.stopVideoRecording();
      setState(() => _recording = false);
      if (mounted) Navigator.of(context).pop(video);
```

```
    } else {
      await c.prepareForVideoRecording();         // iOS: avoids a delay at start
      await c.startVideoRecording();
      setState(() => _recording = true);
    }
  }

  void _switchCamera() {
    if (_cameras.length < 2) return;
    _cameraIndex = (_cameraIndex + 1) % _cameras.length;
    _initController(_cameras[_cameraIndex]);
  }

  @override
  Widget build(BuildContext context) {
    if (_error != null) {
      return Scaffold(appBar: AppBar(), body: Center(child: Text(_error!, textAlign:
TextAlign.center)));
    }
    final c = _controller;
    if (c == null || !c.value.isInitialized) {
      return const Scaffold(backgroundColor: Colors.black, body: Center(child:
CircularProgressIndicator()));
    }
    return Scaffold(
      backgroundColor: Colors.black,
      body: Stack(fit: StackFit.expand, children: [
        Center(child: CameraPreview(c)),         // keeps the camera's aspect ratio
        // Example overlay: a frame for document scanning
        IgnorePointer(
          child: Center(
            child: Container(
              width: 300, height: 190,
              decoration: BoxDecoration(border: Border.all(color: Colors.white, width: 2),
borderRadius: BorderRadius.circular(12)),
            ),
          ),
        ),
        SafeArea(
          child: Align(
            alignment: Alignment.bottomCenter,
            child: Padding(
              padding: const EdgeInsets.all(24),
              child: Row(mainAxisAlignment: MainAxisAlignment.spaceEvenly, children: [
                IconButton(onPressed: _switchCamera, icon: const Icon(Icons.cameraswitch, color:
Colors.white)),
                FloatingActionButton(onPressed: _takePicture, child: const Icon(Icons.camera)),
                IconButton(
                  onPressed: _toggleRecording,
```

```
                  icon: Icon(_recording ? Icons.stop : Icons.videocam, color: _recording ?
Colors.red : Colors.white),
                ),
              ]),
            ),
          ),
        ),
      ]),
    );
  }
}

// Usage: final result = await Navigator.push<XFile>(context, MaterialPageRoute(builder: (_) =>
const CameraPage()));
```

More controller features

```
await c.setZoomLevel(2.0);                       // range: getMinZoomLevel()/getMaxZoomLevel()
await c.setFocusPoint(const Offset(0.5, 0.5));   // normalized 0..1 coordinates
await c.setExposurePoint(const Offset(0.5, 0.5));
await c.setFlashMode(FlashMode.torch);
await c.lockCaptureOrientation(DeviceOrientation.portraitUp);
await c.pausePreview(); await c.resumePreview();

// Frame stream for ML (barcode, faces). It is heavy: process at most one frame at a time.
bool processing = false;
await c.startImageStream((CameraImage image) async {
  if (processing) return;
  processing = true;
  try { await detector.process(image); } finally { processing = false; }
});
await c.stopImageStream();
```

For barcodes or QR codes, use `mobile_scanner` instead of the camera plugin. It wraps ML Kit and AVFoundation and is much easier. For text and faces, use the `google_mlkit_*` packages, converting `CameraImage` to `InputImage` as their README shows.

Common camera bugs

Bug	Fix
Black preview after returning from background	**Handle `didChangeAppLifecycleState` (dispose on inactive, set up again on resumed)**
`CameraException(cameraPermission)`	**Permission denied → show a "go to Settings" message (`openAppSettings`)**
Preview stretched	**Don't force the preview to fill the screen. Keep the aspect ratio, or use `FittedBox(fit: BoxFit.cover)` with a sized box and clipping**
Crash on hot reload	**Camera native state + hot reload is fragile → hot restart**

Bug	Fix
`takePicture` called twice	Check `value.isTakingPicture`
Out-of-memory with image stream	Throttle frames, process in isolates, lower the resolution preset
Emulator issues	Test on real devices

34.5 Video playback: `video_player` (official) + `chewie` (ready-made controls)

Setup

- **Android:** add the `INTERNET` permission for network videos. Cleartext HTTP needs extra config, so use HTTPS.
- **iOS:** HTTPS only (ATS rules).
- **Web:** autoplay rules apply. Browsers usually allow autoplay only when the video is **muted**.

Complete player with controls

```
import 'package:flutter/material.dart';
import 'package:video_player/video_player.dart';

class VideoScreen extends StatefulWidget {
  const VideoScreen({super.key, required this.url});
  final String url;
  @override
  State<VideoScreen> createState() => _VideoScreenState();
}

class _VideoScreenState extends State<VideoScreen> {
  late final VideoPlayerController _controller;
  late final Future<void> _init;

  @override
  void initState() {
    super.initState();
    _controller = VideoPlayerController.networkUrl(
      Uri.parse(widget.url),
      videoPlayerOptions: VideoPlayerOptions(mixWithOthers: true),   // don't stop the user's
music
    );
    _init = _controller.initialize().then((_) {
      _controller.setLooping(false);
      _controller.play();
    });
    // Other sources: VideoPlayerController.asset('assets/intro.mp4'), .file(File(path)),
.contentUri(uri) on Android
  }
```

```
  @override
  void dispose() {
    _controller.dispose();              // ❗ releases native decoders. Leaks here are very
costly
    super.dispose();
  }

  String _fmt(Duration d) =>
      '${d.inMinutes.remainder(60).toString().padLeft(2,
'0')}:${d.inSeconds.remainder(60).toString().padLeft(2, '0')}';

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Video')),
      body: FutureBuilder<void>(
        future: _init,                  // created once in initState (not inside build!)
        builder: (context, snapshot) {
          if (snapshot.hasError) return Center(child: Text('Cannot play video:
${snapshot.error}'));
          if (snapshot.connectionState != ConnectionState.done) {
            return const Center(child: CircularProgressIndicator());
          }
          return Column(children: [
            AspectRatio(
              aspectRatio: _controller.value.aspectRatio,
              child: Stack(alignment: Alignment.bottomCenter, children: [
                VideoPlayer(_controller),
                VideoProgressIndicator(_controller, allowScrubbing: true),
              ]),
            ),
            // Rebuild ONLY the controls whenever the player value changes
            ValueListenableBuilder<VideoPlayerValue>(
              valueListenable: _controller,
              builder: (context, value, _) => Row(children: [
                IconButton(
                  icon: Icon(value.isPlaying ? Icons.pause : Icons.play_arrow),
                  onPressed: () => value.isPlaying ? _controller.pause() : _controller.play(),
                ),
                Text('${_fmt(value.position)} / ${_fmt(value.duration)}'),
                if (value.isBuffering) const Padding(padding: EdgeInsets.all(8), child:
SizedBox.square(dimension: 16, child: CircularProgressIndicator(strokeWidth: 2))),
                const Spacer(),
                IconButton(
                  icon: Icon(value.volume == 0 ? Icons.volume_off : Icons.volume_up),
                  onPressed: () => _controller.setVolume(value.volume == 0 ? 1 : 0),
                ),
              ]),
```

```
            ),
          ]);
        },
      ),
    );
  }
}
```

`VideoPlayerController` is a `ValueNotifier<VideoPlayerValue>`. That's why `ValueListenableBuilder` works with it. Other useful APIs: `seekTo(Duration)`, `setPlaybackSpeed(1.5)`, `value.hasError`, `value.errorDescription`, `value.buffered`, `value.size`, `value.isCompleted`.

Chewie (full-screen, speed menu, subtitles)

```
final chewie = ChewieController(
  videoPlayerController: _controller,
  autoPlay: true,
  looping: false,
  allowFullScreen: true,
  playbackSpeeds: const [0.5, 1, 1.5, 2],
  errorBuilder: (context, msg) => Center(child: Text(msg)),
);
Chewie(controller: chewie);
// dispose both: chewie.dispose(); _controller.dispose();
```

Video feeds (Reels/Shorts style), the senior-level part

- Use a vertical `PageView.builder`. Keep at most **3 controllers alive** (previous, current, next) and dispose the others. Each decoder uses native memory, and devices limit how many can run at once.
- Play only the visible page. Pause when the route is covered (`RouteAware`) or the app goes to the background.
- Preload the next video's `initialize()` early. Cache downloads using `flutter_cache_manager` and play from the file. Use HLS streaming (`.m3u8`) for adaptive bitrate.
- In plain lists, use `visibility_detector` to autoplay the most visible video.
- Alternatives: `media_kit` (libmpv based, desktop + mobile, many formats) and `better_player_plus` (DRM, caching, HLS tracks).
- DRM (Widevine/FairPlay) usually needs a specialized plugin or native SDK.
- Picture-in-picture and background playback need native/platform work or specialized plugins.

34.6 Audio playback: `just_audio` (+ `audio_service` for background)

```
import 'package:just_audio/just_audio.dart';
import 'package:audio_session/audio_session.dart';
```

```
class PlayerService {
  final player = AudioPlayer();

  Future<void> init() async {
    final session = await AudioSession.instance;
    await session.configure(const AudioSessionConfiguration.music());   // handles interruptions like phone calls & ducking
    player.playbackEventStream.listen((_) {}, onError: (Object e, StackTrace st) {
      // network/codec errors arrive here
    });
  }

  Future<void> playUrl(String url) async {
    try {
      await player.setUrl(url);            // also: setAsset, setFilePath, setAudioSource(...)
      await player.play();                 // ⚠️ the Future completes when playback STOPS/pauses, not when it starts!
    } on PlayerException catch (e) {
      // e.code, e.message
    } on PlayerInterruptedException {
      // a new source was loaded before this one finished loading
    }
  }

  Future<void> playlist(List<String> urls) async {
    await player.setAudioSource(ConcatenatingAudioSource(
      children: [for (final u in urls) AudioSource.uri(Uri.parse(u))],
    ));
    // ⚠️ Newer just_audio versions prefer `player.setAudioSources([...])`. Check your version's README.
    player.play();
  }

  Future<void> dispose() => player.dispose();
}
```

Building the UI from streams:

```
StreamBuilder<PlayerState>(
  stream: service.player.playerStateStream,
  builder: (context, snap) {
    final state = snap.data;
    final processing = state?.processingState;
    if (processing == ProcessingState.loading || processing == ProcessingState.buffering) {
      return const CircularProgressIndicator();
    }
    final playing = state?.playing ?? false;
    return IconButton(
```

```
      icon: Icon(playing ? Icons.pause : Icons.play_arrow),
      onPressed: playing ? service.player.pause : service.player.play,
    );
  },
);
StreamBuilder<Duration>(
  stream: service.player.positionStream,
  builder: (_, snap) => Slider(
    value: (snap.data ?? Duration.zero).inMilliseconds.toDouble()
        .clamp(0, (service.player.duration ?? Duration.zero).inMilliseconds.toDouble()),
    max: (service.player.duration ?? Duration.zero).inMilliseconds.toDouble().clamp(1,
double.infinity),
    onChanged: (v) => service.player.seek(Duration(milliseconds: v.toInt())),
  ),
);
```

Background audio & lock-screen controls

Use `audio_service`. Your handler class (extends `BaseAudioHandler`) wraps the player. It publishes `mediaItem` and `playbackState` (controls: play/pause/skip) and runs a foreground service on Android with a media notification. `just_audio_background` is a simpler option for basic cases.

- **Android:** follow the README's manifest entries. These include the media playback foreground service type (`FOREGROUND_SERVICE_MEDIA_PLAYBACK` for Android 14+) and the `AudioService` / `MediaButtonReceiver` declarations. MainActivity may need to extend `AudioServiceActivity`.
- **iOS:** add `UIBackgroundModes` → `audio`.

Short sound effects (button clicks, games): use `audioplayers` or `flame_audio`. They have low latency and can play several sounds at once.

34.7 Recording audio: `record`

```
import 'package:record/record.dart';
import 'package:path_provider/path_provider.dart';

final recorder = AudioRecorder();

Future<void> start() async {
  if (!await recorder.hasPermission()) return;                // asks for permission if needed
  final dir = await getTemporaryDirectory();
  final path = '${dir.path}/note_${DateTime.now().millisecondsSinceEpoch}.m4a';
  await recorder.start(const RecordConfig(encoder: AudioEncoder.aacLc, bitRate: 128000,
sampleRate: 44100), path: path);
}

Future<String?> stop() => recorder.stop();          // returns the file path
```

```
// recorder.onAmplitudeChanged(const Duration(milliseconds: 100)) → animated waveform
// recorder.pause() / resume() / cancel(); recorder.dispose() when done
```

Setup:

- **iOS:** `NSMicrophoneUsageDescription`.
- **Android:** `RECORD_AUDIO`.

34.8 Uploading media reliably

- **Small files:** Dio `FormData` with `onSendProgress` (Chapter 16).
- **Large files and videos:**
  - Get a **pre-signed URL** from your backend, then upload straight to S3/GCS. Your servers never carry the bytes.
  - Use **resumable / multipart uploads** (tus protocol, S3 multipart) so a broken connection doesn't restart from zero.
  - Use `background_downloader` for uploads that continue in the background (URLSession on iOS, WorkManager on Android).
- Compress before uploading. Generate thumbnails with `video_thumbnail`. Show progress and allow cancel and retry.
- Validate type and size on **both** the client and the server.

34.9 Interview questions — media

Q1. `image_picker` vs `camera`? image_picker opens the system picker or camera to choose a single photo or video. It's simple and needs no permission for the gallery on modern Android. The `camera` plugin gives you a live preview and full control for building a custom camera UI, image streams and video recording.

Q2. Why must you dispose the CameraController on `inactive`? The camera is an exclusive OS resource. If you don't release it in the background, other apps can't use it and your preview breaks or crashes on resume. Set it up again on `resumed`.

Q3. How do you reduce image upload size? Pass `maxWidth`/`imageQuality` to the picker, use native compression, convert to WebP/JPEG, strip EXIF data, and upload through pre-signed URLs.

Q4. How do you build a TikTok-style feed efficiently? Use a PageView with at most ~3 live video controllers. Preload the next video and dispose far-away ones. Play only the visible item, use HLS with caching, and pause when the screen is covered or the app is in the background.

Q5. How do you play audio in the background with lock-screen controls? Use audio_service (a foreground service and media session on Android; background mode `audio` on iOS) together with just_audio, and configure audio_session to handle interruptions.

Q6. What does `retrieveLostData` solve? Android can kill your activity while the camera app is open. retrieveLostData recovers the picked result when the app restarts.

Q7. How do you scan QR codes? With mobile_scanner (ML Kit / AVFoundation). It handles the preview, detection, the torch and lifecycle.

Q8. Why doesn't `File` work with picked images on web? dart:io doesn't exist on the web. XFile gives you bytes (`readAsBytes`) and a blob URL instead.

Chapter 35 — Maps & Location: Google Maps, OpenStreetMap, Geolocation, Geocoding, Geofencing & Background Tracking

Goal: Show maps with markers and routes, get the user's location correctly (including permissions and denied cases), turn coordinates into addresses, track location in the background legally, and keep the battery healthy.

35.1 5-year-old picture 🗺️

Your phone has a **GPS compass** that knows where you are. The **map** is a picture book of the world. **Markers** are stickers you put on the pictures. To use the compass you must ask permission. Using it **while the app is closed** needs extra permission and a very good reason.

35.2 Location permissions: the rules

Level	Android	iOS
While using the app	`ACCESS_COARSE_LOCATION` (approximate) and/or `ACCESS_FINE_LOCATION` (precise)	`NSLocationWhenInUseUsageDescription`
Background ("Always")	+ `ACCESS_BACKGROUND_LOCATION` (Android 10+), requested separately, after foreground permission is granted. Since Android 11 the user picks "Allow all the time" in Settings	+ `NSLocationAlwaysAndWhenInUseUsageDescription`, `UIBackgroundModes: location`
Precise vs approximate	Android 12+: the user can grant only approximate	iOS 14+: precise toggle (`NSLocationTemporaryUsageDescriptionDictionary` for temporary precise access)

Store policies:

- Google Play requires a **declaration form** and a demo video for background location. It is only allowed for core features that clearly benefit the user (navigation, fitness tracking, delivery drivers, family safety).
- Apple reviews "Always" usage strictly.

Ask for **when-in-use** first. Ask for background only when the user turns on a feature that needs it, after showing an explanation screen.

35.3 Getting the location: `geolocator`

Manifest / Info.plist

```
<!-- android/app/src/main/AndroidManifest.xml -->
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

```
<!-- ios/Runner/Info.plist -->
<key>NSLocationWhenInUseUsageDescription</key>
<string>We use your location to show nearby stores.</string>
```

A location service that handles every case

```
import 'package:geolocator/geolocator.dart';

sealed class LocationResult {}
final class LocationOk extends LocationResult { LocationOk(this.position); final Position
position; }
final class LocationServiceDisabled extends LocationResult {}
final class LocationDenied extends LocationResult {}
final class LocationDeniedForever extends LocationResult {}

class LocationService {
  Future<LocationResult> current() async {
    // 1. Is GPS/location turned on in the device settings?
    if (!await Geolocator.isLocationServiceEnabled()) return LocationServiceDisabled();

    // 2. Permission
    var permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
      permission = await Geolocator.requestPermission();
      if (permission == LocationPermission.denied) return LocationDenied();
    }
    if (permission == LocationPermission.deniedForever) return LocationDeniedForever();

    // 3. Position. Use the last known position first for speed, then a fresh fix with a timeout.
    final last = await Geolocator.getLastKnownPosition();   // may be null/stale; not supported on
web
    try {
      final fresh = await Geolocator.getCurrentPosition(
        locationSettings: const LocationSettings(
          accuracy: LocationAccuracy.high,
          timeLimit: Duration(seconds: 10),
        ),
      );
      return LocationOk(fresh);
```

```
    } on Exception {
      if (last != null) return LocationOk(last);
      rethrow;
    }
  }

  Stream<Position> track({int distanceFilterMeters = 10}) => Geolocator.getPositionStream(
        locationSettings: LocationSettings(
          accuracy: LocationAccuracy.high,
          distanceFilter: distanceFilterMeters,   // only emit after moving N meters → saves
battery
        ),
      );

  Future<void> openSettings() => Geolocator.openAppSettings();
  Future<void> openLocationSettings() => Geolocator.openLocationSettings();

  double distanceMeters(Position a, Position b) =>
      Geolocator.distanceBetween(a.latitude, a.longitude, b.latitude, b.longitude);
}
```

In the UI, use `switch` on the result:

- `LocationServiceDisabled` → show a "Turn on location" button that calls `openLocationSettings()`.
- `LocationDeniedForever` → show a "Open Settings" button.
- `LocationDenied` → explain why you need it and let the user retry.

Accuracy levels: `lowest`, `low`, `medium`, `high`, `best`, `bestForNavigation`. Higher accuracy uses more battery.

Platform-specific settings

```
final settings = switch (defaultTargetPlatform) {
  TargetPlatform.android => AndroidSettings(
      accuracy: LocationAccuracy.high,
      distanceFilter: 10,
      intervalDuration: const Duration(seconds: 5),
      // Background tracking on Android: a foreground service with a visible notification
      foregroundNotificationConfig: const ForegroundNotificationConfig(
        notificationTitle: 'Tracking your run',
        notificationText: 'Location is used while your workout is active',
        enableWakeLock: true,
      ),
    ),
  TargetPlatform.iOS || TargetPlatform.macOS => AppleSettings(
      accuracy: LocationAccuracy.high,
      distanceFilter: 10,
```

```
      activityType: ActivityType.fitness,
      pauseLocationUpdatesAutomatically: true,
      allowBackgroundLocationUpdates: true,      // requires UIBackgroundModes: location
      showBackgroundLocationIndicator: true,     // blue pill in the status bar
    ),
  _ => const LocationSettings(accuracy: LocationAccuracy.high, distanceFilter: 10),
};
final stream = Geolocator.getPositionStream(locationSettings: settings);
```

For Android background tracking, also declare the foreground service permissions that the geolocator README lists for your version (`FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_LOCATION` on Android 14+) and `ACCESS_BACKGROUND_LOCATION` if tracking continues after the app is closed.

Alternatives: the `location` package (similar API), and `flutter_background_geolocation` (a commercial package with very reliable background tracking, geofencing, activity recognition and battery-conscious motion detection; licensed for Android release builds).

35.4 Geocoding (address ↔ coordinates)

```
import 'package:geocoding/geocoding.dart';

final placemarks = await placemarkFromCoordinates(12.9716, 77.5946);
final p = placemarks.first;
final address = '${p.street}, ${p.locality}, ${p.administrativeArea} ${p.postalCode},
${p.country}';

final locations = await locationFromAddress('MG Road, Bengaluru');
final first = locations.first; // first.latitude, first.longitude
```

- The `geocoding` package uses the platform geocoders. They are free but have rate limits and vary in quality, and the package doesn't support the web.
- Production apps often use a backend proxy to the **Google Places/Geocoding APIs**, Mapbox, or HERE. That gives you autocomplete, keeps the API key on the server and enables caching.

35.5 Google Maps: `google_maps_flutter`

Setup

1. In Google Cloud Console, enable **Maps SDK for Android** and **Maps SDK for iOS** (plus the Maps JavaScript API for web). Create API keys **restricted** by Android package + SHA-1 and by iOS bundle ID. Billing must be enabled.
2. **Android** `AndroidManifest.xml`, inside `<application>`:

```
<meta-data android:name="com.google.android.geo.API_KEY" android:value="${MAPS_API_KEY}" />
```

You can inject the key from `local.properties`/CI with `manifestPlaceholders["MAPS_API_KEY"] = ...` in Gradle, so it isn't committed. 3. **iOS** `AppDelegate.swift`:

```
import GoogleMaps
// inside application(_:didFinishLaunchingWithOptions:) before GeneratedPluginRegistrant
GMSServices.provideAPIKey("YOUR_IOS_KEY")
```

4. **Web:** add the Maps JavaScript script tag with your key in `web/index.html`.

Map with markers, polyline and camera control

```
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';

class StoresMap extends StatefulWidget {
  const StoresMap({super.key, required this.stores});
  final List<Store> stores;
  @override
  State<StoresMap> createState() => _StoresMapState();
}

class _StoresMapState extends State<StoresMap> {
  final _controller = Completer<GoogleMapController>();
  Store? _selected;

  static const _initial = CameraPosition(target: LatLng(12.9716, 77.5946), zoom: 12);

  Set<Marker> get _markers => {
        for (final s in widget.stores)
          Marker(
            markerId: MarkerId(s.id),
            position: LatLng(s.lat, s.lng),
            infoWindow: InfoWindow(title: s.name, snippet: s.address),
            icon: BitmapDescriptor.defaultMarkerWithHue(
              s.id == _selected?.id ? BitmapDescriptor.hueAzure : BitmapDescriptor.hueRed,
            ),
            onTap: () => setState(() => _selected = s),
          ),
      };

  Set<Polyline> get _route => {
        if (widget.stores.length > 1)
          Polyline(
            polylineId: const PolylineId('route'),
            points: [for (final s in widget.stores) LatLng(s.lat, s.lng)],
            width: 5,
            color: Colors.blue,
```

```
          ),
      };

  Future<void> _fitAll() async {
    if (widget.stores.isEmpty) return;
    final lats = widget.stores.map((s) => s.lat);
    final lngs = widget.stores.map((s) => s.lng);
    final bounds = LatLngBounds(
      southwest: LatLng(lats.reduce((a, b) => a < b ? a : b), lngs.reduce((a, b) => a < b ? a :
b)),
      northeast: LatLng(lats.reduce((a, b) => a > b ? a : b), lngs.reduce((a, b) => a > b ? a :
b)),
    );
    final c = await _controller.future;
    await c.animateCamera(CameraUpdate.newLatLngBounds(bounds, 48));   // 48px padding
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: GoogleMap(
        initialCameraPosition: _initial,
        onMapCreated: (c) {
          if (!_controller.isCompleted) _controller.complete(c);
          _fitAll();
        },
        markers: _markers,
        polylines: _route,
        myLocationEnabled: true,          // needs location permission granted first!
        myLocationButtonEnabled: true,
        zoomControlsEnabled: false,
        mapToolbarEnabled: false,
        padding: const EdgeInsets.only(bottom: 80),   // keep the Google logo/buttons above your
bottom UI
        // style: darkMapStyleJson,       // JSON from the Google styling wizard (newer API; older
versions used controller.setMapStyle)
        onTap: (_) => setState(() => _selected = null),
        onCameraIdle: () { /* fetch markers for the visible region */ },
      ),
      floatingActionButton: FloatingActionButton(onPressed: _fitAll, child: const
Icon(Icons.zoom_out_map)),
    );
  }
}
```

Other features:

- **Custom marker icons:** `BitmapDescriptor.asset(ImageConfiguration(size: Size(48, 48)), 'assets/pin.png')` in newer versions (older: `fromAssetImage`). You can also render a widget to bytes and use `BitmapDescriptor.bytes(...)`.
- Circles, polygons, heatmaps, tile overlays.
- Read the visible area with `getVisibleRegion()`.
- **Marker clustering** (built-in `ClusterManager` in recent versions, or `google_maps_cluster_manager`).
- **Gestures inside scroll views:** pass `gestureRecognizers: {Factory<OneSequenceGestureRecognizer>(() => EagerGestureRecognizer())}` so the map gets drag gestures.

Performance and other gotchas

- The map is a **platform view** (Chapter 19). Avoid many maps on one screen or maps inside list items. For thumbnails, use the **Static Maps API** image instead.
- Thousands of markers → cluster them, and load markers by viewport (`onCameraIdle` + `getVisibleRegion`).
- Animating a moving car marker: interpolate positions with an `AnimationController` between updates, and update the marker set at a throttled rate.
- **Grey map** = wrong API key, API not enabled, or billing not set up. Check `adb logcat`, which shows the auth failure.

Directions

The Maps SDK has **no routing built in.** Call the **Directions/Routes API** from your backend, decode the encoded polyline (`flutter_polyline_points` or your own decoder), and draw it as a `Polyline`. To open turn-by-turn navigation in Google/Apple Maps, use `url_launcher`:

```
final uri = Uri.parse('https://www.google.com/maps/dir/?
api=1&destination=12.97,77.59&travelmode=driving');
await launchUrl(uri, mode: LaunchMode.externalApplication);
```

35.6 OpenStreetMap with `flutter_map` (no Google dependency)

```
import 'package:flutter_map/flutter_map.dart';
import 'package:latlong2/latlong.dart';

FlutterMap(
  options: const MapOptions(initialCenter: LatLng(12.9716, 77.5946), initialZoom: 13),
  children: [
    TileLayer(
      urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
      userAgentPackageName: 'com.mycompany.app',     // required by the OSM tile usage policy
    ),
    MarkerLayer(markers: [
```

```
      Marker(
        point: const LatLng(12.9716, 77.5946),
        width: 40, height: 40,
        child: const Icon(Icons.location_pin, color: Colors.red, size: 40),
      ),
    ]),
    PolylineLayer(polylines: [
      Polyline(points: const [LatLng(12.97, 77.59), LatLng(12.98, 77.60)], strokeWidth: 4, color:
Colors.blue),
    ]),
    const RichAttributionWidget(attributions: [TextSourceAttribution('OpenStreetMap
contributors')]),
  ],
);
```

- It's pure Flutter widgets (no platform view), so it works the same on mobile, web and desktop.
- ⚠️ The public OSM tile servers **must not be used for heavy production traffic** (tile usage policy). Use a tile provider (MapTiler, Stadia, Thunderforest, Mapbox raster tiles) or host your own tiles.
- Offline tiles: `flutter_map_tile_caching`.
- Other options: `mapbox_maps_flutter` (official Mapbox SDK, vector maps and navigation) and `apple_maps_flutter`.

35.7 Geofencing (alert me when I enter or leave an area)

- The OS provides geofencing: Android `GeofencingClient` and iOS `CLCircularRegion` monitoring. iOS limits you to **20 regions per app**, and Android to **100 per app per user**.
- In Flutter, use plugins (`native_geofence`, `geofence_service`, or the commercial `flutter_background_geolocation`) or write your own with Pigeon (Chapter 19).
- Geofence events can wake your app in the background. The handler runs in a separate isolate, so mark it with `@pragma('vm:entry-point')` and set up dependencies again (Chapter 18).
- Accuracy is roughly 100 m+. For smaller zones, combine with an active location stream while the app is in the foreground.

35.8 Battery & privacy best practices 🔋

- Request the lowest accuracy that works (city-level search → `low`).
- Use `distanceFilter` and intervals. Stop streams when the screen is hidden (cancel subscriptions in `dispose`, react to app lifecycle).
- Use **significant-change / activity-based** tracking for long periods.
- Send location updates in batches (store locally, upload every N points or minutes).
- Show clearly when tracking is active (the persistent notification on Android, the blue indicator on iOS).

- Treat location as **sensitive personal data**: get consent, keep it only as long as needed, encrypt it at rest, and declare it in the Play Data safety form and App Store privacy labels.
- Mock locations: on Android, `Position.isMocked` can detect fake GPS apps. This matters for delivery and attendance apps.

35.9 Common map/location bugs

Bug	Fix
Grey/blank Google map	API key restrictions/SHA-1, SDK not enabled, billing
Works in debug, fails in Play build	Add the Play App Signing SHA-1 to the key restrictions
`myLocationEnabled` shows nothing	Location permission not granted before building the map
Permission dialog never shows again	`deniedForever` → send the user to app settings
Background updates stop after minutes (Android)	No foreground service, or OEM battery optimizations → foreground service + ask the user to exempt the app
iOS background updates stop	`allowBackgroundLocationUpdates` + `UIBackgroundModes` + "Always" permission
`getCurrentPosition` hangs indoors	Use `timeLimit` and fall back to the last known position
Map gestures don't work inside ListView	`gestureRecognizers` with `EagerGestureRecognizer`
Jank with many markers	Clustering, load by viewport, cache bitmap descriptors

---

35.10 Interview questions — maps & location

Q1. How do you handle location permissions correctly? Check that the location service is on. Check the permission, then request it with an explanation first. Handle denied and denied-forever (open settings) cases. Ask for background permission separately and only when needed. Declare everything in the manifest and Info.plist.

Q2. How do you track location in the background on Android 14? Use a foreground service with the location type, a visible notification, background location permission if tracking continues after the app is closed, and the store policy declaration. Use distance filters and batching to save battery.

Q3. google_maps_flutter vs flutter_map? google_maps_flutter is a native platform view with Google tiles, billing and familiar Google features. flutter_map is pure Flutter: tiles from any provider, works the same on every platform, and is easier to customize. You need a proper tile provider for production.

Q4. How do you show thousands of markers efficiently? Clustering, loading markers by viewport on camera idle, caching marker icons, and throttling updates.

Q5. How do you draw a route between two points? Call a Directions/Routes API (ideally through your backend), decode the polyline, and draw a Polyline. For turn-by-turn, hand off to the native maps apps with url_launcher.

Q6. How do you detect fake GPS? Use `Position.isMocked` on Android, plus server-side checks for impossible speed or jumps, and device integrity attestation.

Chapter 36 — In-App Purchases, Subscriptions & Payment Gateways (Full Code)

Goal: Sell digital products with Apple/Google billing correctly (including server verification, restores and subscription states), and take payments for physical goods with Stripe and Razorpay. Money bugs are the most expensive bugs, so follow the rules in this chapter carefully.

36.1 5-year-old picture 💰

If you sell **stickers that live inside the app** (coins, premium mode), the **app store is the shopkeeper** and takes its share. If you sell **real things** (shoes, pizza, taxi rides), you use a normal **cash register** (payment gateway) like Stripe or Razorpay. And never trust the child (the app) when it says "I paid!". Always ask the bank (your server) to check.

36.2 Which payment system? (store policies)

You sell	Use	Notes
Digital content/features used in the app (premium, coins, subscriptions, ad removal)	Apple IAP / Google Play Billing	Required by App Store guideline 3.1.1 and Google Play's payments policy. There are regional exceptions: e.g., US external purchase links on iOS after the 2025 court ruling, EU DMA alternatives, Play alternative/user-choice billing in some countries. Check current rules before deciding
Physical goods/services (e-commerce, food, rides, tickets, consulting)	Payment gateway (Stripe, Razorpay, PayPal, Adyen, Braintree), Apple Pay / Google Pay via gateway	Store billing not allowed for these
"Reader" apps (Netflix/Spotify style)	Special rules (external link entitlement)	Check Apple's reader app rules
Person-to-person payments, donations to registered charities	Special cases	Check the guidelines

36.3 Store product types

Type	Apple	Google	Example
Consumable	Consumable	In-app product (consume after use)	Coins, gems

Type	Apple	Google	Example
Non-consumable	Non-consumable	In-app product (not consumed)	Remove ads, unlock a level pack
Auto-renewing subscription	Auto-renewable subscription (subscription groups)	Subscription with base plans & offers	Premium monthly/yearly
Non-renewing subscription	Non-renewing	(model as in-app product)	Season pass

Products are set up in **App Store Connect** (with agreements, tax and banking completed) and **Play Console** (the app must be uploaded to a testing track first). Product IDs should match across stores where possible (e.g., `premium_monthly`).

36.4 `in_app_purchase` (official plugin): complete purchase flow

```
import 'dart:async';
import 'package:in_app_purchase/in_app_purchase.dart';

class BillingService {
  BillingService(this._api);
  final PurchaseVerificationApi _api;          // your backend
  final InAppPurchase _iap = InAppPurchase.instance;
  StreamSubscription<List<PurchaseDetails>>? _sub;

  static const premiumMonthly = 'premium_monthly';
  static const coins100 = 'coins_100';
  static const removeAds = 'remove_ads';

  final _products = <String, ProductDetails>{};
  final _status = StreamController<BillingEvent>.broadcast();
  Stream<BillingEvent> get events => _status.stream;

  /// ❗ Call EARLY (e.g., at app start), so purchases completed while the app was
  /// closed or interrupted (pending, renewals, family approvals) are delivered and handled.
  Future<void> init() async {
    _sub = _iap.purchaseStream.listen(
      _onPurchases,
      onError: (Object e) => _status.add(BillingError('$e')),
    );
    if (!await _iap.isAvailable()) {
      _status.add(BillingError('Store not available'));
      return;
    }
    final response = await _iap.queryProductDetails({premiumMonthly, coins100, removeAds});
    if (response.notFoundIDs.isNotEmpty) {
      // Wrong IDs, products not approved yet, app not uploaded to a test track, or agreements
missing
      _status.add(BillingError('Missing products: ${response.notFoundIDs}'));
```

```
    }
    for (final p in response.productDetails) {
      _products[p.id] = p;              // p.title, p.description, p.price (localized string),
p.rawPrice, p.currencyCode
    }
  }

  ProductDetails? product(String id) => _products[id];

  Future<void> buy(String productId, {required String appUserId}) async {
    final product = _products[productId];
    if (product == null) throw StateError('Product $productId not loaded');
    final param = PurchaseParam(
      productDetails: product,
      applicationUserName: appUserId,   // links the purchase to your user (obfuscated account id
on Play)
    );
    if (productId == coins100) {
      await _iap.buyConsumable(purchaseParam: param, autoConsume: true);
    } else {
      await _iap.buyNonConsumable(purchaseParam: param);   // ALSO used for subscriptions
    }
    // The result does NOT come back here. It arrives on purchaseStream.
  }

  Future<void> restore() => _iap.restorePurchases();     // App Store requires a visible "Restore
Purchases" button

  Future<void> _onPurchases(List<PurchaseDetails> purchases) async {
    for (final p in purchases) {
      switch (p.status) {
        case PurchaseStatus.pending:
          _status.add(BillingPending(p.productID));       // e.g., cash payment or parental
approval. Don't grant yet!
        case PurchaseStatus.canceled:
          _status.add(BillingCancelled(p.productID));
        case PurchaseStatus.error:
          _status.add(BillingError(p.error?.message ?? 'Purchase failed'));
        case PurchaseStatus.purchased:
        case PurchaseStatus.restored:
          // ❗ Verify on YOUR server before granting anything
          final ok = await _api.verify(
            platform: p.verificationData.source,                        // 'app_store' /
'google_play'
            productId: p.productID,
            token: p.verificationData.serverVerificationData,           // Play purchase token /
App Store receipt or JWS (depends on plugin/StoreKit version)
          );
          if (ok) {
```

```
            _status.add(BillingSuccess(p.productID));
          } else {
            _status.add(BillingError('Verification failed'));
          }
      }
      // ❗ Always complete. On Android, a purchase not acknowledged within 3 days is
      // AUTOMATICALLY REFUNDED. On iOS, unfinished transactions are delivered again on every
launch.
      if (p.pendingCompletePurchase) {
        await _iap.completePurchase(p);
      }
    }
  }

  Future<void> dispose() async {
    await _sub?.cancel();
    await _status.close();
  }
}

sealed class BillingEvent {}
final class BillingPending extends BillingEvent { BillingPending(this.productId); final String
productId; }
final class BillingCancelled extends BillingEvent { BillingCancelled(this.productId); final String
productId; }
final class BillingSuccess extends BillingEvent { BillingSuccess(this.productId); final String
productId; }
final class BillingError extends BillingEvent { BillingError(this.message); final String message;
}
```

When to complete a purchase: complete it **after** your server has verified and granted the entitlement. If verification fails because of a temporary network error, don't complete it yet. The purchase will be delivered again so you can retry, but on Android remember the 3-day acknowledgement limit. If the purchase is invalid, complete it (or let it be refunded) and don't grant anything.

Paywall UI snippet

```
final product = billing.product(BillingService.premiumMonthly);
FilledButton(
  onPressed: product == null ? null : () => billing.buy(product.id, appUserId: userId),
  child: Text(product == null ? 'Loading…' : 'Go Premium — ${product.price}/month'),  // ALWAYS
show the store's localized price
);
TextButton(onPressed: billing.restore, child: const Text('Restore purchases'));
```

Paywall rules: show the price, the billing period, and the trial terms ("then ₹299/month"). Link to Terms and the Privacy Policy. Don't trick users. Apple and Google both reject misleading paywalls.

Platform-specific extras

- **Android subscriptions with offers/base plans:** cast to `GooglePlayProductDetails` to get `subscriptionOfferDetails`. Upgrades/downgrades use `GooglePlayPurchaseParam(changeSubscriptionParam: ChangeSubscriptionParam(oldPurchaseDetails: ..., replacementMode: ...))`.
- **iOS:** `InAppPurchaseStoreKitPlatformAddition` gives access to the code redemption sheet and the offer code UI. Newer plugin versions support StoreKit 2.
- Use **Xcode StoreKit configuration files** for local testing without App Store Connect. Use sandbox testers, and Play **license testers** plus internal testing tracks.

36.5 Server-side verification & entitlements (the important part) 🔐

```
App ──purchase token──► Your backend ──► Google Play Developer API (purchases.products / purchases.subscriptionsv2)
                                     └──► App Store Server API (transaction info, JWS verification)
        ◄── entitlement (premium until 2026-11-15) ──┘
Store ──server notifications──► Your backend   (renewals, cancellations, refunds, billing retry, grace period)
       • Google: Real-Time Developer Notifications (Pub/Sub)
       • Apple: App Store Server Notifications V2
```

- The **server is the source of truth** for entitlements. The app asks `GET /me/entitlements` at startup and after purchases. It caches the answer locally for offline use, with a short trust window.
- Protect against **replay**: one purchase token per user, and store the token with its order ID.
- Handle every subscription state: active, in **grace period** (payment issue but access continues), **on hold / billing retry** (access paused), cancelled-but-active-until-expiry, expired, refunded/revoked, paused (Play), upgraded/downgraded.
- Never decide "premium" from a client-side flag in SharedPreferences. That can be changed on a rooted device.

36.6 RevenueCat (managed subscriptions backend), the common choice

RevenueCat (`purchases_flutter`) handles receipt validation, entitlements, cross-platform subscription status, webhooks, paywalls (`purchases_ui_flutter`), analytics and experiments.

```
await Purchases.configure(PurchasesConfiguration(Platform.isIOS ? appleApiKey : googleApiKey)..appUserID = userId);
```

```
final offerings = await Purchases.getOfferings();
final package = offerings.current?.monthly;
if (package != null) {
  // Method names changed across major versions (e.g., purchasePackage vs
purchase(PurchaseParams...)). Check your version.
  final result = await Purchases.purchasePackage(package);
}
final info = await Purchases.getCustomerInfo();
final isPremium = info.entitlements.active.containsKey('premium');
Purchases.addCustomerInfoUpdateListener((info) { /* update UI */ });
```

Trade-off: you pay RevenueCat fees above a revenue threshold, and you depend on a vendor, but you save a lot of backend work. Alternatives: Adapty, Qonversion, Glassfy.

36.7 Stripe (`flutter_stripe`) for physical goods/services

Flow (PaymentIntent + Payment Sheet)

```
1. App → your backend: "create order for cart X"
2. Backend → Stripe: create PaymentIntent(amount, currency, customer, idempotency key) → returns
client_secret
   (+ ephemeral key & customer id if saving cards)
3. App: initPaymentSheet(clientSecret) → presentPaymentSheet()
4. Stripe → backend webhook: payment_intent.succeeded → backend marks order PAID (source of truth)
5. App polls/listens for order status (don't rely only on the app callback)
```

Setup

- Android: `MainActivity` must extend `FlutterFragmentActivity` (not FlutterActivity), and the app theme must be a `Theme.AppCompat` / `MaterialComponents` descendant. Otherwise the payment sheet crashes or does nothing.
- iOS: a minimum iOS version per the README. Apple Pay needs a merchant ID capability.

```
import 'package:flutter_stripe/flutter_stripe.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  Stripe.publishableKey = const String.fromEnvironment('STRIPE_PK');   // publishable key only.
NEVER the secret key
  Stripe.merchantIdentifier = 'merchant.com.mycompany.shop';         // Apple Pay
  await Stripe.instance.applySettings();
  runApp(const App());
}

Future<PaymentOutcome> checkout(String orderId) async {
  final intent = await api.createPaymentIntent(orderId);   // backend returns clientSecret,
```

```
customerId, ephemeralKey
  await Stripe.instance.initPaymentSheet(
    paymentSheetParameters: SetupPaymentSheetParameters(
      paymentIntentClientSecret: intent.clientSecret,
      merchantDisplayName: 'My Shop',
      customerId: intent.customerId,
      customerEphemeralKeySecret: intent.ephemeralKey,
      applePay: const PaymentSheetApplePay(merchantCountryCode: 'US'),
      googlePay: const PaymentSheetGooglePay(merchantCountryCode: 'US', testEnv: true),
      style: ThemeMode.system,
      returnURL: 'myshop://stripe-redirect',   // for redirect-based payment methods
    ),
  );
  try {
    await Stripe.instance.presentPaymentSheet();
    // The UI succeeded, but CONFIRM with the backend (webhook-updated order status)
    final paid = await api.waitForOrderPaid(orderId, timeout: const Duration(seconds: 20));
    return paid ? PaymentOutcome.success : PaymentOutcome.processing;
  } on StripeException catch (e) {
    if (e.error.code == FailureCode.Canceled) return PaymentOutcome.cancelled;
    return PaymentOutcome.failed;
  }
}
enum PaymentOutcome { success, processing, cancelled, failed }
```

3-D Secure (SCA) authentication is handled by the Payment Sheet. Use idempotency keys on the server so retries don't create duplicate charges.

36.8 Razorpay (popular in India) 🇮🇳

```
import 'package:razorpay_flutter/razorpay_flutter.dart';

class RazorpayCheckout {
  RazorpayCheckout(this._api) {
    _razorpay.on(Razorpay.EVENT_PAYMENT_SUCCESS, _onSuccess);
    _razorpay.on(Razorpay.EVENT_PAYMENT_ERROR, _onError);
    _razorpay.on(Razorpay.EVENT_EXTERNAL_WALLET, _onWallet);
  }
  final OrdersApi _api;
  final _razorpay = Razorpay();

  Future<void> pay({required String cartId, required String phone, required String email}) async {
    // 1. Backend creates the Razorpay ORDER (amount in paise, receipt id) with its key secret
    final order = await _api.createRazorpayOrder(cartId);
    // 2. Open checkout with the PUBLIC key id + order_id
    _razorpay.open({
      'key': order.keyId,
```

```
      'amount': order.amountPaise,          // e.g. 49900 = ₹499.00
      'currency': 'INR',
      'order_id': order.id,
      'name': 'My Shop',
      'description': 'Order #${order.receipt}',
      'prefill': {'contact': phone, 'email': email},
      'retry': {'enabled': true, 'max_count': 2},
    });
  }

  Future<void> _onSuccess(PaymentSuccessResponse r) async {
    // 3. ❗ Backend verifies the signature HMAC_SHA256(order_id + "|" + payment_id, key_secret)
== r.signature
    await _api.verifyRazorpayPayment(orderId: r.orderId!, paymentId: r.paymentId!, signature:
r.signature!);
  }

  void _onError(PaymentFailureResponse r) { /* r.code, r.message → show retry */ }
  void _onWallet(ExternalWalletResponse r) { /* r.walletName */ }

  void dispose() => _razorpay.clear();   // ❗ removes listeners
}
```

Also handle **payment captured webhooks** (`payment.captured`, `order.paid`) on the server. The app callback can be lost if the app is killed mid-payment. For UPI intent flows, check the plugin docs for the Android `<queries>` entries needed for UPI apps. Other Indian gateways: Cashfree, PayU, PhonePe PG, Paytm.

36.9 Google Pay / Apple Pay buttons (`pay` package)

The `pay` package shows the native Google Pay/Apple Pay buttons and returns a payment token. You send that token to your gateway (Stripe/Adyen/Braintree) on the server. The configuration JSON defines the gateway and merchant info. Apple Pay needs a merchant ID, a processing certificate and a domain or app capability.

36.10 Payment correctness checklist ✅

- ☐ Prices come from the server or store (never hard-coded or computed only on the client)
- ☐ The server creates orders/intents with **idempotency keys**
- ☐ The app never marks an order paid. **Webhooks** do, and the app reads the status
- ☐ Double-tap protection on the pay button
- ☐ Recovery when the app is killed mid-payment (check pending orders at launch)
- ☐ Every store purchase is verified server-side and completed/acknowledged
- ☐ Restore purchases button (iOS); subscription management link (Play / App Store)

- [ ] Refund/revocation notifications handled
- [ ] Testing: sandbox/license testers, test cards (4242...), failure cases, 3DS, slow networks
- [ ] PCI scope minimized (never touch raw card numbers; use SDK UIs/tokenization)
- [ ] Analytics: checkout funnel events; alerts on payment failure spikes

---

36.11 Common payment bugs

Bug	Cause / Fix
Products not found	Product not active/approved, app not uploaded to a Play testing track, App Store agreements/tax/banking incomplete, bundle ID mismatch, waiting for propagation (can take hours)
Users refunded automatically after 3 days (Play)	Purchases never acknowledged → `completePurchase` after granting
"Purchase dialog appears again on every launch" (iOS)	Transactions not finished → `completePurchase`
Premium lost after reinstall	No restore flow / entitlement not fetched from the server
Payment sheet doesn't open on Android (Stripe)	`MainActivity` must be a `FlutterFragmentActivity`; AppCompat theme
Duplicate charges	No idempotency key; double taps
Paid but order shows failed	App killed before the callback → use webhooks + order status polling
Razorpay success but backend unverified	Signature verification missing

---

36.12 Interview questions — payments

Q1. When must you use in-app purchases instead of Stripe? For digital goods or features consumed in the app (premium, coins, subscriptions), store billing is required, with some regional exceptions. Physical goods and real-world services must use regular gateways.

Q2. Why verify purchases on the server? Client data can be tampered with. Server verification against the Apple/Google APIs prevents fraud and replay. The server tracks entitlements and processes renewal and refund notifications.

Q3. What happens if you don't acknowledge a Play purchase? It is automatically refunded after 3 days. On iOS, unfinished transactions keep being delivered again.

Q4. How do you handle subscriptions' lifecycle? With server notifications (RTDN, App Store Server Notifications) that update the entitlement state: renewals, grace period, billing retry or hold, cancellation, expiry, refunds and plan changes. The app reads the entitlements from the backend.

Q5. Describe a reliable Stripe checkout flow. The backend creates a PaymentIntent with an idempotency key. The app shows the Payment Sheet with the client secret. A Stripe webhook marks the order paid, and the app confirms the order status with the backend.

Q6. How does Razorpay verification work? The backend creates the order. After checkout, the app sends payment_id, order_id and signature to the backend, which checks the HMAC-SHA256 with the key secret. Webhooks give extra assurance.

Q7. RevenueCat — why use it? It manages receipt validation, cross-platform entitlements, webhooks, paywalls and analytics, so you don't build a subscription backend yourself. The trade-off is cost and vendor dependency.

Chapter 37 — State Restoration, Custom Slivers & Advanced Widgets (Tables, Overlays, Search, Stepper, Autocomplete)

Goal: Make apps survive Android process death without losing the user's work. Understand how slivers lay out by writing one. Master the "enterprise" widgets used in dashboards and admin apps: data tables, 2D tables, overlays/dropdowns, search and multi-step forms.

Part A — State Restoration ♻️

37.1 5-year-old picture

You're drawing, and Mom says "bath time!" and takes your paper away (the OS kills the app to free memory). When you come back, a good mom gives you **your same drawing back**, not a blank page. **State restoration** is that mom.

37.2 Why it matters

On Android (and sometimes iOS), when your app is in the background and the device needs memory, the OS **kills the process**. When the user returns from the recent apps list, Android recreates the app and expects it to look **as they left it**: same screen, same scroll position, same typed text. Without restoration, Flutter starts from `main()` again with fresh state. The user lands on the home screen and their half-filled form is gone. Common real cases:

- The user opens the camera app or a payment app (UPI, bank 3DS page) from your app, and your app gets killed meanwhile.
- Low-end devices with many apps open.

Test it: Android **Developer options → "Don't keep activities"**, or `adb shell am kill <package>` while the app is in the background. Then reopen it from Recents.

37.3 How it works

- `MaterialApp(restorationScopeId: 'app')` turns on restoration for the app and its Navigator.
- Each widget that wants to restore state uses `RestorationMixin`, defines a `restorationId`, and registers `RestorableProperty` objects in `restoreState`.
- Data is saved into a `RestorationBucket` tree. The engine sends it to the OS (`onSaveInstanceState` on Android, NSCoder on iOS) and gets it back on relaunch.
- Only small, serializable data belongs here (IDs, scroll offsets, text, selected tab). **Do not** put large data or secrets in it. Load data again from the repository using the restored IDs.

37.4 Restoring a counter, a text field and a tab

```
class App extends StatelessWidget {
  const App({super.key});
  @override
  Widget build(BuildContext context) =>
      const MaterialApp(restorationScopeId: 'app', home: EditProfilePage());
}

class EditProfilePage extends StatefulWidget {
  const EditProfilePage({super.key});
  @override
  State<EditProfilePage> createState() => _EditProfilePageState();
}

class _EditProfilePageState extends State<EditProfilePage>
    with RestorationMixin, SingleTickerProviderStateMixin {
  // Built-in restorable types: RestorableInt, RestorableDouble, RestorableBool, RestorableString,
  // RestorableDateTime, RestorableTextEditingController, RestorableEnum, nullable variants
(RestorableIntN, …)
  final RestorableTextEditingController _name = RestorableTextEditingController();
  final RestorableTextEditingController _bio = RestorableTextEditingController();
  final RestorableInt _tab = RestorableInt(0);
  final RestorableBool _agreed = RestorableBool(false);

  // A TabController we own: listener added ONCE (never inside build)
  late final TabController _tabs = TabController(length: 2, vsync: this)
    ..addListener(() => _tab.value = _tabs.index);

  @override
  String get restorationId => 'edit_profile';     // unique among siblings in its restoration
scope

  @override
  void restoreState(RestorationBucket? oldBucket, bool initialRestore) {
    registerForRestoration(_name, 'name');
    registerForRestoration(_bio, 'bio');
    registerForRestoration(_tab, 'tab');
    registerForRestoration(_agreed, 'agreed');
    _tabs.index = _tab.value;                     // apply the restored tab
    // initialRestore == true → values just came back from the OS.
    // If there is nothing saved, the defaults above stay. Load server data now if needed:
    if (_name.value.text.isEmpty) _loadFromServerIfEmpty();
  }

  void _loadFromServerIfEmpty() { /* fetch the profile, then set _name.value.text = ... */ }

  @override
  void dispose() {
```

```
    _tabs.dispose();
    _name.dispose();      // ❗ restorable properties must be disposed
    _bio.dispose();
    _tab.dispose();
    _agreed.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Edit profile'),
        bottom: TabBar(controller: _tabs, tabs: const [Tab(text: 'Basic'), Tab(text: 'More')]),
      ),
      body: TabBarView(controller: _tabs, children: [
        ListView(padding: const EdgeInsets.all(16), children: [
          TextField(controller: _name.value, decoration: const InputDecoration(labelText:
'Name')),
          CheckboxListTile(
            value: _agreed.value,
            onChanged: (v) => setState(() => _agreed.value = v ?? false),
            title: const Text('Show profile publicly'),
          ),
        ]),
        ListView(padding: const EdgeInsets.all(16), children: [
          TextField(controller: _bio.value, maxLines: 5, decoration: const
InputDecoration(labelText: 'Bio')),
        ]),
      ]),
    );
  }
}
```

37.5 Restoring navigation

Routes pushed with plain `Navigator.push` are **not** restorable, because closures can't be serialized. Use the restorable APIs with a **static** route builder and serializable arguments:

```
// Push
Navigator.restorablePush(context, _detailsRouteBuilder, arguments: productId);

// MUST be a static or top-level function
@pragma('vm:entry-point')
static Route<void> _detailsRouteBuilder(BuildContext context, Object? arguments) {
  return MaterialPageRoute<void>(builder: (_) => ProductDetailsPage(id: arguments! as String));
}
// Also: restorablePushNamed, restorablePushReplacement, restorablePopAndPushNamed …
```

```
// Dialogs: showDialog is not restorable → use Navigator.restorablePush with DialogRoute / RawDialogRoute.
```

- **go_router:** set `restorationScopeId` on `GoRouter` (and `MaterialApp.router`). Pages restore from the URL/location. Path parameters restore; `extra` **objects need an** `extraCodec` to be restorable (Chapter 11).
- Scroll positions: give scrollables a `restorationId` (`ListView(restorationId: 'feed')`) inside a restoration scope.
- Custom types: subclass `RestorableValue<T>` (implement `createDefaultValue`, `fromPrimitives`, `toPrimitives`, `didUpdateValue`), or `RestorableListenable`.
- Testing: `await tester.restartAndRestore();` in widget tests (with `restorationScopeId` set) simulates the save → kill → restore cycle. Also `tester.getRestorationData()` / `restoreFrom(data)`.

37.6 Alternatives & complements

- Persist drafts (form data) to local storage as the user types (debounced). This also survives the user swiping the app away, which restoration does **not** cover.
- Hydrated state management (`hydrated_bloc`, Riverpod offline persistence) for app-level state.
- Restore the user's **context** (which screen) through routing restoration, and their **data** through repositories.

Part B — Writing a Custom Sliver (to really understand slivers) 🥓

37.7 The sliver protocol recap

- **In:** `SliverConstraints`
  - `scrollOffset`: how far this sliver's start has scrolled past the viewport's leading edge
  - `remainingPaintExtent`: how much visible space is left
  - `crossAxisExtent`
  - `overlap`: pinned headers above you
  - `cacheOrigin` / `remainingCacheExtent`
  - `axisDirection`, `growthDirection`, `userScrollDirection`
- **Out:** `SliverGeometry`
  - `scrollExtent`: total scrollable length of this sliver
  - `paintExtent`: how much it paints now (≤ remainingPaintExtent)
  - `layoutExtent`: how much space it uses before the next sliver (defaults to paintExtent)
  - `maxPaintExtent`
  - `hitTestExtent`
  - `paintOrigin`: draw offset; negative values allow overlap effects
  - `cacheExtent`, `hasVisualOverflow`

37.8 Example: how `SliverToBoxAdapter` really works

This is essentially the framework's own implementation (`RenderSliverToBoxAdapter`). Reading it teaches you the protocol:

```
class MySliverBox extends SingleChildRenderObjectWidget {
  const MySliverBox({super.key, required Widget super.child});
  @override
  RenderMySliverBox createRenderObject(BuildContext context) => RenderMySliverBox();
}

class RenderMySliverBox extends RenderSliverSingleBoxAdapter {
  @override
  void performLayout() {
    if (child == null) {
      geometry = SliverGeometry.zero;
      return;
    }
    final constraints = this.constraints;
    // 1. Lay out the box child with box constraints: cross axis = viewport width, main axis
unbounded
    child!.layout(constraints.asBoxConstraints(), parentUsesSize: true);
    final childExtent = switch (constraints.axis) {
      Axis.horizontal => child!.size.width,
      Axis.vertical => child!.size.height,
    };
    // 2. How much of [0, childExtent] is visible given the current scroll offset?
    final paintedChildSize = calculatePaintOffset(constraints, from: 0.0, to: childExtent);
    final cacheExtent = calculateCacheOffset(constraints, from: 0.0, to: childExtent);
    // 3. Report geometry
    geometry = SliverGeometry(
      scrollExtent: childExtent,
      paintExtent: paintedChildSize,
      cacheExtent: cacheExtent,
      maxPaintExtent: childExtent,
      hitTestExtent: paintedChildSize,
      hasVisualOverflow: childExtent > constraints.remainingPaintExtent ||
constraints.scrollOffset > 0.0,
    );
    // 4. Position the child (negative offset when partially scrolled off the top)
    setChildParentData(child!, constraints, geometry!);
  }
}
// Usage: CustomScrollView(slivers: [MySliverBox(child: Text('Hello')), SliverList(...)])
```

37.9 Example: a sliver that fades its child as it scrolls away

Building on the same class, override `paint` to apply opacity based on the scroll offset:

```
class RenderFadingSliverBox extends RenderMySliverBox {
  @override
  bool get alwaysNeedsCompositing => child != null;   // we push an OpacityLayer (same idea as
RenderOpacity)

  @override
  void paint(PaintingContext context, Offset offset) {
    if (child == null || geometry == null || !geometry!.visible) {
      layer = null;
      return;
    }
    final extent = geometry!.scrollExtent;
    final progress = extent == 0 ? 0.0 : (constraints.scrollOffset / extent).clamp(0.0, 1.0);
    final alpha = ((1 - progress) * 255).round();
    final childParentData = child!.parentData! as SliverPhysicalParentData;
    layer = context.pushOpacity(
      offset,
      alpha,
      (context, offset) => context.paintChild(child!, offset + childParentData.paintOffset),
      oldLayer: layer as OpacityLayer?,      // reuse the layer between frames
    );
  }
}
```

The sliver is laid out again on every scroll, so `paint` runs with the new `scrollOffset` and the fade updates automatically. (Create a widget class whose `createRenderObject` returns `RenderFadingSliverBox`, just like `MySliverBox`.) For most pinned/floating headers, **don't** write render slivers. Use `SliverPersistentHeader`, `PinnedHeaderSliver`, `SliverFloatingHeader`, `SliverResizingHeader` or `SliverLayoutBuilder`:

```
SliverLayoutBuilder(
  builder: (context, constraints) {
    final collapsed = constraints.scrollOffset > 120;
    return SliverToBoxAdapter(child: AnimatedOpacity(opacity: collapsed ? 0 : 1, duration: const
Duration(milliseconds: 200), child: const Header()));
  },
);
```

Part C — Data tables 📊

37.10 `DataTable` (small tables) with sorting & selection

```
class OrdersTable extends StatefulWidget {
  const OrdersTable({super.key, required this.orders});
```

```
  final List<Order> orders;
  @override
  State<OrdersTable> createState() => _OrdersTableState();
}

class _OrdersTableState extends State<OrdersTable> {
  late List<Order> _rows = [...widget.orders];
  int? _sortColumn;
  bool _ascending = true;
  final _selected = <String>{};

  void _sort<T extends Comparable<T>>(T Function(Order o) key, int column, bool asc) {
    setState(() {
      _rows.sort((a, b) => asc ? key(a).compareTo(key(b)) : key(b).compareTo(key(a)));
      _sortColumn = column;
      _ascending = asc;
    });
  }

  @override
  Widget build(BuildContext context) {
    return SingleChildScrollView(                  // horizontal scroll for wide tables
      scrollDirection: Axis.horizontal,
      child: DataTable(
        sortColumnIndex: _sortColumn,
        sortAscending: _ascending,
        showCheckboxColumn: true,
        columns: [
          DataColumn(label: const Text('Order'), onSort: (i, asc) => _sort((o) => o.id, i, asc)),
          DataColumn(label: const Text('Customer'), onSort: (i, asc) => _sort((o) => o.customer,
i, asc)),
          DataColumn(label: const Text('Total'), numeric: true, onSort: (i, asc) => _sort((o) =>
o.total, i, asc)),
          const DataColumn(label: Text('Status')),
        ],
        rows: [
          for (final o in _rows)
            DataRow(
              key: ValueKey(o.id),
              selected: _selected.contains(o.id),
              onSelectChanged: (v) => setState(() => v! ? _selected.add(o.id) :
_selected.remove(o.id)),
              cells: [
                DataCell(Text(o.id)),
                DataCell(Text(o.customer)),
                DataCell(Text('₹${o.total.toStringAsFixed(2)}')),
                DataCell(Chip(label: Text(o.status))),
              ],
            ),
```

```
        ],
      ),
    );
  }
}
```

⚠️ `DataTable` builds **all rows** at once. It's fine for tens of rows, but slow for thousands.

37.11 `PaginatedDataTable` (pages of rows)

```
class OrdersSource extends DataTableSource {
  OrdersSource(this.orders);
  final List<Order> orders;

  @override
  DataRow? getRow(int index) {
    if (index >= orders.length) return null;
    final o = orders[index];
    return DataRow.byIndex(index: index, cells: [DataCell(Text(o.id)), DataCell(Text(o.customer)),
DataCell(Text('${o.total}'))]);
  }

  @override
  int get rowCount => orders.length;
  @override
  bool get isRowCountApproximate => false;   // true when the server total is unknown
  @override
  int get selectedRowCount => 0;
}

PaginatedDataTable(
  header: const Text('Orders'),
  rowsPerPage: 10,
  availableRowsPerPage: const [10, 25, 50],
  onRowsPerPageChanged: (v) => setState(() => _rowsPerPage = v!),
  onPageChanged: (firstRowIndex) => _maybeFetchMore(firstRowIndex),   // load server pages lazily
  columns: const [DataColumn(label: Text('Order')), DataColumn(label: Text('Customer')),
DataColumn(label: Text('Total'), numeric: true)],
  source: OrdersSource(orders),
);
```

When the data changes, call `notifyListeners()` on the source (`DataTableSource` is a `ChangeNotifier`).

37.12 Huge 2D tables: `TableView` (`two_dimensional_scrollables`, official)

It builds lazily in **both directions**, supports pinned header rows and columns, and handles very large grids such as spreadsheets:

```dart
import 'package:two_dimensional_scrollables/two_dimensional_scrollables.dart';

TableView.builder(
  rowCount: 10000,
  columnCount: 50,
  pinnedRowCount: 1,           // sticky header row
  pinnedColumnCount: 1,        // sticky first column
  columnBuilder: (index) => TableSpan(
    extent: FixedTableSpanExtent(index == 0 ? 160 : 100),
    foregroundDecoration: const TableSpanDecoration(border: TableSpanBorder(trailing:
BorderSide(color: Colors.black12))),
  ),
  rowBuilder: (index) => TableSpan(
    extent: const FixedTableSpanExtent(44),
    backgroundDecoration: TableSpanDecoration(color: index.isEven ? Colors.black.withValues(alpha:
0.03) : null),
  ),
  cellBuilder: (context, vicinity) => TableViewCell(
    child: Padding(
      padding: const EdgeInsets.symmetric(horizontal: 8),
      child: Align(
        alignment: Alignment.centerLeft,
        child: Text(vicinity.row == 0 ? 'Col ${vicinity.column}' :
'R${vicinity.row}C${vicinity.column}'),
      ),
    ),
  ),
);
```

The same package has `TreeView` for hierarchical rows. Other options: `data_table_2` (a DataTable with fixed headers), `syncfusion_flutter_datagrid` and `pluto_grid` / `trina_grid` (feature-rich grids with editing, filtering and export; check licenses).

Part D — Overlays, dropdowns & search 🪟

37.13 `OverlayPortal`: a custom anchored dropdown

```dart
class AnchoredMenu extends StatefulWidget {
  const AnchoredMenu({super.key, required this.items, required this.onSelected});
  final List<String> items;
  final ValueChanged<String> onSelected;
  @override
  State<AnchoredMenu> createState() => _AnchoredMenuState();
}
```

```dart
class _AnchoredMenuState extends State<AnchoredMenu> {
  final _portal = OverlayPortalController();
  final _link = LayerLink();
  final _groupId = Object();           // tapping either the button or the menu doesn't count as "outside"

  @override
  Widget build(BuildContext context) {
    return OverlayPortal(
      controller: _portal,
      overlayChildBuilder: (context) => CompositedTransformFollower(
        link: _link,
        targetAnchor: Alignment.bottomLeft,       // stick the menu's top-left to the button's bottom-left
        followerAnchor: Alignment.topLeft,
        offset: const Offset(0, 4),
        child: Align(
          alignment: AlignmentDirectional.topStart,   // the overlay gives full-screen constraints → let the menu size itself
          child: TapRegion(
            groupId: _groupId,
            onTapOutside: (_) => _portal.hide(),
            child: Material(
              elevation: 6,
              borderRadius: BorderRadius.circular(8),
              child: ConstrainedBox(
                constraints: const BoxConstraints(maxHeight: 240, minWidth: 180),
                child: ListView(
                  shrinkWrap: true,
                  padding: EdgeInsets.zero,
                  children: [
                    for (final item in widget.items)
                      ListTile(
                        dense: true,
                        title: Text(item),
                        onTap: () {
                          _portal.hide();
                          widget.onSelected(item);
                        },
                      ),
                  ],
                ),
              ),
            ),
          ),
        ),
      ),
      child: CompositedTransformTarget(
        link: _link,
```

```
        child: TapRegion(
          groupId: _groupId,
          child: OutlinedButton.icon(
            onPressed: _portal.toggle,
            icon: const Icon(Icons.arrow_drop_down),
            label: const Text('Choose'),
          ),
        ),
      ),
    );
  }
}
```

Why `OverlayPortal` instead of `OverlayEntry`? The overlay child is part of **your widget tree**, so it inherits Theme, Provider/Riverpod scopes and Localizations, and it rebuilds with your state. There is no manual insert/remove, and nothing leaks when the widget is disposed. Use `MenuAnchor` / `DropdownMenu` first if they fit your design.

37.14 `SearchAnchor` + `SearchBar` (Material 3 search)

```
SearchAnchor(
  builder: (context, controller) => SearchBar(
    controller: controller,
    hintText: 'Search products',
    leading: const Icon(Icons.search),
    onTap: controller.openView,
    onChanged: (_) => controller.openView(),
  ),
  suggestionsBuilder: (context, controller) async {     // can be async → fetch from the API
    final query = controller.text.trim();
    if (query.isEmpty) return recentSearches.map((s) => ListTile(leading: const
Icon(Icons.history), title: Text(s), onTap: () => controller.closeView(s)));
    final results = await api.searchSuggestions(query); // ⚠️ add debounce/cancel for real apps
(wrap in your own debouncer)
    return results.map((r) => ListTile(title: Text(r.name), onTap: () {
      controller.closeView(r.name);
      context.push('/product/${r.id}');
    }));
  },
);
```

`SearchAnchor.bar(...)` is a shortcut. For a full-screen search page, use `showSearch` with a `SearchDelegate` (the older API, still valid).

37.15 `Autocomplete` (inline suggestions under a text field)

```
Autocomplete<City>(
  displayStringForOption: (c) => c.name,
  optionsBuilder: (TextEditingValue value) {
    if (value.text.isEmpty) return const Iterable<City>.empty();
    return cities.where((c) => c.name.toLowerCase().startsWith(value.text.toLowerCase()));
  },
  onSelected: (city) => debugPrint('Selected ${city.name}'),
  fieldViewBuilder: (context, controller, focusNode, onSubmit) => TextField(
    controller: controller,
    focusNode: focusNode,
    decoration: const InputDecoration(labelText: 'City'),
  ),
);
```

`optionsBuilder` may return a `Future` for async suggestions. Use `RawAutocomplete` for full control of the options layout.

37.16 `Stepper` (multi-step forms / onboarding / checkout)

```
class CheckoutStepper extends StatefulWidget {
  const CheckoutStepper({super.key});
  @override
  State<CheckoutStepper> createState() => _CheckoutStepperState();
}

class _CheckoutStepperState extends State<CheckoutStepper> {
  int _step = 0;
  final _formKeys = [GlobalKey<FormState>(), GlobalKey<FormState>()];

  bool _validateCurrent() => _step >= _formKeys.length ||
_formKeys[_step].currentState!.validate();

  @override
  Widget build(BuildContext context) {
    return Stepper(
      currentStep: _step,
      type: StepperType.vertical,                    // .horizontal for wide screens
      onStepContinue: () {
        if (!_validateCurrent()) return;
        if (_step < 2) setState(() => _step++); else _placeOrder();
      },
      onStepCancel: _step == 0 ? null : () => setState(() => _step--),
      onStepTapped: (i) { if (i < _step) setState(() => _step = i); },   // only allow going back
      controlsBuilder: (context, details) => Row(children: [
        FilledButton(onPressed: details.onStepContinue, child: Text(_step == 2 ? 'Place order' :
```

```
'Next')),
        const SizedBox(width: 8),
        if (details.onStepCancel != null) TextButton(onPressed: details.onStepCancel, child: const
Text('Back')),
      ]),
      steps: [
        Step(
          title: const Text('Address'),
          isActive: _step >= 0,
          state: _step > 0 ? StepState.complete : StepState.editing,
          content: Form(key: _formKeys[0], child: TextFormField(decoration: const
InputDecoration(labelText: 'Street'), validator: (v) => (v ?? '').isEmpty ? 'Required' : null)),
        ),
        Step(
          title: const Text('Delivery slot'),
          isActive: _step >= 1,
          state: _step > 1 ? StepState.complete : (_step == 1 ? StepState.editing :
StepState.indexed),
          content: Form(key: _formKeys[1], child: DropdownButtonFormField<String>(
            items: const [DropdownMenuItem(value: 'am', child: Text('Morning')),
DropdownMenuItem(value: 'pm', child: Text('Evening'))],
            onChanged: (_) {},
            validator: (v) => v == null ? 'Pick a slot' : null,
          )),
        ),
        Step(title: const Text('Review'), isActive: _step >= 2, content: const Text('Order
summary…')),
      ],
    );
  }

  void _placeOrder() { /* … */ }
}
```

Note: `Stepper` builds every step's `content`. The forms stay in the tree, so their keys stay valid. In production, many teams build a **PageView-based wizard** with their own progress indicator for full design control, and keep the wizard state in their state management, so a back gesture or process death doesn't lose it.

37.17 Interview questions — this chapter

Q1. What is state restoration and when is it needed? It restores UI state after the OS kills a backgrounded app (Android process death). You enable it with `restorationScopeId`, use RestorationMixin with RestorableProperties, and use restorable navigator APIs. Test it with "Don't keep activities".

Q2. Why can't `Navigator.push` routes be restored? They are built from closures, which can't be serialized. Restorable pushes use static route builders and serializable arguments.

Q3. Restoration vs persisting state to storage? Restoration covers process death after the OS kills the app (saved instance state). It does not survive the user swiping the app away or a reboot. Persistence (DB/prefs) survives everything. Use both for important drafts.

Q4. Explain SliverConstraints and SliverGeometry. The constraints describe the scroll offset, remaining paint extent, overlap and cross-axis size. The geometry reports scroll extent, paint extent, layout extent, max paint extent and hit test extent, so the viewport can place the next slivers and know what's visible.

Q5. DataTable vs PaginatedDataTable vs TableView? DataTable builds every row (small data). PaginatedDataTable shows pages from a DataTableSource. TableView builds lazily in both directions with pinned rows and columns (huge grids).

Q6. OverlayPortal vs OverlayEntry? OverlayPortal declares the overlay child inside the widget tree, so it inherits context (theme, providers), updates with state and needs no manual insert/remove. OverlayEntry is imperative.

Q7. How do you anchor a popup to a widget? CompositedTransformTarget + CompositedTransformFollower sharing a LayerLink, rendered in an overlay (OverlayPortal), with TapRegion to close it on outside taps.

Chapter 38 — Code Generation Deep Dive (build_runner, build.yaml, writing a generator) & Publishing Packages to pub.dev

Goal: Configure code generation for big projects (and keep it fast), write your own generator, and publish quality packages and plugins to pub.dev with high pub points and automated releases.

Part A — Code generation

38.1 5-year-old picture 🤖

Some code is boring and repetitive (fromJson, toJson, copyWith, ==). Instead of writing it by hand, you hire a **robot helper** (a generator) that reads little **sticky notes** (annotations like `@JsonSerializable()`) on your classes and **writes the boring code for you** into separate files (`*.g.dart`). **build_runner** is the robot's boss: it decides which robots run on which files.

38.2 Why codegen in Dart?

Flutter disables `dart:mirrors` (runtime reflection), because it would break tree shaking and AOT size optimizations. Dart macros were **cancelled** (January 2025). So **build-time code generation** is the standard way to remove boilerplate:

- `json_serializable`, `freezed`, `riverpod_generator`, `go_router_builder`, `retrofit_generator`, `drift_dev`, `injectable_generator`, `mockito`, `hive_ce_generator`, `flutter_gen_runner`, `auto_route_generator`, `envied_generator`, `isar_community_generator`, `dart_mappable_builder`, `slang_build_runner`...

38.3 How it fits together

```
your_file.dart              part 'your_file.g.dart';   ← you declare the part
   @JsonSerializable()      (annotation from a *_annotation package in dependencies)
          |
build_runner (dev_dependency) runs the builders from generator packages (dev_dependencies)
          |
          ▼
your_file.g.dart            part of 'your_file.dart';  ← generated, contains _$UserFromJson…
```

- Annotation packages (`json_annotation`, `freezed_annotation`, `riverpod_annotation`) go in **dependencies**. Generators (`json_serializable`, `freezed`, `riverpod_generator`) and `build_runner` go in **dev_dependencies**.
- Generated files use `part` / `part of`, so they can access private members of the library.
- Many `source_gen`-based generators write into **one combined** `.g.dart` per file through the `source_gen:combining_builder`. That's why json_serializable and riverpod_generator output can share `user.g.dart`. Freezed writes a separate `.freezed.dart`.

38.4 Commands

```
dart run build_runner build --delete-conflicting-outputs   # one-time build
dart run build_runner watch -d                             # rebuild on save (-d = delete
conflicting)
dart run build_runner clean                                # clear the build cache
(.dart_tool/build)
dart run build_runner build --build-filter="lib/features/auth/**"   # only some outputs (advanced;
check docs)
```

- "Conflicting outputs" happens when `.g.dart` files already exist that build_runner didn't create (e.g., committed or from another branch). `-d` deletes and regenerates them.
- `build_runner` versions from 2025 onward made big **speed improvements** (e.g., compiling builders ahead of time and faster incremental builds). Keep it up to date in large repos.

38.5 `build.yaml`: configure generators project-wide

Put a `build.yaml` next to `pubspec.yaml`:

```
targets:
  $default:
    builders:
      # Configure json_serializable for the whole package (no need to repeat options in every
annotation)
      json_serializable:
        options:
          explicit_to_json: true
          field_rename: snake
          include_if_null: false
          any_map: false
          checked: false            # true → better error messages when JSON has the wrong type
(slower)
          create_to_json: true
        generate_for:               # ⚡ only run on model files → much faster builds
          include:
            - lib/**/models/**.dart
            - lib/**/dto/**.dart
```

```
    freezed:
      generate_for:
        include:
          - lib/**/models/**.dart
          - lib/**/state/**.dart
      # freezed-specific options (e.g., turning off generated features) are listed in the
freezed README

    riverpod_generator:
      generate_for:
        include:
          - lib/**/providers/**.dart
          - lib/**/*_provider.dart

    # Turn a generator off completely
    # some_package:some_builder:
    #   enabled: false

    source_gen:combining_builder:
      options:
        ignore_for_file:          # add ignores at the top of generated .g.dart files
          - type=lint
          - implicit_dynamic_parameter
```

- `generate_for` **is the #1 performance fix** in big projects. Without it, every builder analyzes every Dart file.
- Annotation-level options override build.yaml: `@JsonSerializable(fieldRename: FieldRename.none)`.
- In monorepos, each package has its own `build.yaml` and runs build_runner separately. Use melos scripts to run codegen only in packages that depend on `build_runner`.
- Also exclude generated files from the analyzer and coverage (Chapter 2 and 22).

38.6 Should generated files be committed?

Commit `*.g.dart`	Don't commit (generate in CI)
✅ Fresh clones build immediately; reviewers see generated diffs; CI is faster	✅ Smaller diffs; no merge conflicts in generated code
❌ Merge conflicts; must remember to regenerate	❌ Everyone and CI must run build_runner; slower start
Either way, add a CI check: run build_runner and fail if `git diff --exit-code` shows changes (when committing), or make sure the CI build always generates first.	

38.7 Writing your own generator (`source_gen`)

Example: annotate an enum with `@EnumLabels()` and generate a `label` extension.

Package layout (a monorepo with 3 packages):

```
packages/
  enum_labels_annotation/lib/enum_labels_annotation.dart
  enum_labels_generator/
    lib/builder.dart
    lib/src/enum_labels_generator.dart
    build.yaml
  app/  (uses both)
```

Annotation package

```dart
// enum_labels_annotation.dart
class EnumLabels {
  const EnumLabels();
}
const enumLabels = EnumLabels();
```

Generator

```dart
// lib/src/enum_labels_generator.dart
import 'package:analyzer/dart/element/element.dart';
import 'package:build/build.dart';
import 'package:enum_labels_annotation/enum_labels_annotation.dart';
import 'package:source_gen/source_gen.dart';

class EnumLabelsGenerator extends GeneratorForAnnotation<EnumLabels> {
  @override
  String generateForAnnotatedElement(Element element, ConstantReader annotation, BuildStep
buildStep) {
    if (element is! EnumElement) {
      throw InvalidGenerationSourceError('@EnumLabels can only be used on enums.', element:
element);
    }
    final name = element.name;
    final cases = element.fields.where((f) => f.isEnumConstant).map((f) {
      final label = f.name.replaceAllMapped(RegExp('([a-z])([A-Z])'), (m) => '${m[1]} ${m[2]}');
      final pretty = label[0].toUpperCase() + label.substring(1);
      return "      $name.${f.name} => '$pretty',";
    }).join('\n');

    return '''
extension ${name}Labels on $name {
  String get label => switch (this) {
$cases
  };
}
''';
```

```
  }
}
```

The `analyzer` package's element API has changed across major versions (a new "element2" model was introduced in 2025). Pin compatible `analyzer` / `source_gen` versions and follow their migration guides.

Builder factory

```
// lib/builder.dart
import 'package:build/build.dart';
import 'package:source_gen/source_gen.dart';
import 'src/enum_labels_generator.dart';

Builder enumLabelsBuilder(BuilderOptions options) =>
    SharedPartBuilder([EnumLabelsGenerator()], 'enum_labels');   // writes into the shared .g.dart
```

Generator's build.yaml

```
builders:
  enum_labels:
    import: "package:enum_labels_generator/builder.dart"
    builder_factories: ["enumLabelsBuilder"]
    build_extensions: {".dart": [".enum_labels.g.part"]}
    auto_apply: dependents           # applies automatically to packages that depend on this
generator
    build_to: cache                  # the .g.part goes to cache; combining_builder writes the
final .g.dart
    applies_builders: ["source_gen:combining_builder"]
```

Usage in the app

```
import 'package:enum_labels_annotation/enum_labels_annotation.dart';
part 'status.g.dart';

@enumLabels
enum OrderStatus { pending, outForDelivery, delivered }

// OrderStatus.outForDelivery.label == 'Out For Delivery'
```

Testing generators: `package:build_test`'s `testBuilder(...)` with input assets and expected outputs. Also add a golden-file test of the generated code.

Other generator types: `LibraryBuilder` (a standalone `.dart` output), and plain `Builder` implementations for non-Dart outputs such as assets and JSON. Keep generators **deterministic and fast**: no network, and no timestamps in the output.

38.8 Codegen problems & fixes

Problem	Fix
`_$UserFromJson isn't defined`	`part 'user.g.dart';` missing/misspelled, or build_runner not run
`Could not generate fromJson code for X`	Nested type not serializable → annotate it, add `explicitToJson`, or add a `JsonConverter`
Builds take minutes	`generate_for`, fewer annotated files, update build_runner, split into packages
Generated code changed after upgrading a generator	Expected; regenerate & review; pin versions in the lock file
Version solving fails between analyzer/source_gen/generators	Upgrade all generators together; check `flutter pub deps`; rarely, `dependency_overrides` temporarily
Freezed: "missing implementations"	Wrong class form for your freezed version (`abstract class X with _$X` in freezed 3) or the `.freezed.dart` part is missing
Riverpod generator: `_$Counter` not found	Class name/provider naming, `part`, `riverpod_annotation` import

Part B — Publishing packages & plugins to pub.dev 📦

38.9 Before you publish: checklist

1. **Name:** lowercase_with_underscores, unique on pub.dev, not misleading (don't use `flutter_` names that imply official packages unless you're allowed).
2. **pubspec.yaml:**

```yaml
name: smart_debouncer
description: >-
  A tiny, well-tested debouncer and throttler for Dart and Flutter with
  cancellation, leading/trailing edges and stream transformers.   # 60-180 characters recommended
version: 1.0.0
repository: https://github.com/you/smart_debouncer
issue_tracker: https://github.com/you/smart_debouncer/issues
documentation: https://github.com/you/smart_debouncer#readme   # optional
topics: [debounce, throttle, streams, utilities]               # up to 5 topics, lowercase
screenshots:
  - description: Debounced search demo
    path: doc/screenshots/search.gif
funding:
  - https://github.com/sponsors/you
environment:
  sdk: ^3.5.0
  flutter: ">=3.24.0"        # only if it depends on Flutter
dependencies:
```

```
  meta: ^1.15.0              # keep constraints WIDE (caret), so apps can resolve versions
dev_dependencies:
  test: ^1.25.0
  lints: ^5.0.0
```

3. **README.md**: what it does, a quick example at the top, installation, features, platform support table, limitations, a contributing link. pub.dev renders this as your home page.
4. **CHANGELOG.md**: an entry for every version (`## 1.0.0` + bullet points). It's required for full pub points.
5. **LICENSE**: an OSI-approved license (MIT, BSD-3, Apache-2.0). pub.dev detects it.
6. **example/**: `example/lib/main.dart` (or `example/example.dart` for Dart packages). pub.dev shows it in the Example tab.
7. **Public API docs**: `///` on every public member. Check with `dart doc`. pub points include doc coverage (20%+ is the minimum for full points, but aim for 100%).
8. **`lib/` structure**: public API in `lib/smart_debouncer.dart` (exports), implementation in `lib/src/` (private by convention).
9. **Tests & CI** (format, analyze, test on multiple SDK versions).
10. `.pubignore` to exclude large files from the archive (works like .gitignore; if present it replaces .gitignore for publishing).

38.10 Pub points (max 160) and how to earn them

pub.dev scores packages with **pana**. Run it locally: `dart pub global activate pana && pana .`

Category	Requirements
Follow Dart file conventions	**Valid pubspec, README, CHANGELOG, OSI license**
Provide documentation	**Example + ≥20% documented public API**
Platform support	**Works on multiple platforms (no unconditional dart:io for "all platforms"), declared plugin platforms**
Pass static analysis	**No errors/warnings/lints; formatted code**
Support up-to-date dependencies	**Works with the latest versions of dependencies & SDK**
Null safety / modern SDK	**Sound null safety & a recent SDK constraint (Dart 3 compatible)**
Popularity is now shown as downloads. Likes are separate.	

38.11 Publish

```
dart format .
dart analyze            # or flutter analyze
dart test               # or flutter test
dart pub publish --dry-run   # validates & shows the files that will be uploaded
dart pub publish             # asks for confirmation; first time opens Google login
```

- **Publishing is permanent.** You can't delete a version. You can:

    - **retract** a version within **7 days** of publishing (pub stops picking it for new resolutions)
    - mark the package **discontinued** (optionally pointing to a replacement)
    - mark it unlisted
- **Verified publisher**: create one on pub.dev for your domain (domain verified through Google Search Console) and transfer packages to it. It shows a ✓ badge and adds trust, which companies care about.
- **Prereleases**: `2.0.0-dev.1`, `2.0.0-beta.2`. Users must opt in explicitly.
- **SemVer discipline**:
    - breaking API change → major
    - new feature → minor
    - bug fix → patch
    - For `0.x`, a minor bump means breaking.

38.12 Automated publishing from GitHub Actions (OIDC, no secrets)

1. On pub.dev → package **Admin** → **Automated publishing** → enable GitHub Actions, set the repository and a tag pattern such as `v{{version}}`.
2. Add a workflow:

```yaml
# .github/workflows/publish.yml
name: Publish to pub.dev
on:
  push:
    tags: ['v[0-9]+.[0-9]+.[0-9]+*']
jobs:
  publish:
    permissions:
      id-token: write          # required for OIDC authentication with pub.dev
    uses: dart-lang/setup-dart/.github/workflows/publish.yml@v1
    # with:
    #   working-directory: packages/smart_debouncer   # for monorepos
```

For Flutter packages, use a custom job: install Flutter (`subosito/flutter-action`), then run `flutter pub publish --force` with the same `id-token: write` permission (follow the pub.dev automated publishing docs for Flutter). 3. Release: bump the version in pubspec + CHANGELOG → commit → `git tag v1.1.0 && git push --tags`.

Monorepos: tag per package (`smart_debouncer-v1.1.0`) with a matching pattern, or use **melos version/publish** to version and publish many packages, with changelogs generated from conventional commits.

38.13 Publishing a plugin: extra rules

- Declare platforms under `flutter: plugin: platforms:` with `pluginClass`/`package` (Android), `pluginClass` (iOS/macOS), `fileName` (web), `dartPluginClass` for Dart-only implementations.

- Federated plugins: publish `x_platform_interface` first, then the implementations, then the app-facing `x` package (with `default_package` endorsements).
- **iOS:** support **Swift Package Manager** (`ios/<name>/Package.swift`) and keep a CocoaPods podspec while the ecosystem transitions. Include a `PrivacyInfo.xcprivacy` privacy manifest if your plugin uses required-reason APIs or collects data.
- **Android:** a `namespace` in Gradle, minimal permissions (apps inherit them!), ProGuard/R8 consumer rules (`consumer-rules.pro`) for reflection-based code, and the 16 KB page-size compatibility of native libraries.
- Example app with integration tests. Document permissions/setup steps for every platform clearly in the README.
- Keep platform interface changes **non-breaking**: add methods with default implementations that throw `UnimplementedError`.

38.14 Maintaining packages (and judging others' packages)

- Respond to issues; label them; have a contributing guide and issue templates.
- Run CI against **stable and beta** Flutter so you catch breakages early.
- Keep dependency constraints wide but tested (`dart pub downgrade` tests the lower bounds).
- Deprecate APIs with `@Deprecated('Use x instead. Will be removed in 3.0')` before removing them.
- Security: enable 2FA on your Google account. Automated publishing via OIDC means no long-lived tokens.

38.15 Interview questions — codegen & packages

Q1. Why does Flutter rely on code generation instead of reflection? dart:mirrors is unavailable in Flutter because reflection defeats tree shaking and AOT optimizations, and macros were cancelled. Build-time generation produces plain Dart code with no runtime cost.

Q2. How do you speed up build_runner in a large project? Restrict builders with `generate_for` in build.yaml, split the code into packages, keep build_runner/generators updated, use watch mode locally, and avoid unnecessary annotated files.

Q3. What is `SharedPartBuilder` / combining builder? It lets several source_gen generators write `.g.part` fragments that the combining builder merges into a single `.g.dart` part file.

Q4. How do you publish a package and ensure quality? Complete pubspec metadata, README/CHANGELOG/LICENSE/example, documented API, analysis-clean code, tests, `pana` checks, `dart pub publish --dry-run`, then publish, preferably through automated OIDC publishing on tags.

Q5. Can you delete a published version? No. You can retract it within 7 days, discontinue or unlist the package, and publish fixed versions.

Q6. What are pub points? A score (max 160) from pana analysis: conventions, documentation, platform support, static analysis, up-to-date dependencies, and modern SDK/null safety support.

Q7. dependencies vs dev_dependencies for codegen packages? Annotation packages go in dependencies (used in source). Generators and build_runner go in dev_dependencies (build tools only).

Chapter 39 — Release Automation: fastlane (Android & iOS), Codemagic, GitHub Actions for iOS, and Patrol E2E Tests

Goal: Give you complete, copy-adaptable configuration files for automated builds and store uploads, plus end-to-end tests that can handle native UI (permission dialogs, notifications). Chapter 26 explained the concepts. This chapter has the actual files.

Tool options and flags change over time. Treat these files as solid templates, and check the official docs of fastlane, Codemagic and Patrol for the versions you install.

39.1 5-year-old picture 🏭

Releasing an app by hand is like baking 100 cakes yourself, step by step, every week, and sometimes forgetting the sugar. **Automation** is a **cake factory**: you write the recipe once (config files), press a button (push a git tag), and it bakes, checks, boxes and delivers the cakes the same way every time.

39.2 fastlane setup

```
# macOS: Ruby/Bundler recommended so everyone uses the same fastlane version
cd ios && bundle init && echo 'gem "fastlane"' >> Gemfile && bundle install && bundle exec
fastlane init
cd ../android && bundle init && echo 'gem "fastlane"' >> Gemfile && bundle install && bundle exec
fastlane init
```

This creates `ios/fastlane/` and `android/fastlane/`, each with an `Appfile` and a `Fastfile`. Commit the `Gemfile` + `Gemfile.lock`.

Credentials (never commit them)

- **iOS:** an **App Store Connect API key** (.p8): key ID, issuer ID and key content (base64) stored in CI secrets. Signing certificates & profiles go through **match** (an encrypted git repo or S3/GCS bucket, with `MATCH_PASSWORD`).
- **Android:** a **Google Play service account JSON** (Play Console → API access), stored as a CI secret, plus the upload keystore + passwords (Chapter 26).

39.3 `ios/fastlane/Fastfile`

```
default_platform(:ios)

ROOT = File.expand_path("../..", __dir__)   # __dir__ = ios/fastlane → ROOT = Flutter project root

platform :ios do
  before_all do
    setup_ci if ENV["CI"]                  # creates a temporary keychain on CI machines
  end

  def asc_key
    app_store_connect_api_key(
      key_id: ENV["ASC_KEY_ID"],
      issuer_id: ENV["ASC_ISSUER_ID"],
      key_content: ENV["ASC_KEY_CONTENT_BASE64"],
      is_key_content_base64: true
    )
  end

  desc "Sync signing certificates & profiles (read-only on CI)"
  lane :certs do
    asc_key
    match(type: "appstore", app_identifier: "com.mycompany.shop", readonly: is_ci)
  end

  desc "Build and upload to TestFlight"
  lane :beta do
    asc_key
    match(type: "appstore", app_identifier: "com.mycompany.shop", readonly: true)

    build_number = latest_testflight_build_number(app_identifier: "com.mycompany.shop") + 1

    Dir.chdir(ROOT) do
      sh("flutter", "pub", "get")
      sh("flutter", "build", "ipa", "--release",
         "--flavor", "prod", "-t", "lib/main_prod.dart",
         "--build-number=#{build_number}",
         "--obfuscate", "--split-debug-info=build/symbols",
         "--export-options-plist=ios/ExportOptions.plist")
    end

    ipa = Dir["#{ROOT}/build/ios/ipa/*.ipa"].first
    UI.user_error!("IPA not found") unless ipa

    upload_to_testflight(
      ipa: ipa,
      skip_waiting_for_build_processing: true,
      changelog: ENV["RELEASE_NOTES"] || "Bug fixes and improvements"
```

```
    )
  end

  desc "Submit the latest TestFlight build for App Store review"
  lane :release do
    asc_key
    deliver(
      submit_for_review: true,
      automatic_release: false,
      phased_release: true,              # 7-day gradual rollout
      skip_binary_upload: true,          # uses the build already on TestFlight
      skip_screenshots: true,
      force: true,                       # skip the HTML preview
      precheck_include_in_app_purchases: false
    )
  end
end
```

`ios/ExportOptions.plist` (App Store distribution with manual signing from match):

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-
1.0.dtd">
<plist version="1.0">
<dict>
  <key>method</key><string>app-store-connect</string>   <!-- older Xcode versions: app-store -->
  <key>teamID</key><string>ABCDE12345</string>
  <key>signingStyle</key><string>manual</string>
  <key>provisioningProfiles</key>
  <dict>
    <key>com.mycompany.shop</key><string>match AppStore com.mycompany.shop</string>
  </dict>
  <key>uploadSymbols</key><true/>
</dict>
</plist>
```

For manual signing, the Xcode project's Release configuration must also be set to use the match profile. Running `update_code_signing_settings` in the lane can do this. Extensions (notification service, widgets) need their own profiles in the dictionary.

39.4 `android/fastlane/Fastfile`

```
default_platform(:android)

ROOT = File.expand_path("../..", __dir__)   # android/fastlane → project root
```

```
platform :android do
  def play_json
    ENV["PLAY_SERVICE_ACCOUNT_JSON"]          # the JSON content stored as a CI secret
  end

  desc "Build AAB and upload to the internal testing track"
  lane :internal do
    codes = google_play_track_version_codes(track: "internal", json_key_data: play_json)
    version_code = (codes.max || 0) + 1

    Dir.chdir(ROOT) do
      sh("flutter", "pub", "get")
      sh("flutter", "build", "appbundle", "--release",
         "--flavor", "prod", "-t", "lib/main_prod.dart",
         "--build-number=#{version_code}",
         "--obfuscate", "--split-debug-info=build/symbols")
    end

    upload_to_play_store(
      track: "internal",
      aab: "#{ROOT}/build/app/outputs/bundle/prodRelease/app-prod-release.aab",
      json_key_data: play_json,
      release_status: "completed",          # "draft" for apps that were never published
      skip_upload_metadata: true,
      skip_upload_images: true,
      skip_upload_screenshots: true
    )
  end

  desc "Promote internal → production with a staged rollout"
  lane :promote do |options|
    upload_to_play_store(
      track: "internal",
      track_promote_to: "production",
      rollout: options[:rollout] || "0.1",  # 10%
      json_key_data: play_json,
      skip_upload_changelogs: false
    )
  end

  desc "Increase the production rollout percentage"
  lane :rollout do |options|
    upload_to_play_store(track: "production", rollout: options[:percent], json_key_data:
play_json,
                         skip_upload_aab: true, skip_upload_apk: true, skip_upload_metadata: true)
  end
end
```

Run it: `cd android && bundle exec fastlane internal`, or `bundle exec fastlane promote rollout:0.2`.

Release notes per version: `android/fastlane/metadata/android/<locale>/changelogs/<versionCode>.txt`. iOS metadata and screenshots: `ios/fastlane/metadata/` and `ios/fastlane/screenshots/` (managed by `deliver`).

39.5 GitHub Actions workflow using these lanes (iOS + Android)

```
name: Release
on:
  push:
    tags: ['v*']

jobs:
  android:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '17' }
      - uses: subosito/flutter-action@v2
        with: { channel: stable, cache: true }
      - uses: ruby/setup-ruby@v1
        with: { ruby-version: '3.3', bundler-cache: true, working-directory: android }
      - name: Signing files
        run: |
          echo "${{ secrets.ANDROID_KEYSTORE_BASE64 }}" | base64 -d > android/app/upload-
keystore.jks
          printf "storePassword=%s\nkeyPassword=%s\nkeyAlias=upload\nstoreFile=upload-
keystore.jks\n" \
            "${{ secrets.KEYSTORE_PASSWORD }}" "${{ secrets.KEY_PASSWORD }}" >
android/key.properties
      - name: Upload to Play internal
        working-directory: android
        env:
          PLAY_SERVICE_ACCOUNT_JSON: ${{ secrets.PLAY_SERVICE_ACCOUNT_JSON }}
        run: bundle exec fastlane internal

  ios:
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
        with: { channel: stable, cache: true }
      - uses: ruby/setup-ruby@v1
        with: { ruby-version: '3.3', bundler-cache: true, working-directory: ios }
      - name: TestFlight
        working-directory: ios
        env:
```

```
          ASC_KEY_ID: ${{ secrets.ASC_KEY_ID }}
          ASC_ISSUER_ID: ${{ secrets.ASC_ISSUER_ID }}
          ASC_KEY_CONTENT_BASE64: ${{ secrets.ASC_KEY_CONTENT_BASE64 }}
          MATCH_PASSWORD: ${{ secrets.MATCH_PASSWORD }}
          MATCH_GIT_BASIC_AUTHORIZATION: ${{ secrets.MATCH_GIT_BASIC_AUTHORIZATION }}   # base64
"user:token" for the certs repo
          RELEASE_NOTES: ${{ github.ref_name }}
        run: bundle exec fastlane beta
      - name: Upload Crashlytics symbols (optional)
        run: echo "firebase crashlytics:symbols:upload --app=$FIREBASE_IOS_APP_ID build/symbols"
```

Tips:

- Pin the Xcode version on macOS runners (`sudo xcode-select -s /Applications/Xcode_XX.app`) so builds don't break when the runner image updates.
- Cache CocoaPods (`ios/Pods`) keyed on `Podfile.lock`, and Gradle (`actions/setup-java` with `cache: gradle`).
- Use `concurrency:` groups to avoid two releases running at the same time.

39.6 Codemagic (`codemagic.yaml` at the repo root)

```
workflows:
  release:
    name: Android & iOS release
    instance_type: mac_mini_m2
    max_build_duration: 90
    integrations:
      app_store_connect: ASC API Key          # name of the API key added in Codemagic team
settings
    environment:
      flutter: stable
      xcode: latest
      cocoapods: default
      android_signing:
        - upload_keystore                     # keystore reference uploaded in Codemagic UI
      ios_signing:
        distribution_type: app_store
        bundle_identifier: com.mycompany.shop
      groups:
        - google_play                         # contains GCLOUD_SERVICE_ACCOUNT_CREDENTIALS
        - firebase                            # optional
      vars:
        FLAVOR: prod
    triggering:
      events: [tag]
      tag_patterns:
        - pattern: 'v*'
```

```
      include: true
scripts:
  - name: Get packages
    script: flutter pub get
  - name: Static analysis & tests
    script: |
      flutter analyze
      flutter test
  - name: Set up iOS signing in the Xcode project
    script: xcode-project use-profiles
  - name: Build Android App Bundle
    script: |
      flutter build appbundle --release --flavor $FLAVOR -t lib/main_prod.dart \
        --build-number=$BUILD_NUMBER --obfuscate --split-debug-info=build/symbols
  - name: Build iOS IPA
    script: |
      flutter build ipa --release --flavor $FLAVOR -t lib/main_prod.dart \
        --build-number=$BUILD_NUMBER \
        --export-options-plist=/Users/builder/export_options.plist
artifacts:
  - build/**/outputs/bundle/**/*.aab
  - build/ios/ipa/*.ipa
  - build/symbols/**
  - flutter_drive.log
publishing:
  google_play:
    credentials: $GCLOUD_SERVICE_ACCOUNT_CREDENTIALS
    track: internal
    submit_as_draft: false
  app_store_connect:
    auth: integration
    submit_to_testflight: true
    beta_groups:
      - Internal QA
  email:
    recipients: [mobile-team@mycompany.com]
    notify: { success: true, failure: true }
```

- `$BUILD_NUMBER` is Codemagic's incrementing build counter. You can also fetch the latest store build number with Codemagic CLI tools (`google-play get-latest-build-number`, `app-store-connect get-latest-testflight-build-number`) and add 1.
- `xcode-project use-profiles` applies the fetched signing profiles and writes `/Users/builder/export_options.plist`.
- Codemagic also has a **workflow editor UI**, but YAML is easier to review and version, which is better for teams.

39.7 Other pipeline essentials

- **Firebase App Distribution** for QA builds on every merge:
  - fastlane plugin: `firebase_app_distribution(app: ENV["FIREBASE_APP_ID"], groups: "qa", android_artifact_type: "AAB")`
  - or the CLI: `firebase appdistribution:distribute build/app/outputs/flutter-apk/app-release.apk --app $APP_ID --groups qa`
- **Version from tag:** parse `v2.4.1` → `--build-name=2.4.1`.
- **Changelog** from conventional commits (`git-cliff`, `release-please`).
- **Notifications:** Slack webhook on success or failure.
- **Secrets rotation:** keystore passwords and API keys in a secret manager. Restrict who can trigger release workflows (protected tags/environments with required reviewers).

39.8 Patrol — E2E tests that can tap native UI 🤖

`integration_test` can't tap system permission dialogs, notifications or settings. **Patrol** (LeanCode) can, and it also adds shorter finders.

Setup (summary — follow the Patrol docs for your version)

```yaml
dev_dependencies:
  patrol: ^x.y.z     # check pub.dev
patrol:
  app_name: My Shop
  android:
    package_name: com.mycompany.shop
  ios:
    bundle_id: com.mycompany.shop
```

```
dart pub global activate patrol_cli
patrol doctor
```

Native setup: an Android instrumentation test runner class (`MainActivityTest.java`) and Gradle `testInstrumentationRunner` config; an iOS `RunnerUITests` target. These steps are documented step by step in Patrol's setup guide.

A test

```dart
// patrol_test/checkout_test.dart   (older Patrol versions use integration_test/)
import 'package:flutter/material.dart';
import 'package:patrol/patrol.dart';
import 'package:my_shop/main_test.dart' as app;   // app entry with fake backend / test config

void main() {
  patrolTest(
```

```
    'user scans a product with the camera and adds it to the cart',
    ($) async {
      await app.main();
      await $.pumpAndSettle();

      await $('Log in').tap();
      await $(#emailField).enterText('qa@example.com');        // #emailField → Key('emailField')
      await $(#passwordField).enterText('Passw0rd!');
      await $(#loginButton).tap();
      await $('Home').waitUntilVisible();

      await $(#scanButton).tap();

      // Native permission dialog: Flutter-only tests can't do this
      // (Patrol 4 moved native APIs to $.platform.mobile; older versions use $.native)
      if (await $.platform.mobile.isPermissionDialogVisible()) {
        await $.platform.mobile.grantPermissionWhenInUse();
      }

      await $(#fakeScanResult).tap();                              // test hook in the scanner
screen
      await $('Add to cart').tap();
      expect($(#cartBadge).$('1'), findsOneWidget);

      // Notifications
      await $.platform.mobile.pressHome();
      await $.platform.mobile.openNotifications();
      await $.platform.mobile.tapOnNotificationBySelector(
        Selector(textContains: 'Order confirmed'),   // selector class names also differ between
Patrol versions
      );
    },
  );
}
```

Run it: `patrol test -t patrol_test/checkout_test.dart` (add `--device <id>`). Patrol runs each test in isolation, works with Firebase Test Lab / BrowserStack / emulator.wtf, and supports **hot restart** while developing tests (`patrol develop`).

E2E best practices

- A dedicated **test entry point** (`main_test.dart`) with a fake or staging backend, seeded data and disabled analytics/animations.
- Stable selectors: `Key`s or semantics identifiers, not localized text.
- Keep E2E tests few and focused on **critical journeys**: login, purchase, onboarding, deep link. Cover the rest with widget tests.
- Run them nightly or on release branches if they're slow. Use sharding across devices.
- Record videos and screenshots on failure as CI artifacts.

39.9 Interview questions — release automation

Q1. How do you automate iOS signing in CI? fastlane match (encrypted certificates/profiles repo) or Codemagic/Xcode Cloud managed signing, an App Store Connect API key for authentication, and `setup_ci` for a temporary keychain.

Q2. How do you avoid version code conflicts? Compute the build number from the store's latest build + 1, or from the CI run number. Never set it by hand.

Q3. What does a release pipeline look like for you? Tag → CI: analyze/test → build signed AAB/IPA with obfuscation → upload symbols → upload to internal track/TestFlight → QA → promote with staged rollout → monitor crash-free rate → increase the rollout.

Q4. Why Patrol over integration_test? It can interact with native UI (permission dialogs, notifications, system settings) and has better finders and test isolation, which makes real-world E2E flows testable.

Q5. Where do you store signing keys? In CI secret storage or a secret manager (base64-encoded files), never in the repo. Restrict access with protected environments, and back up keys securely.

Chapter 40 — Games with Flame, Advanced FFI (structs, callbacks, ffigen) & Custom Stream Transformers

Goal: Build a small 2D game with Flame (game loop, components, sprites, input, collisions, overlays, audio). Go deeper into native interop (structs, memory, async callbacks from native threads, generated bindings). Master stream transformation by writing your own transformers.

Part A — 2D games with Flame 🎮

40.1 5-year-old picture

A normal app redraws only when something changes. A **game** is a **cartoon that never stops**: about 60 times every second it **moves** everything a little (`update`) and **draws** everything again (`render`). That never-ending heartbeat is the **game loop**.

40.2 Flame's building blocks

Concept	What it is
`FlameGame`	The root game: owns the loop, the `world` and the `camera`
`GameWidget`	The Flutter widget that shows a game (place it anywhere in your widget tree)
`Component`	Everything in the game is a component tree (like widgets, but mutable and updated every frame)
`PositionComponent`	Has position, size, angle, scale, anchor
`SpriteComponent` / `SpriteAnimationComponent`	Shows images / animations
`World` + `CameraComponent`	The world holds game objects; the camera decides what part of the world you see (viewport, follow player)
`update(double dt)`	Called every frame; `dt` = seconds since the last frame → multiply speeds by `dt`
`render(Canvas canvas)`	Custom drawing (sprites render themselves)
Mixins	`TapCallbacks`, `DragCallbacks`, `KeyboardHandler`, `CollisionCallbacks`, `HasGameReference<T>`
Hitboxes	`RectangleHitbox`, `CircleHitbox`, `PolygonHitbox` + `HasCollisionDetection` on the game
Effects	`MoveEffect`, `ScaleEffect`, `RotateEffect`, `OpacityEffect`, `SequenceEffect` with `EffectController`
Overlays	Flutter widgets (menus, HUD) on top of the game

Concept	What it is
Bridge packages	`flame_audio`, `flame_forge2d` **(physics),** `flame_tiled` **(maps),** `flame_bloc` / `flame_riverpod`

40.3 A complete mini game: "Catch the stars" ⭐

```
dependencies:
  flame: ^x.y.z       # check pub.dev for the latest 1.x
flutter:
  assets:
    - assets/images/  # Flame's default image prefix is assets/images/
```

```
import 'dart:math';
import 'package:flame/collisions.dart';
import 'package:flame/components.dart';
import 'package:flame/events.dart';
import 'package:flame/game.dart';
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    home: Scaffold(
      body: GameWidget<StarGame>.controlled(
        gameFactory: StarGame.new,
        overlayBuilderMap: {
          'hud': (context, game) => Hud(game: game),
          'gameOver': (context, game) => GameOverMenu(game: game),
        },
        initialActiveOverlays: const ['hud'],
      ),
    ),
  ));
}

class StarGame extends FlameGame with HasCollisionDetection, DragCallbacks {
  late Basket basket;
  final score = ValueNotifier<int>(0);       // Flutter overlays listen to this
  final lives = ValueNotifier<int>(3);
  final _random = Random();
  double _spawnTimer = 0;

  @override
  Color backgroundColor() => const Color(0xFF0B1026);

  @override
  Future<void> onLoad() async {
    // The camera looks at the world; with the default viewfinder, world (0,0) is the screen
```

```
centre.
    camera.viewfinder.anchor = Anchor.topLeft;  // make (0,0) the top-left corner, like normal
screen coordinates
    basket = Basket()..position = Vector2(size.x / 2, size.y - 80);
    await world.add(basket);
  }

  @override
  void update(double dt) {
    super.update(dt);
    if (lives.value <= 0) return;
    _spawnTimer += dt;
    final interval = max(0.35, 1.2 - score.value * 0.02);   // gets harder over time
    if (_spawnTimer >= interval) {
      _spawnTimer = 0;
      world.add(Star(speed: 150 + _random.nextDouble() * 150 + score.value * 3)
        ..position = Vector2(_random.nextDouble() * (size.x - 40) + 20, -20));
    }
  }

  @override
  void onDragUpdate(DragUpdateEvent event) {
    basket.position.x = (basket.position.x + event.localDelta.x).clamp(40, size.x - 40);
  }

  void caught() => score.value++;

  void missed() {
    lives.value--;
    if (lives.value <= 0) {
      pauseEngine();
      overlays.add('gameOver');
    }
  }

  void restart() {
    world.children.whereType<Star>().forEach((s) => s.removeFromParent());
    score.value = 0;
    lives.value = 3;
    overlays.remove('gameOver');
    resumeEngine();
  }

  @override
  void onRemove() {
    score.dispose();
    lives.dispose();
    super.onRemove();
  }
```

```
}

class Basket extends RectangleComponent with HasGameReference<StarGame> {
  Basket() : super(size: Vector2(80, 24), anchor: Anchor.center, paint: Paint()..color =
Colors.amber);

  @override
  Future<void> onLoad() async => add(RectangleHitbox());
}

class Star extends CircleComponent with HasGameReference<StarGame>, CollisionCallbacks {
  Star({required this.speed}) : super(radius: 12, anchor: Anchor.center, paint: Paint()..color =
Colors.white);
  final double speed;

  @override
  Future<void> onLoad() async => add(CircleHitbox());

  @override
  void update(double dt) {
    super.update(dt);
    position.y += speed * dt;                    // ❗ frame-rate independent movement
    if (position.y > game.size.y + 20) {
      removeFromParent();
      game.missed();
    }
  }

  @override
  void onCollisionStart(Set<Vector2> intersectionPoints, PositionComponent other) {
    super.onCollisionStart(intersectionPoints, other);
    if (other is Basket) {
      removeFromParent();
      game.caught();
    }
  }
}

class Hud extends StatelessWidget {
  const Hud({super.key, required this.game});
  final StarGame game;
  @override
  Widget build(BuildContext context) => SafeArea(
        child: Padding(
          padding: const EdgeInsets.all(16),
          child: Row(children: [
            ValueListenableBuilder<int>(
              valueListenable: game.score,
              builder: (_, s, __) => Text('Score: $s', style: const TextStyle(color: Colors.white,
```

```
fontSize: 20)),
            ),
            const Spacer(),
            ValueListenableBuilder<int>(
              valueListenable: game.lives,
              builder: (_, l, __) => Text('❤️' * l, style: const TextStyle(fontSize: 20)),
            ),
          ]),
        ),
      );
}

class GameOverMenu extends StatelessWidget {
  const GameOverMenu({super.key, required this.game});
  final StarGame game;
  @override
  Widget build(BuildContext context) => Center(
        child: Card(
          child: Padding(
            padding: const EdgeInsets.all(24),
            child: Column(mainAxisSize: MainAxisSize.min, children: [
              Text('Game over! Score: ${game.score.value}', style:
Theme.of(context).textTheme.titleLarge),
              const SizedBox(height: 16),
              FilledButton(onPressed: game.restart, child: const Text('Play again')),
            ]),
          ),
        ),
      );
}
```

Shapes are used so the example runs without image files. To use images, switch to `SpriteComponent` and load sprites in `onLoad`:

```
class StarSprite extends SpriteComponent with HasGameReference<StarGame> {
  @override
  Future<void> onLoad() async {
    sprite = await game.loadSprite('star.png');    // assets/images/star.png
    size = Vector2.all(32);
  }
}
// Animations from a sprite sheet:
final animation = await game.loadSpriteAnimation(
  'player_run.png',
  SpriteAnimationData.sequenced(amount: 8, stepTime: 0.08, textureSize: Vector2(32, 32)),
);
```

Audio: `flame_audio` → `FlameAudio.play('catch.wav')`, `FlameAudio.bgm.play('music.mp3')`. Preload sounds in `onLoad`.

40.4 Game performance & architecture tips

- Movement in `update` must use `dt`. Never assume 60 fps (120 Hz screens and slow devices exist).
- Avoid allocations in `update` (reuse `Vector2`s), because GC pauses cause stutters.
- Use sprite batching (`SpriteBatch`) for many identical sprites. Use object pools for bullets and particles.
- Use the `world` + `camera` structure. Put HUD components in `camera.viewport` or use Flutter overlays.
- Game state such as score and level can live in Bloc/Riverpod via `flame_bloc` / `flame_riverpod`, or `ValueNotifier`s as above.
- Physics: `flame_forge2d` (Box2D). Tile maps: `flame_tiled`. Particles: `ParticleSystemComponent`.
- Test components with `flame_test` (`testWithFlameGame`).
- For 3D or console-quality games, use Unity, Godot or Unreal.

Part B — Advanced Dart FFI ⚙️

40.5 Structs, arrays & memory ownership

```
// native/geo.h
typedef struct { double lat; double lng; } Point;
double distance_km(Point a, Point b);
int32_t sum_array(const int32_t* values, int32_t length);
char* greet(const char* name);       // returns malloc'ed string → caller must free with free_string
void free_string(char* s);
```

```
import 'dart:ffi';
import 'package:ffi/ffi.dart';

final class Point extends Struct {
  @Double()
  external double lat;
  @Double()
  external double lng;
}

final DynamicLibrary _lib = DynamicLibrary.open('libgeo.so');   // per-platform loading (see Chapter 19)
```

```dart
final _distance = _lib.lookupFunction<Double Function(Point, Point), double Function(Point,
Point)>('distance_km');
final _sumArray = _lib.lookupFunction<Int32 Function(Pointer<Int32>, Int32), int
Function(Pointer<Int32>, int)>('sum_array');
final _greet = _lib.lookupFunction<Pointer<Utf8> Function(Pointer<Utf8>), Pointer<Utf8>
Function(Pointer<Utf8>)>('greet');
final _freeString = _lib.lookupFunction<Void Function(Pointer<Utf8>), void
Function(Pointer<Utf8>)>('free_string');

double distanceKm(double lat1, double lng1, double lat2, double lng2) {
  return using((arena) {                              // everything allocated with arena is freed
at the end
    final a = arena<Point>()..ref.lat = lat1..ref.lng = lng1;
    final b = arena<Point>()..ref.lat = lat2..ref.lng = lng2;
    return _distance(a.ref, b.ref);                   // structs are passed by value
  });
}

int sum(List<int> values) {
  final ptr = calloc<Int32>(values.length);
  try {
    ptr.asTypedList(values.length).setAll(0, values);  // fast copy into native memory
    return _sumArray(ptr, values.length);
  } finally {
    calloc.free(ptr);                                 // ❗ Dart's GC does NOT manage native
memory
  }
}

String greet(String name) {
  final namePtr = name.toNativeUtf8();
  try {
    final resultPtr = _greet(namePtr);
    try {
      return resultPtr.toDartString();
    } finally {
      _freeString(resultPtr);                         // free with the SAME allocator that
allocated it (native side)
    }
  } finally {
    malloc.free(namePtr);                             // toNativeUtf8 uses malloc by default
  }
}
```

Rules of memory ownership:

1. Whoever allocates must free, with the **matching allocator**.
2. Document ownership for every function that returns a pointer.
3. Use `Arena` / `using` for temporary allocations.

4. For native objects tied to Dart objects' lifetimes, attach a `NativeFinalizer` (calls a native free function when the Dart object is garbage collected) and implement `Finalizable`.

40.6 Callbacks from native to Dart

API	Use
`Pointer.fromFunction`	**Older API: synchronous callbacks on the same thread only, static functions only**
`NativeCallable.isolateLocal`	**Sync callbacks invoked on the isolate's own thread; can capture closures; must call `close()`**
`NativeCallable.listener`	**Callbacks invoked from any native thread. The call is posted asynchronously to the Dart isolate. Return type must be `void`. Must `close()` when done**

```
// native/worker.h
typedef void (*ProgressCallback)(int32_t percent);
void start_long_task(ProgressCallback cb);   // runs on a background native thread and calls cb repeatedly
```

```
typedef ProgressNative = Void Function(Int32);

final _startLongTask = _lib.lookupFunction<
    Void Function(Pointer<NativeFunction<ProgressNative>>),
    void Function(Pointer<NativeFunction<ProgressNative>>)>('start_long_task');

Stream<int> runLongTask() {
  late final NativeCallable<ProgressNative> callable;
  var closed = false;
  void closeCallable() {                // close exactly once
    if (closed) return;
    closed = true;
    callable.close();                   // ❗ otherwise the isolate stays alive and leaks
  }

  final controller = StreamController<int>(onCancel: closeCallable);
  callable = NativeCallable<ProgressNative>.listener((int percent) {
    if (controller.isClosed) return;
    controller.add(percent);
    if (percent >= 100) {
      closeCallable();
      controller.close();
    }
  });
  _startLongTask(callable.nativeFunction);
  return controller.stream;
}
```

After `close()`, the native side must not call the function pointer again. Design your C API so it stops calling (or unregisters) the callback before you close it.

40.7 Generating bindings

- **`ffigen`** (C/Objective-C): configure in `ffigen.yaml` and run `dart run ffigen`.

```yaml
# ffigen.yaml
name: GeoBindings
description: Bindings for libgeo
output: lib/src/geo_bindings.g.dart
headers:
  entry-points:
    - native/geo.h
```

Use the generated class: `final bindings = GeoBindings(DynamicLibrary.open(...)); bindings.distance_km(a, b);`. Newer setups can generate `@Native` external functions resolved through **native assets** instead of manual `DynamicLibrary.open`.

- **`jnigen`** (Java/Kotlin on Android): generates Dart classes that call JNI directly. For example, you can call Android SDK APIs synchronously without platform channels. Configure the classes to generate in `jnigen.yaml`.
- **Swift:** expose `@objc` APIs and use ffigen's Objective-C support (Swift-specific tooling, "swiftgen", is evolving).
- **Native assets / build hooks** (`hook/build.dart`, stable since Dart 3.10 / Flutter 3.38): your package compiles C/C++ (with `native_toolchain_c`) or downloads prebuilt binaries at build time, and Flutter bundles them for every target automatically. No more manual CMake/Gradle/Podspec wiring for pure-native packages.
- **Rust:** `flutter_rust_bridge` (high-level codegen: structs, async, streams) or native assets with cargo.

40.8 Threading & performance with FFI

- An FFI call blocks the calling isolate until it returns. Run slow native functions in a helper isolate (`Isolate.run(() => nativeHeavy())`). The `DynamicLibrary` lookups must happen inside that isolate (top-level lazy finals are per isolate, which is fine).
- `@Native(isLeaf: true)` / `lookupFunction(isLeaf: true)` marks calls that don't call back into Dart or block for long. Leaf calls are cheaper.
- Pass large data as `Pointer<Uint8>` + length, or typed data views. Avoid per-element conversions.
- Use `Uint8List` views over native memory (`asTypedList`) for zero-copy reads. Be careful with lifetimes: the view is invalid once the memory is freed. You can attach a finalizer to the view with `asTypedList(..., finalizer:)`.

Part C — Custom Stream transformers 🔁

40.9 Three ways to transform streams

1. Built-in methods: `map`, `where`, `expand`, `asyncMap`, `asyncExpand`, `distinct`, `take`, `skip`, `timeout`, `handleError`.
2. `StreamTransformer.fromHandlers` for simple element-by-element transforms.
3. `StreamTransformer.fromBind` (or implementing `StreamTransformerBase`) for anything with state or timing (debounce, throttle, buffer, switchMap).

`fromHandlers`: drop invalid items and convert errors

```
final parseInts = StreamTransformer<String, int>.fromHandlers(
  handleData: (value, sink) {
    final n = int.tryParse(value);
    if (n != null) sink.add(n);          // skip non-numbers
  },
  handleError: (error, stack, sink) => sink.addError(FormatException('Upstream: $error'), stack),
  handleDone: (sink) => sink.close(),
);
// Stream.fromIterable(['1', 'x', '3']).transform(parseInts) → 1, 3
```

`fromBind`: debounce (with correct pause/cancel handling and flushing on done)

```
import 'dart:async';

StreamTransformer<T, T> debounce<T>(Duration duration) =>
    StreamTransformer<T, T>.fromBind((Stream<T> input) {
      late StreamController<T> controller;
      StreamSubscription<T>? subscription;
      Timer? timer;
      T? pending;
      var hasPending = false;

      void flush() {
        if (hasPending) {
          hasPending = false;
          controller.add(pending as T);
        }
      }

      controller = StreamController<T>(
        sync: true,
        onListen: () {
          subscription = input.listen(
            (event) {
```

```
              pending = event;
              hasPending = true;
              timer?.cancel();
              timer = Timer(duration, flush);
            },
            onError: controller.addError,
            onDone: () {
              timer?.cancel();
              flush();                      // emit the last value instead of losing it
              controller.close();
            },
          );
        },
        onPause: () => subscription?.pause(),
        onResume: () => subscription?.resume(),
        onCancel: () {
          timer?.cancel();
          return subscription?.cancel();
        },
      );
      return controller.stream;
    });

// Usage:
// searchQueries.transform(debounce(const Duration(milliseconds: 300))).listen(search);
```

(If the input is a broadcast stream and you want the output broadcast too, use `StreamController.broadcast` and handle multiple listeners. Libraries like `stream_transform` do this for you.)

`switchMap`: only the latest async request matters

```
StreamTransformer<S, T> switchMap<S, T>(Stream<T> Function(S value) convert) =>
    StreamTransformer<S, T>.fromBind((input) {
      late StreamController<T> controller;
      StreamSubscription<S>? outer;
      StreamSubscription<T>? inner;
      var outerDone = false;

      controller = StreamController<T>(
        onListen: () {
          outer = input.listen(
            (value) {
              inner?.cancel();                        // ❗ cancel the previous inner stream
              inner = convert(value).listen(
                controller.add,
                onError: controller.addError,
                onDone: () {
```

```
                inner = null;
                if (outerDone) controller.close();
              },
            );
          },
          onError: controller.addError,
          onDone: () {
            outerDone = true;
            if (inner == null) controller.close();
          },
        );
      },
      onCancel: () async {
        await inner?.cancel();
        await outer?.cancel();
      },
    );
    return controller.stream;
  });

// queries.transform(debounce(300.ms)).transform(switchMap((q) =>
Stream.fromFuture(api.search(q))))
```

This is exactly the **"search-as-you-type without out-of-order results"** pattern from Chapter 16. In production, prefer the tested implementations in `stream_transform` (`debounce`, `throttle`, `switchMap`, `combineLatest`, `buffer`) or `rxdart`.

Testing transformers

```
test('debounce emits only the last value', () {
  fakeAsync((async) {
    final controller = StreamController<int>();
    final results = <int>[];
    controller.stream.transform(debounce(const Duration(milliseconds: 300))).listen(results.add);
    controller..add(1)..add(2)..add(3);
    async.elapse(const Duration(milliseconds: 299));
    expect(results, isEmpty);
    async.elapse(const Duration(milliseconds: 1));
    expect(results, [3]);
    controller.close();
  });
});
```

40.10 Interview questions — this chapter

Q1. Explain the game loop in Flame. On every frame, Flame calls `update(dt)` on the component tree (state changes scaled by elapsed time) and then `render(canvas)` (drawing). The FlameGame is driven by a Ticker inside GameWidget.

Q2. How do you make movement frame-rate independent? Multiply velocities by `dt` (seconds since the last frame).

Q3. How do Flutter UI and a Flame game communicate? Overlays (Flutter widgets above the game, toggled with `game.overlays`), shared notifiers/state management (flame_bloc/flame_riverpod), and the game reference passed to widgets.

Q4. Who frees memory returned from a C function? Whoever allocated it, using the matching allocator. Expose native free functions, use Arena for temporary allocations, and use NativeFinalizer for objects tied to Dart lifetimes.

Q5. How do you receive callbacks from native background threads? `NativeCallable.listener`, which posts calls to the Dart isolate asynchronously. Close it when finished.

Q6. What are native assets/build hooks? A Dart/Flutter mechanism where packages declare build hooks (`hook/build.dart`) that compile or download native code, which the toolchain bundles automatically for each target platform.

Q7. How would you implement debounce on a stream? A StreamTransformer (fromBind) with a Timer reset on each event, emitting the latest value when the timer fires, plus handling pause/resume/cancel and flushing on done. Or use stream_transform/rxdart.

Q8. What does switchMap solve? It cancels the previous inner async operation when a new value arrives, so only the latest request's results are emitted. This prevents out-of-order search results.

Chapter 41 — Interview Question Bank, Part 4: 120 Extra Questions (All Levels)

These questions are **new**. They don't repeat Chapters 29–31, and they cover the topics added in Part VII as well as trickier corners of Dart and Flutter. Level tags: 🟢 junior, 🟡 mid, 🔴 senior.

A. Dart corners (1–25)

1. 🟢 What's the difference between `List<int>.empty()`, `[]` and `const []`? `List.empty()` is fixed-length by default (can't add; use `growable: true` to allow it). `[]` is a growable empty list. `const []` is an immutable canonical list.

2. 🟢 What does `Iterable.followedBy` do? It lazily concatenates two iterables: `[1, 2].followedBy([3])` → (1, 2, 3).

3. 🟡 What is `Object.hash` / `Object.hashAll`? Helpers that combine field hash codes correctly when you override `hashCode`.

4. 🟡 What is the difference between `as` and pattern casting `case final x as int`? Both throw if the type is wrong. Inside patterns, `as` casts *and* binds. Use `case int x` when you want a non-throwing type test.

5. 🟡 Why does `print(1 == 1.0)` print true? Dart's numeric equality compares values across int/double. `identical(1, 1.0)` is false on native platforms but true on the web, where both are JS numbers.

6. 🟡 What happens when an int overflows on native vs web? Native: 64-bit two's-complement wraparound. Web: JS numbers lose precision beyond 2^53. Use `BigInt` when you need arbitrary precision.

7. 🟡 Explain `covariant` keyword. It allows an overriding method's parameter to have a narrower type than the parent's (e.g., `didUpdateWidget(covariant MyWidget old)`), with a runtime check.

8. 🟡 What's `noSuchMethod` used for? It is called for unknown member accesses (usually with `dynamic`). It's used by mock frameworks (mockito's generated mocks rely on it).

9. 🔴 How do generics and `is` checks interact at runtime? Generics are reified, so `list is List<int>` checks the actual type argument. A `List<dynamic>` containing only ints is still not a `List<int>`.

10. 🟡 `Future.value` vs `Future.sync` vs `Future.microtask` vs `Future()`?

- `Future.value(x)`: a future already completed with x (callbacks run as microtasks).
- `Future.sync(fn)`: runs fn immediately.

- `Future.microtask(fn)`: runs fn in a microtask.
- `Future(fn)`: runs fn in a later event (Timer.run).

11. 🟡 What does `await` on a non-Future value do? It wraps the value and suspends until the next microtask, so the code still yields to the event loop.

12. 🔴 What is a `Zone`-local value and a real use case? A value stored with `runZoned(zoneValues: {...})` and read with `Zone.current[key]`. It's used for request-scoped context such as correlation IDs in logs, and by test frameworks.

13. 🟢 What is `typedef` for function types used for in Flutter? Callback types like `VoidCallback`, `ValueChanged<T>`, `ValueGetter<T>`, `ValueSetter<T>`, `WidgetBuilder`.

14. 🟡 How do `sync*` generators help performance? They produce values lazily on demand, avoiding building whole lists.

15. 🟡 What's the difference between `Stream.listen` `cancelOnError: true/false`? With true, the subscription cancels on the first error. With false (the default), it keeps receiving events after errors.

16. 🟡 `StreamController(sync: true)` — when? When you want events delivered immediately during `add`, for example in transformers forwarding events. Be careful about re-entrancy.

17. 🔴 How can two isolates share large binary data efficiently? `TransferableTypedData` (transfer instead of copy), `Isolate.exit` to send results without copying, or native memory through FFI.

18. 🟡 What are wildcard variables (Dart 3.7)? `_` no longer creates a binding, so you can have several `_` parameters, e.g. `(_, _) => ...`.

19. 🟡 What do null-aware elements (Dart 3.8) look like? `[?a, ?b]` adds an element only if it's non-null. In maps: `{'k': ?v}`.

20. 🔴 What are primary constructors (Dart 3.13)? A class header that declares the constructor parameters and fields together, e.g. `class Point(var int x, var int y);`. It needs an SDK lower bound of 3.13.

21. 🟡 What are dot shorthands and when can't you use them? `.value` instead of `Type.value` when the type is known from context. You can't use them where no context type exists (e.g., `var x = .center;`).

22. 🟡 What's the difference between `final class` and `sealed class`? Final: can be instantiated, but can't be extended or implemented outside its library. Sealed: abstract, with a known set of subtypes in the same library, so switches over it are exhaustive.

23. 🔴 Why can't you `await` inside `initState` directly, and what's the right pattern? `initState` must be synchronous (the framework doesn't await it). Start an async method (`_load()`) and handle the result with `mounted` checks, or use FutureBuilder/state management.

24. 🟡 How do you write an extension on a generic type? `extension ListX<T> on List<T> { T? get secondOrNull => length > 1 ? this[1] : null; }`

25. 🔴 **Explain `Finalizer` vs `NativeFinalizer`.** Finalizer runs a Dart callback some time after an object becomes unreachable (no guarantees, not for critical cleanup). NativeFinalizer calls a native function and is more reliable for freeing native resources. Explicit `close`/`dispose` APIs are still preferred.

B. Widgets, UI & layout (26–50)

26. 🟢 **What is `FittedBox` used for?** Scaling a child down (or up) to fit its parent, e.g. preventing long labels from overflowing.

27. 🟢 **How do you give rounded corners to an image?** `ClipRRect(borderRadius: ..., child: Image(...))`, or use a `DecoratedBox`/`Container` with a `DecorationImage` and `borderRadius`.

28. 🟡 **`Flexible(fit: FlexFit.tight)` vs `Expanded`?** They are identical. Expanded is a shortcut.

29. 🟡 **Why does `Column(mainAxisAlignment: center)` not center inside a `SingleChildScrollView`?** The scroll view gives unbounded height, so the Column shrinks to its content. Use `ConstrainedBox(minHeight: constraints.maxHeight)` with a LayoutBuilder, or `SliverFillRemaining(hasScrollBody: false)` with `Center`.

30. 🟡 **How do you make two widgets in a Row the same height?** `IntrinsicHeight` with `CrossAxisAlignment.stretch` (expensive), or fixed heights, or a Table.

31. 🟡 **What is `ColoredBox` and why prefer it over `Container(color:)`?** It's a single lightweight render object that only paints a color, and it can be const.

32. 🟡 **`Material` widget — why does InkWell need it?** Ink splashes are painted on the nearest `Material`'s ink layer. Without it (or under an opaque child), the splashes are hidden.

33. 🟡 **What's the difference between `showModalBottomSheet` and `Scaffold.showBottomSheet`?** Modal: a route with a barrier, returns a Future result, dismissed on outside tap. Persistent: part of the Scaffold, no barrier, controlled with a PersistentBottomSheetController.

34. 🟡 **How do you lock text scaling for one widget (and should you)?** `MediaQuery.withClampedTextScaling(maxScaleFactor: 1.3, child: ...)` or override `textScaler`. Only clamp where layout truly breaks, because users rely on large text.

35. 🟡 **What is `SliverFillRemaining(hasScrollBody: false)` for?** It fills the remaining viewport with non-scrolling content, e.g. pushing a footer or button to the bottom when content is short.

36. 🔴 **Why can `LayoutBuilder` cause extra rebuilds?** It builds during layout, so its builder runs again whenever the incoming constraints change (for example on resize, or keyboard insets in some layouts). Keep its builder light.

37. 🟡 **What does `RepaintBoundary.toImage` enable?** Capturing a widget as an image (`(key.currentContext!.findRenderObject() as RenderRepaintBoundary).toImage(pixelRatio: 3)`), e.g. for sharing a receipt.

38. 🟡 **How do you render a widget offscreen to an image without showing it?** Use the `screenshot` package's `captureFromWidget`, or build it in an Overlay/Offstage with a RepaintBoundary, or use a `PictureRecorder` with your own painting.

39. 🟢 **What is `Badge` widget?** A Material 3 widget that shows a small label or count on top of an icon.

40. 🟡 **How do you implement a custom page transition for all routes?** `ThemeData.pageTransitionsTheme` with builders per platform, or `PageRouteBuilder`/go_router `CustomTransitionPage` per route.

41. 🟡 **How do you handle very long text with "Read more"?** Measure with `TextPainter` (`didExceedMaxLines`) inside a LayoutBuilder, toggle `maxLines`, and animate with AnimatedSize.

42. 🔴 **How does `Flow` differ from `Stack` for animated menus?** Flow's delegate repositions children with transforms at **paint time**, without relayout, so it's efficient for animating many children.

43. 🟡 **What's `ListWheelScrollView`?** A 3D wheel-like list (picker style). `CupertinoPicker` uses it.

44. 🟡 **How do you show a widget only on wide screens without rebuilding the whole page on resize?** Put a small `LayoutBuilder`/`MediaQuery.sizeOf` check around just that part, or use separate layout widgets selected high up while keeping their children const.

45. 🔴 **Why might `const` not work on your widget?** One of its constructor arguments isn't a compile-time constant (runtime values, callbacks referencing instance members, non-const objects), or the class lacks a const constructor or has non-final fields.

46. 🟡 **What is `KeyedSubtree`?** A widget that attaches a key to a subtree without adding layout. It's used to preserve identity when wrapping children dynamically.

47. 🟡 **How do you detect when a widget becomes visible on screen?** The `visibility_detector` package (for impressions and video autoplay), or scroll listeners combined with item positions.

48. 🔴 **How do you implement a sticky "Add to cart" bar that hides when scrolling down and shows when scrolling up?** Listen to `UserScrollNotification`/ScrollController direction and animate the bar with AnimatedSlide/SizeTransition. Alternatively, use `SliverFloatingHeader`-style slivers for headers.

49. 🟡 **What is `SafeArea(minimum: ...)`?** It guarantees minimum padding even on devices without system intrusions.

50. 🟡 **Explain `MediaQuery.removePadding`.** It removes inherited padding (e.g., the status bar top padding) for a subtree, such as a ListView inside a Drawer that would otherwise add top padding.

C. State, architecture & data (51–75)

51. 🟡 **How do you share state between two unrelated screens in Provider?** Provide it above MaterialApp (app scope), or above the navigator branch that contains both screens.

52. 🟡 In Riverpod, how do you reset all user-specific providers on logout? Make them depend on (`ref.watch`) an auth/user-ID provider so they rebuild when the user changes. Or invalidate them explicitly, or scope them in a nested ProviderScope that is recreated (with a new key) on login.

53. 🔴 How do you avoid "provider dependency cycles"? Restructure: extract shared logic into a third provider, pass data as parameters, or break cycles with events. Riverpod throws CircularDependencyError when it detects a cycle.

54. 🟡 In Bloc, how do you initialize data when a screen opens? Add an event in the BlocProvider's `create`: `create: (_) => ProductsBloc(repo)..add(ProductsRequested())`.

55. 🟡 When should a Cubit be global vs per-screen? Global for app-wide state (auth, cart, settings). Per-screen for feature state that should reset when the user leaves. Scope it with route-level providers.

56. 🔴 How do you keep a long form's state across navigation and process death? Form state in a state holder scoped to the flow (not the widget), drafts persisted to storage with a debounce, and restoration IDs for the current step.

57. 🟡 What is the Unit of Work / transaction concern on mobile? Group local DB writes in transactions so related data stays consistent, e.g. order + order items.

58. 🟡 How do you model "loading more" while showing existing items? State with `items`, `isLoadingMore`, `hasMore` and `loadMoreError`, separate from the initial `status`.

59. 🔴 How do you implement optimistic updates with rollback? Update the UI state immediately, call the API, and restore the previous state on failure with a message. Server responses reconcile the final values.

60. 🟡 How do you store dates correctly? Store UTC timestamps (ISO-8601 or epoch milliseconds) and convert to local time only for display (`toLocal()`, intl formatting). Take care with time zones in scheduling (the `timezone` package).

61. 🟡 How do you handle API pagination when items can be added while scrolling? Cursor-based pagination and deduplication by ID.

62. 🔴 How would you design a caching policy for product details? Memory cache with a TTL, a persistent cache for offline use, stale-while-revalidate, invalidation on updates (events/push), ETags for validation, and cache keys that include locale/currency.

63. 🟡 Why separate DTOs from domain models? To isolate the app from API changes, add computed fields, use proper types (DateTime, enums) and validate data at the boundary.

64. 🔴 How do you handle schema migrations for an offline-first app shipped to millions? Versioned migrations tested from every older schema version (Drift migration tests). Never edit past migrations. Back up or recover on failure. Make migrations idempotent. Watch crash rates after release.

65. 🟡 What's the difference between `ref.invalidate` and `ref.refresh`? invalidate marks the provider for recomputation the next time it is read. refresh invalidates and reads it immediately, returning the new value.

66. 🟡 How do you test a widget that depends on a Riverpod provider with async data? `ProviderScope(overrides: [provider.overrideWith(...)])` with fake data. Pump, then pump again (or pumpAndSettle) to get past loading. Assert on data, loading and error states.

67. 🔴 What is "server-driven UI" and its trade-offs? The backend sends a UI description (JSON) and the app renders it with predefined components (`rfw`, custom renderers). Benefits: changes without app releases, A/B tests. Costs: complexity, limited flexibility, versioning, performance and debugging difficulty.

68. 🟡 How do you implement multi-language content coming from the API? Send `Accept-Language` headers, cache data per locale, and refresh data when the locale changes.

69. 🟡 Explain the outbox pattern on mobile. User actions are stored locally as pending operations and synced in order with retries. The UI reflects their local state immediately.

70. 🔴 How do you guarantee exactly-once processing of user actions with retries? Client-generated idempotency keys (UUIDs) stored with each operation. The server deduplicates by key.

71. 🟡 How do you securely refresh tokens on app resume? On `resumed`, check the token's expiry, then refresh proactively (single-flight). Handle refresh failure by logging out.

72. 🟡 What's dependency inversion in a Flutter repository layer? Blocs/view models depend on an abstract repository interface. Implementations are injected (real, or fake in tests).

73. 🔴 How would you split a 2,000-line Bloc? Split by sub-feature into smaller blocs/cubits. Move business rules into domain services or use cases. Compose them through the presentation layer or shared repositories.

74. 🟡 How do you prevent memory leaks with get_it singletons holding user data? Use session scopes (`pushNewScope` / `popScope`) or reset and recreate them on logout. Avoid caching user data in app-wide singletons.

75. 🟡 What does `AsyncValue.guard` replace? A manual try/catch that converts exceptions into an AsyncError.

D. Platform, media, maps & payments (76–95)

76. 🟡 How do you keep the screen awake during video playback or navigation? `wakelock_plus` (`WakelockPlus.enable()` / `disable()`), released when the screen is no longer used.

77. 🟡 How do you share an image to WhatsApp or other apps? `share_plus` with an `XFile` (save the bytes to a temp file first).

78. 🟡 How do you open the app settings page? `openAppSettings()` (permission_handler) or `Geolocator.openAppSettings()`.

79. 🟡 What's the difference between `ResolutionPreset.high` and `max` in the camera plugin? High is about 720p–1080p depending on the platform (check the docs). Max is the highest the device supports. Higher presets use more memory and bandwidth.

80. 🔴 How do you process camera frames with ML in real time without jank? Image stream with throttling (one frame in flight), a lower resolution, processing in native ML libraries or isolates, and reusing buffers.

81. 🟡 How do you autoplay only the visible video in a feed? visibility_detector or scroll position math. Keep one active controller and pause the others.

82. 🟡 What are restrictions for background location on Google Play? Declaration form, prominent disclosure, core-feature justification and a video demo. Without approval the app can be rejected.

83. 🟡 Why might a Google map show but markers be missing? Markers set before the map is created, an empty marker set, wrong coordinates (lat/lng swapped), or invalid custom icon loading.

84. 🔴 How would you animate a moving driver marker smoothly? Interpolate between GPS updates with an AnimationController/Tween over LatLng and bearing, update the marker at a throttled rate, and use dead reckoning for gaps.

85. 🟡 Why must `completePurchase` be called? Android auto-refunds purchases not acknowledged within 3 days. On iOS, unfinished transactions are delivered again repeatedly.

86. 🟡 What is a subscription grace period? After a renewal payment fails, the user keeps access for a configured time while the store retries the payment.

87. 🔴 How do you prevent users from sharing premium access across accounts? Server-side binding of purchase tokens or original transaction IDs to one account. Reject or transfer them per policy, and handle restores carefully.

88. 🟡 Why does Stripe's payment sheet require FlutterFragmentActivity? The Stripe Android SDK uses fragment-based UI (AndroidX Activity Result APIs), which needs a FragmentActivity host.

89. 🟡 How do you open WhatsApp/phone/email from Flutter? `url_launcher` with `https://wa.me/<number>`, `tel:`, `mailto:` URIs. Add the `<queries>` entries (Android 11+) and `LSApplicationQueriesSchemes` (iOS) when you check with `canLaunchUrl`.

90. 🟡 How do you create home-screen widgets for a Flutter app? Native widget UI (Glance/RemoteViews on Android, WidgetKit/SwiftUI on iOS). Share data through `home_widget` (App Groups / SharedPreferences) and trigger updates from Flutter.

91. 🔴 How would you integrate a native SDK that requires `Application.onCreate` initialization? A custom Android `Application` class (registered in the manifest) that initializes the SDK, or the plugin's `onAttachedToEngine`. For iOS, AppDelegate `didFinishLaunching`. Expose functionality through Pigeon.

92. 🟡 What is `FlutterFragmentActivity` vs `FlutterActivity`? FlutterFragmentActivity extends FragmentActivity, which some plugins need (Stripe, local_auth biometric prompts on Android). FlutterActivity is lighter.

93. 🟡 How do you implement biometric login with `local_auth`? `canCheckBiometrics` / `isDeviceSupported` → `authenticate(localizedReason: ...)` → on success, unlock a credential from secure storage. On Android, MainActivity must be a FlutterFragmentActivity.

94. 🔴 How do you test payments end to end? Sandbox environments (Stripe test cards, Razorpay test mode, Play license testers, Apple sandbox/StoreKit configs), backend webhooks against a staging environment, and Patrol tests for native sheets where possible.

95. 🟡 How do you handle the Android "Don't keep activities" scenario during payments? Recover pending orders at startup by checking the backend order status. Use restoration for UI state, and rely on webhooks for the truth.

E. Tooling, testing, release & teams (96–120)

96. 🟡 What is `build.yaml` used for? Configuring code generators: options, which files to generate for (`generate_for`), enabling/disabling builders, and defining your own builders.

97. 🟡 How do you write a custom lint rule? With the analyzer plugin system (Dart 3.10+ supports first-party analyzer plugins) or `custom_lint` (older approach). Define rules that visit the AST and report diagnostics, optionally with quick fixes.

98. 🟡 How do you test a widget's appearance across themes and text scales? Golden tests over a matrix of light/dark themes, text scales 1.0/2.0, RTL and screen sizes (e.g., alchemist scenarios).

99. 🟡 What is `flutter test --tags` for? Running subsets of tests marked with `@Tags(['golden'])` or `tags:` (e.g., skip slow tests locally).

100. 🔴 How do you reduce flakiness in integration tests? Avoid fixed delays (wait for conditions), use deterministic test data and backends, disable animations, use stable keys, isolate tests, and add retries only as diagnostics while fixing root causes.

101. 🟡 How do you run tests on multiple Flutter versions in CI? A matrix strategy over Flutter versions/channels (stable + beta) with subosito/flutter-action.

102. 🟡 What is `dart fix`? It applies automated fixes for deprecated APIs and lints. Run `dart fix --dry-run` to preview and `--apply` to apply.

103. 🔴 How would you measure and enforce app size budgets? `--analyze-size` in CI producing JSON, comparing against a baseline and failing when a threshold is exceeded. Track download sizes from store consoles.

104. 🟡 What's the difference between internal testing, closed testing and open testing on Play? Internal: up to 100 testers, quick availability. Closed: invite lists/groups. Open: public opt-in beta. All come before production.

105. 🟡 What is phased release on the App Store? An automatic 7-day gradual rollout of an update to users with automatic updates, which can be paused.

106. 🔴 How do you handle a breaking backend change while old app versions exist? API versioning, keeping old endpoints during a deprecation window, a minimum version gate (force update), and monitoring version adoption.

107. 🟡 How do you add a debug-only menu without shipping it in release? Guard it with `kReleaseMode`/flavors (const conditions get tree-shaken), or put it in a separate entry point used only for dev flavors.

108. 🟡 What is a CODEOWNERS file? A GitHub/GitLab file mapping paths to owners, who automatically become required reviewers for changes in their areas.

109. 🔴 How do you onboard 10 new developers quickly to a Flutter monorepo? One-command setup scripts (FVM + melos bootstrap), architecture docs/ADRs, a sample feature walkthrough, good first issues, pairing, consistent lints/templates (mason bricks).

110. 🟡 What is `mason`? A template generator (bricks) to scaffold features, files and tests consistently, e.g. a new feature folder with bloc + repository + tests.

111. 🟡 How do you write a good bug report as a mobile developer? Steps to reproduce, expected vs actual behavior, device/OS/app version/build number, logs/stack traces, screenshots/video, frequency, and regression info.

112. 🔴 How do you prioritize performance work vs features? Measure the user impact (crash/ANR/jank metrics, conversion correlation), set performance budgets/SLOs, fix the worst offenders first, and reserve capacity every sprint.

113. 🟡 How do you keep design and implementation consistent? A design token pipeline, a component library, widgetbook, golden tests, and design reviews on PRs with screenshots.

114. 🟡 What do you check before upgrading a major package version? Changelog breaking changes, migration guides, transitive dependency impact, platform requirement changes (minSdk/iOS version), and tests on a branch.

115. 🔴 How would you migrate an app from GetX to a different architecture without stopping feature work? Strangler pattern: new features use the new architecture, adapters bridge shared services, migrate screen by screen with tests, remove GetX routing last, and track progress.

116. 🟡 What metrics matter after a release? Crash-free users/sessions, ANR rate, startup time, key funnel conversions, rating/review trends, error rates, adoption of the new version.

117. 🟡 How do you communicate a delay to stakeholders? Early and with facts: what's done, what's blocked, impact, options (scope cuts, phased release), and new estimates with confidence levels.

118. 🔴 Design question: offline-capable attendance app with geofencing and anti-fraud. Key points? Geofence + GPS checks, mock location detection, device attestation, selfie with liveness check (ML Kit/native SDK), local queue with signed timestamps (server time sync), background sync, encryption at rest, admin rules from remote config.

119. 🔴 Design question: real-time collaborative notes app. Key points? CRDTs or operational transforms for conflict-free merging, local-first DB, WebSocket sync, presence indicators, offline edits merged later, version history, end-to-end tests of merge cases.

120. 🟡 What would you improve first in a legacy Flutter app you just joined? Get it building reliably (version pinning, CI). Add crash reporting/analytics if missing. Fix critical lints/deprecations. Add tests around the riskiest flows. Then improve architecture incrementally, without a rewrite.